

UNIVERSIDADE DO MINHO

TEXTO PEDAGÓGICO DE APOIO

Investigação Operacional

Cláudio M. Martins Alves

2008

Índice

1	Introdução	1
1.1	Caracterização	1
1.2	Notas Históricas e Desenvolvimento	4
1.3	Metodologia da Investigação Operacional	10
1.4	Estrutura	14
2	Programação Linear	17
2.1	Introdução	17
2.2	Modelos de Programação Linear	18
2.2.1	Exemplo	18
2.2.2	Elementos de um Modelo de Programação Linear	20
2.2.3	Formulação Geral	23
2.2.4	Soluções de um Modelo de Programação Linear	25
2.3	Resolução de Modelos de Programação Linear	31
2.3.1	Aspectos Práticos	31
2.3.2	O Método Simplex	31
2.3.3	Interpretação dos Resultados	39
2.3.4	Determinar uma Primeira Solução Válida de Base	40
2.3.4.1	Solução Imediata	41
2.3.4.2	Método das 2-Fases	42
2.3.4.3	Método do Grande M	46
2.3.5	Degenerescência	48
2.4	Representação Matricial	51
2.5	Método Simplex Revisto	52
2.6	Métodos de Ponto Interior	55
2.7	Dualidade	55
2.7.1	Correspondência Primal-Dual	56
2.7.2	O Método Simplex Dual	61
2.8	Análise de Sensibilidade	64
2.9	Conclusões	68
2.10	Exercícios	69
3	Programação Inteira	77
3.1	Introdução	77
3.2	Modelos de Programação Inteira	77
3.2.1	Exemplos	78

3.2.2	Tipos de Modelos	81
3.2.3	Modelação com Variáveis Binárias	86
3.2.3.1	Condições Lógicas	86
3.2.3.2	Restrições Impostas Condicionalmente	90
3.2.3.3	Funções Não-Lineares: um Exemplo	94
3.3	Qualidade dos Modelos	95
3.3.1	Relaxação Linear e Intervalo de Integralidade	95
3.3.2	Propriedade da Total Unimodularidade	98
3.4	Melhoria dos Modelos	99
3.4.1	Limites nas Variáveis	100
3.4.2	Análise de Coeficientes	102
3.4.3	Restrições de Cobertura	104
3.4.4	Teste e Fixação do Valor das Variáveis	106
3.4.5	Restrições Desagregadas	107
3.5	Planos de Corte	108
3.5.1	Funções Superaditivas	109
3.5.2	Inequações de Chvátal-Gomory	110
3.5.3	Corte Fraccionário de Gomory	112
3.5.4	Corte Inteiro Misto de Gomory	113
3.6	Resolução de Modelos de Programação Inteira	114
3.6.1	Método de Partição e Avaliação	115
3.6.2	Métodos Baseados em Planos de Corte	127
3.7	Conclusões	130
3.8	Exercícios	130
4	Fluxos em Rede	145
4.1	Introdução	145
4.2	Problema de Transportes	146
4.2.1	Definição do Problema	146
4.2.2	Formulação de Programação Linear	149
4.2.3	Método Simplex para Problemas de Transportes	151
4.2.4	Problema de Transbordo	162
4.3	Problema de Afectação	164
4.3.1	Definição do Problema	164
4.3.2	Algoritmo Húngaro	167
4.4	Problema de Caminho Mais Curto	170
4.4.1	Definição do Problema	170
4.4.2	Algoritmo de Dijkstra	171
4.4.3	Algoritmo de Bellman-Ford	174
4.5	Problema de Fluxo de Custo Mínimo	179
4.5.1	Definição do Problema	179
4.5.2	Formulação de Programação Linear	182
4.5.3	Método Simplex de Rede	184
4.6	Aplicações	189
4.6.1	Gestão de Projectos	189
4.6.2	Planeamento de Circuitos de Recolha e Distribuição	193

4.7	Conclusões	197
4.8	Exercícios	198
5	Programação Dinâmica	205
5.1	Introdução	205
5.2	Exemplo	206
5.3	Terminologia e Definições	209
5.4	Princípio de Optimalidade	212
5.5	Algoritmo de Programação Dinâmica	216
5.6	Outras Relações de Recorrência	216
5.6.1	Relações de Recorrência com Contribuições Descontadas	217
5.6.2	Contribuições Multiplicativas	221
5.6.3	Funções MinMax e MaxMin	222
5.7	Variantes	224
5.7.1	Espaço de Estados com Várias Dimensões	224
5.7.2	Espaço Contínuo de Estados	227
5.8	Aspectos Computacionais	229
5.9	Aplicações	231
5.9.1	Problemas de Substituição de Equipamentos	231
5.9.2	Problemas de Comparação de Sequências	236
5.10	Conclusões	240
5.11	Exercícios	240
6	Métodos Heurísticos	249
6.1	Introdução	249
6.2	Avaliação de uma Heurística	250
6.3	Métodos de Construção	251
6.3.1	Descrição Geral	251
6.3.2	Exemplo I: o Problema da Árvore de Suporte de Custo Mínimo	252
6.3.3	Exemplo II: o Problema do Caixeiro Viajante	254
6.4	Pesquisa Local	257
6.4.1	Descrição Geral	257
6.4.2	Funções de Vizinhança	259
6.4.3	Exemplo: o Problema do Caixeiro Viajante	262
6.5	Pesquisa Tabu	264
6.5.1	Descrição Geral	264
6.5.2	Memória de Curto Prazo	267
6.5.3	Memória de Longo Prazo	268
6.5.4	Exemplo I: Problema de Programação Inteira	269
6.5.5	Exemplo II: Escalonamento de Tarefas numa Máquina	273
6.6	Pesquisa por Arrefecimento Simulado	276
6.6.1	Descrição Geral	276
6.6.2	Exemplo: o Problema de Coloração de Grafos	278
6.7	Conclusões	284
6.8	Exercícios	285
	Bibliografia	289

Capítulo 1

Introdução

1.1 Caracterização

Apesar de suscitar frequentemente algumas dúvidas, a expressão *Investigação Operacional*¹ é no entanto sugestiva: essa designação engloba todo o esforço em usar resultados de investigação (em geral) na melhoria de processos e organizações. Dito de outra forma, e usando expressões comuns na época do seu surgimento oficial, a Investigação Operacional procura trazer os cientistas para um nível operacional, pondo-os ao serviço das organizações de modo a resolver diferentes questões de ordem técnica e económica.

A Investigação Operacional enquanto disciplina reconhecida como tal tem data oficial de nascimento. O termo *Operational Research* terá sido usado pela primeira vez em 1936 pelo especialista em radares Robert Watson-Watt (1892-1973). O advento oficial da disciplina coincide com o período da Segunda Guerra Mundial. Em 1940, Patrick Blackett² (1897-1974), um físico experimental da Universidade de Manchester, especialista na detecção de trajectórias de partículas ionizadas, torna-se líder de uma equipa multi-disciplinar de cientistas (matemáticos, físicos, astrofísicos) a convite do estado-maior britânico para trabalhar em problemas de defesa aérea e costeira. É também a partir dessa época que a disciplina se começa a organizar. A designação *operacional* deve muito ao contexto militar. Não deixando de fazer sentido noutros contextos, a expressão perdurará até hoje.

A Segunda Grande Guerra marca mais o reconhecimento de uma categoria disciplinar e a necessidade da sua organização do que uma descoberta esclarecida do potencial das áreas científicas em resolver problemas organizacionais. Muito antes, a

¹ *Operations Research* na terminologia anglo-saxónica. Na Europa, a designação de *Operational Research* é usada preferencialmente.

² Patrick Blackett será galardoado com o Prémio Nobel de Física em 1948 pelos seus trabalhos em física nuclear e radiações cósmicas.

Ciência tinha já sido naturalmente aplicada com esse fim. Alguns autores veem em Arquimedes um dos exemplos mais antigos. No 3º século a. C., o célebre matemático grego (287 a. C.- 212 a. C.) terá posto os seus conhecimentos científicos ao serviço da defesa da sua cidade de Siracusa contra os ataques da frota romana, concebendo catapultas e outros engenhos de guerra. O cerco durará três anos.

Outros nomes ilustres seguiram-lhe os passos: Blaise Pascal (1623-1662) contribuiu para a resolução de problemas de decisão desenvolvendo a teoria das probabilidades, Leonhard Euler (1707-1783) resolveu o problema das sete pontes de Königsberg¹ naquela que é considerada por muitos a aplicação mais antiga da teoria de grafos, Gaspard Monge (1746-1818) resolveu problemas de transporte óptimo de massas com aplicações em obras públicas², Charles Babbage (1791-1871) permitiu a extensão a toda a Inglaterra do sistema postal conhecido por *Penny Post* em 1840 graças aos seus trabalhos no domínio do transporte e triagem de correio, Frederick Taylor (1856-1915) introduziu formas de organização e gestão científica do trabalho dando origem ao chamado *taylorismo*.

Ao longo do tempo, vários métodos e técnicas nasceram e desenvolveram-se no seio da comunidade ligada à Investigação Operacional. Essa realidade leva a que essa disciplina seja hoje consensualmente definida como o conjunto de métodos e técnicas racionais usados na resolução de problemas reais em sistemas complexos. A área é naturalmente multi-disciplinar quer devido às técnicas que emprega quer do ponto de vista dos problemas que pretende resolver. Na sua génese, as equipas que a praticavam incluíam cientistas de vários quadrantes.

A Investigação Operacional tem uma forte orientação ao problema (à sua resolução) o que pressupõe uma metodologia ajustada que tenha em consideração tanto a compreensão do verdadeiro problema que se põe a montante, como a possibilidade de implementação (ou adequação) das soluções a jusante. Apesar de não haver um consenso forte no que toca à metodologia da Investigação Operacional, várias características são no entanto comuns a todas as propostas que são descritas na literatura. A correcta identificação do problema é obviamente primordial. Falhando nesse passo, corre-se o risco de mais tarde se resolver bem o problema errado. Para ser útil, a formulação do

¹O resultado do matemático e físico suíço foi publicado em 1736 num artigo em latim intitulado *Solutio problematis ad geometriam situs pertinentis*, também considerado como a primeira publicação de Investigação Operacional.

²Esses trabalhos foram publicados em 1781 numa obra intitulada *Mémoire sur la théorie des déblais et des remblais*. Monge descreve nessa obra o problema de transporte conhecido por problema de Monge-Kantorovich que só seria resolvido em 1942 pelo matemático e economista russo Leonid Kantorovitch. Esse último receberá em 1975 o prémio do Banco da Suécia em Ciências Económicas em memória de Alfred Nobel (o “Prémio Nobel” de economia).

problema deve também ter em conta aspectos inerentes à sua resolução. Caindo noutro extremo, a formulação de um problema através de modelos tecnicamente intratáveis não acrescenta qualquer espécie de valor.

A abordagem é necessariamente sistémica, privilegiando uma visão global do contexto onde surge o problema a uma visão parcelar e estanque. O carácter multidisciplinar não pode ser esquecido no momento de passar à prática. Deve-se favorecer a formação de equipas com competências diferentes desde que com potencial claro para a resolução do problema. Outro factor importante a ter em consideração prende-se com a implementação das soluções. Uma solução tecnicamente correcta que obedece às hipóteses de partida pode ser no entanto rejeitada pelas direcções se não forem devidamente acarretados aspectos de ordem social, por exemplo, ou quaisquer outros factores de risco. A determinação de uma solução não é um fim em si. Só com um acompanhamento dessa passagem *do papel* ao ambiente real se poderá ter a certeza que as soluções geradas melhoram efectivamente a situação inicial. A componente de acesso à informação e de controlo deve por isso fazer parte do exercício da Investigação Operacional. Mais adiante (na Secção 1.3) voltaremos aos aspectos metodológicos associados à execução de projectos de Investigação Operacional.

O recurso preferencial a técnicas racionais para a resolução de problemas é outra das características importantes da Investigação Operacional. A modelização do problema surge na interface entre o uso dessas técnicas e a definição formal do problema. Em muitos casos, o problema é formulado como um problema de optimização e resolvido através de métodos de optimização apropriados. A componente de optimização ocupa um lugar importante na área da Investigação Operacional. Neste texto, focamos a nossa atenção precisamente nos modelos e algoritmos¹ de optimização em detrimento de outros métodos analíticos baseados por exemplo em análise matemática.

Os algoritmos são procedimentos constituídos por uma série de passos que em conjunto realizam uma determinada tarefa. Quando executado em ciclo, um algoritmo termina num número finito de iterações logo que se verifique um determinado critério de paragem. No nosso caso, o algoritmo realiza operações matemáticas com vista a determinar boas soluções (idealmente as melhores – as soluções óptimas) para problemas de optimização. Um algoritmo que não garanta que a solução óptima seja determinada no final da sua execução é designado por *heurística*². As heurísticas são procedimentos de pesquisa de soluções que são definidas muitas vezes para resolver problemas específicos. As chamadas *meta-heurísticas* distinguem-se das outras heurísticas pelo

¹O termo *algoritmo* deve-se ao matemático árabe Al-Khuwarismi que no século IX descreveu a primeira abordagem sistemática para a resolução de equações lineares.

²O termo tem origem no grego *heuriskêin* que significa *encontrar*.

facto de possuírem uma estrutura geral, que é independente da definição particular de um problema. As meta-heurísticas podem ser potencialmente usadas na resolução de qualquer problema de optimização.

Terminada a Segunda Guerra Mundial, a Investigação Operacional passou a ser usada também em contexto civil nomeadamente na indústria e nas empresas. Contribuiu com resultados notáveis em diversas áreas de aplicação, algumas mais tradicionais do que outras. Na produção, obtiveram-se resultados no domínio da alocação de recursos, do escalonamento de tarefas, da gestão de stocks e das filas de espera, por exemplo. Outros exemplos de aplicação podem ser encontrados em áreas como os transportes, a logística, a gestão de recursos humanos, ou ainda as telecomunicações.

Finalmente, uma nota para as ferramentas ligadas ao exercício da Investigação Operacional. O rápido incremento das capacidades de cálculo dos computadores teve um impacto determinante na transferência de técnicas e conhecimentos dos laboratórios de Investigação Operacional para a sociedade em geral. Aproveitando essa tendência, nasceram e floresceram empresas de software dedicadas à implementação de software dito de Investigação Operacional. Algumas delas têm mesmo uma dimensão considerável (ILOG, ou ainda a portuguesa SISCOG de menor dimensão). Foi criada uma plataforma de desenvolvimento de software livre (COIN-OR – Computational Infrastructure for Operations Research) onde se desenvolvem diferentes projectos e através da qual são disponibilizados diversos recursos na forma de bibliotecas de software. Outras ferramentas foram integradas em aplicações tradicionais como é o caso das folhas de cálculo (Solver do Microsoft EXCEL).

Ao longo deste capítulo, voltaremos com mais detalhe a alguns dos aspectos que focamos nesta introdução.

1.2 Notas Históricas e Desenvolvimento

A criação daquilo que passará em breve a designar-se por Investigação Operacional está directamente ligada ao desenvolvimento do radar na Inglaterra no período que antecede a Segunda Grande Guerra. O grande impulso dá-se no ano de 1937 e é protagonizado por Albert Rowe (1898-1976), o então director da Bawdsey Research Station. Os recursos em homens e material sendo inevitavelmente limitados e as exigências da época sendo naturalmente extremas, havia que distribuir esses recursos de modo a garantir o melhor nível de serviço possível. Os radares e as equipas que os operavam deveriam ser usados de forma a garantir a maior eficácia na defesa aérea da Inglaterra. Na prática, pretendia-se determinar a localização dos equipamentos e a quantidade de recursos humanos a distribuir entre os serviços de detecção, transmissão e contra-ofensiva. Out-

ras questões tal como a gestão das falhas do equipamento e a sua manutenção eram também abordadas pelas equipas de Investigação Operacional.

Com início da Batalha de Inglaterra, numa altura em que grande parte da Europa está já sob controlo alemão, a formação de equipas de Investigação Operacional acelera-se dos dois lados do Atlântico. No final da guerra, a força aérea norte-americana contará com cerca de 30 equipas de Investigação Operacional.

Os problemas que essas equipas resolvem durante a guerra são diversos. Um deles consiste no transporte de cargas e tropas através do Atlântico. A urgência nessa época está em minimizar os danos causados pelos submarinos alemães (os *U-boats*), e organizar uma contra-ofensiva eficaz. Os ataques desferidos pelos *U-boats* obrigam a que os navios de transporte sejam escoltados por navios de guerra. O problema está em determinar o tamanho desses grupos de navios. Grupos pequenos são mais velozes mas obrigam a distribuir os navios de defesa por um maior número de grupos, aumentando assim a vulnerabilidade de cada um deles. Grupos maiores deslocam-se mais lentamente, mas em contrapartida aumentam as suas capacidades de defesa. A segunda opção será finalmente preferida à primeira. Na organização da contra-ofensiva, as equipas de Investigação Operacional aconselharam também os seus estados-maiores sob a forma como deveriam ser detonadas as minas anti-submarinos. A profundidade a que essas bombas deveriam ser detonadas foi ajustada. A eficácia dos ataques duplicou.

Nos Estados Unidos, um dos nomes marcantes que esteve na génese da Investigação Operacional e que mais tarde viria a contribuir de forma notável para o seu progresso é o de George Dantzig (1914-2005). Dantzig é considerado por muitos como um dos pais da Programação Linear juntamente com John von Neumann (1903-1957) e Leonid Kantorovich (1912-1986).

No final da guerra, muitos dos que tinham colaborado em equipas de Investigação Operacional continuam a sua actividade em instituições governamentais e na indústria. Os especialistas começam a organizar-se em associações. Em 1948, é criado o *Operational Research Club of Britain*, que em 1954 se tornará a *Operational Research Society*. A *International Federation of Operational Research Societies* (IFORS) reúne hoje as associações de Investigação Operacional de mais de 45 países da Europa, América e Ásia. Entre as maiores estão a norte-americana *Institute for Operations Research and the Management Sciences* (INFORMS) e a europeia *European Operational Research Societies* (EURO) que reúne por seu lado as associações de cerca de 30 países, incluindo a *Associação Portuguesa de Investigação Operacional* (APDIO).

Diversos resultados científicos passam a ser explicitamente associados ao campo agora formalmente criado da Investigação Operacional. A primeira revista da especiali-

dade (a *Operational Research Quarterly*) é criada em 1950 na Inglaterra¹. Actualmente, entre as revistas de topo encontram-se as seguintes:

- Operations Research (INFORMS);
- Management Science (INFORMS);
- Mathematical Programming (Mathematical Programming Society);
- European Journal of Operational Research (EURO);
- INFORMS Journal on Computing;
- Mathematics of Operations Research (INFORMS);
- Interfaces (INFORMS).

Em Portugal, a APDIO publica desde 1981 uma revista científica intitulada *Investigação Operacional*.

Ao longo dos últimos 60 anos, a definição de um número considerável de técnicas e abordagens marcaram a história da Investigação Operacional. Abaixo, listam-se por ordem cronológica algumas das principais contribuições feitas no pós-guerra¹.

1947 George Dantzig formula o problema de Programação Linear na sequência da sua actividade como consultor na US Air Force para o planeamento dos programas de treino e de abastecimento. O termo “programação” presta geralmente a confusões uma vez que não tem nenhuma relação directa com programas de computador. Na altura, era usado pelos militares para designar um plano de operações. Os programas de computador eram designados por “códigos”. A expressão “Programação Linear” permaneceu até hoje. O termo “programação” passou mesmo a ser usado para designar outras técnicas de optimização.

Apesar de Dantzig ser reconhecido como o investigador que contribuiu de forma mais decisiva para o desenvolvimento da Programação Linear, outras figuras de renome antecederam-no no estudo dessa técnica, nomeadamente o matemático russo Leonid Kantorovich. Em 1939, este último usa a Programação Linear para resolver um problema de corte de placas de madeira².

¹Hoje essa revista tem o título de *Journal of the Operational Research Society*.

¹Fonte: OR/MS Today, Great Moments in HistORy, Outubro de 2002. Uma referência actualizada e bem mais completa é a seguinte: Saul I. Gass, Arjang A. Assad. An Annotated Timeline of Operations Research: An Informal History. ISBN 1402081162, 9781402081163, Springer, 2006.

²Num artigo que voltou a ser publicado recentemente, o próprio Dantzig relembra a história do nascimento da Programação Linear (G. Dantzig. Linear Programming. *Operations Research*, 50(1):42–47, 2002).

- 1947 Dantzig desenvolve o método Simplex para a resolução de problemas de Programação Linear.
- 1949 Desenvolvimento do método de Monte Carlo que permite efectuar aproximações numéricas recorrendo a procedimentos aleatórios. O método foi definido por Nicholas Metropolis em 1947, e descrito num artigo publicado juntamente com Stanislas Ulam em 1949. O desenvolvimento desses métodos deve-se em grande parte à sua importância nas simulações realizadas no âmbito do Projecto Manhattan que conduziu à primeira bomba atómica. John von Neumann é também reconhecido como um dos principais actores do seu desenvolvimento.
- 1950 O estatístico norte-americano William Deming introduz o controlo estatístico de processos, o controlo da qualidade, e outras técnicas da sua autoria a engenheiros e gestores de topo japoneses, a convite da União Japonesa de Cientistas e Engenheiros. As recomendações de Deming serão seguidas, o que levará a produtividade da indústria japonesa, e a qualidade dos seus produtos, a aumentar de forma significativa. Em poucos anos, os produtos japoneses (mais baratos e de melhor qualidade) passam a inundar o mercado norte-americano. A contribuição de Deming para o desenvolvimento económico do Japão será reconhecida ao mais alto nível.
- 1951 Harold Kuhn e Albert Tucker publicam um artigo numa conferência em Berkeley (Califórnia) intitulado *Nonlinear programming* onde descrevem as condições necessárias para a optimalidade de uma solução de um problema de programação não-linear com restrições. O mérito dessa descoberta deve-se em grande parte a William Karush que as descreveu pela primeira vez na sua tese de mestrado em 1939. Por essa razão, essas condições são também conhecidas por condições de Karush-Kuhn-Tucker.
- 1953 Richard Bellman desenvolve a Programação Dinâmica. Essa técnica é usada para resolver problemas de decisão sequenciais, para os quais é possível obter uma solução de forma recursiva a partir das soluções de subproblemas de menor dimensão. Muitos problemas relevantes de optimização combinatória são resolvidos de forma eficiente usando Programação Dinâmica.
- 1953 Thomson Whitin publica o livro *The Theory of Inventory Management* onde descreve vários métodos de controlo de inventários. Em 1957, o autor publicará uma segunda edição do livro com novos resultados dos seus trabalhos de investigação.
- 1954 Carl Lemke desenvolve o método Simplex-dual que permite resolver um problema

- dual de Programação Linear sem que seja necessário reformular o problema original (o problema primal). O conceito da dualidade é comum na matemática. Em Programação Linear, a teoria da dualidade determina um conjunto de relações entre dois problemas (primal e dual). No mesmo ano, Martin Beale descreve uma variante do método de Lemke que é equivalente em termos computacionais.
- 1954 Lester Ford e Ray Fulkerson desenvolvem a teoria dos fluxos em rede, associando os seus resultados à teoria de grafos. Contratados pela US Air Force para resolver um problema de tráfego ferroviário, Ford e Fulkerson começam por abordar o problema usando Programação Linear, mas os seus trabalhos conduzi-los-ão rapidamente a algoritmos especializados. Em 1962, Ford e Fulkerson publicam um livro de referência nesse domínio intitulado *Flows in networks*.
- 1955 Dantzig publica o artigo *Linear Programming under Uncertainty* que marca o início da Programação Estocástica. A incerteza relativamente ao valor exacto dos parâmetros de um problema de optimização é considerada de forma explícita. Nos modelos determinísticos, assume-se que todos os parâmetros são conhecidos, o que em muitos casos pode não se ajustar à realidade.
- 1956 James Kelley e Morgan Walker desenvolvem o método do caminho crítico (o *Critical Path Method*, ou CPM, na terminologia anglo-saxónica). Esse método, que começou por ser designado por técnica de Kelley-Walker, é uma ferramenta de gestão de projectos que permite identificar as actividades críticas que não poderão sofrer atrasos. Em 1959, D. Malcolm, J. Roseboom, C. Clark e W. Fazar desenvolvem a técnica de avaliação e revisão de programas (o *Program Evaluation and Review Technique*, ou PERT), que considera o carácter aleatório dos tempos de execução das actividades.
- 1958 Ralph Gomory contribui para a criação da Programação Inteira, também designada por Programação Linear Inteira. Os problemas de Programação Inteira caracterizam-se por envolverem variáveis (incógnitas) que só podem tomar valores inteiros. Gomory derivou uma família de restrições (planos de corte) que ao serem adicionados a um modelo de Programação Linear garantem que a solução óptima seja sempre inteira.
- 1959 George Dantzig e Philip Wolfe desenvolvem um princípio de decomposição que permite resolver problemas de Programação Linear de grande dimensão (o método de decomposição de Dantzig-Wolfe). Em 1962, J. Benders descreve a forma dual desse método, aplicando-a na resolução de problemas de Programação Inteira.

- 1960 A. Land e A. Doig desenvolvem o método de partição e avaliação sucessivas (*branch-and-bound*, na terminologia anglo-saxónica) para resolver vários problemas de Programação Inteira (com todas as variáveis inteiras, com apenas um subconjunto de variáveis inteiras e com variáveis binárias).
- 1961 Clarence Zener lança os fundamentos de um novo campo da optimização não-linear designado por Programação Geométrica.
- 1962 Lotfi Zadeh começa a desenvolver a teoria dos conjuntos difusos. Em 1965, esse autor publica um artigo pioneiro intitulado *Fuzzy sets*. Nos conjuntos difusos, as relações de pertença são diferentes daquelas que se verificam em conjuntos clássicos. Em vez de uma relação de pertença do tipo binário (um elemento pertence ou não pertence a um conjunto), definem-se graus de pertença expressos entre 0 e 1. Esses conjuntos são usados para modelar a incerteza ou a falta de precisão da informação.
- 1965 Juris Hartmanis e Richard Stearns contribuem para o advento da teoria da complexidade, com base na qual irá ser possível classificar problemas de decisão e optimização, e algoritmos, segundo grupos específicos de complexidade.
- 1970 Michael Held e Richard Karp desenvolvem a técnica da relaxação lagrangeana.
- 1970 Início do desenvolvimento de técnicas para a tomada de decisão multi-critério a partir dos trabalhos de Milan Zeleny, Stanley Zionts, Jyrki Wallenius, Ward Edwards, Bernard Roy.
- Alguns autores fazem coincidir a data do surgimento dessa disciplina com a data de realização do 7º Simpósio em Programação Matemática que teve lugar em 1970 em Haia (Países Baixos), onde teve lugar a primeira sessão dedicada à tomada de decisão multi-critério.
- 1972 Peter Checkland desenvolve a metodologia dos *Soft Systems* para a modelação e resolução de problemas reais. Essa metodologia aplica-se aos casos em que a própria definição inicial do problema não é consensual entre as partes envolvidas. A metodologia defende uma abordagem sistémica à resolução de problemas.
- 1976 Genichi Taguchi desenvolve os chamados métodos de Taguchi com base na estatística para o controlo da qualidade.
- 1978 Abraham Charnes, William Cooper e Edward Rhodes introduzem o método de análise da envolvente de dados (*Data Envelopment Analysis*, ou DEA) no artigo *Measuring the efficiency of decision making units*. Esse método permite avaliar

a eficiência relativa de diferentes organismos (hospitais, escolas), recorrendo à Programação Linear.

1983 S. Kirkpatrick, Charles Gelatt e Mario Vecchi desenvolvem uma meta-heurística inspirada de processos da indústria metalúrgica e vidreira designada por método de pesquisa por arrefecimento simulado (*simulated annealing*).

1984 Narendra Karmarkar desenvolve um algoritmo que permite resolver problemas de Programação Linear em tempo polinomial. O algoritmo de Karmarkar é um método de ponto interior.

Antes dele, em 1978, Leonid Khachian tinha já desenvolvido um método de ponto interior (o método da elipsóide) para problemas de Programação Linear. Apesar de também ser polinomial, o grau do polinómio do algoritmo de Khachian é elevado, o que o torna ineficiente na prática.

1989 Fred Glover desenvolve a meta-heurística designada por método de pesquisa tabu.

1990 Desenvolvimento da área da gestão da cadeia de abastecimento.

1994 Lançamento do *Network-Enabled Optimization System* (NEOS) que permite resolver problemas de optimização através da Internet usando um servidor remoto.

2000 Lançamento da plataforma COIN-OR (*Computational Infrastructure for Operations Research*, inicialmente designada por *Common Optimization INterface for Operations Research*). Essa biblioteca de software livre implementa diversos métodos de Investigação Operacional.

1.3 Metodologia da Investigação Operacional

Apesar da sua história ser marcada por muitos desenvolvimentos de ordem técnica, os especialistas em Investigação Operacional não esqueceram o pragmatismo dos primeiros anos. Na prática, em projectos de Investigação Operacional, apenas uma pequena parte do tempo e do esforço acaba por ser dispendido na componente de resolução dos modelos. Nesses projectos, é particularmente relevante a fase de análise do sistema e da interacção entre as suas partes, de formulação do problema e de análise das condições de implementação das soluções obtidas.

Por volta do anos 70, ganhou força um ramo da Investigação Operacional que colocava uma maior ênfase na análise e estruturação dos problemas. Os trabalhos desenvolvidos nesse domínio deram corpo a um conjunto de metodologias e abordagens conhecidas por *soft-OR*. Neste texto, passamos em revista alguns aspectos da metodologia

geral da Investigação Operacional que engloba todas as fases desde o primeiro contacto com o problema até à implementação das soluções.

A prática da Investigação Operacional implica uma abordagem científica à resolução de problemas. Embora não haja uma metodologia unanimemente reconhecida por todos os seus praticantes, a existência de alguns passos gerais parecem não levantar muitas dúvidas. A metodologia da Investigação Operacional pode assim ser caracterizada pelas seguintes fases:

- Identificação do problema;
- Construção do modelo;
- Resolução do modelo;
- Validação do modelo;
- Implementação e controlo.

Identificação do problema

Os problemas estão raramente definidos à partida. A primeira fase de um projecto de Investigação Operacional consiste assim em identificar todos os elementos que os caracterizam, o que implica necessariamente um conhecimento do contexto em que eles se colocam. Esse conhecimento é adquirido através de uma análise do sistema, durante a qual devem ser determinadas as verdadeiras expectativas e necessidades da organização. É possível distinguir entre duas grandes categorias de sistemas: os que são determinísticos, e os que são estocásticos. Nos sistemas determinísticos, os dados são perfeitamente conhecidos, e uma determinada acção produzirá sempre o mesmo resultado, desde que se mantenham as condições iniciais. Nos sistemas estocásticos, pode ser impossível determinar com certeza o valor de alguns dos parâmetros do problema. O estudo destes sistemas deve por isso integrar factores de carácter probabilístico.

O acesso a fontes de informação para a recolha de dados assume aqui um duplo papel. Por um lado, ajuda a precisar a envolvente. Por outro, irá permitir alimentar o modelo que será construído mais tarde. Formular correctamente o problema é primordial na medida em que condiciona o sucesso de todo o projecto. Tipicamente, os erros cometidos nesta fase apenas são detectados bem mais tarde, numa altura em que se já discute o modo de implementação das soluções.

A formulação do problema passa por distinguir os elementos controláveis, as incógnitas ou variáveis passíveis de serem fixadas pelo decisor, dos elementos não controláveis

(os parâmetros) que devem ser, na medida do possível, determinados da forma mais realística. A noção de problema implica que existam graus de liberdade que definam um espaço de alternativas. As relações que possam existir entre as incógnitas do problema condicionam esse espaço. Em alguns casos, essas relações podem ser expressas na forma de equações ou inequações. O critério, ou objectivo, surge aqui como uma forma de ponderar soluções, permitindo que uma delas sobressaia e possa ser considerada como sendo a melhor opção. A fixação dos objectivos deverá ser sempre da responsabilidade do decisor.

Construção do modelo

O modelo consiste numa representação da realidade com o qual se pretende reproduzir as propriedades da situação em análise. Os modelos definem relações entre parâmetros de entrada e variáveis de saída, e podem ser usados com diferentes objectivos: para efectuar previsões, para explicar uma situação identificando relações de causa-efeito, ou para definir políticas de intervenção num sistema. Este último objectivo é talvez um dos mais comuns em Investigação Operacional. Esses modelos são usados para determinar o valor que as variáveis do problema devem assumir para que seja alcançada a situação desejada.

Ao construir um modelo, um analista assume tipicamente um conjunto de suposições, o mais próximo possível da realidade, e que lhes permitem explorar o modelo com as técnicas disponíveis. A título de exemplo, podemos citar a independência estatística entre acontecimentos, a ausência de memória segundo a qual o conhecimento de uma situação presente é suficiente para efectuar previsões, ou ainda a linearidade da qual dependem muitos dos modelos que iremos analisar neste texto.

Existem várias abordagens que podem ser usadas para construir modelos. A Programação Matemática é um exemplo. Os modelos de Programação Linear e de Programação Inteira pertencem a essa classe de modelos. Outra ferramenta muito expressiva com a qual é possível construir modelos para muitos problemas relevantes são os grafos. Os modelos de fluxo em rede que abordamos mais adiante baseiam-se nesse tipo de representações.

Resolução do modelo

Nesta fase, procura-se determinar o procedimento que irá ser usado para obter soluções a partir do modelo que acabou de ser construído. Um modelo é explorado usando

métodos específicos que dependem da própria natureza do modelo. Podemos recorrer a métodos analíticos baseados em álgebra ou análise matemática, a métodos determinísticos, ou a experiências numéricas (simulação, métodos probabilísticos). Os métodos analíticos são usados, por exemplo, para resolver equações. Entre os métodos determinísticos mais usados em Investigação Operacional encontram-se, por exemplo, técnicas de Programação Matemática como a Programação Linear ou a Programação Inteira, e algoritmos de optimização combinatória. A simulação numérica é usada para reproduzir fenómenos em computador, e baseia-se numa caracterização estatística da situação que se pretende estudar. Essa caracterização inclui factores tais como a distribuição da chegada de entidades ao sistema ou das durações de serviço.

Contudo, em alguns casos, pode ser necessário definir um procedimento de resolução específico para o modelo que foi construído. Isso acontece, por exemplo, quando os procedimentos existentes não permitem obter soluções de forma eficiente (em tempo útil).

O desenvolvimento de ferramentas informáticas veio facilitar consideravelmente o uso de técnicas de optimização. Algumas aplicações tradicionais como as folhas de cálculo passaram a integrar módulos de optimização com os quais é possível formular e resolver problemas de optimização, e analisar as soluções obtidas. Também no campo da simulação numérica, por exemplo, foram desenvolvidas linguagens específicas que vieram acelerar significativamente a fase de construção e tratamento dos modelos.

Validação do modelo

Nesta fase, procura-se confirmar que o modelo representa de forma correcta a realidade. Alguns autores comparam esse processo com a validação de um programa de computador. Uma vez terminada a implementação, o programador testa a sua aplicação para confirmar que os resultados que obtém correspondem efectivamente ao resultado esperado. O teste do modelo é feito medindo os desvios entre os dados observados e a informação que é produzida pelo modelo.

Ao construir um modelo, vários erros podem ocorrer. Um deles prende-se com a qualidade dos parâmetros. Uma amostragem deficiente pode conduzir a avaliações incorrectas. Os parâmetros do modelo podem ser validados usando, por exemplo, testes de hipóteses. A quantidade de parâmetros usados para construir o modelo pode ser também questionada. Outros erros frequentes prendem-se com a estrutura do próprio modelo. Estabelecer relações entre as variáveis de um problema nem sempre é uma tarefa trivial. A definição dessas relações deve ser conduzida com cuidado uma vez que elas condicionam de forma decisiva as soluções que são geradas pelo modelo.

Implementação e controlo

A análise e validação das soluções geradas pelos modelos antecede a sua implementação prática. Uma forma de validar uma solução consiste em tentar avaliar o impacto que teria tido se tivesse sido aplicada no passado recente da organização. Se elas não tivessem melhorado a situação nessa altura, poderá estar em causa a qualidade do modelo, ou até mesmo a correcta identificação do problema.

A implementação das soluções deve envolver de forma activa os decisores da organização. A probabilidade de sucesso de um projecto de Investigação Operacional aumenta com o grau de envolvimento dos decisores em todo o processo. Os analistas devem explicar a solução que preconizam, enquadrando-a no contexto operacional da organização. As mudanças devem ser comunicadas aos colaboradores da organização que irão ser afectados.

O controlo do processo de implementação permite corrigir falhas e desvios relativamente às recomendações dos analistas. Um controlo feito com frequência durante um período de tempo razoável permitirá também detectar alterações às condições iniciais que conduziram às soluções preconizadas, e possibilitar a aplicação de medidas correctivas.

1.4 Estrutura

Como vimos, a Investigação Operacional constitui hoje um corpo disciplinar vasto, rico dos seus métodos, aplicações e ferramentas. Neste texto, analisamos alguns modelos e métodos de Investigação Operacional que pertencem à classe das abordagens determinísticas. A lista não é exaustiva, mas inclui algumas das ferramentas mais importantes.

No Capítulo 2, descrevemos os modelos e métodos de Programação Linear. Analisamos a estrutura desses modelos, as suas soluções e os métodos de resolução existentes. Apresentamos a teoria da dualidade que permite relacionar modelos de Programação Linear aparentemente distintos. Abordamos a questão da análise de sensibilidade das soluções obtidas através desses métodos. A análise de sensibilidade é uma análise pós-otimização que se destina, por exemplo, a avaliar a robustez das soluções perante modificações ou desvios dos parâmetros do modelo.

O Capítulo 3 descreve uma extensão da Programação Linear designada por Programação Inteira. Os modelos de Programação Inteira consideram explicitamente o facto das incógnitas dos problemas poderem assumir apenas valores inteiros. Em termos de

modelação e de resolução, as diferenças com a Programação Linear são consideráveis. Os modelos de Programação Inteira são mais difíceis de resolver do que os modelos de Programação Linear. Nesse capítulo, abordamos a questão da qualidade dos modelos, descrevemos estratégias para melhorar essa qualidade, e apresentamos métodos alternativos de resolução.

No Capítulo 4, analisamos os modelos de fluxos em rede. Esses modelos são definidos a partir de grafos, e podem ser representados também usando modelos de Programação Linear ou Inteira. A estrutura particular destes modelos faz com que seja possível resolvê-los usando métodos específicos, mais eficientes do que os métodos genéricos de Programação Linear e Programação Inteira. Descrevemos alguns problemas clássicos, nomeadamente os problemas de transportes, de afectação, de caminho mais curto, e o problema geral de fluxo de custo mínimo. Os algoritmos de resolução desses problemas são descritos nesse capítulo, e algumas aplicações são também apresentadas.

O Capítulo 5 descreve uma abordagem de modelação e resolução de problemas de decisão e optimização sequenciais: a Programação Dinâmica. Descrevemos os elementos que caracterizam esses modelos e o procedimento de resolução. Apresentamos variantes do modelo mais simples, e discutimos também algumas aplicações desta técnica.

No Capítulo 6, apresentamos métodos alternativos aos métodos exactos de optimização: os métodos heurísticos. Esses métodos procuram boas soluções para problemas de optimização, e são usados para resolver problemas cuja complexidade inviabiliza o uso de um método exacto. Os métodos heurísticos não garantem que a solução óptima do problema seja encontrada. Nesse capítulo, exploramos diferentes tipos de métodos heurísticos. Descrevemos os métodos de construção que são usados para resolver problemas específicos, exploramos o princípio da pesquisa local, e analisamos algumas meta-heurísticas que são procedimentos genéricos que podem ser aplicados a diversos problemas.

Capítulo 2

Programação Linear

2.1 Introdução

A Programação Linear foi uma das primeiras técnicas a ser desenvolvida no âmbito da Investigação Operacional. John Von Neumann, Leonid Kantorovich e George Dantzig foram pioneiros nesse domínio, mas é a este último que se devem os principais desenvolvimentos. É Dantzig, por exemplo, que definirá o primeiro método de resolução de problemas de Programação Linear: o método Simplex. Por altura da 2ª Guerra Mundial, a Programação Linear é usada para resolver vários problemas de planeamento a pedido das autoridades militares aliadas. O sucesso que granjeou nessa altura fez com que a técnica passasse mais tarde a ser usada nas organizações. Actualmente, a Programação Linear é usada para resolver problemas reais em diversas áreas como a logística, a indústria ou a saúde.

Em termos genéricos, os modelos de Programação Linear representam problemas de optimização nos quais se pretende determinar o óptimo de uma função linear num espaço definido por restrições que são também elas lineares nas variáveis. Os métodos de resolução que foram desenvolvidos para estes modelos permitem determinar a solução óptima para problemas com uma dimensão considerável. Em 1979, Leonid Khachian viria mesmo a mostrar que estes problemas podem ser resolvidos em tempo polinomial, *i.e.*, através de algoritmos (eficientes) cujo tempo máximo de computação pode ser expresso na forma de um polinómio em ordem à dimensão do problema.

A aplicabilidade da Programação Linear faz desta técnica uma ferramenta de optimização muito relevante. Por outro lado, e como veremos nos próximos capítulos, a sua importância deve-se também ao facto de outras técnicas de optimização terem fortes ligações com a Programação Linear. É o caso da Programação Inteira e dos modelos de fluxos em rede.

Neste Capítulo, analisamos em detalhe vários aspectos ligados à formulação e res-

olução de modelos de Programação Linear. Descrevemos a teoria da dualidade que permite analisar o mesmo problema sob perspectivas diferentes, e mostramos como podem ser conduzidas análises de sensibilidade a partir destes modelos.

2.2 Modelos de Programação Linear

2.2.1 Exemplo

Para ilustrar a nossa apresentação, vamos considerar um exemplo simples.

Exemplo 2.1 Considere o caso de uma empresa que se dedica ao fabrico de 3 artigos diferentes designados por A_1 , A_2 e A_3 . A empresa trabalha 8 horas por dia, e 5 dias por semana. Os artigos são vendidos ao preço de 5, 15 e 10 euros por unidade, respectivamente. A empresa dispõe de uma única máquina para produzir esses artigos, e com a qual consegue produzir por hora 60 unidades de A_1 , 15 unidades de A_2 e 40 unidades de A_3 . A empresa sabe que não conseguirá vender por semana mais de 1000, 1200 e 800 unidades de cada um dos artigos, respectivamente. Com base nessa informação, a empresa pretende determinar o plano de produção semanal que maximize os seus lucros.

Neste caso, é possível resolver facilmente o problema usando um argumento simples. O problema da empresa está em determinar como deve ser distribuída a capacidade da máquina entre os 3 artigos. Para maximizar o seu lucro, a empresa deve produzir preferencialmente os artigos com maior rendibilidade, *i.e.* aqueles que gerem o maior lucro por unidade de tempo. Vamos usar a hora como unidade de tempo. O lucro gerado numa hora por cada um dos artigos é facilmente calculado: o artigo A_1 rende 60×5 euros = 300 euros/h, o artigo A_2 rende 15×15 euros = 225 euros/h e o artigo A_3 rende 40×10 euros = 400 euros/h. O artigo A_3 é por isso o mais atractivo. Atendendo à capacidade da máquina, a empresa pode produzir até 2400 unidades de A_3 , mas dado o limite máximo de vendas desse artigo, a empresa deve optar por produzir apenas 800 unidades (acima desse valor, o lucro é 0). Isso implica que a máquina esteja ocupada durante $800/40 = 20$ h na produção do artigo A_3 . O tempo que resta para produzir os outros artigos é de apenas 20h. Usando o mesmo raciocínio, concluimos que a empresa deve produzir 1000 unidades de A_1 e 50 unidades de A_2 .

Esta abordagem é adequada à resolução deste caso simples, mas falha no caso geral em que o número de restrições, por exemplo, pode ser bem maior. Neste capítulo, analisamos uma abordagem geral para a resolução de problemas deste género. Essa abordagem assenta numa representação algébrica do problema: um modelo de Programação Linear.

Um dos primeiros passos para construir esse modelo consiste em identificar as variáveis do problema. Vimos que o problema da empresa consiste em determinar a forma como deve ser distribuída a capacidade da máquina para a produção de cada artigo. Esse problema é equivalente a determinar o número de unidades de cada artigo que devem ser produzidas numa determinada unidade de tempo. Como o plano é semanal, vamos usar a semana como unidade de tempo. Podemos definir as nossas variáveis da forma seguinte:

Variável : Significado

x_1	: número de unidades de A_1 que devem ser produzidas por semana;
x_2	: número de unidades de A_2 que devem ser produzidas por semana;
x_3	: número de unidades de A_3 que devem ser produzidas por semana.

Outro elemento importante do problema são as suas restrições. São elas que condicionam o valor que as variáveis podem tomar. Uma das restrições do problema diz respeito à capacidade da máquina. Numa semana, a máquina funciona 8 dias $\times$ 5h/dia = 40h. Esse valor define um limite superior que não pode ser excedido, e que condiciona portanto o número de unidades de cada artigo que podem ser produzidas. Por cada unidade do artigo A_1 , é necessário $\frac{1}{60}$ h, enquanto que são necessários respectivamente $\frac{1}{15}$ h e $\frac{1}{40}$ h para produzir o artigo A_2 e A_3 . Assumindo que o número total de horas que são necessárias para produzir um tipo de artigos é directamente proporcional ao número de unidades desse artigo que são produzidas, podemos exprimir a restrição de capacidade semanal da máquina da forma seguinte:

$$\frac{1}{60}x_1 + \frac{1}{15}x_2 + \frac{1}{40}x_3 \leq 40,$$

o que é equivalente a

$$2x_1 + 8x_2 + 3x_3 \leq 4800.$$

Por definição, as variáveis deste problema são não-negativas. Por seu lado, o limite de vendas define um valor máximo para cada uma das variáveis:

$$0 \leq x_1 \leq 1000$$

$$0 \leq x_2 \leq 1200$$

$$0 \leq x_3 \leq 800$$

O objectivo da empresa é maximizar o seu lucro. Esse objectivo define um critério de selecção entre as várias soluções válidas para o problema. A expressão do lucro

depende directamente das unidades que são produzidas. Neste caso, essa expressão é também linear nas variáveis:

$$\max z = 5x_1 + 15x_2 + 10x_3.$$

□

2.2.2 Elementos de um Modelo de Programação Linear

Os modelos de Programação Matemática nos quais se incluem os modelos de Programação Linear são compostos por:

- variáveis;
- restrições;
- objectivo(s).

As variáveis correspondem às incógnitas do problema cujo valor é susceptível de ser definido pelo decisor. Podem representar quantidades, valores monetários, ordens de preferência, ou localizações, por exemplo. As variáveis estão associadas a decisões, e por essa razão são também designadas por *variáveis de decisão*. Determinar as variáveis de um modelo passa por identificar os tipos de decisões que podem ser tomadas. Espera-se que uma vez calculado o valor das variáveis, a informação que representam permita responder claramente ao problema original.

As variáveis dos modelos de Programação Linear são contínuas, e geralmente não-negativas. Variáveis que possam tomar apenas valores negativos são facilmente substituídas por variáveis não-negativas. Uma variável y negativa pode assim ser substituída por uma variável y' não-negativa fazendo:

$$y' = -y.$$

Em alguns casos, as incógnitas do problema não têm restrições de sinal, podendo tomar valores positivos, negativos ou nulos. Essas incógnitas podem ser modeladas usando um par de variáveis não-negativas. Se y for uma variável sem restrição de sinal e y' e y'' forem duas variáveis não-negativas, a variável y poderá ser substituída fazendo:

$$y = y' - y''.$$

Em modelos de maior dimensão, as variáveis são tipicamente agrupadas em conjuntos de variáveis que partilham características comuns. Um exemplo típico é o caso de um artigo que é produzido em diferentes períodos de tempo. Para representar o número

de unidades produzidas, podemos usar um conjunto de variáveis que designamos por x , e indexar cada uma das variáveis desse conjunto usando um índice t , por exemplo, com $t = 1, \dots, T$, sendo T o índice do último período. Assim, a variável x_1 representará o número de unidades do artigo produzido no 1º período, e assim sucessivamente. Essa representação facilita muito a escrita dos modelos. É mais fácil fazer referência a uma variável, e evitam-se as expressões longas e sujeitas a erros como por exemplo a seguinte:

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 + x_{10},$$

que pode ser substituída pela expressão bem mais compacta

$$\sum_{i=1}^{10} x_i.$$

As restrições traduzem relações entre as variáveis, limitando o valor que essas variáveis podem tomar. Dividem o espaço de soluções em duas partes: num sub-espaço de soluções válidas onde se encontram todas as combinações de valores admissíveis para as variáveis, e num sub-espaço de soluções não admissíveis. A forma geral de uma restrição é a seguinte:

$$\langle \text{LADO ESQUERDO} \rangle \quad \text{RELAÇÃO} \quad \langle \text{LADO DIREITO} \rangle$$

O lado esquerdo define uma expressão nas variáveis do modelo. Essa expressão deve ser linear. As variáveis são multiplicadas por coeficientes (os chamados *coeficientes tecnológicos*), que poderão ser iguais a 0, e os termos resultantes são adicionados entre eles. O lado direito da restrição é o termo independente. Quanto à relação, ela pode ser do tipo *maior ou igual* ($\geq$), *menor ou igual* ($\leq$), ou simplesmente do tipo igualdade ($=$).

Apesar de existirem muitos tipos diferentes de restrições, é possível identificar algumas restrições mais comuns. Uma delas são as restrições de disponibilidade. Essas restrições são do tipo *menor ou igual*, e limitam o valor daquilo que pode ser adquirido ou consumido, por exemplo. Essas restrições estão muitas vezes associadas a recursos escassos. No Exemplo 2.1, o limite de vendas de cada um dos artigos define uma restrição de disponibilidade.

Outro tipo de restrições são as restrições de capacidade. Essas restrições estão normalmente associadas a restrições técnicas que ocorrem no seio da empresa ou organização onde se coloca o problema. Do ponto de vista algébrico, são semelhantes às restrições de disponibilidade. Diferem no sentido em que as quantidades que representam podem ser mais facilmente alteradas pelo gestor ou decisor da empresa. No

Exemplo 2.1, existe uma restrição de capacidade associada à máquina, que define um limite máximo de horas semanais durante as quais a máquina poderá estar a funcionar. O gestor pode eventualmente aumentar o número de horas de funcionamento da máquina se essa opção for economicamente atractiva. O mesmo já não acontece com os limites de vendas dos artigos que serão muito mais difíceis de alterar por parte do gestor.

As restrições de equilíbrio definem normalmente uma relação entre duas quantidades definidas como expressões lineares nas variáveis. Se no Exemplo 2.1 fosse imposto que o número de artigos A_1 produzidos não fosse maior do que o número de unidades de A_2 , e que o número de unidades de A_2 não fosse maior que o número de unidades de A_3 , essas condições seriam respectivamente formuladas usando as seguintes restrições de equilíbrio:

$$\begin{aligned}x_1 - x_2 &\leq 0, \\x_2 - x_3 &\leq 0.\end{aligned}$$

Outro grupo de restrições são aquelas que definem requisitos a cumprir tal como uma procura, uma quantidade a produzir, ou um lucro a alcançar, por exemplo. Essas restrições definem níveis mínimos que devem ser satisfeitos, e são do tipo *maior ou igual*.

Restrições que definem limites inferiores ou superiores relativamente ao valor que as variáveis podem tomar são também usuais em modelos de Programação Linear. As expressões seguintes definem respectivamente um limite inferior e superior numa variável de decisão designada por x :

$$\begin{aligned}x &\geq 15, \\x &\leq 50.\end{aligned}$$

Num modelo de Programação Linear, o objectivo do problema (o critério de optimização) é formulado através de uma expressão linear nas variáveis designada por *função objectivo*. Essa função pode ser de minimização ou de maximização. Alguns exemplos tradicionais de funções objectivo são a maximização do lucro, a minimização dos custos, ou ainda a minimização das distâncias percorridas.

A função objectivo de um modelo de Programação Linear está normalmente associada a um único critério. Contudo, na prática, são frequentes as situações em que vários critérios são usados para avaliar uma solução. É possível tratar esses casos de diferentes formas. Uma delas consiste em usar o critério mais importante e avaliar a solução resultante à luz dos outros critérios. Outra abordagem consiste em usar rotativamente os vários critérios, e comparar as várias soluções entre elas. A ponderação

dos vários critérios expressos na forma de uma combinação linear é outra alternativa. Em alguns casos, após uma re-análise do problema, pode ser possível transformar um critério numa restrição. Um exemplo típico é o caso de um custo que, em vez de se procurar minimizar, se pode limitar a um valor máximo.

Ao construir um modelo de Programação Linear, devemos ter em atenção o facto de estarmos a fazer um conjunto de suposições relativamente aos dados e às propriedades do problema. Algumas dessas suposições foram já referidas de forma implícita na apresentação que fizemos. Podemos resumi-las da forma seguinte:

- a proporcionalidade: uma variável contribui para o valor da função objectivo e das restrições do modelo numa quantidade que é directamente proporcional ao seu valor;
- a aditividade: a contribuição do conjunto das variáveis consiste na soma das contribuições individuais de cada variável;
- a divisibilidade: as variáveis representam quantidades que podem ser divididas, podendo assumir qualquer valor fraccionário,
- o carácter determinístico: os parâmetros do problema são perfeitamente conhecidos.

2.2.3 Formulação Geral

Um modelo de Programação Linear com n variáveis e m restrições é representado na sua forma canónica conforme segue:

$$\begin{array}{ll}
 \max \ z = & c_1x_1 \quad + \quad c_2x_2 \quad + \quad c_3x_3 \quad + \quad \dots \quad + \quad c_nx_n \\
 \text{sujeito a} & \\
 & a_{11}x_1 \quad + \quad a_{12}x_2 \quad + \quad a_{13}x_3 \quad + \quad \dots \quad + \quad a_{1n}x_n \leq b_1 \\
 & a_{21}x_1 \quad + \quad a_{22}x_2 \quad + \quad a_{23}x_3 \quad + \quad \dots \quad + \quad a_{2n}x_n \leq b_2 \\
 & \dots \\
 & a_{m1}x_1 \quad + \quad a_{m2}x_2 \quad + \quad a_{m3}x_3 \quad + \quad \dots \quad + \quad a_{mn}x_n \leq b_m \\
 & x_1 \quad , \quad x_2 \quad , \quad x_3 \quad , \quad \dots \quad , \quad x_n \quad \geq \quad 0
 \end{array}$$

Os modelos de *minimização* terão restrições do tipo *maior ou igual*. De notar que uma função objectivo f do tipo *minimização* é facilmente transformada numa função do

tipo *maximização* atendendo à seguinte correspondência:

$$\max f = -\min(-f).$$

Na forma standard (ou normal), as restrições de um modelo de Programação Linear são expressas na forma de equações:

$$\begin{array}{llllllllll} \max z = & c_1x_1 & + & c_2x_2 & + & c_3x_3 & + & \dots & + & c_nx_n \\ \text{sujeito a} & & & & & & & & & \\ & a_{11}x_1 & + & a_{12}x_2 & + & a_{13}x_3 & + & \dots & + & a_{1n}x_n & = & b_1 \\ & a_{21}x_1 & + & a_{22}x_2 & + & a_{23}x_3 & + & \dots & + & a_{2n}x_n & = & b_2 \\ & \dots & & & & & & & & & & \\ & a_{m1}x_1 & + & a_{m2}x_2 & + & a_{m3}x_3 & + & \dots & + & a_{mn}x_n & = & b_m \\ & x_1 & , & x_2 & , & x_3 & , & \dots & , & x_n & \geq & 0 \end{array}$$

Uma inequação é transformada numa equação adicionando variáveis não-negativas ao termo do lado esquerdo. No caso de uma restrição do tipo *menor ou igual*, as variáveis adicionais designam-se por *variáveis de folga*. No Exemplo 2.1, a restrição ao número de horas que a máquina pode funcionar

$$2x_1 + 8x_2 + 3x_3 \leq 4800$$

pode ser transformada numa equação usando uma variável de folga F_1 :

$$2x_1 + 8x_2 + 3x_3 + F_1 = 4800,$$

com $F_1 \geq 0$. No caso de uma restrição do tipo *maior ou igual*, as variáveis adicionais designam-se por *variáveis de excesso*. Se ainda no Exemplo 2.1 houvesse uma restrição que obrigasse a que fossem produzidas pelo menos 100 unidades de A_2 , a inequação resultante

$$x_2 \geq 100$$

poderia ser transformada numa equação adicionando uma variável de excesso não-negativa E_1 :

$$x_2 + E_1 = 100.$$

2.2.4 Soluções de um Modelo de Programação Linear

A solução de um modelo de Programação Linear consiste num conjunto de valores atribuídos às variáveis de decisão, e ao qual corresponde um valor para a função objectivo. Uma solução é válida se satisfizer todas as restrições do modelo. O conjunto de todas as soluções válidas é designado por espaço de soluções válidas. Através de um modelo de Programação Linear, procuramos a solução válida que corresponde ao melhor valor da função objectivo, *i.e.* a solução óptima. De notar que essa solução não é necessariamente única.

A atenção dada aos modelos de Programação Linear deve-se em grande parte à facilidade em calcular as suas soluções óptimas comparativamente com outros modelos como os não-lineares, por exemplo. A linearidade do modelo conduz a espaços de soluções que são poliedros convexos¹, e para os quais é fácil caracterizar os pontos óptimos. Para ilustrarmos as propriedades dos espaços de soluções, vamos recorrer à representação gráfica de um modelo de pequenas dimensões (para modelos com mais de 3 variáveis, essa representação é impossível).

Exemplo 2.2 Considere o seguinte modelo de Programação Linear com 2 variáveis de decisão e 3 restrições:

$$\begin{array}{rcll} \max z = & x_1 & + & 2x_2 \\ \text{sujeito a} & & & \\ & x_1 & + & 4x_2 \leq 16 \\ & x_1 & + & x_2 \leq 7 \\ & x_1 & & \leq 6 \\ & x_1 & , & x_2 \geq 0 \end{array}$$

O modelo pode ser representado num espaço a 2 dimensões no qual os pontos correspondem a pares de valores das variáveis x_1 e x_2 (Figura 2.1).

A zona a cinzento representa o espaço de soluções válidas. Cada restrição divide o espaço em dois semi-espaços: um que contém todos os pontos que verificam a restrição, e outro que inclui todas as soluções que violam essa mesma restrição. Na fronteira, estão todos os pontos (x_1, x_2) para os quais o termo do lado esquerdo da restrição tem um valor igual ao termo independente. O espaço de soluções válidas corresponde à intersecção dos semi-espaços válidos definidos por cada uma das restrições. Para o

¹A convexidade é um conceito importante na matemática, e em particular na optimização. De forma genérica, um objecto diz-se *convexo* se o segmento que unir dois pontos que estão no interior desse objecto está também ele totalmente contido nesse objecto.

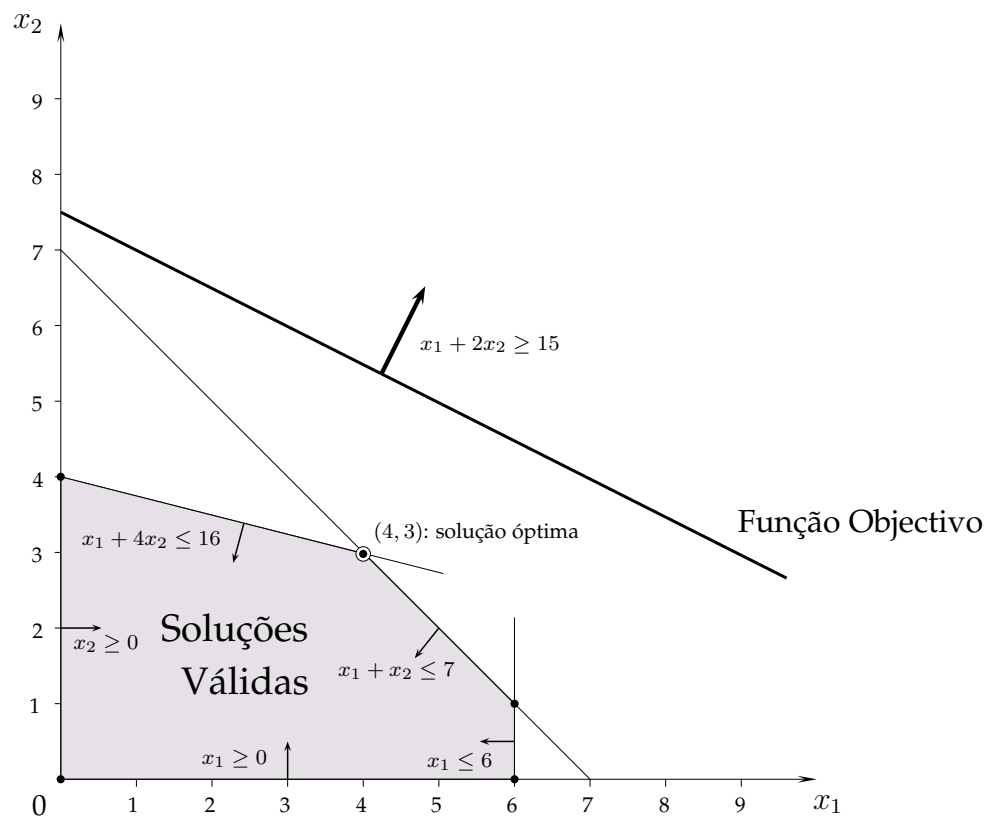


Figura 2.1: Representação gráfica do modelo do Exemplo 2.2

representar, devemos por isso desenhar a recta que determina a fronteira dos semi-espacos de cada uma das restrições, e ter em atenção o sinal da restrição (ver o sentido das setas na Figura 2.1).

A função objectivo permite escolher a solução óptima entre as diferentes soluções válidas. Na Figura 2.1, representámos a recta que corresponde a um valor de $z = 15$, *i.e.* todos os pontos (x_1, x_2) dessa recta têm um valor na função objectivo igual a 15. O valor de z aumenta no sentido apontado pela seta que está junto a essa recta. Assim, a recta que representa a função objectivo $z = 16$ estará “acima” da recta representada na Figura 2.1, e será também paralela a essa recta. O mesmo acontece qualquer que seja o valor de z , visto que o declive das rectas é definido pelos coeficientes das variáveis de decisão na função objectivo.

Maximizar a função objectivo consiste em determinar o ponto (x_1, x_2) do espaço de soluções válidas que corresponde ao maior valor de z . Esse ponto é necessariamente o último ponto intersectado por uma recta que representa a função objectivo no sentido crescente dos seus valores. Claramente, esse ponto deverá ser um ponto extremo do espaço de soluções válidas. Neste Exemplo, a solução óptima do modelo é dada pelo ponto $(4, 3)$ da Figura 2.1, e corresponde assim a termos $x_1 = 4$ e $x_2 = 3$ para um valor da função objectivo de $z = 10$. $\square$

A solução óptima encontra-se na fronteira do espaço de soluções válidas. É um ponto extremo desse espaço se for única. Se existirem soluções óptimas alternativas (com o mesmo valor da função objectivo), essas soluções estarão num segmento de recta¹ ou numa das faces do poliedro² de soluções válidas. Neste último caso, o número de soluções será sempre infinito.

Exemplo 2.3 Considere o modelo do Exemplo 2.2 com a nova função objectivo seguinte:

$$\max x_1 + x_2.$$

Essa função objectivo é paralela a uma das restrições do problema. Na Figura 2.2, podemos observar que todos os pontos que pertencem ao segmento a negro correspondem ao mesmo valor da função objectivo $z = 7$, que é também o melhor valor da função objectivo que pode ser alcançado. Qualquer um desses pontos é por isso uma solução óptima do problema. $\square$

Um modelo de Programação Linear poderá também não ter nenhuma solução óptima. Distinguem-se dois casos:

¹No caso dos modelos com 2 variáveis.

²Definido por um plano para modelos com 2 variáveis, e por um hiperplano para modelos com mais de 3 variáveis.

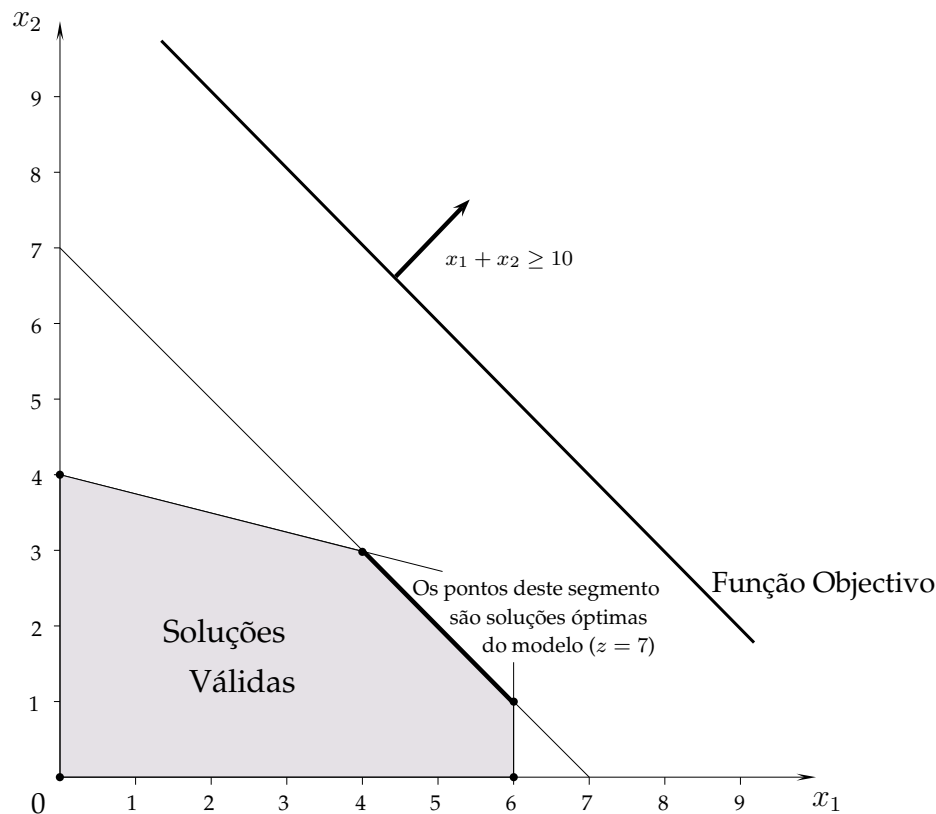


Figura 2.2: Representação gráfica de um modelo com várias soluções ótimas

- o modelo é impossível, não tem nenhuma solução válida (a intersecção dos semi-espacos de soluções válidas definidos pelas restrições é um conjunto vazio),
- o valor da função objectivo é ilimitado, e não é possível determinar uma solução que seja melhor que todas as outras.

O segundo caso é ilustrado no Exemplo seguinte.

Exemplo 2.4 Considere o modelo de Programação Linear seguinte:

$$\begin{aligned}
 \max \quad z = & \quad x_1 \quad + \quad x_2 \\
 \text{sujeito a} \quad & \\
 & -x_1 \quad + \quad x_2 \leq 4 \\
 & x_1 \quad - \quad 2x_2 \leq 4 \\
 & x_1 \quad , \quad x_2 \geq 0
 \end{aligned}$$

A Figura 2.3 representa graficamente o espaço de soluções válidas desse modelo. O valor da função objectivo não tem limite superior. Para qualquer valor de z da função

objectivo, existe sempre um par correspondente de valores (x_1, x_2) . Consequentemente, não é possível identificar uma solução óptima que domine todas as restantes.

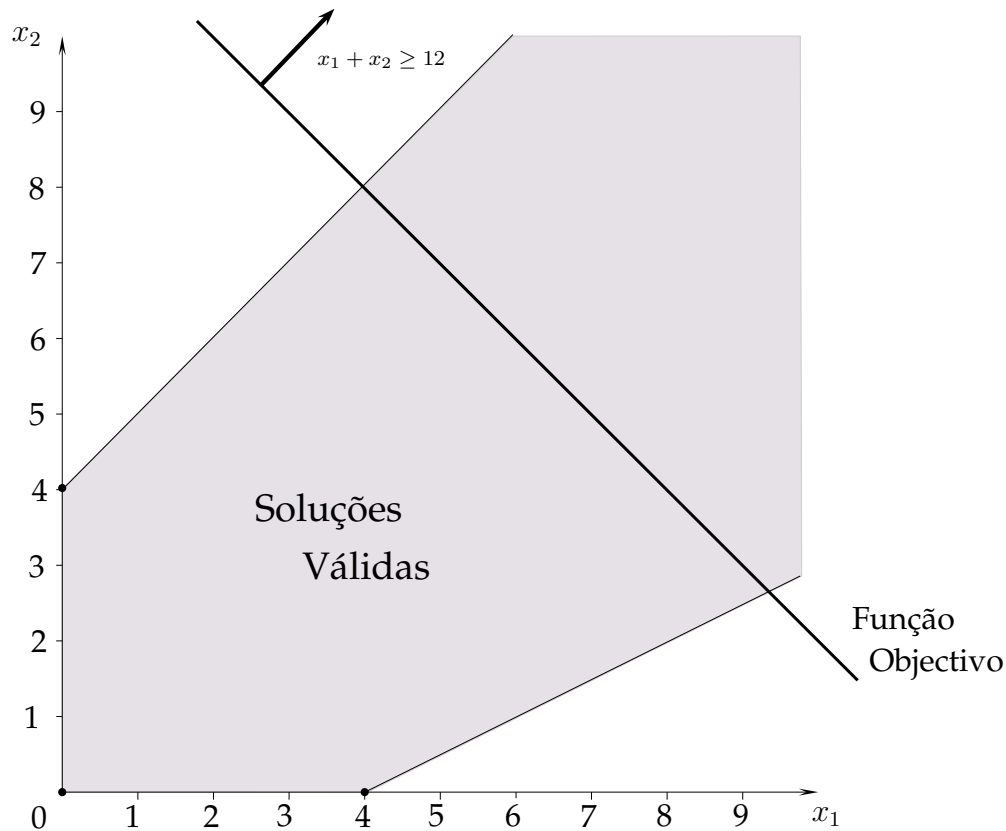


Figura 2.3: Representação gráfica de um modelo com um espaço de soluções válidas ilimitado

□

A par dos conceitos de solução válida e de solução óptima, existe um conjunto de soluções que é particularmente relevante em Programação Linear. Essas soluções são as chamadas *soluções de base*. Do ponto de vista geométrico, uma solução de base corresponde a um ponto que está na intersecção das fronteiras definidas por um par de restrições. Uma solução de base não tem de ser necessariamente válida para o modelo. No Exemplo 2.2, as rectas $x_1 + 4x_2 = 16$ e $x_1 = 6$ que definem a fronteira de duas das restrições do modelo intersectam-se fora do espaço de soluções válidas. No entanto, a solução correspondente é considerada uma solução de base. Uma solução de base que é também válida para o modelo global é designada por *solução válida de base*. Uma solução válida de base corresponde a um vértice do poliedro que define o espaço de

soluções válidas. Por definição, a solução óptima de um modelo Programação Linear é uma solução válida de base.

As soluções de base são definidas a partir de um modelo de Programação Linear originalmente na sua forma canónica ao qual são adicionadas variáveis de folga ou de excesso de modo a que todas as restrições estejam na forma de equações. Para facilitar a apresentação, vamos considerar um modelo do tipo *maximização*. Na forma canónica, todas as restrições desse modelo devem ser do tipo *menor ou igual*. Assim, podemos considerar todas as variáveis adicionais como sendo variáveis de folga. Se o modelo tiver n variáveis de decisão e m restrições do tipo *menor ou igual*, teremos de adicionar m variáveis de folga. No final, teremos um sistema com $n + m$ variáveis e m equações. Da mesma forma, as soluções de base são definidas por $n + m$ variáveis.

As soluções de base distinguem-se pelo facto de serem compostas por n variáveis de valor nulo (as *variáveis não-básicas*). O valor das restantes $n + m$ variáveis dessas soluções (as *variáveis básicas*, ou a *base*) é obtido com base no sistema de equações resultante. Uma solução é válida de base se todas as variáveis básicas forem não-negativas. De notar que o número de variáveis do sistema de equações original é superior ao número de equações. Ao arbitrarmos o valor de n variáveis de decisão ($(n + m)$ variáveis- m restrições), obtemos um sistema de equações com um número idêntico de variáveis e restrições.

É interessante observar a estrutura de soluções válidas de base adjacentes. Voltando ao Exemplo 2.2, e designando respectivamente por F_1 , F_2 e F_3 as variáveis de folga associadas a cada uma das restrições, a solução de base que corresponde à solução óptima desse modelo é dada por:

$$(x_1, x_2, F_1, F_2, F_3) = (4, 3, 0, 0, 2).$$

As variáveis básicas associadas a essa solução são as variáveis x_1 , x_2 e F_3 . Uma das soluções adjacentes a essa solução é a seguinte:

$$(x_1, x_2, F_1, F_2, F_3) = (0, 4, 0, 3, 6).$$

Nesse caso, as variáveis básicas são respectivamente x_2 , F_2 e F_3 . A segunda solução adjacente à solução óptima corresponde a:

$$(x_1, x_2, F_1, F_2, F_3) = (6, 1, 6, 0, 0),$$

e tem x_1 , x_2 e F_1 como variáveis básicas. O conjunto de variáveis básicas de duas soluções válidas de base adjacentes diferem apenas numa variável, o mesmo acontecendo com as variáveis não-básicas dessas soluções. Essa propriedade verifica-se para todas as soluções válidas de base adjacentes dos modelos de Programação Linear.

2.3 Resolução de Modelos de Programação Linear

2.3.1 Aspectos Práticos

Já vimos que o sucesso da Programação Linear como técnica usada efectivamente na resolução de problemas reais de planeamento e gestão se devia essencialmente à facilidade com que as soluções óptimas desses modelos eram calculadas. Por volta do início dos anos 80, demonstrou-se mesmo que os modelos de Programação Linear podiam ser resolvidos por algoritmos que corriam em tempo polinomial. Como não pode deixar de ser, a dificuldade de resolução cresce com a dimensão do modelo (número de variáveis e restrições), mas a um ritmo “moderado”. Esse facto permite que esta técnica possa ser usada para resolver problemas reais de média e grande dimensão.

Existem duas grandes abordagens de resolução para modelos de Programação Linear: o método Simplex e os métodos de ponto interior. Nesta Secção, analisamos com maior profundidade o método Simplex. Algumas notas relativas aos métodos de ponto interior são dadas no final desta Secção. No pior caso, o método Simplex pode terminar apenas após ter realizado um número exponencial de iterações¹. Curiosamente, o método Simplex continua a ser na prática um dos métodos de resolução mais eficientes. No capítulo dos métodos de ponto interior, o método proposto por Narendra Karmarkar em 1984 compete directamente com o método Simplex, podendo mesmo ser melhor em problemas de muito grande dimensão.

A facilidade de resolução dos modelos de Programação Linear deve-se também em parte à sua estrutura. Uma das características recorrentes dos modelos de Programação Linear é o facto das matrizes de coeficientes desses modelos serem *esparsas*: as variáveis de decisão têm coeficientes positivos num número reduzido de restrições (muitos dos coeficientes do termo do lado esquerdo das restrições são nulos). Nesses casos, os cálculos tornam-se bem mais fáceis.

2.3.2 O Método Simplex

Para ilustrar o funcionamento do método Simplex, vamos usar o modelo descrito no Exemplo 2.2. O problema é do tipo *maximização*. Lembramos que minimizar uma função é equivalente a maximizar a função simétrica. Em primeiro lugar, começamos por reduzir o modelo à sua forma standard adicionando uma variável de folga (não-negativa) a cada uma das restrições:

¹A descoberta data de 1970 e deve-se a Victor Klee and George Minty. O resultado foi publicado em 1972 num artigo intitulado “How good is the simplex algorithm?”.

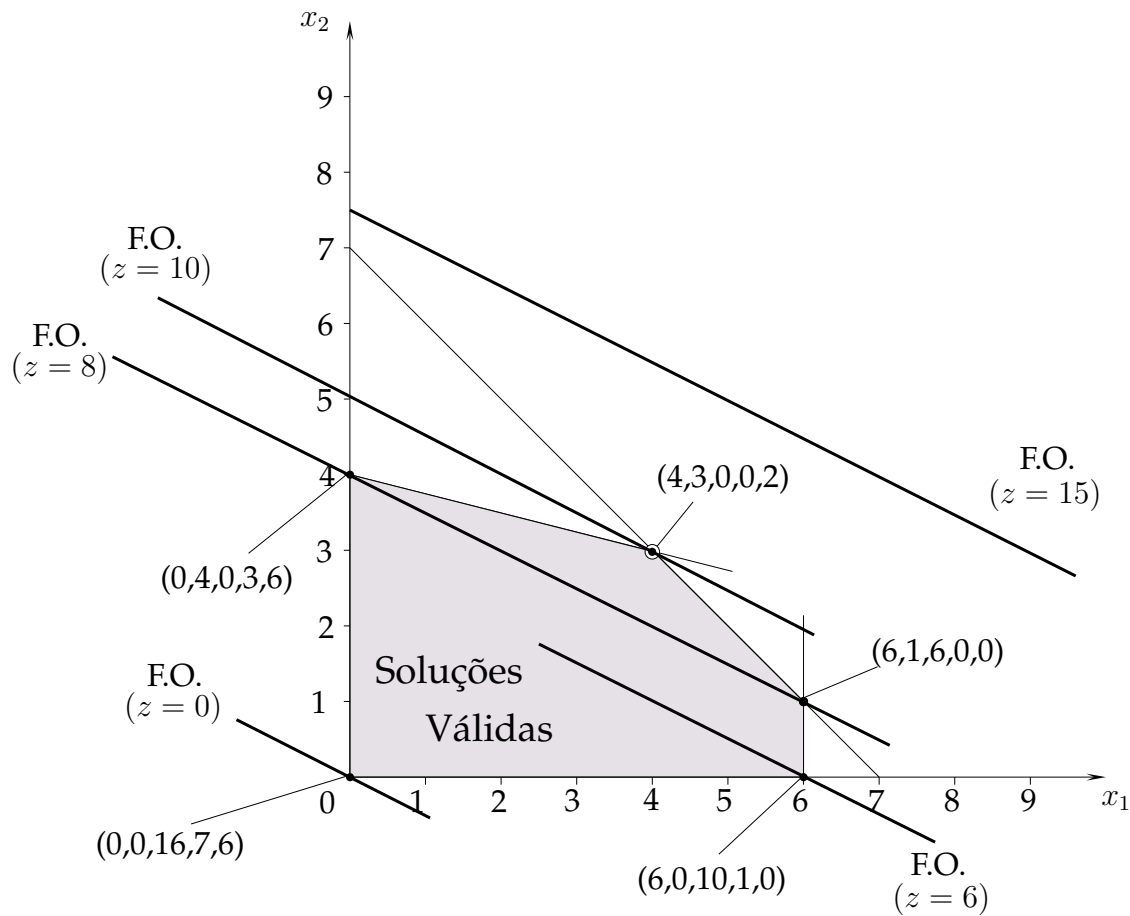


Figura 2.4: Soluções válidas de base (modelo do Exemplo 2.2)

$$\begin{aligned}
 \max \quad & z = x_1 + 2x_2 \\
 \text{sujeito a} \quad & \\
 & x_1 + 4x_2 + F_1 = 16 \\
 & x_1 + x_2 + F_2 = 7 \\
 & x_1 + F_3 = 6 \\
 & x_1, x_2, F_1, F_2, F_3 \geq 0
 \end{aligned}$$

Vamos assumir que as variáveis de folga têm coeficiente nulo na função objectivo. Em termos práticos, isso significa que a função objectivo não é penalizada (nem melhorada) pelo facto de não serem atingidos os limites impostos pelos termos independentes das restrições. Na Figura 2.4, assinalámos o valor de todas as soluções válidas de base do modelo (na forma $(x_1, x_2, F_1, F_2, F_3)$). Por definição, essas soluções são pontos extremos do espaço de soluções válidas.

A resolução parte de um ponto inicial que deve uma solução válida de base para o modelo. Vamos usar a solução que corresponde à origem do gráfico, com $x_1 = 0$ e

$x_2 = 0$. Nesse ponto, o valor das 3 variáveis de folga atinge o seu valor máximo. A solução válida de base correspondente é a seguinte:

$$(x_1, x_2, F_1, F_2, F_3) = (0, 0, 16, 7, 6).$$

As variáveis básicas dessa solução são as variáveis de folga F_1 , F_2 e F_3 , enquanto que x_1 e x_2 são as variáveis não-básicas. Nesse ponto, o valor de z na função objectivo é igual a 0. As variáveis básicas dessa solução e a função objectivo z podem ser expressas em ordem às variáveis não-básicas da forma seguinte:

$$\begin{array}{rclclcl} F_1 & = & 16 & - & x_1 & - & 4x_2 \\ F_2 & = & 7 & - & x_1 & - & x_2 \\ F_3 & = & 6 & - & x_1 & & \\ z & = & 0 & + & x_1 & + & 2x_2 \end{array}$$

Para aumentarmos o valor de z , temos de aumentar o valor das variáveis não-básicas. Vamos optar por aumentar apenas o valor de uma dessas variáveis, mantendo o valor das outras variáveis não-básicas a zero. Escolhemos x_2 por ter o maior coeficiente na função objectivo. O próximo passo consiste em determinar o valor que deve ser atribuído a essa variável. O princípio consiste em atribuir-lhe o maior valor possível, *i.e.* o maior entre todos aqueles que garantem que nenhuma restrição de não-negatividade é violada no sistema de equações acima. Dado que o valor de x_1 se mantém a zero, podemos reescrever essas equações da forma seguinte:

$$\begin{array}{rclcl} F_1 & = & 16 & - & 4x_2 \\ F_2 & = & 7 & - & x_2 \\ F_3 & = & 6 & & \\ z & = & 0 & + & 2x_2 \end{array}$$

Tendo em atenção apenas a primeira das equações, o valor máximo que x_2 pode tomar é 4. Acima desse valor, a variável de folga F_1 toma valores negativos. Para evitar que a restrição de não-negatividade de F_2 seja violada, x_2 não pode ser maior que 7 (segunda equação). A terceira equação não condiciona o valor máximo que x_2 pode tomar. Assim, o valor que deve ser atribuído à variável x_2 é igual a 4. Fazendo $x_2 = 4$, e mantendo $x_1 = 0$, os valores das restantes variáveis passa a ser o seguinte:

$$\begin{array}{rcl} F_1 & = & 0 \\ F_2 & = & 3 \\ F_3 & = & 6 \end{array}$$

O valor z da função objectivo passa a ser igual a 8. Na Figura 2.4, podemos observar que passámos do ponto de origem do gráfico para o ponto adjacente seguinte:

$$(x_1, x_2, F_1, F_2, F_3) = (0, 4, 0, 3, 6).$$

A variável x_2 passou a ser uma variável básica no lugar de F_1 que passou a não-básica.

À semelhança do que fizemos num dos passos iniciais, vamos exprimir as variáveis básicas em ordem às variáveis não-básicas. Para isso, usamos o último sistema de equações que neste caso corresponde ao ponto de origem do gráfico. A variável básica x_2 passa a ser expressa como segue:

$$x_2 = 4 - \frac{1}{4}x_1 - \frac{1}{4}F_1.$$

Para chegarmos a essa expressão, usámos a primeira equação do sistema, por ser a equação associada a F_1 , *i.e.* aquela que deixou o seu lugar na base à variável x_2 . Substituindo x_2 por essa expressão nas restantes equações do sistema, obtemos

$$\begin{array}{rclclcl} x_2 & = & 4 & - & \frac{1}{4}x_1 & - & \frac{1}{4}F_1 \\ F_2 & = & 3 & - & \frac{3}{4}x_1 & + & \frac{1}{4}F_1 \\ F_3 & = & 6 & - & x_1 & & \\ z & = & 8 & + & \frac{1}{2}x_1 & - & \frac{1}{2}F_1 \end{array}$$

Usando o mesmo procedimento que usámos anteriormente, ainda é possível aumentar o valor da função objectivo dando um valor positivo à variável não-básica x_1 . Não é interessante aumentar o valor de F_1 , visto que por cada unidade que lhe for atribuída o valor de z diminuirá de $\frac{1}{2}$. Mantendo o valor de F_1 a zero, podemos reescrever o sistema de equações acima da forma seguinte:

$$\begin{array}{rclcl} x_2 & = & 4 & - & \frac{1}{4}x_1 \\ F_2 & = & 3 & - & \frac{3}{4}x_1 \\ F_3 & = & 6 & - & x_1 \\ z & = & 8 & + & \frac{1}{2}x_1 \end{array}$$

O maior valor que x_1 pode tomar é igual a 4. Acima desse valor, a restrição de não-negatividade de F_2 será violada. O valor z da função objectivo atinge o valor de 10. A nova solução válida de base é igual a:

$$(x_1, x_2, F_1, F_2, F_3) = (4, 3, 0, 0, 2).$$

A variável básica F_2 deixou o seu lugar na base à variável x_1 .

Com base na equação relativa a F_2 no último sistema de equações, a nova variável básica x_1 passa a ser expressa em ordem às variáveis não-básicas da forma seguinte:

$$x_1 = 4 + \frac{1}{3}F_1 - \frac{4}{3}F_2.$$

Podemos assim reescrever novamente o último sistema de equações de modo a que as variáveis básicas sejam todas elas expressas em ordem às variáveis não-básicas:

$$\begin{array}{rccccrcl} x_2 & = & 3 & - & \frac{1}{3}F_1 & + & \frac{1}{3}F_2 \\ x_1 & = & 4 & + & \frac{1}{3}F_1 & - & \frac{4}{3}F_2 \\ F_3 & = & 2 & - & \frac{1}{3}F_1 & + & \frac{4}{3}F_2 \\ z & = & 10 & - & \frac{1}{3}F_1 & - & \frac{2}{3}F_2 \end{array}$$

Dado que agora não é possível melhorar o valor da função objectivo aumentando o valor de qualquer uma das variáveis não-básicas, podemos concluir que a solução válida de base $(x_1, x_2, F_1, F_2, F_3) = (4, 3, 0, 0, 2)$ é a solução óptima do modelo. Esta prova algébrica confirma o resultado que já tínhamos obtido usando a representação gráfica.

O método Simplex consiste num conjunto de passos executados iterativamente até que um critério de paragem se verifique. Neste caso, o critério é um critério de optimalidade que nos garante que a solução válida de base da iteração é optima. Nesse sentido, o método Simplex é um algoritmo. Podemos sistematizá-lo conforme descrito no Algoritmo 3.1. Os passos no ciclo *enquanto* consistem em mover-se para uma solução válida de base adjacente à solução da iteração. Se não existir nenhuma solução válida de base adjacente que seja melhor do que a solução da iteração, então esta última é garantidamente a solução óptima.

Os passos do método Simplex que apresentámos permitem perceber a execução do método em termos algébricos. Outra forma de executar o método consiste em recorrer aos chamados *quadros Simplex*. Para ilustrar essa abordagem, vamos usar novamente o modelo do Exemplo 2.2. A solução válida inicial que usámos foi a origem que corresponde ao ponto $(x_1, x_2, F_1, F_2, F_3) = (0, 0, 16, 7, 6)$. As variáveis básicas dessa solução são as variáveis de folga F_1 , F_2 e F_3 . As expressões das variáveis básicas em ordem às não-básicas para essa solução podem ser reescritas da forma seguinte:

$$\begin{array}{rccccrcl} x_1 & + & 4x_2 & + & F_1 & & = & 16 \\ x_1 & + & x_2 & & & + & F_2 & = & 7 \\ x_1 & & & & & & + & F_3 & = & 6 \\ z & - & x_1 & - & 2x_2 & & & & = & 0 \end{array}$$

Algoritmo 3.1: Método Simplex

```

1 Input: modelo de Programação Linear ;
2 Formular o modelo na sua forma standard usando variáveis de folga ou excesso ;
3 Determinar uma primeira solução válida de base ;
4 se não existir nenhuma solução válida de base então
5   | Output: “Modelo não tem soluções válidas” ;
6   | Goto 20;
7 senão
8   | Expressar as variáveis básicas e o valor da função objectivo em ordem às
   |   variáveis não-básicas;
9   | Enquanto existir uma variável não-básica com potencial para melhorar o
   |   valor da função objectivo fazer
10  |   | Escolher a variável não-básica que vai entrar para a base ;
11  |   | Determinar o valor (máximo) que essa variável pode tomar ;
   |   | /* A variável básica que sai da base é também escolhida */
12  |   | se a variável não-básica que entra para a base tiver valor finito então
13  |   |   | Actualizar a solução válida de base ;
14  |   |   | Expressar a nova variável básica em ordem à variável que saiu da base ;
15  |   |   | Reescrever as restantes equações com base na expressão definida no
   |   |   | passo anterior ;
16  |   | senão
17  |   |   | Output: “Modelo tem solução ilimitada” ;
18  |   |   | Goto 20;
19 Output: “A solução válida de base corrente é solução óptima do modelo” ;
20 Stop ;

```

Partindo dessa representação, construímos o seguinte quadro Simplex:

	z	x_1	x_2	F_1	F_2	F_3	
F_1	0	1	4	1	0	0	16
F_2	0	1	1	0	1	0	7
F_3	0	1	0	0	0	1	6
z	1	-1	-2	0	0	0	0

No topo do quadro, estão listadas todas as variáveis do modelo. Os coeficientes dos termos do lado esquerdo das equações estão no interior do quadro. As variáveis básicas

associadas a cada equação estão na coluna do lado esquerdo. A variável z pode ser interpretada como a variável básica associada à última equação. O valor das variáveis básicas lê-se na coluna do lado direito do quadro. As variáveis que não estão na coluna do lado esquerdo (a base) são variáveis não-básicas. O valor de cada uma dessas variáveis é nulo.

Após ter introduzido as necessárias variáveis de folga ou excesso no modelo, é necessário determinar uma primeira solução válida de base para poder iniciar o método Simplex. No Exemplo que estamos a considerar, escolhemos a origem do gráfico ($x_1 = 0$, $x_2 = 0$) que corresponde à base representada no quadro acima. O passo seguinte consiste em verificar se o critério de optimalidade já foi alcançado, *i.e.* se não existe nenhuma variável não-básica com potencial para melhorar o valor da função objectivo. Uma solução desse género deve ter um coeficiente negativo na última linha do quadro. Se todos os coeficientes forem positivos ou nulos, a solução válida de base representada no quadro Simplex é óptima. No quadro acima, existem coeficientes negativos na última linha do quadro, logo a solução não é óptima.

Quando a solução válida de base do quadro não é óptima, efectua-se uma iteração do método Simplex: escolhemos uma nova solução válida de base adjacente à actual, e que melhora o valor da função objectivo. Para isso, começamos por determinar a variável que vai entrar para a base (que corresponde à chamada *coluna pivot*). Escolhemos aquela que tem o coeficiente negativo com o maior valor absoluto. No nosso Exemplo, escolhemos a variável não-básica x_2 . De seguida, determinamos a variável básica que vai sair da base. Para cada uma das linhas do quadro, dividimos o coeficiente da coluna do lado direito do quadro Simplex pelos elementos estritamente positivos (> 0) da coluna pivot (se todos os elementos da coluna pivot forem negativos ou nulos, o modelo terá uma solução ilimitada). Escolhemos a linha que corresponde à menor razão. Essa linha é designada por *linha pivot*. No nosso Exemplo, a linha associada à menor razão é aquela que corresponde à variável básica F_1 ($\frac{16}{4} = 4$). Na intersecção entre a linha pivot e a coluna pivot, temos o *elemento pivot*. O quadro abaixo ilustra essas operações.

	z	x_1	x_2	F_1	F_2	F_3	
F_1	0	1	4	1	0	0	16
F_2	0	1	1	0	1	0	7
F_3	0	1	0	0	0	1	6
z	1	-1	-2	0	0	0	0

$\uparrow$
coluna pivot

$\frac{16}{4} = 4 \leftarrow \text{linha pivot}$
 $\frac{7}{1} = 7$
 $\times$
 $\times$

Por definição, em qualquer quadro Simplex deve existir uma matriz identidade associada às variáveis básicas. Ao mudar a base, temos de actualizar o quadro em conformidade usando operações elementares sobre linhas. Podemos multiplicar uma linha por uma constante não nula, e adicionar a uma linha do quadro uma outra linha eventualmente multiplicada por uma constante. O próximo passo consiste assim em actualizar o quadro para a nova solução válida de base usando eliminação de Gauss. A linha pivot deve ser dividida pelo valor do elemento pivot. A cada uma das outras linhas devemos subtrair a nova linha pivot multiplicada pelo coeficiente dessa linha na coluna pivot. O quadro seguinte ilustra o resultado dessas operações.

	z	x_1	x_2	F_1	F_2	F_3	
x_2	0	$\frac{1}{4}$	1	$\frac{1}{4}$	0	0	4
F_2	0	$\frac{3}{4}$	0	$-\frac{1}{4}$	1	0	3
F_3	0	1	0	0	0	1	6
z	1	$-\frac{1}{2}$	0	$\frac{1}{2}$	0	0	8

Dado que ainda existem coeficientes negativos na última linha do quadro, a solução ainda não é óptima. O método prossegue com uma nova iteração. A variável que deverá entrar para a base é a variável x_1 . Dividindo os coeficientes da coluna do lado direito pelos coeficientes da coluna pivot, determina-se a nova linha pivot, que neste caso será a linha associada a F_2 .

	z	x_1	x_2	F_1	F_2	F_3	
x_2	0	$\frac{1}{4}$	1	$\frac{1}{4}$	0	0	4
F_2	0	$\frac{3}{4}$	0	$-\frac{1}{4}$	1	0	3
F_3	0	1	0	0	0	1	6
z	1	$-\frac{1}{2}$	0	$\frac{1}{2}$	0	0	8

$\uparrow$
coluna pivot

$\frac{4}{\frac{1}{4}} = 16$
 $\frac{3}{\frac{3}{4}} = 4 \leftarrow \text{linha pivot}$
 $\frac{6}{1} = 6$
 $\times$

A actualização do quadro nos moldes descritos acima resulta no quadro seguinte:

	z	x_1	x_2	F_1	F_2	F_3	
x_2	0	0	1	$\frac{1}{3}$	$-\frac{1}{3}$	0	3
x_1	0	1	0	$-\frac{1}{3}$	$\frac{4}{3}$	0	4
F_3	0	0	0	$\frac{1}{3}$	$-\frac{4}{3}$	1	2
z	1	0	0	$\frac{1}{3}$	$\frac{2}{3}$	0	10

Para esse quadro, o critério de optimalidade é satisfeito dado que não há nenhum coeficiente negativo na última linha. Podemos concluir que a solução válida de base $(x_1, x_2, F_1, F_2, F_3) = (4, 3, 0, 0, 2)$ é também solução óptima do problema.

2.3.3 Interpretação dos Resultados

A resolução de um modelo de Programação Linear através do método Simplex permite-nos não só obter a solução óptima do problema (se existir) como também um conjunto de informação útil relativamente a essa solução. Relativamente à solução do problema (e do modelo que o representa), vimos já que podiam ocorrer quatro situações:

- o problema tem uma única solução: corresponde à última solução válida de base dada pelo método Simplex (se forem usados quadros Simplex, a solução pode ser extraída do último quadro associando as variáveis básicas da coluna do lado esquerdo com os valores da coluna do lado direito);
- o problema tem várias soluções: o método Simplex termina com uma solução válida de base que satisfaz o critério de optimalidade, e existe pelo menos uma variável não-básica que tem coeficiente nulo na correspondente expressão da função objectivo (ou na última linha do quadro Simplex óptimo). Nesse caso, podemos modificar a base introduzindo essa variável não-básica, sem alterar o valor da função objectivo;
- o problema é ilimitado: no método Simplex, esses casos podem ser detectados quando, numa dada iteração, a variável não-básica que deveria entrar para a base pode tomar qualquer valor positivo (todos os elementos da coluna pivot são negativos ou nulos);
- o problema é impossível: nesse caso, obviamente, não é sequer possível determinar uma solução válida de base para iniciar o método Simplex.

Além dessa informação, podemos também extrair dois grandes tipos de informação associados à solução, nomeadamente:

- os custos reduzidos associados às variáveis,
- os preços sombra associados às restrições.

O custo reduzido mede a variação que deve ser aplicada ao coeficiente na função objectivo de uma variável de decisão para que ela se torne atractiva. Numa solução óptima, as variáveis básicas já são variáveis atractivas, dado que têm valor positivo.

O custo reduzido de uma variável básica é por isso igual a 0. As variáveis não-básicas têm valor nulo, e têm, na maior parte dos casos, um custo reduzido positivo. Em problemas de minimização, o custo reduzido de uma variável não-básica determina o valor que deveria ser subtraído ao custo original da variável para que ela se tornasse atractiva, *i.e.* para que ela passasse a básica. Em problemas de maximização, esse custo reduzido mede o aumento que é necessário aplicar ao coeficiente na função objectivo de uma variável não-básica para que ela se torne atractiva. Nos quadros Simplex, o custo reduzido das variáveis pode ser lido na última linha do quadro na coluna que corresponde a essa variável.

O preço sombra associado a uma restrição mede a variação do valor da função objectivo correspondente a um aumento de uma unidade no termo independente original dessa restrição. Quando, na solução óptima, a variável de folga ou excesso de uma restrição tem valor positivo, o preço sombra é igual a zero. Tomando como exemplo um problema de maximização e uma das suas restrições do tipo *menor ou igual*, se a solução óptima corresponde a não saturar essa restrição, o valor da variável de folga correspondente será positivo. Aumentar o valor do termo independente dessa restrição não irá alterar a solução, apenas irá aumentar o valor da folga associada a essa restrição.

Aumentar marginalmente o termo independente de uma restrição que tem uma variável de folga ou excesso associada igual a 0 irá alterar o valor da função objectivo de uma quantidade igual ao seu preço sombra. Por exemplo, num problema de maximização, aumentar a disponibilidade de um recurso escasso permite obter soluções com maior valor de lucro. Ainda a título de exemplo, num problema de produção com restrições de procura e no qual se pretende minimizar o custo total, se aumentarmos o valor de uma procura que era satisfeita exactamente na solução óptima, o custo aumentará de uma quantidade igual ao preço sombra associado à respectiva restrição.

Nos quadros Simplex, o preço sombra de uma restrição pode ser lido na última linha do quadro na coluna que corresponde à variável de folga ou excesso dessa restrição.

2.3.4 Determinar uma Primeira Solução Válida de Base

Para poder aplicar o método Simplex, é necessário ter uma primeira solução válida de base. Nos exemplos que usámos acima, foi fácil determinar essa base inicial: escolhemos o ponto de origem por ser um ponto extremo do espaço de soluções válidas. Porém, em muitos casos, determinar uma base inicial pode não ser óbvio. Podemos pensar num problema de minimização por exemplo. Se todas as variáveis forem não-negativas, tipicamente, o ponto de origem não é uma solução válida, senão seria obviamente a solução óptima. Nesses casos, temos de determinar uma solução que seja não só válida mas também que esteja na intersecção da fronteira definida por um par de restrições

(ver Secção 2.2.4).

Num modelo com n variáveis e m restrições, uma solução válida de base é composta no máximo por m variáveis positivas (básicas). Os vectores coluna associados aos coeficientes nas restrições dessas variáveis devem também ser linearmente independentes formando uma matriz regular (com inversa). Do ponto de vista algébrico, determinar uma primeira solução válida de base consiste em encontrar uma solução que verifique essas condições. Para iniciar o método Simplex, deveremos ainda exprimir as variáveis básicas da solução em ordem às variáveis não-básicas.

Nesta Secção, começamos por rever um caso simples em que determinar uma base inicial é imediato. Introduzimos também duas técnicas que permitem determinar uma base inicial para os casos mais difíceis.

2.3.4.1 Solução Imediata

O caso mais favorável quando se procura uma solução válida de base inicial acontece quando a matriz de coeficientes dos termos do lado esquerdo das restrições contém a matriz identidade. Para ilustrar esse caso, vamos considerar o caso de um modelo générico de maximização na forma canónica:

$$\begin{aligned}
 \max \quad z = & \quad c_1x_1 \quad + \quad c_2x_2 \quad + \quad \dots \quad + \quad c_nx_n \\
 \text{s. a} & \\
 & a_{11}x_1 \quad + \quad a_{12}x_2 \quad + \quad \dots \quad + \quad a_{1n}x_n \leq b_1 \\
 & a_{21}x_1 \quad + \quad a_{22}x_2 \quad + \quad \dots \quad + \quad a_{2n}x_n \leq b_2 \\
 & \dots \\
 & a_{m1}x_1 \quad + \quad a_{m2}x_2 \quad + \quad \dots \quad + \quad a_{mn}x_n \leq b_m \\
 & x_1 \quad , \quad x_2 \quad , \quad \dots \quad , \quad x_n \quad \geq 0
 \end{aligned}$$

Adicionando variáveis de folga em cada uma das restrições, obtemos o modelo na forma standard:

$$\begin{aligned}
 \max \quad z = & \quad c_1x_1 \quad + \quad c_2x_2 \quad + \quad \dots \quad + \quad c_nx_n \\
 \text{s. a} & \\
 & a_{11}x_1 \quad + \quad a_{12}x_2 \quad + \quad \dots \quad + \quad a_{1n}x_n \quad + \quad F_1 \quad = \quad b_1 \\
 & a_{21}x_1 \quad + \quad a_{22}x_2 \quad + \quad \dots \quad + \quad a_{2n}x_n \quad + \quad F_2 \quad = \quad b_2 \\
 & \dots \\
 & a_{m1}x_1 \quad + \quad a_{m2}x_2 \quad + \quad \dots \quad + \quad a_{mn}x_n \quad + \quad \dots \quad + \quad F_m = b_m \\
 & x_1 \quad , \quad x_2 \quad , \quad \dots \quad , \quad x_n \quad , \quad F_1 \quad , \quad F_2 \quad , \quad \dots \quad , \quad F_m \geq 0
 \end{aligned}$$

Se representarmos esse modelo genérico através de um quadro Simplex, a matriz identidade aparece claramente:

	z	x_1	x_2	$\dots$	x_n	F_1	F_2	$\dots$	F_m	
F_1	0	a_{11}	a_{12}	$\dots$	a_{1n}	1	0	$\dots$	0	b_1
F_2	0	a_{21}	a_{22}	$\dots$	a_{2n}	0	1	$\dots$	0	b_2
$\dots$										
F_m	0	a_{m1}	a_{m2}	$\dots$	a_{mn}	0	0	$\dots$	1	b_m
z	1	c_1	c_2	$\dots$	c_n	0	0	$\dots$	0	0

Nesses casos, escolhemos para a base as variáveis associadas à matriz identidade. A solução que obtemos é composta no máximo por m variáveis positivas, e os vectores coluna compostos pelos coeficientes nas restrições dessas variáveis definem uma matriz regular (a inversa da matriz identidade é outra matriz identidade). Assumindo que todos os b_i , $i = 1, \dots, m$, são positivos, a solução é válida, sendo por isso uma solução válida de base. Este exemplo traduz as situações que encontramos até agora.

Quando a matriz identidade não é facilmente “isolada”, recorre-se a um dos dois métodos descritos abaixo. O ponto de partida de ambos os métodos é o mesmo: adicionam-se variáveis ao modelo (variáveis artificiais) de modo a fazer facilmente surgir essa matriz identidade, e forçam-se essas variáveis a serem nulas na solução final. Os dois métodos variam precisamente no procedimento que é usado para obrigar as variáveis artificiais a tomarem valores nulos.

2.3.4.2 Método das 2-Fases

As variáveis artificiais devem ser adicionadas às restrições para as quais não é possível encontrar facilmente uma variável com coeficiente unitário na restrição e nulo nas outras restrições. Vamos considerar por exemplo a restrição do tipo *menor ou igual* seguinte:

$$5x_1 + 3x_2 + 7x_3 \leq 15.$$

Adicionando uma variável de folga a essa restrição, obtemos:

$$5x_1 + 3x_2 + 7x_3 + F_1 = 15.$$

A variável F_1 só é usada nessa restrição: tem coeficiente unitário nessa restrição e nulo nas restantes. A variável F_1 pode por isso ser usada como variável básica associada a essa restrição.

Se existir uma variável com coeficiente unitário numa restrição e coeficiente nulo nas restantes, essa variável pode também ser usada como variável básica da restrição.

Exemplo 2.5 Considere o modelo de Programação Linear seguinte:

$$\begin{array}{ll} \min z = & 4x_1 + 12x_2 + 16x_3 \\ \text{s. a} & \\ & 2x_1 + x_2 + 4x_3 \geq 10 \\ & + x_2 + x_3 \geq 4 \\ & + 2x_2 + x_3 \geq 8 \\ & x_1, x_2, x_3 \geq 0 \end{array}$$

Esse modelo é transformado para a forma standard subtraindo uma variável de excesso a cada uma das restrições

$$\begin{array}{ll} \min z = & 4x_1 + 12x_2 + 16x_3 \\ \text{s. a} & \\ & 2x_1 + x_2 + 4x_3 - E_1 = 10 \\ & + x_2 + x_3 - E_2 = 4 \\ & + 2x_2 + x_3 - E_3 = 8 \\ & x_1, x_2, x_3, E_1, E_2, E_3 \geq 0 \end{array}$$

A variável x_1 tem coeficiente nulo na 2ª e 3ª restrição. Dividindo a primeira restrição por 2, essa variável fica com coeficiente unitário nessa restrição. Nessas condições, a variável x_1 pode ser usada como variável básica para a 1ª restrição. Já para as restantes, não é possível identificar uma variável com essas características. Por esse motivo, adiciona-se uma variável artificial a cada uma delas.

$$\begin{array}{ll} \min z = & 4x_1 + 12x_2 + 16x_3 \\ \text{s. a} & \\ & 2x_1 + x_2 + 4x_3 - E_1 = 10 \\ & + x_2 + x_3 - E_2 + A_1 = 4 \\ & + 2x_2 + x_3 - E_3 + A_2 = 8 \\ & x_1, x_2, x_3, E_1, E_2, E_3, A_1, A_2 \geq 0 \end{array}$$

Com esse esquema, já é possível construir uma base válida. Neste caso, a base é constituída pelas variáveis x_1 , A_1 e A_2 . $\square$

Em relação às variáveis artificiais (que também são não-negativas) é importante notar que se numa dada solução alguma dessas variáveis for positiva, a restrição a que

está associada será automaticamente violada. No Exemplo acima, se $A_1 > 0$ teremos que:

$$x_2 + x_3 - E_2 < 4.$$

Dado que a variável de excesso E_2 é não-negativa, temos que:

$$x_2 + x_3 < 4.$$

o que é contrário à restrição original. Logo, uma solução válida de base com variáveis artificiais básicas é de facto válida para o modelo “aumentado” mas não para o modelo original.

Partindo de uma base com variáveis artificiais, o método das 2-fases procura retirar essas variáveis da base, determinando uma solução válida de base na qual todas as variáveis artificiais são nulas. O método divide-se em 2 partes (as 2 fases):

- usando o método Simplex, resolve-se o modelo original, ao qual foram acrescentadas as variáveis de folga ou excesso e as variáveis artificiais, com uma função objectivo que consiste no somatório das variáveis artificiais. Com este procedimento, procura-se uma solução para a qual as variáveis básicas estão ao seu nível mais baixo (atendendo às restrições de não-negatividade esse valor é 0). Dois casos poderão então ocorrer:
 - na solução final, todas as variáveis artificiais estão fora da base (a solução óptima obtida tem valor 0 dado que as variáveis artificiais são todas nulas);
 - a solução final tem valor positivo, *i.e.* ainda existe pelo menos uma variável artificial na base.

No primeiro caso, encontrou-se uma solução válida de base que é também válida para o modelo original sem variáveis artificiais: prossegue-se com a 2ª fase do método. No segundo caso, não vai ser possível eliminar todas as variáveis artificiais da base: o problema é impossível.

- as variáveis artificiais são eliminadas do modelo, exprimimos a função objectivo original em ordem às variáveis não-básicas da solução válida de base que determinámos na 1ª fase, e resolve-se o modelo resultante usando o método Simplex.

O Exemplo seguinte ilustra o funcionamento do método das 2-fases. Antes de o introduzirmos, deixamos aqui uma nota relativamente à forma como a maioria dos autores aborda os problemas de Programação Linear de *minimização*. É possível actualizar o critério de optimalidade para o caso dos problemas de *minimização* (para os

problemas de *maximização*, todos os coeficientes na última linha de um quadro Simplex devem ser não-negativos). No entanto, a maioria dos autores prefere manter esse critério, e usar em vez da função de *minimização* uma função objectivo equivalente de *maximização*. Para isso, recorrem à equivalência seguinte:

$$\min z = \sum_{i=1}^n c_i x_i \quad \text{é equivalente a} \quad \max -z = \sum_{i=1}^n (-c_i) x_i.$$

Exemplo 2.6 Considere o modelo descrito no Exemplo 2.5. O quadro Simplex do modelo “aumentado” (com variáveis artificiais) que corresponde à 1ª fase do método das 2-fases é o seguinte:

	z	x_1	x_2	x_3	E_1	E_2	E_3	A_1	A_2	
x_1	0	1	$\frac{1}{2}$	2	$-\frac{1}{2}$	0	0	0	0	5
A_1	0	0	1	1	0	-1	0	1	0	4
A_2	0	0	2	1	0	0	-1	0	1	8
$-z$	-1	0	0	0	0	0	0	1	1	0
	0	0	-1	-1	0	1	0	-1	0	-4
	0	0	-2	-1	0	0	1	0	-1	-8
$-z$	-1	0	-3	-2	0	1	1	0	0	-12

A função objectivo passou a ser $\min z = A_1 + A_2$. Os cálculos na parte inferior do quadro permitem exprimir a função objectivo em ordem às variáveis não-básicas. São operações elementares sobre linhas da matriz que representa o sistema de equações. Imediatamente abaixo da linha da função objectivo, temos os coeficientes da 2ª restrição multiplicados por -1 . A 2ª linha corresponde aos coeficientes da 3ª restrição multiplicados também por -1 . Somando os coeficientes da função objectivo com esses coeficientes obtemos a última linha do quadro, que é uma expressão em ordem às variáveis não-básicas. As iterações do método Simplex resultam nos quadros seguintes:

	z	x_1	x_2	x_3	E_1	E_2	E_3	A_1	A_2	
x_1	0	1	0	$\frac{3}{2}$	$-\frac{1}{2}$	$\frac{1}{2}$	0	$-\frac{1}{2}$	0	3
x_2	0	0	1	1	0	-1	0	1	0	4
A_2	0	0	0	-1	0	2	-1	-2	1	0
$-z$	-1	0	0	1	0	-2	1	3	0	0

	z	x_1	x_2	x_3	E_1	E_2	E_3	A_1	A_2	
x_1	0	1	0	$\frac{7}{4}$	$-\frac{1}{2}$	0	$\frac{1}{4}$	0	$\frac{1}{4}$	3
x_2	0	0	1	$\frac{1}{2}$	0	0	$-\frac{1}{2}$	0	$\frac{1}{2}$	4
E_2	0	0	0	$-\frac{1}{2}$	0	1	$-\frac{1}{2}$	-1	$\frac{1}{2}$	0
$-z$	-1	0	0	0	0	0	0	1	1	0

O último quadro é óptimo. Dado que a solução válida de base não tem nenhuma variável artificial positiva, o modelo original tem solução. Passamos assim para a 2ª fase.

As variáveis artificiais são retiradas do quadro, e a última linha é substituída pelos coeficientes das variáveis na função objectivo original. As operações listadas no quadro abaixo, tal como acima, têm por objectivo garantir que a função objectivo é expressa em ordem às variáveis não-básicas. No final dessas operações, dado que o critério de optimalidade é satisfeito, concluimos que a solução válida de base representada no quadro é solução óptima do problema.

	z	x_1	x_2	x_3	E_1	E_2	E_3	
x_1	0	1	0	$\frac{7}{4}$	$-\frac{1}{2}$	0	$\frac{1}{4}$	3
x_2	0	0	1	$\frac{1}{2}$	0	0	$-\frac{1}{2}$	4
E_2	0	0	0	$-\frac{1}{2}$	0	1	$-\frac{1}{2}$	0
$-z$	-1	4	12	16	0	0	0	0
	0	-4	0	-7	2	0	-1	-12
	0	0	-12	-6	0	0	6	-48
$-z$	-1	0	0	3	2	0	5	-60

□

2.3.4.3 Método do Grande M

Uma alternativa ao método das 2-fases consiste em resolver o modelo com variáveis artificiais numa única fase, penalizando essas variáveis na função objectivo. Usamos a letra M para designar um coeficiente de valor elevado (bem maior do que os outros coeficientes do modelo). Vamos considerar um problema de *minimização* com n variáveis de decisão ($x_i, i = 1, \dots, n$), m restrições e a função objectivo seguinte:

$$\min z = \sum_{i=1}^n c_i x_i.$$

Vamos assumir que foram adicionadas $m' \leq m$ variáveis artificiais ($A_j, j = 1, \dots, m'$) ao modelo, nas restrições que levantam problemas. O método do grande M consiste em resolver através do método Simplex o mesmo modelo com a função objectivo seguinte:

$$\min z = \sum_{i=1}^n c_i x_i + \sum_{j=1}^{m'} M \times A_j.$$

O Exemplo seguinte ilustra o funcionamento desse método.

Exemplo 2.7 Vamos usar outra vez o modelo do Exemplo 2.5. As duas variáveis artificiais que é necessário adicionar ao modelo são penalizadas na função objectivo com um coeficiente de valor M . O quadro Simplex inicial é o seguinte.

	z	x_1	x_2	x_3	E_1	E_2	E_3	A_1	A_2	
x_1	0	1	$\frac{1}{2}$	2	$-\frac{1}{2}$	0	0	0	0	5
A_1	0	0	1	1	0	-1	0	1	0	4
A_2	0	0	2	1	0	0	-1	0	1	8
$-z$	-1	4	12	16	0	0	0	M	M	0
	0	-4	-2	-8	2	0	0	0	0	-20
	0	0	$-M$	$-M$	0	M	0	$-M$	0	$-4M$
	-1	0	$-2M$	$-M$	0	0	M	0	$-M$	$-8M$
$-z$	-1	0	$10 - 3M$	$8 - 2M$	2	M	M	0	0	$-20 - 12M$

Mais uma vez, é necessário exprimir a função objectivo em ordem às variáveis não-básicas. As operações listadas na parte inferior do quadro descrevem esses passos. Os passos seguintes consistem numa aplicação directa do método Simplex. No quadro acima, a coluna pivot está associada à variável x_2 devido ao facto do coeficiente $10 - 3M$ ser o mais negativo.

	z	x_1	x_2	x_3	E_1	E_2	E_3	A_1	A_2	
x_1	0	1	0	$\frac{3}{2}$	$-\frac{1}{2}$	$\frac{1}{2}$	0	$-\frac{1}{2}$	0	3
x_2	0	0	1	1	0	-1	0	1	0	4
A_2	0	0	0	-1	0	2	-1	-2	1	0
$-z$	-1	0	0	$-2 + M$	2	$10 - 2M$	M	$-10 + 3M$	0	-60

	z	x_1	x_2	x_3	E_1	E_2	E_3	A_1	A_2	
x_1	0	1	0	$\frac{7}{4}$	$-\frac{1}{2}$	0	$\frac{1}{4}$	0	$\frac{1}{4}$	3
x_2	0	0	1	$\frac{1}{2}$	0	0	$-\frac{1}{2}$	0	$\frac{1}{2}$	4
E_2	0	0	0	$-\frac{1}{2}$	0	1	$-\frac{1}{2}$	-1	$\frac{1}{2}$	0
$-z$	-1	0	0	3	2	0	5	M	$-5 + M$	-60

Naturalmente, o valor da solução óptima corresponde àquele que obtivemos através do método das 2-fases. As variáveis artificiais são não-básicas. A solução é por isso válida para o modelo original. $\square$

2.3.5 Degenerescência

A degenerescência em Programação Linear corresponde a um conjunto de situações particulares relativamente à estrutura do espaço de soluções válidas. Os casos de degenerescência podem ser interpretados do ponto de vista algébrico e geométrico. Distinguem-se dois casos:

- a mesma solução válida de base pode ser representada usando várias bases diferentes;
- existem várias soluções alternativas (*degenerescência dual*).

O primeiro caso ocorre quando existem soluções válidas de base com pelo menos uma variável básica igual a 0. Se trocarmos essa variável na base por outra variável não-básica, esta última ficará com valor 0 na base seguinte. Do ponto de vista geométrico, esse caso ocorre quando um ponto extremo de um problema com n variáveis está na intersecção de mais de n hiperplanos. Num modelo com 2 variáveis, por exemplo, se um ponto extremo do espaço de soluções válidas estiver na intersecção de mais do que 2 rectas associadas a restrições do problema, esse ponto será um ponto extremo degenerado. O Exemplo seguinte ilustra esse caso.

Exemplo 2.8 Considere o modelo de Programação Linear seguinte:

$$\begin{aligned}
 \max \quad & z = \quad x_1 \quad + \quad 2x_2 \\
 \text{s. a} \quad & \\
 & x_1 \quad + \quad 4x_2 \leq 16 \\
 & x_1 \quad + \quad x_2 \leq 7 \\
 & x_1 \leq 6 \\
 & x_1 \quad + \quad 2x_2 \leq 10 \\
 & x_1 \quad , \quad x_2 \geq 0
 \end{aligned}$$

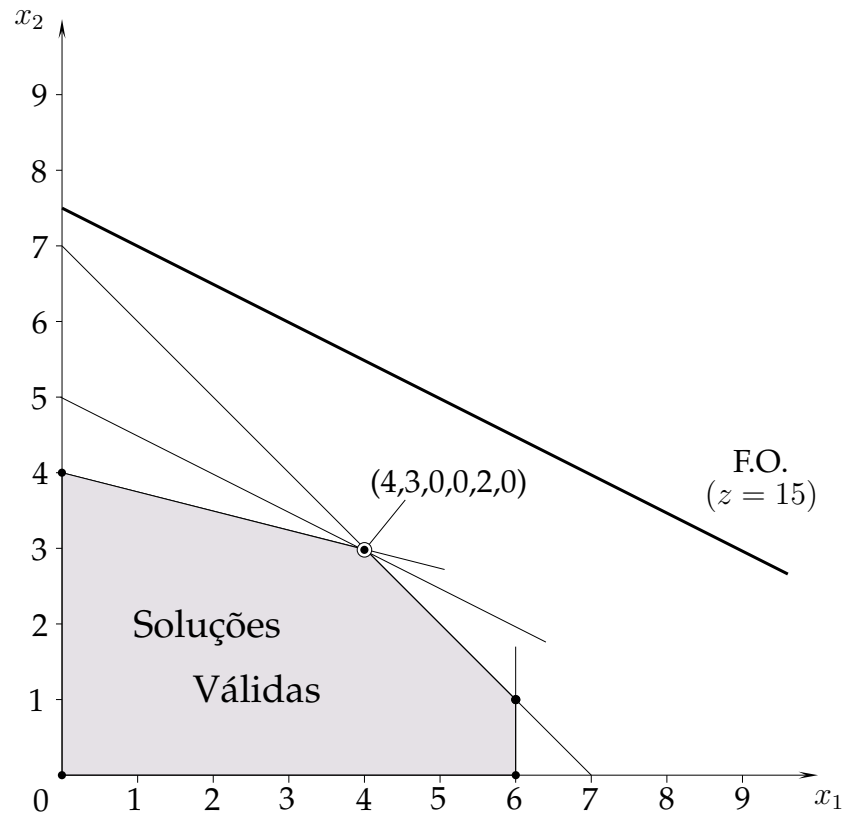


Figura 2.5: Representação gráfica (modelo do Exemplo 2.8)

Adicionando variáveis de folga às restrições, obtemos o modelo na sua forma standard:

$$\begin{array}{llllllll}
 \max \ z = & x_1 & + & 2x_2 & & & & \\
 \text{s. a} & & & & & & & \\
 & x_1 & + & 4x_2 & + & F_1 & & = \ 16 \\
 & x_1 & + & x_2 & & & + & F_2 = \ 7 \\
 & x_1 & & & & & + & F_3 = \ 6 \\
 & x_1 & + & 2x_2 & & & & + F_4 = 10 \\
 & x_1 & , & x_2 & , & F_1 & , & F_2 , F_3 , F_4 \geq 0
 \end{array}$$

A solução ótima desse modelo é a seguinte:

$$(x_1, x_2, F_1, F_2, F_3, F_4) = (4, 3, 0, 0, 2, 0).$$

A Figura 2.5 ilustra o espaço de soluções válidas desse modelo. Conforme se pode observar, o ponto extremo que corresponde à solução ótima está na intersecção das rectas associadas à 1ª, 2ª e 4ª restrição.

É possível representar a solução óptima através das 3 bases ilustradas nos quadros Simplex seguintes:

	z	x_1	x_2	F_1	F_2	F_3	F_4	
x_2	0	0	1	$\frac{1}{3}$	$-\frac{1}{3}$	0	0	3
x_1	0	1	0	$-\frac{1}{3}$	$\frac{4}{3}$	0	0	4
F_3	0	0	0	$\frac{1}{3}$	$-\frac{4}{3}$	1	0	2
F_4	0	0	0	$-\frac{1}{3}$	$-\frac{2}{3}$	0	1	0
z	1	0	0	$\frac{1}{3}$	$\frac{2}{3}$	0	0	10

	z	x_1	x_2	F_1	F_2	F_3	F_4	
x_2	0	0	1	$\frac{1}{2}$	0	0	$-\frac{1}{2}$	3
F_2	0	0	0	$\frac{1}{2}$	1	0	$-\frac{3}{2}$	0
F_3	0	0	0	1	0	1	-2	2
x_1	0	1	0	-1	0	0	2	4
z	1	0	0	0	0	0	1	10

	z	x_1	x_2	F_1	F_2	F_3	F_4	
x_2	0	0	1	0	-1	0	1	3
F_1	0	0	0	1	2	0	-3	0
F_3	0	0	0	0	$-\frac{1}{2}$	1	$\frac{5}{4}$	2
x_1	0	0	0	0	2	0	-1	4
z	1	0	0	0	0	0	1	10

□

Na prática, esta situação pode levar o método Simplex a entrar em ciclo em que as bases associadas a soluções degeneradas são repetidas sem fim. No entanto, essas situações são muito raras. Uma forma de os evitar consiste em mudar a sequência de variáveis básicas que são retiradas da base.

O segundo caso de degenerescência está ligado à existência de soluções alternativas. Do ponto de vista geométrico, essas situações ocorrem quando o hiperplano associado a uma restrição é paralelo à função objectivo. Nesse caso, o custo reduzido de algumas variáveis não-básicas é nulo, e é possível efectuar uma iteração do método Simplex no sentido de uma solução válida de base adjacente sem que o valor da função objectivo se altere.

2.4 Representação Matricial

Por definição, uma solução válida de base de um modelo de Programação Linear com n variáveis e m restrições está associada a m colunas linearmente independentes que correspondem às suas variáveis básicas. Do ponto de vista algébrico, deve ser possível exprimir essas m variáveis básicas em ordem às restantes $n - m$ variáveis não-básicas. Se nos reportarmos à representação através de um quadro Simplex da solução válida de base inicial de um modelo de Programação Linear, observamos facilmente que é possível dividir os vários coeficientes em matrizes conforme descrito abaixo:

x_B	A	I	b
	$-c$	0	0

A matriz A (com m linhas e m colunas) designa a matriz de coeficientes das variáveis não-básicas, enquanto a matriz identidade I (com m linhas e $n - m$ colunas) está associada às variáveis básicas da solução. A existência dessa matriz identidade traduz o facto das variáveis básicas serem expressas em ordem às variáveis não-básicas. O vector x_B é o vector das variáveis básicas, cujos valores correspondem respectivamente aos elementos do vector b . O vector de coeficientes das variáveis não-básicas na função objectivo é designado por $-c$ (assumindo que a função objectivo é de *maximização*).

Para uma solução válida de base qualquer, as m colunas linearmente independentes associadas às variáveis básicas formam uma matriz que é designada por B . As restantes $n - m$ colunas associadas às variáveis não-básicas são designadas por N (as variáveis associadas são designadas por x_N). Os vectores c_B e c_N designam os respectivos vectores de coeficientes na função objectivo dessas variáveis. Na solução inicial representada acima, temos que $B = I$ e $N = A$.

De forma genérica, podemos reescrever um modelo de Programação Linear usando notação matricial da forma seguinte:

$$\begin{aligned} \max \quad & z = c_N x_N + c_B x_B, \\ \text{s. a} \quad & N x_N + B x_B = b, \\ & x_N, x_B \geq 0. \end{aligned}$$

Multiplicando por B^{-1} as matrizes de coeficientes das restrições, passamos a poder

exprimir as variáveis básicas em ordem às não-básicas:

$$B^{-1}Nx_N + B^{-1}Bx_B = B^{-1}b \quad \text{e} \quad B^{-1}Nx_N + x_B = B^{-1}b. \quad (2.1)$$

Fazendo o mesmo com a função objectivo, podemos exprimir z em ordem às variáveis não-básicas substituindo na função objectivo x_B por

$$x_B = B^{-1}b - B^{-1}Nx_N,$$

que se obtém através da equação (2.1). O resultado é o seguinte:

$$z = c_Nx_N + c_B(B^{-1}b - B^{-1}Nx_N) = (c_N - c_BB^{-1}N)x_N + c_BB^{-1}b.$$

Dado que as variáveis não-básicas são nulas ($x_N = 0$), o valor das variáveis básicas x_B é igual a $B^{-1}b$ (equação (2.1)). Se $x_B \geq 0$, então a solução definida por B é válida de base.

Partindo da solução válida de base inicial representada acima, uma solução válida de base que é definida por um conjunto de colunas B será representada na forma matricial da forma seguinte:

x_B	$B^{-1}A$	B^{-1}	$B^{-1}b$
z	$c_BB^{-1}A - c$	c_BB^{-1}	$c_BB^{-1}b$

2.5 Método Simplex Revisto

Na sua definição original, o método Simplex obriga ao cálculo da totalidade dos coeficientes que compõem cada um dos quadros. Tendo em atenção a estrutura desses quadros, é possível tornar o método mais eficiente reduzindo o número de operações efectuadas e diminuindo o espaço de memória necessário à sua execução. O método que resulta dessas simplificações é designado por *método Simplex revisto*. Os pacotes de software comercial mais competitivos implementam essa versão do método Simplex.

Se observarmos o último quadro Simplex da Secção anterior, concluímos rapidamente sobre a importância da matriz B^{-1} na definição do quadro Simplex associado a uma determinada solução válida de base. Toda a informação do quadro depende dessa matriz e da informação do modelo original (matriz A , e vectores b e c). Lembremos que

a matriz B^{-1} é a inversa da matriz formada pelos vectores de coeficientes das variáveis básicas (B). Assim, sempre que fazemos referência à matriz B^{-1} , estamos também a referir-nos implicitamente a uma base (x_B). O método Simplex revisto baseia-se em grande parte no cálculo dessa matriz B^{-1} .

O Algoritmo 3.2 resume a sequência de operações que definem o método Simplex revisto. Uma vez mais, assumimos que a função objectivo é de *maximização*.

Para podermos efectuar uma iteração do método Simplex, não necessitamos de toda a informação contida no quadro. Para escolhermos a variável não-básica que irá entrar para a base, é necessário calcular os coeficientes na última linha do quadro dessas variáveis. Esse cálculo é feito a partir da matriz B^{-1} e da informação do modelo original. Se não existir nenhuma variável básica com coeficiente negativo nessa linha, concluímos que a solução válida de base corrente é óptima. Uma vez conhecida a variável que vai entrar para a base, temos de identificar aquela que irá deixar o seu lugar na base. Para isso, temos de calcular a razão entre os coeficientes da coluna à direita do quadro com os coeficientes da coluna associada à variável que entra para a base. Conhecida a variável que vai sair da base, fica completamente definida a estrutura da base seguinte (x_B). A matriz B pode ser actualizada, B^{-1} é recalculada e o processo pode ser repetido.

Na prática, não é necessário recalcular explicitamente a matriz B^{-1} por inversão directa da matriz B . As operações elementares sobre linhas que são feitas em cada iteração do método Simplex permitem obter rapidamente essa matriz B^{-1} . Tipicamente, a matriz B^{-1} é recalculada apenas explicitamente ao cabo de um certo número de iterações para evitar a propagação de erros de truncatura.

É importante notar que em cada uma das iterações não é necessário calcular os coeficientes das colunas (parte interior do quadro) associadas às variáveis não-básicas. Em modelos de grandes dimensões em que o número de variáveis pode ser considerável, o facto de não calcular todos esses coeficientes (e de não os armazenar em memória) tem um impacto significativa no desempenho do método. Em muitos casos, a matriz original A é uma matriz esparsa (com muitos 0) que é possível armazenar em memória de forma eficiente e a partir da qual é fácil realizar operações algébricas. Contudo, à medida que vão sendo realizadas iterações do método Simplex, essas vantagens vão sendo anuladas uma vez que a matriz resultante $B^{-1}A$ vai ficando cada vez mais densa.

No método Simplex revisto, os coeficientes das variáveis não-básicas na última linha dos quadros Simplex são calculados sequencialmente. Em vez de calcular todos os coeficientes dessa linha para determinar o menor coeficiente negativo, podemos optar por colocar na base a primeira variável não-básica com coeficiente negativo. Em todo o caso, para confirmar a optimalidade de uma solução, terão de ser calculados sempre

Algoritmo 3.2: Método Simplex Revisto

```

1 Input: modelo de Programação Linear ;
2 Determinar uma solução válida de base inicial ;
   /* os vectores coluna associados aos coeficientes nas restrições do
   modelo original das variáveis básicas definem a matriz  $B$  */
3 se não existir nenhuma solução válida de base então
4   | Output: “Modelo não tem soluções válidas” ;
5   | Goto 26 ;
6 senão
7   | Repetir
8   |   Calcular  $B^{-1}$ ;
9   |   Para todas as variáveis  $j$  não-básicas fazer
10  |   |  $d_j := c_B B^{-1} A_j - c_j$  ;
    |   | /*  $A_j$ : vector coluna associado aos coeficientes nas
    |   | restrições do modelo original da variável não-básica  $j$ ,
    |   |  $c_j$ : coeficiente na função objectivo dessa variável */
11  |   | se existe pelo menos uma variável não-básica  $j$  tal que  $d_j < 0$  então
12  |   |   Seja  $j'$  a variável não-básica tal que  $d_{j'} = \min_j \{d_j\}$  ;
13  |   |   Para todas as variáveis  $i$  básicas fazer
14  |   |   | se  $(B^{-1} A_{j'})_i > 0$  então
15  |   |   |   |  $e_i := \frac{(B^{-1} b)_i}{(B^{-1} A_{j'})_i}$  ;
16  |   |   |   senão
17  |   |   |   |  $e_i := M$  ;
18  |   |   | se existe pelo menos uma variável básica  $i$  tal que  $e_i \neq M$  então
19  |   |   |   | Seja  $i'$  a variável básica tal que  $e_{i'} = \min_i \{e_i\}$  ;
20  |   |   | senão
21  |   |   |   | Output: “O modelo tem solução ilimitada” ;
22  |   |   |   | Goto 26;
23  |   |   | Actualizar a base e a matriz  $B$  ;
    |   |   | /* a variável  $j'$  entra para a base,  $i'$  sai da base */
24  |   | Até todos os  $d_j$  serem positivos ou nulos ;
25 Output: “A solução  $x_B = B^{-1}b$  é solução óptima do modelo” ;
26 Stop ;

```

todos os coeficientes das variáveis não-básicas.

2.6 Métodos de Ponto Interior

Uma alternativa ao método Simplex para a resolução de modelos de Programação Linear são os chamados *métodos de ponto interior*. Como vimos no Capítulo 1, o advento desses métodos data do final dos anos 80, e deve-se fundamentalmente a Narendra Karmarkar. Os métodos de ponto interior contrastam com o método Simplex em termos de complexidade computacional. Enquanto que, no pior caso, o tempo que o método Simplex leva a determinar a solução ótima é expresso como uma função exponencial em ordem à dimensão do problema, os métodos de ponto interior correm em tempo polinomial. No entanto, os dois tipos de métodos têm desempenhos médios comparáveis. Tipicamente, o Simplex necessita de um maior número de iterações, mas as operações em cada iteração são simples. Os métodos de ponto interior convergem num menor número de iterações, mas dependem de cálculos mais complexos. As diferenças entre os métodos de ponto interior e o método Simplex fazem-se notar essencialmente em problemas de grandes dimensões para os quais o número de pontos extremos é muito elevado. Poderão também existir vantagens em recorrer a métodos de ponto interior para problemas com um elevado grau de degenerescência.

Os métodos de ponto interior, tal como o método Simplex, são métodos iterativos. Diferem do método Simplex na forma como exploram o espaço de soluções. Como vimos, o método Simplex percorre esse espaço caminhando entre pontos extremos (soluções válidas de base). Por seu lado, os métodos de ponto interior partem de uma solução válida no interior do espaço de soluções válidas, e mantêm-se estritamente no interior desse espaço convergindo em direcção à solução ótima. Os métodos de ponto interior não são penalizados pela existência de pontos degenerados, já que por definição esses pontos são pontos extremos do espaço de soluções válidas.

Desde a descoberta de Karmarkar em 1984, foram propostos diferentes métodos de ponto interior. As variantes diferem em aspectos como o comprimento do passo que conduz de uma solução à solução seguinte, ao tipo de soluções exploradas em cada iteração ou ainda ao espaço onde são pesquisadas as soluções. Apesar dessas diferenças, todos os métodos partilham as características de base que enunciámos acima.

2.7 Dualidade

A dualidade é um dos conceitos mais importantes da Programação Linear. Permite associar a um problema de Programação Linear um outro problema, chamado o *prob-*

lema dual, a partir da informação do primeiro. O problema inicial é normalmente designado por *problema primal*. Os dois problemas estão estreitamente ligados. Partilham propriedades importantes nomeadamente no que toca às suas soluções válidas e em particular às suas soluções óptimas. Como vimos atrás, ao usar o método Simplex para resolver um modelo de Programação Linear, conseguimos obter mais informação para além dos valores óptimos das variáveis de decisão do problema inicial (primal). Essa informação diz respeito aos custos reduzidos das variáveis não-básicas e aos preços sombra associados às restrições. Esses valores estão directamente relacionados com a formulação do problema dual. Nesta Secção, descrevemos essas correspondências em detalhe.

2.7.1 Correspondência Primal-Dual

O problema dual é construído a partir da informação do problema primal. Para simplificarmos a apresentação, vamos designar o problema primal por P e o problema dual correspondente por D . Vamos também assumir que P é um problema de maximização. O exemplo seguinte ilustra a forma como é construído o problema dual com base na informação do problema primal.

Exemplo 2.9 Considere o modelo de Programação Linear descrito no Exemplo 2.2:

$$\begin{array}{llllll}
 P : & \max & z_P = & x_1 & + & 2x_2 \\
 & s. & a & & & \\
 & & & x_1 & + & 4x_2 & \leq & 16 \\
 & & & x_1 & + & x_2 & \leq & 7 \\
 & & & x_1 & & & \leq & 6 \\
 & & & x_1 & , & x_2 & \geq & 0
 \end{array}$$

Vamos designar esse problema por *problema primal* P por uma questão de convenção uma vez que é o problema de partida. O número de variáveis de decisão e de restrições do problema D é igual respectivamente ao número de restrições e de variáveis de P . Enquanto que o problema P tem 2 variáveis de decisão e 3 restrições, o problema D terá 3 variáveis de decisão e 2 restrições.

O tipo de função objectivo de D depende do tipo de função objectivo de P : uma vez que P um problema de maximização, D será um problema de minimização. Por seu lado, os coeficientes da função objectivo de D correspondem respectivamente aos coeficientes do termo independente das restrições de P . O coeficiente do termo independente da primeira restrição será o coeficiente da primeira variável de decisão de D ,

e assim sucessivamente. O problema P está expresso na forma canónica: sendo um problema de maximização, as restrições são do tipo *menor ou igual*. Da mesma forma, o problema D , que é um problema de minimização, irá ter restrições do tipo *maior ou igual*. Se considerarmos, como é o caso, o problema primal expresso na forma canónica, as variáveis do problema dual serão não-negativas.

Finalmente, as restrições de D são definidas a partir dos vectores coluna de coeficientes associados às variáveis de P . O termo independente dessas restrições corresponde respectivamente ao coeficiente na função objectivo associado a essa coluna de P .

O problema dual D é definido da forma seguinte:

$$\begin{aligned}
 D : \quad \min \quad z_D = & \quad 16y_1 \quad + \quad 7y_2 \quad + \quad 6y_3 \\
 \text{s. a} \quad & \\
 & y_1 \quad + \quad y_2 \quad + \quad y_3 \quad \geq \quad 1 \\
 & 4y_1 \quad + \quad y_2 \quad \geq \quad 2 \\
 & y_1 \quad , \quad y_2 \quad , \quad y_3 \quad \geq \quad 0
 \end{aligned}$$

□

Listamos na tabela abaixo as regras que são usadas para formular o problema dual D a partir do modelo associado ao problema primal P .

	P	D	
Função Objectivo	max	min	
Coeficientes	A	A	
Coeficientes na FO	c	b	
Termo independente	b	c	
Variável x_i	não-negativa	$\geq$	Restrição i
Variável x_i	não-positiva	$\leq$	Restrição i
Variável x_i	sem restrição de sinal	$=$	Restrição i
Restrição j	$\leq$	não-negativa	Variável y_j
Restrição j	$\geq$	não-positiva	Variável y_j
Restrição j	$=$	sem restrição de sinal	Variável y_j

A correspondência entre problema primal e dual pode ser representada usando também a notação matricial que introduzimos acima. Ao problema primal P representado

como segue:

$$\begin{aligned}
 P : \quad & \max \quad z_P = cx, \\
 & s. \ a \\
 & Ax \leq b, \\
 & x \geq 0,
 \end{aligned}$$

corresponderá assim o problema dual D seguinte:

$$\begin{aligned}
 D : \quad & \min \quad z_D = yb, \\
 & s. \ a \\
 & yA \geq c, \\
 & y \geq 0.
 \end{aligned}$$

Essa correspondência traduz-se também nos quadros Simplex. Um quadro Simplex reproduz o sistema de equações do modelo para uma determinada solução válida de base do problema primal P . Cada linha do quadro, excepto a última, representa os coeficientes de todas as variáveis numa determinada restrição. A última linha corresponde aos coeficientes das variáveis na função objectivo. As colunas do quadro estão associadas às variáveis do modelo, excepto a coluna da direita que representa o valor das respectivas variáveis básicas. A representação do problema dual obtém-se “rodando” o quadro: as colunas do quadro passam a representar os coeficientes das variáveis duais nas restrições, enquanto que as linhas passam a estar associadas às variáveis duais. O valor das variáveis duais corresponde agora aos coeficientes da última linha do quadro Simplex. O Exemplo seguinte ilustra essa correspondência.

Exemplo 2.10 A solução $(0, 4, 0, 3, 6)$ é uma solução válida de base para o modelo do Exemplo 2.9. O quadro Simplex associado a essa solução é o seguinte:

	z	x_1	x_2	F_1	F_2	F_3	
x_2	0	$\frac{1}{4}$	1	$\frac{1}{4}$	0	0	4
F_2	0	$\frac{3}{4}$	0	$-\frac{1}{4}$	1	0	3
F_3	0	1	0	0	0	1	6
z	1	$-\frac{1}{2}$	0	$\frac{1}{2}$	0	0	8

O valor das variáveis do problema primal pode ser lido na coluna do lado direito do quadro. As variáveis não-básicas têm valor nulo. Vamos designar por E_1 e E_2 as variáveis de excesso do modelo dual que apresentámos no Exemplo 2.9. O quadro

Simplex acima apresenta também uma solução para o problema dual. O valor das variáveis duais pode ser lido na última linha do quadro Simplex, e correspondem a

$$(y_1, y_2, y_3, E_1, E_2) = \left(\frac{1}{2}, 0, 0, -\frac{1}{2}, 0 \right).$$

Essa solução dual viola a condição de não-negatividade da variável de excesso E_1 . Ao mesmo tempo, a solução primal correspondente não é ótima, precisamente devido à existência desse coeficiente negativo na linha da função objectivo. Existe de facto uma relação entre essas duas situações, à qual voltaremos no final desta Secção. $\square$

Existem relações muito estreitas entre as soluções dos problemas primal e dual. Assim, é possível afirmar que o valor da função objectivo de P para qualquer solução válida x é sempre inferior ao valor da função objectivo de D para qualquer solução válida y , *i.e.*

$$cx \leq yb.$$

Se atendermos às restrições de não-negatividade das variáveis primais x e das variáveis duais y , chegamos facilmente a esse resultado. No problema P , se multiplicarmos os vectores Ax e b por y , obtemos

$$yAx \leq yb = z_D.$$

Por definição, $yA \geq c$, logo

$$z_P = cx \leq yAx \leq yb = z_D.$$

Essa propriedade é conhecida por *propriedade da dualidade fraca*.

A propriedade da dualidade fraca permite-nos, por exemplo, determinar limites para o valor ótimo da função objectivo de D com base numa solução válida para P : se x for uma solução válida de P e z_D^* o valor ótimo de D , pela propriedade da dualidade fraca, podemos afirmar que $cx \leq z_D^*$, *i.e.* o ótimo de D nunca será inferior a cx .

Podemos tirar a mesma conclusão relativamente ao valor ótimo do problema P . Se z_P^* designar o valor ótimo de P , para qualquer solução válida y do problema D , teremos que $z_P^* \leq yb$. No caso extremo, quando o problema P não tem solução limitada, o correspondente dual D é impossível. Contudo, é importante notar que essa relação não é simétrica: se D for um problema impossível, P não é necessariamente um problema com solução ilimitada conforme ilustrado no exemplo seguinte.

Exemplo 2.11 Considere o problema de Programação Linear seguinte:

$$\begin{aligned}
P : \max z_P = & \quad 3x_1 + 2x_2 \\
s. a & \\
& x_1 - x_2 \leq 5 \\
& -x_1 + x_2 \leq -7 \\
& x_1, x_2 \geq 0
\end{aligned}$$

O problema dual D associado a P é o seguinte:

$$\begin{aligned}
D : \min z_D = & \quad 5y_1 - 7y_2 \\
s. a & \\
& y_1 - y_2 \geq 3 \\
& -y_1 + y_2 \geq 2 \\
& y_1, y_2 \geq 0
\end{aligned}$$

É fácil verificar que ambos os problemas são impossíveis: se adicionarmos as duas restrições em cada um dos modelos, obtemos respectivamente $0 \leq -2$ e $0 \geq 5$. $\square$

Nos casos em que P é um problema possível e com solução de valor finito, é possível relacionar os valores das soluções óptimas x^* e y^* de P e D , respectivamente, da forma seguinte:

$$cx^* = y^*b. \quad (2.2)$$

Esse resultado é mais forte que aquele que é enunciado pela propriedade da dualidade fraca, no sentido em que estabelece um valor específico para o valor ótimo dos dois problemas, em vez de definir simplesmente um limite para esses valores. Por esse motivo, essa propriedade é conhecida por *propriedade da dualidade forte*.

Partindo dessa propriedade da dualidade forte, é possível derivar uma relação interessante entre os valores das variáveis de decisão e os valores das variáveis de folga ou excesso dos dois problemas no ponto ótimo. Assim, se subtrairmos a quantidade y^*Ax^* a ambos os termos de (2.2), obtemos a relação seguinte

$$cx^* - y^*Ax^* = y^*b - y^*Ax^*,$$

a partir da qual chegamos facilmente à expressão seguinte

$$x^*(y^*A - c) + y^*(b - Ax^*) = 0. \quad (2.3)$$

Os vectores $(y^*A - c)$ e $(b - Ax^*)$ representam respectivamente o valor das variáveis de folga e excesso associados aos pontos óptimos x^* e y^* . Dado que todas as variáveis de P e D , incluindo as de folga e de excesso, são não-negativas, a relação (2.3) só se irá verificar se as condições seguintes forem satisfeitas:

- se a variável x_i^* for positiva, a variável de excesso da restrição i de D deve ter valor nulo; da mesma forma, se a variável y_j^* for positiva, a variável de folga da restrição j de P deve ter valor nulo;
- se a variável de folga da restrição j de P for positiva, a variável y_j^* de D deve ter valor nulo; da mesma forma, se a variável de excesso da restrição i de D for positiva, a variável x_i^* de P deve ter valor nulo.

Essa relação é conhecida por *propriedade da folga complementar*.

É interessante notar a tradução dessas propriedades no método Simplex. Em cada iteração, o método Simplex caminha para uma solução válida de base para o problema primal que melhore o valor da função objectivo. Ao mesmo tempo, o método determina uma solução para o problema dual que é complementar à solução primal, *i.e.* no sentido definido pela relação (2.3). Voltando ao Exemplo 2.10, podemos confirmar que as soluções primais e duais representadas no quadro Simplex verificam de facto essa relação.

Por outro lado, a solução dual representada no quadro não é válida uma vez que viola uma das condições de não-negatividade, enquanto que a solução do problema primal não é óptima dado que existem coeficientes negativos na última linha do quadro. Uma solução válida de base para o problema primal é óptima se todos os coeficientes da última linha do quadro forem não-negativos, *i.e.* se a solução dual complementar for também uma solução válida. Em suma, o método Simplex procura uma solução válida de base cuja solução dual complementar seja válida para o problema dual. Quando isso acontece, as soluções são óptimas para os seus respectivos problemas.

2.7.2 O Método Simplex Dual

No método Simplex, as soluções em cada iteração são soluções válidas de base para o problema primal, enquanto que as respectivas soluções duais complementares violam as restrições do problema dual (excepto no caso da solução óptima). No método Simplex Dual, acontece precisamente o contrário. O método parte de uma solução que é válida para o problema dual, mas cuja solução primal complementar viola as restrições do problema primal. Em cada iteração, o método procura uma solução dual válida que melhore o valor da função objectivo usando operações semelhantes às usadas no método Simplex. Ambos os métodos terminam quando as soluções primais e duais são simultaneamente válidas para os seus respectivos problemas.

O Algoritmo 3.3 descreve os passos do método Simplex Dual. Ilustramos o funcionamento do método através do Exemplo 2.12.

Algoritmo 3.3: Método Simplex Dual

```

1 Input: modelo de Programação Linear ;
2 Formular o modelo de modo a que todas as restrições sejam do tipo  $\leq$  ;
3 Transformar as restrições em equações adicionando variáveis de folga ;
4 Determinar uma primeira solução de base ;
   /* Seguindo as regras que usámos para construir os quadros Simplex,
      na última linha do quadro, as variáveis básicas deverão ter
      coeficientes não-negativos e as variáveis não-básicas
      coeficientes nulos */
5 se as variáveis básicas forem todas não-negativas então
6   | Output: “A solução de base corrente é solução óptima do modelo” ;
7   | Goto 15 ;
8 senão
9   | Enquanto houver variáveis básicas negativas fazer
10  |   Escolher a variável básica que vai sair da base: essa variável deve ser
11  |   negativa e ter o maior valor absoluto ;
12  |   Escolher a variável não-básica que vai entrar para a base: essa variável
13  |   deve ter coeficiente negativo na equação da variável básica que sai da
14  |   base, e a menor razão em valor absoluto entre o coeficiente na equação da
15  |   função objectivo e esse coeficiente ;
16  |   Expressar a nova variável básica em ordem à variável que saiu da base ;
17  |   Reescrever as restantes equações com base na expressão definida no passo
18  |   anterior ;
19  |   /* Os dois últimos passos são comuns ao método Simplex.
20  |      Consistem numa sequência de operações elementares sobre
21  |      linhas de um sistema de equações. */
22  |
23 Output: “A solução válida de base corrente é solução óptima do modelo” ;
24 Stop ;

```

Exemplo 2.12 Considere o modelo de Programação Linear seguinte:

$$\begin{aligned}
 \min \quad & z_D = 16y_1 + 7y_2 + 6y_3 \\
 \text{s. a} \quad & \\
 & y_1 + y_2 + y_3 \geq 1 \\
 & 4y_1 + y_2 \geq 2 \\
 & y_1, y_2, y_3 \geq 0
 \end{aligned}$$

Multiplicando cada um dos termos das restrições por -1 , e adicionando as variáveis E_1 e E_2 , chegamos ao seguinte quadro Simplex inicial.

	z	y_1	y_2	y_3	E_1	E_2	
E_1	0	-1	-1	-1	1	0	-1
E_2	0	-4	-1	0	0	1	-2
$-z$	-1	16	7	6	0	0	0

A solução de base representada nesse quadro não é válida para o problema primal, enquanto que a correspondente solução dual é válida para o problema dual. Na primeira iteração do método Simplex Dual, escolhemos a variável básica E_2 para sair da base por ser negativa e ter o maior valor absoluto. A variável não-básica y_1 vai entrar para a base: tem coeficiente negativo na linha associada a E_2 , e a menor razão em valor absoluto entre o coeficiente na última linha do quadro e esse coeficiente negativo ($|\frac{16}{-4}|$).

	z	y_1	y_2	y_3	E_1	E_2	
E_1	0	-1	-1	-1	1	0	-1
E_2	0	-4	-1	0	0	1	-2
$-z$	-1	16	7	6	0	0	0

$\leftarrow$ linha pivot

$$\begin{array}{c} \left| \frac{16}{-4} \right| \quad \left| \frac{7}{-1} \right| \quad \left| \frac{6}{-1} \right| \quad \times \quad \times \\ \uparrow \\ \text{coluna pivot} \end{array}$$

As iterações do método Simplex Dual conduzem aos quadros seguintes.

	z	y_1	y_2	y_3	E_1	E_2	
E_1	0	0	$-\frac{3}{4}$	-1	1	$-\frac{1}{4}$	$-\frac{1}{2}$
y_1	0	1	$\frac{1}{4}$	0	0	$-\frac{1}{4}$	$\frac{1}{2}$
$-z$	-1	0	3	6	0	4	-8

$\leftarrow$ linha pivot

$$\begin{array}{c} \times \quad \left| \frac{3}{-\frac{3}{4}} \right| \quad \left| \frac{6}{-1} \right| \quad \times \quad \left| \frac{4}{-\frac{1}{4}} \right| \\ \uparrow \\ \text{coluna pivot} \end{array}$$

	z	y_1	y_2	y_3	E_1	E_2	
y_2	0	0	1	$\frac{4}{3}$	$-\frac{4}{3}$	$\frac{1}{3}$	$\frac{2}{3}$
y_1	0	1	0	$-\frac{1}{3}$	$\frac{1}{3}$	$-\frac{1}{3}$	$\frac{1}{3}$
$-z$	-1	0	0	2	4	3	-10

O problema é o dual do problema que foi resolvido através do método Simplex na Secção 2.3.2. Podemos facilmente observar a correspondência entre o quadro óptimo que obtivemos nessa secção e o quadro óptimo acima. $\square$

2.8 Análise de Sensibilidade

Como vimos no Capítulo 1, a resolução de um problema não acaba quando se obtém a solução do modelo que o procura representar. Não nos podemos esquecer que a construção do modelo assenta num conjunto de pressupostos. Se é certo que a solução que obtivemos é óptima para o modelo que construímos, o risco de ela não o ser para o problema original é real.

Um dos pressupostos mais fortes em Programação Linear consiste em assumir que todos os parâmetros são valores fixos e perfeitamente conhecidos. Existem várias razões que nos podem levar a pôr em causa esses parâmetros: o facto de terem sido avaliados num determinado instante de tempo e poderem estar desactualizados, ou a possibilidade de terem sido simplesmente mal avaliados. Por essa razão, é importante perceber em que medida a solução óptima do modelo pode ser afectada por variações no valor dos parâmetros. Procuramos fazê-lo analisando a sensibilidade da solução óptima a alterações nos dados de entrada do modelo.

Em termos gerais, uma análise de sensibilidade pode ser conduzida tendo por base um dos seguintes objectivos:

- avaliar o impacto na solução óptima corrente da alteração de um dos parâmetros do modelo (em A , b ou c);
- determinar os intervalos de variação dos parâmetros do modelo que garantem que a estrutura da base óptima corrente não é alterada.

Para percebermos como pode ser realizada essa análise, vamos recorrer à representação matricial do quadro Simplex de uma solução válida de base:

x_B	$B^{-1}A$	B^{-1}	$B^{-1}b$
z	$c_B B^{-1}A - c$	$c_B B^{-1}$	$c_B B^{-1}b$

A título de exemplo, vamos considerar a alteração do coeficiente de uma variável numa restrição. Essa alteração afecta a matriz A do modelo original. Vamos designar por A'

a matriz que resulta dessa alteração. Podemos reportá-la no quadro acima da forma seguinte:

x_B	$B^{-1}A'$	B^{-1}	$B^{-1}b$
z	$c_B B^{-1}A' - c$	$c_B B^{-1}$	$c_B B^{-1}b$

Apenas as componentes que dependiam inicialmente de A são alteradas. Ao actualizarmos essas componentes, as variáveis básicas podem deixar de estar expressas em ordem às variáveis não-básicas. Se for o caso, aplicam-se as operações elementares sobre linhas de um sistema de equações (eliminação de Gauss) que já usámos atrás. Finalmente, se a solução obtida não for óptima, procede-se à reoptimização através do método Simplex, ou do método Simplex Dual.

Os passos que acabamos de descrever definem o procedimento geral da análise de sensibilidade. O Exemplo seguinte ilustra esse procedimento.

Exemplo 2.13 Vamos considerar novamente o modelo de Programação Linear que usámos em exemplos anteriores:

$$\begin{aligned}
 \max \quad z = & \quad x_1 \quad + \quad 2x_2 \\
 \text{s. a} \quad & \\
 & x_1 \quad + \quad 4x_2 \leq 16 \\
 & x_1 \quad + \quad x_2 \leq 7 \\
 & x_1 \leq 6 \\
 & x_1 \quad , \quad x_2 \geq 0
 \end{aligned}$$

O quadro Simplex associado à solução óptima desse modelo é o seguinte:

	z	x_1	x_2	F_1	F_2	F_3	
x_2	0	0	1	$\frac{1}{3}$	$-\frac{1}{3}$	0	3
x_1	0	1	0	$-\frac{1}{3}$	$\frac{4}{3}$	0	4
F_3	0	0	0	$\frac{1}{3}$	$-\frac{4}{3}$	1	2
z	1	0	0	$\frac{1}{3}$	$\frac{2}{3}$	0	10

A matriz A , e os vectores b e c associados ao modelo original têm os seguintes valores:

$$A = \begin{bmatrix} 1 & 4 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad b = \begin{bmatrix} 16 \\ 7 \\ 6 \end{bmatrix}, \quad c = \begin{bmatrix} 1 & 2 \end{bmatrix}.$$

Usando a representação matricial, podemos também dividir o quadro Simplex nas seguintes matrizes e vectores:

$$B^{-1}A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} \frac{1}{3} & -\frac{1}{3} & 0 \\ -\frac{1}{3} & \frac{4}{3} & 0 \\ \frac{1}{3} & -\frac{4}{3} & 1 \end{bmatrix}, \quad B^{-1}b = \begin{bmatrix} 3 \\ 4 \\ 2 \end{bmatrix},$$

$$c_B B^{-1}A - c = \begin{bmatrix} 0 & 0 \end{bmatrix}, \quad c_B B^{-1} = \begin{bmatrix} \frac{1}{3} & \frac{2}{3} & 0 \end{bmatrix}, \quad c_B B^{-1}b = \begin{bmatrix} 0 \end{bmatrix}.$$

Vamos assumir em primeiro lugar que o coeficiente da variável x_2 na primeira restrição, cujo valor no modelo inicial é igual a 4, passa a ser igual a 2. No modelo original, a matriz que é afectada por essa alteração é a matriz A . Vamos designar por A' a matriz resultante:

$$A' = \begin{bmatrix} 1 & \mathbf{2} \\ 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

Essa alteração afecta apenas as componentes do quadro Simplex da solução óptima que dependem de A , *i.e.* $B^{-1}A$ e $C_B B^{-1}A - c$ que passam a ter os valores seguintes:

$$B^{-1}A' = \begin{bmatrix} \frac{1}{3} & -\frac{1}{3} & 0 \\ -\frac{1}{3} & \frac{4}{3} & 0 \\ \frac{1}{3} & -\frac{4}{3} & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{3} \\ 1 & \frac{2}{3} \\ 0 & -\frac{2}{3} \end{bmatrix},$$

$$C_B B^{-1}A' - c = \begin{bmatrix} 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{3} & -\frac{1}{3} & 0 \\ -\frac{1}{3} & \frac{4}{3} & 0 \\ \frac{1}{3} & -\frac{4}{3} & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 1 \\ 1 & 0 \end{bmatrix} - \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & -\frac{2}{3} \end{bmatrix}.$$

O quadro final completo que resulta dessa alteração é o seguinte:

	z	x_1	x_2	F_1	F_2	F_3	
x_2	0	0	$\frac{1}{3}$	$\frac{1}{3}$	$-\frac{1}{3}$	0	3
x_1	0	1	$\frac{2}{3}$	$-\frac{1}{3}$	$\frac{4}{3}$	0	4
F_3	0	0	$-\frac{2}{3}$	$\frac{1}{3}$	$-\frac{4}{3}$	1	2
z	1	0	$-\frac{2}{3}$	$\frac{1}{3}$	$\frac{2}{3}$	0	10

O quadro não está na forma correcta: as variáveis básicas não estão expressas em ordem às variáveis não-básicas. O passo seguinte consiste em validar o quadro usando operações elementares sobre linhas de um sistema de equações.

	z	x_1	x_2	F_1	F_2	F_3	
x_2	0	0	1	1	-1	0	9
x_1	0	1	0	-1	$\frac{2}{3}$	0	-2
F_3	0	0	0	1	-2	1	8
z	1	0	0	1	0	0	16

A solução de base representada no quadro não é válida para o problema primal, enquanto a solução dual complementar é válida para o problema dual. Usamos o algoritmo Simplex Dual para determinar a solução óptima do modelo alterado a partir deste último quadro Simplex. O quadro final é obtido após uma iteração.

	z	x_1	x_2	F_1	F_2	F_3	
x_2	0	1	1	0	$-\frac{1}{3}$	0	7
F_1	0	-1	0	1	$-\frac{2}{3}$	0	2
F_3	0	1	0	0	$-\frac{4}{3}$	1	6
z	1	1	0	0	$\frac{2}{3}$	0	14

A nova solução óptima tem um valor igual a 14, e corresponde à solução válida de base:

$$(x_1, x_2, F_1, F_2, F_3) = (0, 7, 2, 0, 6).$$

Outra vertente de uma análise de sensibilidade consiste em determinar os intervalos dentro dos quais poderão variar os coeficientes do modelo sem que a estrutura da base seja alterada (de forma a que as variáveis básicas continuem na base). Vamos ter em atenção o coeficiente c_1 de x_1 na função objectivo. Esse valor pertence ao vector c do modelo original, e ao vector c_B do quadro Simplex final:

$$c = \begin{bmatrix} c_1 & 2 \end{bmatrix}, \quad c_B = \begin{bmatrix} 2 & c_1 & 0 \end{bmatrix}.$$

No quadro final, esses vectores afectam os valores da última linha do quadro. Para que a base desse quadro não se altere, os coeficientes das variáveis não-básicas F_1 e F_2 nessa linha devem ser não-negativos, *i.e.*

$$\begin{bmatrix} 2 & c_1 & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{3} \\ -\frac{1}{3} \\ \frac{1}{3} \end{bmatrix} = \frac{2}{3} - \frac{1}{3}c_1 \geq 0,$$

$$\begin{bmatrix} 2 & c_1 & 0 \end{bmatrix} \begin{bmatrix} -\frac{1}{3} \\ \frac{4}{3} \\ -\frac{4}{3} \end{bmatrix} = -\frac{2}{3} + \frac{4}{3}c_1 \geq 0.$$

Temos assim que o intervalo para c_1 que garante que a base não se altera é $[\frac{1}{2}; 2]$. Grandes intervalos significam uma menor sensibilidade da solução de base perante variações do valor do parâmetro estudado. Essa situação é ideal no sentido em que reflecte uma maior robustez da solução. $\square$

Para além dos casos abordados, também se poderá avaliar o impacto da adição de uma nova restrição ao modelo, ou de uma nova variável. Ambos os casos são tratados da mesma forma que os casos anteriores.

2.9 Conclusões

Os modelos de Programação Linear representam problemas de optimização definidos a partir de uma função objectivo e de restrições que são lineares nas variáveis. A relativa facilidade com que estes modelos são resolvidos, comparativamente com outros como os modelos não-lineares, tem motivado o interesse por estes modelos. O aumento da capacidade dos computadores tem contribuído também para que problemas reais de grande dimensão pudessem ser resolvidos em tempos muito reduzidos. Os melhores *softwares* comerciais da actualidade permitem hoje resolver problemas com vários milhares de variáveis e restrições.

No entanto, as suposições nas quais se baseiam os modelos de Programação Linear podem limitar a sua aplicabilidade. Desde já, a própria linearidade da função objectivo e das restrições pode não representar de forma adequada a realidade dos problemas. Estes modelos são também determinísticos, o que significa que se assume

que os parâmetros do problema (os coeficientes dos modelos) são perfeitamente conhecidos, o que nem sempre se verifica na prática. Por último, o facto das variáveis serem contínuas exclui muitos problemas reais extremamente relevantes em que as quantidades envolvidos correspondem apenas a valores inteiros. Esses problemas podem ser tratados através de modelos e métodos de Programação Inteira. Essas abordagens são o tema do próximo capítulo.

2.10 Exercícios

1. A Pastel&Nata é uma empresa de média dimensão que se dedica ao fabrico de produtos de pastelaria. A empresa recebeu recentemente uma encomenda de 200kg de umas das suas especialidades: o creme de pasteleiro *Pastelissimo*. O segredo do *Pastelissimo* reside no equilíbrio dos seus nutrientes, e numa receita original que não foi alterada desde a criação da empresa. O creme é composto por 20% de açúcar, 10% de leite e 15% de água, e contém pelo menos 20% de matéria gorda e 25% de ovos.

Para realizar o *Pastelissimo*, a Pastel&Nata dispõe de vários ingredientes que se distinguem pela sua composição e pelo seu preço. A tabela seguinte indica o peso (em kg) de cada um dos elementos constituintes presentes em 1kg do respectivo ingrediente:

	Gorduras vegetais	Óleo	Natas	Xarope de açúcar	Leite em pó	Leite fresco	Ovo em pó	Ovos frescos
Matéria gorda	0.80	0.70	0.35	0.05	0.10	0.15	0.30	0.15
Açúcar			0.30	0.80	0.05			
Ovos							0.60	0.75
Leite			0.20		0.70	0.75		
Água	0.20	0.30	0.15	0.15	0.15	0.10	0.10	0.10

O preço (em euros/Kg) de cada um dos ingredientes é indicado na tabela seguinte:

	Gorduras vegetais	Óleo	Natas	Xarope de açúcar	Leite em pó	Leite fresco	Ovo em pó	Ovos frescos
Preço	0.60	0.80	1.00	0.30	0.50	0.90	0.40	0.60

A Pastel&Nata pretende determinar em que quantidades deverá usar cada um dos ingredientes de modo a minimizar o custo total de produção.

- a) Formule este problema usando um modelo de Programação Linear.
- b) Determine a solução óptima do problema usando uma ferramenta (*software*) de optimização. Qual é a percentagem de matéria gorda e de ovos na receita final?
2. A RIR dedica-se ao fabrico e à comercialização de vários tipos de açúcar que produz através de um processo de refinação. As encomendas de açúcar branco passadas pelos clientes da empresa para os próximos 3 meses são de 4000, 8000 e 10000 toneladas, respectivamente. Os custos de produção da empresa variam ao longo dos meses, fruto essencialmente das flutuações do preço das matérias-primas. No próximo mês, prevê-se que esse custo de produção seja de 300 euros/tonelada, passando para 500 euros/tonelada no mês seguinte, e para 400 euros/tonelada no último mês.

Em cada mês, a empresa pode armazenar o açúcar para o vender eventualmente em períodos subsequentes. Por cada tonelada que é armazenada durante um mês, a empresa incorre num custo de 15 euros.

Formule este problema usando um modelo de Programação Linear.

3. Considere um problema de distribuição de energia eléctrica desde uma central localizada num ponto geográfico A até dois outros locais designados por D e E. A energia é encaminhada através de uma rede de distribuição que interliga os vários locais. Nessa rede, existem dois pontos intermédios B e C que recebem a energia vinda de A, e a redistribuem a D e E.

Em cada ligação da rede, existem tipicamente perdas de energia. A tabela seguinte indica o valor dessas perdas (em percentagem) para cada uma das ligações existentes na rede:

	A	B	C	D	E
A		40	10		
B			10	20	7
C		10		5	5
D					15
E					

O custo de produção de 1 megawatt na central localizada em A é de 100 euros. Os custos unitários de transporte entre cada ponto da rede são dados na tabela seguinte (em euros/megawatt):

	A	B	C	D	E
A		2	8		
B			3	4	10
C		3		12	9
D					5
E					

No próximo período de planeamento, irá ser necessário transportar 100 e 250 megawatts para os pontos D e E da rede, respectivamente.

Sabendo que os pontos B e C da rede não podem receber mais de 300 megawatts em cada período, pretende-se determinar qual deve ser a quantidade de energia que deve ser produzida em A, e como deve ser encaminhada pela rede de modo a minimizar o custo total.

- a) Formule este problema usando um modelo de Programação Linear.
 - b) Recorrendo a um *software* de optimização, determine a solução óptima do problema.
4. A companhia SPAGS produz 2 modelos de sacos desportivos: o modelo standard e o modelo de luxo. Diariamente, a SPAGS pode produzir até 450 unidades do modelo standard, e 325 unidades do modelo de luxo. No final do seu ciclo de produção, os sacos são sujeitos a operações de controlo da qualidade. Os sacos do modelo de luxo demoram 3 vezes mais tempo a serem controlados do que os sacos standard. A secção de controlo da qualidade dispõe diariamente de 660 horas.homem.

Um saco standard é vendido por 260 euros, enquanto que um saco do modelo de luxo é vendido por 390 euros. A SPAGS pretende determinar o seu plano diário de produção de modo a maximizar o seu lucro.

- a) Formule este problema usando um modelo de Programação Linear.
- b) Represente graficamente o espaço de soluções do modelo que definiu em a):
 - i. Identifique todas as soluções válidas de base.
 - ii. Determine a solução óptima do problema.
- c) Resolva o problema usando o método Simplex.

5. Uma companhia produz 2 artigos diferentes. Cada um desses artigos deve ser processado em 3 máquinas distintas, sendo que cada máquina necessita de pelo menos um operador para poder funcionar. O tempo (em horas) necessário ao processamento de cada artigo em cada uma das máquinas e o lucro unitário desses artigos estão indicados na tabela seguinte:

	Artigo 1	Artigo 2
Máquina 1	3	6
Máquina 2	5	
Máquina 3	4	8

A companhia dispõe actualmente de 7 operadores, e de 4, 2 e 3 máquinas do tipo 1, 2 e 3, respectivamente. A área de produção está aberta 40 horas por semana, e cada operador trabalha 35 horas por semana.

Nessas condições, a companhia pretende determinar o plano de produção semanal que maximize o seu lucro total.

- Formule este problema usando um modelo de Programação Linear.
 - Represente graficamente o espaço de soluções do modelo que definiu em a). Identifique a solução óptima do problema.
 - Resolva o problema usando o método Simplex. Interprete todos os passos que efectuou no gráfico que construiu em b).
6. Um florista comercializa 4 ramos de flores diferentes designados por A, B, C e D. Os ramos são feitos a partir de rosas, tulipas, cravos e margaridas. Cada ramo é composto por uma determinada combinação de flores conforme indicado na tabela seguinte:

	Rosas	Tulipas	Cravos	Margaridas
Ramo A	3	6	2	4
Ramo B	5		2	4
Ramo C	4	8	2	
Ramo D	4		2	6

O preço unitário de venda dos ramos A, B, C e D é respectivamente de 7, 12, 5 e 10 euros. Neste momento, o florista dispõe de 600 rosas, 400 tulipas, 500 cravos

e 400 margaridas. O florista pretende determinar em que quantidades deverá produzir cada tipo de ramo de modo a maximizar o lucro total, assumindo que os conseguirá vender todos.

- a) Formule este problema usando um modelo de Programação Linear.
- b) Determine a solução óptima do problema usando o método Simplex:
 - i. Quais são restrições que têm folga?
 - ii. Quais são os valores dos preços sombra associados às restrições do modelo? Interprete esses valores.
 - iii. Qual seria o impacto na solução óptima de uma diminuição de 1 Euro no preço de venda do ramo A? Justifique.
- c) Assuma que por cada flor que não é usada num ramo, o florista incorre num custo de c unidades monetárias. Como integraria esse dado no modelo que formulou em a).

7. Considere um problema de investimento no qual 100 milhões euros devem ser investidos por 3 aplicações financeiras que iremos designar por A, B e C. Os retornos esperados em percentagem do capital investido são de 15, 8 e 6% para cada uma das aplicações, respectivamente.

Para garantir um nível mínimo de investimento nas aplicações de menor rendimento, são impostas regras quanto aos montantes que poderão ser investidos em cada aplicação. Assim, por cada euro investido na aplicação A, 2 deverão ser investidos na aplicação C. Por outro lado, é imposto que pelo menos 20% do capital total seja investido nas aplicações B ou C, atendendo a que cada euro investido na aplicação B conta a dobrar.

- a) Formule este problema usando um modelo de Programação Linear.
 - b) Determine a solução óptima do problema usando o método Simplex.
8. Descreva os problemas que podem surgir na resolução de um problema de Programação Linear altamente degenerado.
9. Considere o modelo de Programação Linear seguinte:

$$\begin{array}{rcllcl}
 \max z = & 3x_1 & + & 6x_2 & + & 5x_3 \\
 \text{sujeito a} & & & & & \\
 & 4x_1 & + & 4x_2 & + & 5x_3 & \leq & 30 \\
 & 4x_1 & + & 5x_2 & + & 5x_3 & \leq & 25 \\
 & x_1 & + & 3x_2 & + & x_3 & \leq & 15 \\
 & x_1 & , & x_2 & , & x_3 & \geq & 0
 \end{array}$$

Determine a solução óptima usando o método Simplex revisto.

10. Diga, justificando, se as seguintes afirmações são verdadeiras ou falsas:

- a) uma variável que sai da base numa determinada iteração não pode voltar a entrar para a base na iteração imediatamente seguinte;
- b) uma variável que entra para a base numa determinada iteração pode sair da base na iteração imediatamente seguinte.

11. Considere o modelo de Programação Linear seguinte:

$$\begin{array}{rcllcl}
 \min z = & 4x_1 & + & 5x_2 & + & 7x_3 \\
 \text{sujeito a} & & & & & \\
 & 5x_1 & + & 6x_2 & + & 3x_3 & \geq & 25 \\
 & 3x_1 & + & 4x_2 & + & 6x_3 & \geq & 32 \\
 & 5x_1 & + & 4x_2 & + & 8x_3 & \geq & 44 \\
 & x_1 & , & x_2 & , & x_3 & \geq & 0
 \end{array}$$

a) Determine a solução óptima deste modelo usando:

- i. o método das 2-fases
- ii. o método do grande M
- iii. o método Simplex dual

b) Formule o modelo dual associado.

c) Determine a solução óptima do modelo dual usando o método Simplex.

12. Considere o modelo de Programação Linear seguinte:

$$\begin{array}{rcllcl}
\min z = & 8x_1 & + & 16x_2 & + & 20x_3 \\
\text{sujeito a} & & & & & \\
& 3x_1 & + & 5x_2 & + & 3x_3 & \geq & 37 \\
& & & 4x_2 & + & 4x_3 & \geq & 23 \\
& 2x_1 & + & 2x_2 & + & 3x_3 & = & 17 \\
& x_1 & , & x_2 & , & x_3 & \geq & 0
\end{array}$$

- a) Resolva o modelo dual associado usando o método Simplex.
- b) Determine a solução ótima do modelo primal a partir do quadro ótimo que obteve em b).
- 13.** Considere o modelo de Programação Linear seguinte:

$$\begin{array}{rcllcl}
\max z = & 25x_1 & + & 5x_2 \\
\text{sujeito a} & & & & & \\
& 2x_1 & + & 3x_2 & \leq & 10 \\
& 3x_1 & + & x_2 & \leq & 15 \\
& x_1 & , & x_2 & \geq & 0
\end{array}$$

- a) Determine a solução ótima usando o método Simplex.
- b) Em que intervalo pode variar o valor do termo independente da primeira restrição sem que a base associada à solução ótima seja alterada?
- c) Qual deve ser o valor mínimo do coeficiente da variável x_2 na função objetivo para que essa variável entre para a base?
- d) Suponha que a 2ª restrição do modelo passava a ser a seguinte:

$$10x_1 + x_2 \leq 15.$$

Analise o impacto dessa alteração na solução do modelo original.

- 14.** Considere o caso de uma empresa que se dedica ao fabrico de 3 artigos diferentes designados por A, B e C. Para produzir esses artigos, a empresa dispõe de 3 máquinas que pode usar até 800 horas por mês. Só existe uma forma de processar o artigo B nas máquinas, enquanto os artigos A e C podem ser processados de duas formas distintas. A tabela seguinte indica o tempo (em horas) que é necessário em cada máquina para produzir uma unidade de cada um dos artigos:

	Artigo A		Artigo B	Artigo C	
Tipo de processamento	Tipo 1	Tipo 2	Tipo 1	Tipo 1	Tipo 2
Máquina 1	2	1	1	6	8
Máquina 2	3	3	2	5	3
Máquina 3		3	4	6	1

O preço unitário de venda dos artigos A, B e C é respectivamente de 15, 10 e 35 euros. A empresa pretende determinar o plano de produção mensal que maximize o lucro total.

- a) Formule este problema usando um modelo de Programação Linear.
 - b) Determine a solução óptima do problema usando o método Simplex.
 - c) Qual é o preço que a empresa estaria disposta a pagar por uma hora adicional na máquina 1? E na máquina 3? Justifique.
 - d) Se a máquina 1 for sujeita a operações de manutenção que a obriguem a estar parada durante 50 horas no próximo mês, qual será a perda líquida da empresa com esta paragem?
 - e) Suponha que os tempos de processamento (tipo 1) do artigo A são revistos em baixa passando para 1 e 2 horas nas máquinas A e B, respectivamente. Analise as implicações desta alteração.
 - f) Quais são as implicações no plano óptimo de produção de uma redução de 3 euros no preço de venda do artigo A?
- 15.** Considere novamente o problema de investimento descrito no exercício 7.
- a) Determine os intervalos dentro dos quais poderá variar a taxa de rentabilidade de cada um dos planos sem que haja uma alteração na base do quadro óptimo.
 - b) Assuma que o montante mínimo a investir nos planos B e C passa a ser de 75% do capital total. Analise as implicações desta alteração.
 - c) Suponha que existe a possibilidade de investir num 4º plano cuja rentabilidade é de 10%. A abertura desse plano tem um custo fixo associado de 500 mil euros. Determine se o investidor deve investir nesse 4º plano.

Capítulo 3

Programação Inteira

3.1 Introdução

Um dos prolongamentos da Programação Linear que iremos abordar neste capítulo surge quando algumas das incógnitas dos problemas só podem tomar valores inteiros. Essa extensão da Programação Linear é designada por Programação Inteira (ou também por Programação Linear Inteira). As primeiras contribuições nessa área datam dos finais dos anos 50, e são protagonizadas por investigadores como Ralph Gomory.

Os modelos de Programação Inteira são usados para resolver vários problemas reais de optimização. As aplicações surgem em diversas áreas. A Programação Inteira é usada na resolução de problemas de transportes, de planeamento da produção, ou ainda de gestão de recursos humanos. Além de considerarem explicitamente a integralidade das variáveis, esses modelos permitem ainda formular restrições importantes (condições lógicas, por exemplo), que não podem ser modeladas usando apenas variáveis contínuas.

A estrutura geral de um modelo de Programação Inteira é essencialmente a mesma do que a estrutura de um modelo de Programação Linear. Já em termos de resolução, as diferenças são significativas. Em regra geral, um modelo de Programação Inteira é bem mais difícil de resolver do que um modelo de Programação Linear. Nesse contexto, é importante ter em atenção aspectos relacionados com a qualidade dos modelos. Neste capítulo, analisamos essas questões e apresentamos várias estratégias para melhorar a qualidade dos modelos. Descrevemos também métodos de resolução de Programação Inteira que assentam em técnicas de Programação Linear.

3.2 Modelos de Programação Inteira

Quando num modelo de Programação Linear, todas ou algumas das variáveis podem apenas assumir valores inteiros, o modelo passa a ser um modelo de Programação

Inteira. Apesar da aparente proximidade entre os dois tipos de modelos, existem diferenças importantes entre eles tanto no que toca aos detalhes ligados à sua construção, como no que toca à dificuldade da sua resolução. O sucesso da Programação Inteira deve-se ao facto de muitos problemas reais poderem ser formulados através de modelos desse género.

Existem diferentes tipos de modelos de Programação Inteira. Os modelos mais simples são muito próximos dos modelos de Programação Linear. A única diferença reside no facto de algumas quantidades serem inteiras. A título de exemplo, podemos pensar em problemas que envolvam variáveis associadas ao número de trabalhadores ou de produtos. Muitas vezes, uma solução razoável para este tipo de modelos passa por resolver o modelo de Programação Linear associado (sem as restrições que obrigam as variáveis a serem inteiras, as restrições de integralidade), e arredondar o valor dessas variáveis. Modelos menos óbvios são aqueles que envolvem variáveis binárias (que podem tomar o valor 0 ou 1), e com as quais é possível, por exemplo, modelar condições lógicas.

3.2.1 Exemplos

Vamos considerar dois problemas simples que podem ser formulados usando um modelo de Programação Inteira. Não consideramos aqui os casos mais óbvios que em termos de modelação não são mais do que uma simples extensão dos modelos de Programação Linear.

Exemplo 3.1 Considere o problema que consiste em determinar a carga que deve ser transportada por um camião. A capacidade do camião é de 15 toneladas, e existem 5 cargas diferentes que poderão ser transportadas. Existem 5 cargas diferentes que pesam respectivamente 3, 5, 7 e 8 toneladas. As cargas têm valores diferentes: o valor de uma carga corresponde ao benefício que a transportadora obtém se transportar a carga. Vamos assumir que as cargas valem respectivamente 1500, 2000, 2700 e 4200 euros.

O problema consiste em determinar a combinação de cargas cujo valor total é máximo, e cujo peso total é inferior à capacidade do camião. Para cada uma das cargas, deve-se decidir se a carga deve ou não ser transportada. As variáveis de decisão do modelo x_i , $i = 1, \dots, 4$, são variáveis binárias que tomam o valor 1 se a carga i é transportada, e 0 caso contrário:

$$x_i = \begin{cases} 1, & \text{a carga } i \text{ é transportada pelo camião,} \\ 0, & \text{caso contrário.} \end{cases}$$

O problema pode ser formulado através do modelo de Programação Inteira seguinte:

$$\begin{array}{llllllll} \max z = & 1500x_1 & + & 2000x_2 & + & 2700x_3 & + & 4200x_4 \\ \text{s. a} & & & & & & & \\ & 3x_1 & + & 5x_2 & + & 7x_3 & + & 8x_4 & \leq & 15 \\ & x_1 & , & x_2 & , & x_3 & , & x_4 & \in & \{0, 1\} \end{array}$$

O modelo tem apenas uma restrição relativa à capacidade do camião. Este é um problema clássico de Programação Inteira, conhecido por *problema de mochila*.

O modelo pode ser resolvido de diferentes formas. Uma delas consiste em enumerar todas as soluções possíveis. Com 4 variáveis binárias, o espaço de soluções inclui 16 soluções diferentes. Algumas dessas soluções não são válidas dado que violam a restrição de capacidade do camião (por exemplo, a solução $(x_1, x_2, x_3, x_4) = (1, 1, 1, 1)$ tem um peso associado igual a 23). A solução óptima é determinada por inspecção das soluções válidas. Se esta abordagem é viável para problemas de pequena dimensão, é claramente inadequada para problemas com um grande número de variáveis para os quais o espaço de soluções inclui um número considerável de soluções. Esses casos podem ser resolvidos recorrendo aos métodos de resolução de modelos de Programação Inteira que analisaremos mais adiante. $\square$

Outro problema clássico de Programação Inteira é o caso em que se consideram custos fixos. Esses custos são contabilizados no máximo uma vez, ao contrário dos custos variáveis que são directamente proporcionais a variáveis gerais. Se tivermos uma variável x à qual está associada um custo variável c_1 que depende directamente do valor de x , e um custo fixo em que se incorre apenas se x for superior a 0, o custo total associado à variável x será o seguinte:

$$\begin{array}{ll} 0 & \text{se } x = 0, \\ c_1x + c_2 & \text{se } x > 0. \end{array}$$

O custo total não é linear em ordem à variável x conforme ilustrado na Figura 3.1. No Exemplo seguinte, apresentamos a estrutura geral dos modelos de Programação Inteira que são usados para formular problemas com custos fixos.

Exemplo 3.2 Uma companhia produz 3 tipos de artigos diferentes (artigo 1, 2 e 3), e usa para esse efeito máquinas dedicadas. O custo semanal de aluguer das máquinas é de 150, 175 e 100 euros para os artigos 1, 2 e 3, respectivamente. Para produzir os artigos, é necessária mão-de-obra e matéria-prima:

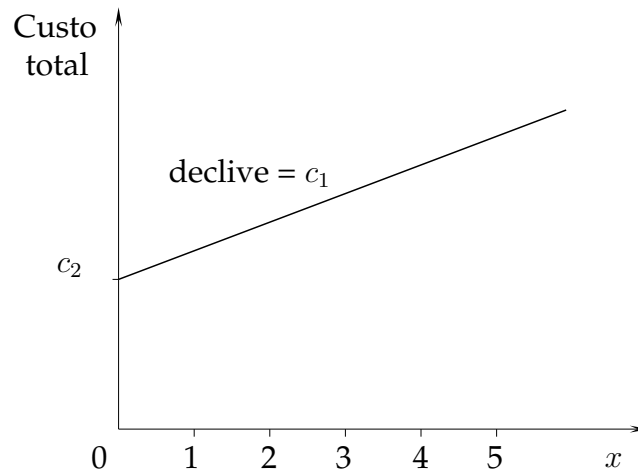


Figura 3.1: Relação entre o custo total e o valor de uma variável com custo fixo

	Mão-de-obra (em h/artigo)	Matéria-prima (em unidades/artigo)
Artigo 1	4	2
Artigo 2	6	4
Artigo 3	1	2

Semanalmente, estão disponíveis 230h de mão-de-obra e 200 unidades de matéria-prima. Os artigos são vendidos ao preço unitário de 20, 25 e 10 euros, respectivamente. A companhia pretende determinar que artigos deve produzir, e em que quantidades, de modo a maximizar o seu lucro total.

O número de artigos de cada tipo que devem ser produzidos é uma das incógnitas do problema. Uma vez que se trata de quantidades inteiras, usamos variáveis do tipo inteiro para as representar:

x_i : número de artigos i produzidos, com $x_i \geq 0$ e inteiro, $i = 1, \dots, 3$.

O custo fixo de aluguer de uma máquina é pago apenas se o tipo de artigo associado a essa máquina for produzido. Vamos usar variáveis binárias para representar a situação de produção ou não produção de artigos de um determinado tipo:

$$y_i = \begin{cases} 1, & \text{se o artigo } i \text{ for produzido } (x_i > 0), \\ 0, & \text{caso contrário.} \end{cases} \quad i = 1, \dots, 3.$$

Com essas variáveis, o custo fixo é facilmente formulado na função objectivo:

$$\max z = 20x_1 + 25x_2 + 10x_3 - 150y_1 - 175y_2 - 100y_3.$$

Por definição, as variáveis binárias estão directamente ligadas às variáveis que representam o nível de produção de cada artigo. Se $x_i > 0$, devemos ter $y_i = 1$, $i = 1, \dots, 3$. Essa relação é expressa através do conjunto de restrições seguintes:

$$x_i \leq My_i, \quad i = 1, \dots, 3,$$

em que M representa um valor numérico elevado. Uma solução com $x_i = 0$ e $y_i = 1$ é válida se tivermos em conta esse conjunto de restrições. Contudo, nunca será óptima dado que as variáveis y_i têm coeficientes negativos na função objectivo (a mesma solução com $y_i = 0$ é válida, e tem um valor da função objectivo associado maior).

Em vez do valor M , podemos usar valores inferiores nessas restrições. No caso da variável x_1 , por exemplo, podemos usar o valor $\frac{115}{2}$. Devido à restrição sobre a mão-de-obra, essa variável nunca será superior a esse valor. Veremos mais adiante que a escolha desse valor pode resultar em modelos de melhor qualidade.

As restrições relativas à mão-de-obra e à matéria-prima são formuladas do mesmo modo que num modelo tradicional de Programação Linear:

$$\begin{aligned} 4x_1 + 6x_2 + 1x_3 &\leq 230 && \text{(mão-de-obra),} \\ 2x_1 + 4x_2 + 2x_3 &\leq 200 && \text{(matéria-prima).} \end{aligned}$$

□

3.2.2 Tipos de Modelos

Os modelos de Programação Inteira podem ser classificados em dois grandes grupos:

- os modelos de programação inteira pura, nos quais todas as variáveis devem assumir valores inteiros;
- os modelos de programação inteira mista que usam variáveis contínuas (que podem tomar valores fraccionários) e variáveis inteiras.

Os modelos de Programação Inteira são usados frequentemente na prática devido à sua capacidade em representar de forma correcta a realidade dos problemas. As variáveis de tipo inteiro, e em particular as variáveis binárias, permitem modelar diversas situações que não é possível formular usando variáveis contínuas. Como exemplos, podemos citar:

- a contagem de elementos: é o caso mais simples que pode ser ilustrado com exemplos como a contagem de pessoas, de produtos, ou de veículos;

- a modelação de decisões lógicas: usando variáveis binárias, é possível modelar decisões do tipo *verdadeiro* ou *falso*;
- a formulação de condições lógicas: como a implicação, condições do tipo *se e só se* ou outros operadores mais simples como a disjunção;
- a disjunção de restrições (ou dicotomias): enquanto que um modelo de Programação Linear obriga a que todas as restrições sejam satisfeitas em simultâneo, usando variáveis binárias, é possível “activar” subconjuntos de restrições do modelo, e “desactivar” outras, o que acaba por representar uma disjunção entre conjuntos de restrições;
- a formulação de funções não-lineares simples: como o valor absoluto ou o produto de variáveis binárias.

Alguns modelos especiais de Programação Inteira têm merecido uma particular atenção devido ao facto de representarem muitas situações reais, ou porque são frequentemente embebidos noutros modelos. Um deles é o problema da mochila que abordámos no Exemplo 3.1. O modelo correspondente pode ser usado para representar, por exemplo, problemas de investimento ou de gestão de portfolios, e problemas de corte e empacotamento. Outros problemas tradicionais são os problemas de cobertura de conjuntos, os problemas de partição de conjuntos e os problemas de empacotamento de conjuntos que são formulados através de modelos que são conceptualmente simples, e que passamos a descrever.

Vamos considerar um conjunto E constituído por m elementos $e_1, e_2, e_3, \dots, e_m$, e um conjunto S composto por subconjuntos de E . Para $m = 15$, o conjunto S poderia ser definido, por exemplo, da forma seguinte:

$$S = \{\{e_1, e_2, e_6, e_7, e_{11}, e_{12}\}, \{e_3, e_8, e_{13}\}, \{e_4, e_9, e_{14}\}, \{e_5, e_{10}, e_{15}\}\}.$$

Vamos designar por P_i o $i^{\text{ésimo}}$ elemento do conjunto S , com $i = 1, \dots, n$. Para o exemplo acima, temos que $P_1 = \{e_1, e_2, e_6, e_7, e_{11}, e_{12}\}$. A Figura 3.2 ilustra graficamente um exemplo do que poderiam ser o conjunto E e S para $m = 15$, e subconjuntos P_i com $i = 1, \dots, 10$.

Os três problemas baseiam-se nessa definição de conjuntos e subconjuntos. Em cada um deles, pretende-se determinar quais os subconjuntos P_i que devem ser escolhidos de forma a satisfazer restrições que estão directamente relacionadas com os elementos e_j de E , $j = 1, \dots, m$. Vamos começar pelo problema de partição de conjuntos. Nesse problema, procura-se escolher os conjuntos P_i de modo a que todos os elementos e_j de E sejam incluídos uma e uma única vez. O resultado é uma partição do conjunto E

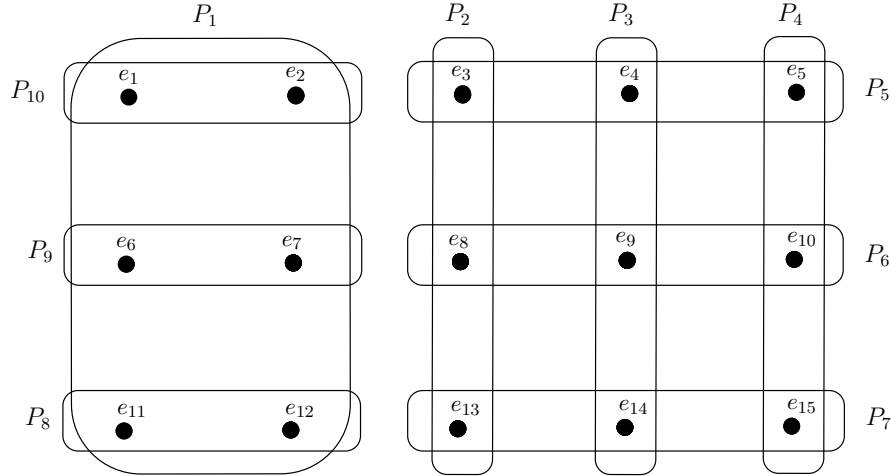


Figura 3.2: Problemas de partição, empacotamento e cobertura de conjuntos

em conjuntos P_i que são disjuntos. Para o conjunto E ilustrado na Figura 3.2, uma partição possível de E poderia ser definida pelos subconjuntos P_1, P_2, P_3 e P_4 .

O problema de partição de conjuntos pode ser formulado usando um modelo de Programação Inteira em que as variáveis de decisão determinam se um conjunto P_i faz ou não parte da partição. Vamos designar por x_i a variável de decisão associada ao conjunto P_i com $i = 1, \dots, m$:

$$x_i = \begin{cases} 1, & \text{se o conjunto } P_i \text{ pertence à partição,} \\ 0, & \text{caso contrário.} \end{cases}$$

O modelo tem assim uma variável (ou coluna) para cada subconjunto P_i . As restrições dizem respeito aos elementos do conjunto E : qualquer que sejam os subconjuntos P_i que sejam escolhidos, cada elemento e_j de E só poderá ser incluído uma vez. As restrições do modelo são por isso restrições de *igualdade*. Para o exemplo representado na Figura 3.2, o correspondente modelo de Programação Inteira é definido da forma seguinte:

$$\min / \max \quad z = x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 + x_{10}$$

s. a

$$\begin{array}{rcl} x_1 & & + x_{10} = 1 \quad (e_1) \\ x_1 & & + x_{10} = 1 \quad (e_2) \\ & x_2 & + x_5 = 1 \quad (e_3) \\ & & x_3 + x_5 = 1 \quad (e_4) \end{array}$$

$$\begin{array}{rclcl}
& x_4 + x_5 & = 1 & (e_5) \\
x_1 & & + x_9 & = 1 & (e_6) \\
x_1 & & + x_9 & = 1 & (e_7) \\
& x_2 & + x_6 & = 1 & (e_8) \\
& x_3 & + x_6 & = 1 & (e_9) \\
& x_4 & + x_6 & = 1 & (e_{10}) \\
x_1 & & + x_8 & = 1 & (e_{11}) \\
x_1 & & + x_8 & = 1 & (e_{12}) \\
& x_2 & + x_7 & = 1 & (e_{13}) \\
& x_3 & + x_7 & = 1 & (e_{14}) \\
& + x_4 & + x_7 & = 1 & (e_{15}) \\
x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10} & \in \{0, 1\}
\end{array}$$

Funções objectivo de *minimização* ou *maximização* fazem ambas sentido neste tipo de modelos.

Para ilustrarmos a aplicabilidade destes modelos, vamos considerar o caso de uma empresa de telecomunicações que pretende determinar a melhor localização para as suas antenas de transmissão. A empresa quer garantir a cobertura de uma dada área geográfica usando o menor número possível de antenas. Os pontos onde existe procura nessa área (as cidades, por exemplo) podem ser representados de forma abstracta através dos elementos e_j do conjunto E . Vamos considerar ainda que a empresa identificou potenciais locais para instalar as suas antenas, sendo que cada um deles apenas garante a cobertura de um número limitado de cidades. A área coberta por cada um desses locais pode ser representada através dos subconjuntos P_i . Se nos referirmos aos dados da Figura 3.2, o local associado ao subconjunto P_1 permitiria cobrir as cidades $e_1, e_2, e_6, e_7, e_{11}$ e e_{12} . Se, por razões técnicas, cada cidade só deve ser coberta por uma antena, teremos um problema de partição de conjuntos com uma função objectivo de *minimização*.

Outro exemplo de sucesso de aplicação destes modelos é caso do escalonamento de tripulações de bordo no transporte aéreo de passageiros. Nesses casos, os elementos do conjunto E representam vôos entre dois destinos, e os subconjuntos P_i representam rotas formadas por sequências de vôos que garantem que as tripulações regressam sempre à sua cidade de partida. O objectivo consiste em escolher as rotas que irão ser efectivamente usadas de modo a que todos os vôos se realizem, com recurso ao menor número possível de tripulações. Muitas vezes, esse problema é resolvido como um problema de cobertura de conjuntos (que analisaremos mais adiante) pelo simples facto do problema de partição de conjuntos ser mais restrito (restrições de igualdade),

e poder não ter sequer uma solução válida.

Nos problemas de empacotamento de conjuntos, procura-se escolher os subconjuntos P_i de forma a maximizar o valor total desses subconjuntos (ou o número de subconjuntos escolhidos, se os pesos forem iguais a 1). Os elementos e_j de E não podem ser incluídos mais do que uma vez. Os subconjuntos escolhidos devem por isso ser disjuntos, à semelhança do que acontece nos problemas de partição de conjuntos. O modelo de Programação Inteira para o problema de empacotamento de conjuntos retratado na Figura 3.2 é o seguinte:

$$\begin{aligned}
 \max \quad & z = x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 + x_{10} \\
 \text{s. a} \quad & \\
 & x_1 \qquad \qquad \qquad + x_{10} \leq 1 \qquad (e_1) \\
 & x_1 \qquad \qquad \qquad + x_{10} \leq 1 \qquad (e_2) \\
 & \qquad \dots \qquad \qquad \dots \qquad \qquad \dots \leq 1 \qquad \dots \\
 & \qquad \qquad x_3 \qquad \qquad \qquad + x_7 \qquad \qquad \leq 1 \qquad (e_{14}) \\
 & \qquad \qquad + x_4 \qquad \qquad + x_7 \qquad \qquad \leq 1 \qquad (e_{15}) \\
 & x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10} \in \{0, 1\}
 \end{aligned}$$

O modelo para o problema empacotamento de conjuntos pode ser facilmente convertido num modelo de partição de conjuntos, adicionando variáveis de folga em cada uma das restrições. Essas variáveis são necessariamente iguais a 0 ou 1. Também é possível converter um problema de partição de conjuntos num problema de empacotamento de conjuntos, adicionando variáveis artificiais nas restrições do modelo, e forçando-as a tomarem o valor 0 penalizando-as na função objectivo.

O problema de cobertura de conjuntos é o oposto do problema de empacotamento de conjuntos. A escolha dos subconjuntos P_i é feita de modo a garantir que todos os elementos e_j de E são incluídos *pelo menos* uma vez. Os subconjuntos escolhidos não têm de ser disjuntos, mas a sua intersecção deve ser igual ao conjunto E . Para esses casos, a função objectivo é necessariamente do tipo *minimização*. O problema de cobertura de conjuntos associado ao exemplo retratado na Figura 3.2 pode ser formulado através do seguinte modelo de Programação Inteira:

$$\begin{aligned}
 \min \quad & z = x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 + x_9 + x_{10} \\
 \text{s. a} \quad & \\
 & x_1 \qquad \qquad \qquad + x_{10} \geq 1 \quad (e_1)
 \end{aligned}$$

$$\begin{array}{rcll}
x_1 & & + x_{10} & \geq 1 & (e_2) \\
\ldots & \ldots & \ldots & \geq 1 & \ldots \\
& x_3 & + x_7 & \geq 1 & (e_{14}) \\
& + x_4 & + x_7 & \geq 1 & (e_{15})
\end{array}$$

$$x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10} \in \{0, 1\}$$

3.2.3 Modelação com Variáveis Binárias

Em termos de modelação, não existem grandes diferenças entre os modelos de Programação Linear e os modelos de Programação Inteira com variáveis inteiras gerais, *i.e.* variáveis que podem tomar qualquer valor inteiro. Em regra geral, essas variáveis representam quantidades que são simplesmente indivisíveis (pessoas, artigos), e não permitem modelar situações que sejam significativamente diferentes daquelas que poderiam ser modeladas através de um modelo de Programação Linear. Pelo contrário, as variáveis binárias (0-1) aumentam significativamente a expressividade dos modelos, permitindo modelar condições que estão fora do alcance dos modelos de Programação Linear. O revês da medalha é uma maior complexidade dos modelos que acaba por tornar o exercício de modelação mais difícil. Nesta secção, descrevemos algumas regras gerais que permitem traduzir algumas condições na forma de relações entre variáveis binárias.

3.2.3.1 Condições Lógicas

Usando variáveis binárias, é possível representar expressões lógicas (relações que dependem de variáveis que podem tomar os valores lógicos *verdadeiro* ou *falso*) através de expressões algébricas lineares. A correspondência entre essas variáveis e as variáveis binárias é imediata: o valor *falso* corresponde ao valor 0 de uma variável binária, enquanto que o valor *verdadeiro* é representado pelo valor 1.

Para ilustrarmos a equivalência entre expressões lógicas e expressões lineares baseadas em variáveis binárias, vamos começar por usar um exemplo simples de produção. Essa correspondência será generalizada logo de seguida.

Exemplo 3.3 Vamos considerar um modelo de Programação Inteira que representa o problema de uma companhia que produz 6 tipos diferentes de artigos. Vamos assumir que o modelo tem variáveis inteiras x_i , $i = 1, \dots, 6$ que representam o nível de produção do artigo i . Adicionalmente, são impostas as seguintes restrições:

- a) a companhia só pode produzir 3 tipos de artigos diferentes;

- b) se a companhia produzir o artigo 1, não poderá produzir o artigo 6;
- c) se a companhia produzir o artigo 4, terá de produzir o artigo 1 ou o artigo 3;
- d) a companhia só poderá produzir o artigo 6 se produzir também o artigo 2;
- e) a companhia não pode produzir simultaneamente o artigo 5 e o artigo 6.

Essas restrições definem relações lógicas entre proposições do tipo “a companhia produz o artigo i ” (à qual iremos associar a variável lógica R_i). A restrição *b*), por exemplo, corresponde à seguinte expressão lógica:

$$R_1 \Rightarrow \neg R_6,$$

i.e. o facto da companhia produzir o artigo 1 implica que ela não produza o artigo 6 (o operador “ $\neg$ ” representa a negação). Claramente, qualquer uma dessas restrições não é susceptível de ser formulada usando apenas variáveis contínuas ou variáveis inteiras gerais.

Para poder formular essas restrições, é necessário ter alguma indicação sobre se um dado artigo está ou não a ser produzido. Esse caso foi introduzido no Exemplo 3.2, e consiste em associar às variáveis x_i uma variável binária y_i definida da seguinte forma:

$$y_i = \begin{cases} 1, & \text{se o artigo } i \text{ é produzido,} \\ 0, & \text{caso contrário.} \end{cases} \quad \left| \quad i = 1, \dots, 6. \right.$$

À semelhança do que acontecia com o problema dos custos fixos, as restrições que definem a relação entre as variáveis x_i e y_i são definidas como segue:

$$x_i \leq M y_i, \quad i = 1, \dots, 6.$$

As variáveis y_i permitem fazer a contagem do número de artigos diferentes que são efectivamente produzidos. É com base nessa contagem que formulamos a restrição *a*):

$$\sum_{i=1}^6 y_i \leq 3.$$

O termo do lado esquerdo corresponde ao número total de artigos diferentes que são produzidos.

A restrição *b*) traduz-se algebricamente numa relação entre as variáveis y_1 e y_6 . A ideia está em forçar a variável y_6 a tomar o valor 0 sempre que a variável y_1 é igual a 1:

$$y_1 \leq 1 - y_6.$$

A validade da restrição pode ser confirmada inspeccionando todas as combinações de valores para y_1 e y_6 , e correspondentes valores lógicos de R_1 e R_6 (proposições introduzidas atrás), e comparando a satisfação ou não da restrição algébrica com o resultado da respectiva condição lógica. A tabela de verdade associada a essa expressão lógica é a seguinte:

R_1	R_6	$R_1 \Rightarrow \neg R_6$
F	F	V
F	V	V
V	F	V
V	V	F

O valor *falso* e o valor *verdadeiro* são designados respectivamente por F e V . Se enumerarmos todos os valores possíveis para as variáveis y_1 e y_6 , verificamos que o resultado final é consistente com os valores obtidos através da expressão lógica:

y_1	y_2	$y_1 \leq 1 - y_6$
0	0	✓
0	1	✓
1	0	✓
1	1	×

A restrição $c)$ só permite que o artigo 4 seja produzido se o artigo 1 ou o artigo 3 também o forem. Uma forma de traduzir essa restrição algebricamente consiste em limitar superiormente o valor de y_4 , fazendo-o depender do valor das variáveis y_1 e y_3 :

$$y_4 \leq y_1 + y_3.$$

Traduzida algebricamente, a restrição $d)$ impede a variável y_6 de ser igual a 1 se a variável y_2 não for ela também igual a 1. Se y_2 for igual a 0, a variável y_6 deverá ser forçada a tomar o valor 0. Essa condição lógica é formulada através da seguinte expressão algébrica:

$$y_6 \leq y_2.$$

Finalmente, a restrição $e)$ traduz-se no facto das duas variáveis y_5 e y_6 não poderem ambas tomar o valor 1. Nesse caso, a soma das variáveis não poderá exceder o valor 1:

$$y_5 + y_6 \leq 1.$$

□

O Exemplo 3.3 mostra-nos como é que alguma experiência acerca da construção de modelos de Programação Inteira nos pode ajudar a expressar na forma de restrições lineares um conjunto de condições lógicas. Algumas correspondências podem ser contudo generalizadas (por razões de clareza, usamos a mesma notação do Exemplo 3.3):

1. $\neg R_1$ corresponde a $1 - y_1$;
2. $R_1 \wedge R_2$ corresponde a $y_1 + y_2 = 2$;
3. $R_1 \wedge R_2 \wedge R_3 \wedge \dots \wedge R_n$ corresponde a $y_1 + y_2 + y_3 + \dots + y_n = n$;
4. $R_1 \vee R_2$ corresponde a $y_1 + y_2 \geq 1$;
5. $R_1 \vee R_2 \vee R_3 \vee \dots \vee R_n$ corresponde a $y_1 + y_2 + y_3 + \dots + y_n \geq 1$;
6. $R_1 \vee (R_2 \wedge R_3)$ corresponde a $y_1 + y_2 \geq 1$ e $y_1 + y_3 \geq 1$;
7. $R_1 \wedge (R_2 \vee R_3)$ corresponde a $y_1 = 1$ e $y_2 + y_3 \geq 1$;
8. $R_1 \Rightarrow R_2$ corresponde a $y_1 \leq y_2$;
9. $R_1 \Rightarrow (R_2 \wedge R_3)$ corresponde a $y_1 \leq y_2$ e $y_1 \leq y_3$;
10. $R_1 \Rightarrow (R_2 \vee R_3)$ corresponde a $y_1 \leq y_2 + y_3$;
11. $(R_1 \wedge R_2) \Rightarrow R_3$ corresponde a $y_1 + y_2 \leq 1 + y_3$;
12. $(R_1 \vee R_2) \Rightarrow R_3$ corresponde a $y_1 \leq y_3$ e $y_2 \leq y_3$;
13. $R_1 \Leftrightarrow R_2$ corresponde a $y_1 = y_2$.

Algumas das correspondências descritas na tabela podem ser deduzidas a partir de outras, e por aplicação de algumas regras de equivalência entre expressões lógicas. Por exemplo, sabendo que

$$A \Rightarrow B,$$

é equivalente a

$$\neg A \vee B,$$

podemos deduzir a correspondência nº11 com base nas correspondências 1 e 5 da forma seguinte:

$$(R_1 \wedge R_2) \Rightarrow R_3$$

é equivalente a

$$\neg(R_1 \wedge R_2) \vee R_3,$$

i.e.

$$\neg R_1 \vee \neg R_2 \vee R_3.$$

De acordo com as correspondências 1 e 5 da lista acima, esta última expressão pode ser representada algebricamente da forma seguinte:

$$(1 - y_1) + (1 - y_2) + y_3 \geq 1,$$

o que nos conduz a

$$1 + y_3 \geq y_1 + y_2.$$

3.2.3.2 Restrições Impostas Condicionalmente

Da mesma forma que podem ser formuladas condições lógicas sobre decisões elementares (“se produzirmos o artigo 1, não poderemos produzir o artigo 3”, “os artigos 2 e 4 não poderão ser produzidos simultaneamente”), é possível também fazer depender a imposição de uma restrição de uma condição. Essas restrições podem ser do tipo “se o artigo 4 for produzido, não poderemos produzir mais de 3 artigos diferentes” ou “se o artigo 2 ou o artigo 3 forem produzidos, o artigo 4 e 5 deverão obrigatoriamente ser produzidos”. Em qualquer caso, a restrição só é imposta se se verificar uma determinada condição.

Vamos considerar os dois tipos de restrições que é possível fazer depender de uma condição, *i.e.* as restrições do tipo *maior ou igual* e as restrições do tipo *menor ou igual*:

$$\sum_{j=1}^n a_j x_j \geq b,$$

$$\sum_{j=1}^n a_j x_j \leq b.$$

A condição da qual depende a imposição de uma dessas restrições vai ser representada através da variável binária y . Essa condição pode corresponder a uma decisão elementar (“produzir ou não um dado artigo”, por exemplo), ou pode estar associada a um conjunto mais complexo de decisões (“são produzidos pelo menos 3 artigos diferentes”, por exemplo).

O procedimento que é usado para formular a dependência entre a verificação de uma condição e a imposição de uma restrição baseia-se no princípio simples que consiste em “eliminar” uma restrição de um modelo tornando-a redundante. Se a condição não se verificar ($y = 0$), a restrição que depende dessa condição deverá ser transformada numa restrição redundante. Se, pelo contrário, a condição se verificar ($y = 1$) a restrição imposta no modelo deverá corresponder exactamente à restrição original.

Uma restrição do tipo *maior ou igual* é redundante se o termo independente do lado direito é menor do que o valor do termo do lado esquerdo para qualquer combinação de valores das variáveis de decisão. Recorrendo novamente ao coeficiente M que representa um valor numérico muito elevado, podemos formular a imposição de uma restrição do tipo *maior ou igual* condicionada ao valor de y da seguinte forma:

$$\sum_{j=1}^n a_j x_j \geq b - M(1 - y).$$

Quando a condição representada pela variável binária y não se verificar, teremos que $y = 0$, e

$$\sum_{j=1}^n a_j x_j \geq b - M(1 - 0) = b - M \approx -M.$$

A restrição é redundante, e não tem por isso qualquer impacto no valor da solução óptima do modelo (eliminámos implicitamente a restrição do modelo). Se a condição se verificar, teremos que $y = 1$, e

$$\sum_{j=1}^n a_j x_j \geq b - M(1 - 1) = b.$$

Nesse caso, a restrição corresponde à restrição original.

A abordagem no caso das restrições do tipo *menor ou igual* é essencialmente a mesma. A diferença reside no facto de uma restrição desse tipo tornar-se redundante se o termo independente do lado direito da restrição for maior que o termo do lado esquerdo para qualquer combinação de valores das variáveis de decisão. A imposição de uma restrição do tipo *menor ou igual* condicionada ao valor de y é formulada da seguinte forma:

$$\sum_{j=1}^n a_j x_j \leq b + M(1 - y).$$

Essa abordagem permite-nos também formular disjunções de restrições em modelos de Programação Inteira. Nos modelos de Programação Linear e Inteira, o espaço de soluções válidas é definido com base na intersecção dos espaços definidos por cada uma

das restrições. Todas as restrições do modelo devem ser satisfeitas simultaneamente. Usando variáveis binárias, podemos deixar em aberto a escolha de um conjunto de restrições em detrimento de outro. A condição para a imposição de uma restrição passa a ser do tipo “se a restrição A não for satisfeita, a restrição B terá de o ser (e vice-versa)” o que equivale à seguinte formulação “uma das restrições A ou B deve ser satisfeita”. Vamos considerar o caso simples em que se pretende impôr pelo menos uma das restrições seguintes:

$$D_1 : \sum_{j=1}^n a_{1j}x_j \leq b_1,$$

ou

$$D_2 : \sum_{j=1}^n a_{2j}x_j \leq b_2.$$

Podemos condicionar a imposição de cada uma dessas restrições ao valor tomado respectivamente por duas variáveis binárias que vamos designar por y_1 e y_2 :

$$y_i = \begin{cases} 1, & \text{se a restrição } D_i \text{ for satisfeita,} \\ 0, & \text{caso contrário.} \end{cases} \quad \left| \quad i \in \{1, 2\}.$$

Tal como fizemos acima, as restrições de partida são modificadas para terem em conta o seu carácter condicional:

$$\begin{aligned} D_1 : \quad & \sum_{j=1}^n a_{1j}x_j \leq b_1 + M(1 - y_1), \\ D_2 : \quad & \sum_{j=1}^n a_{2j}x_j \leq b_2 + M(1 - y_2). \end{aligned}$$

Finalmente, a disjunção é formulada forçando uma e apenas uma das variáveis binárias a ser igual a 1:

$$y_1 + y_2 = 1.$$

A extensão aos casos em que se considera a disjunção entre vários conjuntos de restrições de dimensão maior que 1 é imediata. Sem perda de generalidade, vamos assumir o caso em que todas as restrições são do tipo *menor ou igual*. Os conjuntos de restrições podem ser expressos da forma seguinte:

$$\begin{aligned}
D_1 &= \left\{ \begin{array}{l} \sum_{j=1}^n a_{1j}^1 x_j \leq b_1^1 \\ \sum_{j=1}^n a_{2j}^1 x_j \leq b_2^1 \\ \dots \\ \sum_{j=1}^n a_{m_1 j}^1 x_j \leq b_{m_1}^1 \end{array} \right\} = \left\{ x : \sum_{j=1}^n a_{ij}^1 x_j \leq b_i^1, \ i = 1, \dots, m_1 \right\}, \\
D_2 &= \left\{ \begin{array}{l} \sum_{j=1}^n a_{1j}^2 x_j \leq b_1^2 \\ \sum_{j=1}^n a_{2j}^2 x_j \leq b_2^2 \\ \dots \\ \sum_{j=1}^n a_{m_2 j}^2 x_j \leq b_{m_2}^2 \end{array} \right\} = \left\{ x : \sum_{j=1}^n a_{ij}^2 x_j \leq b_i^2, \ i = 1, \dots, m_2 \right\}, \\
&\dots \\
D_p &= \left\{ \begin{array}{l} \sum_{j=1}^n a_{1j}^p x_j \leq b_1^p \\ \sum_{j=1}^n a_{2j}^p x_j \leq b_2^p \\ \dots \\ \sum_{j=1}^n a_{m_p j}^p x_j \leq b_{m_p}^p \end{array} \right\} = \left\{ x : \sum_{j=1}^n a_{ij}^p x_j \leq b_i^p, \ i = 1, \dots, m_p \right\}.
\end{aligned}$$

A cada um desses conjuntos associamos uma variável binária y_k , $k = 1, \dots, p$, que é igual a 1 apenas no caso em que todas as restrições de D_k são satisfeitas. Tal como fizemos atrás, fazemos depender a imposição do conjunto de restrições D_k do valor da variável y_k da seguinte forma:

$$D_k = \left\{ x : \sum_{j=1}^n a_{ij}^k x_j \leq b_i^k + M(1 - y_k), \ i = 1, \dots, m_k \right\}.$$

A disjunção é garantida através da restrição seguinte:

$$\sum_{k=1}^p y_k = 1.$$

Algumas variações dessa disjunção de restrições podem facilmente ser formuladas a partir das expressões acima. Assim, se quisermos impôr que pelo menos p' conjunto de restrições sejam satisfeitas (com $p' \leq p$), bastará substituir a última expressão pela seguinte:

$$\sum_{k=1}^p y_k \geq p'.$$

3.2.3.3 Funções Não-Lineares: um Exemplo

Como foi referido atrás, é possível formular algumas funções não-lineares através de expressões lineares usando variáveis binárias. Nesta secção, exploramos o caso particular da função de valor absoluto $|z|$, em que z pode ser representada na forma de uma expressão linear. A função de valor absoluto $|z|$ é definida da seguinte forma:

$$|z| = \begin{cases} z, & \text{se } z \geq 0, \\ -z, & \text{se } z \leq 0. \end{cases}$$

A variável z pode tomar um valor positivo, negativo ou nulo. Em qualquer um dos casos, podemos exprimir essa variável como uma diferença entre duas variáveis positivas v^+ e v^- :

$$z = v^+ - v^-.$$

Se garantirmos que apenas uma das variáveis v^+ ou v^- é positiva, a função $|z|$ poderá ser formulada desta forma:

$$|z| = v^+ + v^-.$$

Usando o procedimento que descrevemos para a disjunção de restrições, podemos forçar a que apenas uma das variáveis v^+ ou v^- seja positiva. Para isso, recorreremos a duas variáveis binárias y^+ e y^- que associamos respectivamente a v^+ e v^- :

$$y^+ = \begin{cases} 1, & \text{se a variável } v^+ \text{ for positiva,} \\ 0, & \text{caso contrário,} \end{cases}$$

$$y^- = \begin{cases} 1, & \text{se a variável } v^- \text{ for positiva,} \\ 0, & \text{caso contrário.} \end{cases}$$

A definição de restrições do tipo “se v^+ é positiva, v^- deve ser nula, ou vice-versa” é feita impondo as seguintes restrições:

$$\begin{aligned} v^+ &\leq M(1 - y^-), \\ v^- &\leq M(1 - y^+), \\ y^+ + y^- &= 1. \end{aligned}$$

Substituindo y^- por $1 - y^+$, podemos ainda reduzir o número de variáveis e restrições necessárias:

$$\begin{aligned} v^+ &\leq My^+, \\ v^- &\leq M(1 - y^+). \end{aligned}$$

3.3 Qualidade dos Modelos

Na maior parte dos casos, os modelos de Programação Inteira são bem mais difíceis de resolver do que os modelos de Programação Linear. Muitas vezes, para problemas com alguma dimensão, pode mesmo ser impossível determinar a solução óptima em tempo razoável. Essa realidade obriga-nos a ter um especial cuidado com a forma como formulamos um modelo de Programação Inteira. A noção de qualidade de um modelo é usada aqui para distinguir as boas formulações das formulações que dificilmente poderão ser resolvidas em tempo útil.

3.3.1 Relaxação Linear e Intervalo de Integralidade

A qualidade de um modelo de Programação Inteira é avaliada essencialmente com base no chamado *intervalo de integralidade*. Essa medida condiciona de forma decisiva a eficiência de métodos de resolução como o de partição e avaliação que analisaremos mais adiante. Para a definirmos completamente, precisamos de introduzir previamente alguns conceitos. Para isso, vamos usar como referência o modelo genérico de Programação Inteira representado da forma seguinte:

$$\begin{aligned} \min \quad & z_{PI} = \sum_{j=1}^n c_j x_j, \\ \text{s. a} \quad & \sum_{j=1}^n a_{ij} x_j \geq b_i, \quad i = 1, \dots, m, \\ & x_j \geq 0 \text{ e inteiro, } j = 1, \dots, n, \end{aligned}$$

e cuja solução óptima será designada por z_{PI}^* . O espaço de soluções válidas desse modelo para o caso em que existem apenas duas variáveis inteiras pode ser representado de forma esquemática conforme ilustramos na Figura 3.3. As soluções válidas correspondem aos pares de coordenadas inteiras assinalados pelos pontos a negro. As rectas representam a fronteira definida pelas restrições do modelo. O espaço de soluções

contínuas definido por essas restrições (a cinzento na Figura 3.3) corresponde ao espaço de soluções do modelo de Programação Inteira sem as restrições de integralidade (que obrigam cada uma das variáveis a serem inteiras). Esse modelo é designado por *relaxação linear*:

$$\begin{aligned} \min \quad & z_{RL} = \sum_{j=1}^n c_j x_j, \\ \text{s. a} \quad & \sum_{j=1}^n a_{ij} x_j \geq b_i, \quad i = 1, \dots, m, \\ & x_j \geq 0, \quad j = 1, \dots, n. \end{aligned}$$

Vamos designar a sua solução óptima por z_{RL}^* . Para o caso ilustrado na Figura 3.3, a resolução da relaxação linear não nos permite obter a solução óptima do modelo de Programação Inteira associado. Nenhum dos pontos extremos do espaço de soluções da relaxação linear é uma solução inteira. Arredondar a solução da relaxação linear também não nos garante que possamos obter essa solução óptima inteira.

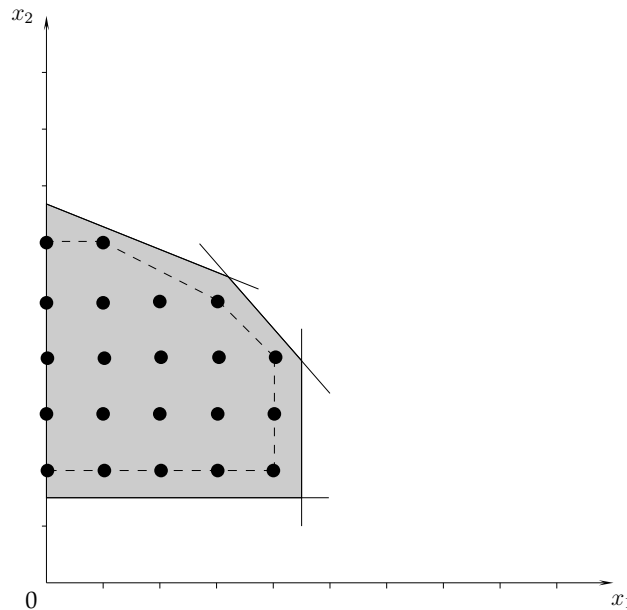


Figura 3.3: Espaço de soluções válidas para um modelo de Programação Inteira

No melhor dos casos, os pontos extremos do espaço de soluções da relaxação linear de um modelo de Programação Inteira são pontos inteiros. Quando isso acontece, a solução óptima do modelo de Programação Inteira pode ser obtida resolvendo simplesmente a sua relaxação linear. Como vimos, essa resolução é feita normalmente de

forma eficiente, o que garante que a solução óptima inteira seja ela também obtida rapidamente. Os modelos cuja relaxação linear estejam nessas condições possuem a chamada *propriedade da integralidade*. Na Figura 3.3, se o espaço de soluções válidas da relaxação linear fosse definido através das rectas a tracejado, o modelo teria a propriedade da integralidade. Esse espaço convexo de soluções é definido por pontos extremos inteiros, o que garante que o ponto óptimo seja também um ponto inteiro. Os métodos de planos de corte que abordaremos mais adiante procuram aproximar esse espaço adicionando restrições (planos de corte) à relaxação linear. Essas restrições *cortam* porções do espaço onde se encontram soluções com valores fraccionários, mas não excluem nenhuma solução inteira.

Na maior parte dos casos, os modelos de Programação Inteira não possuem a propriedade da integralidade. A resolução das relaxações lineares desses modelos apenas nos permite derivar limites relativamente ao valor do óptimo inteiro. Para o modelo geral definido no início desta secção, sabemos que $z_{RL}^* \leq z_{PI}^*$, dado que a relaxação linear corresponde ao modelo de Programação Inteira ao qual foram retirados um conjunto de restrições. No caso dos problemas de maximização, verifica-se a relação inversa. A Figura 3.4 ilustra essa relação.

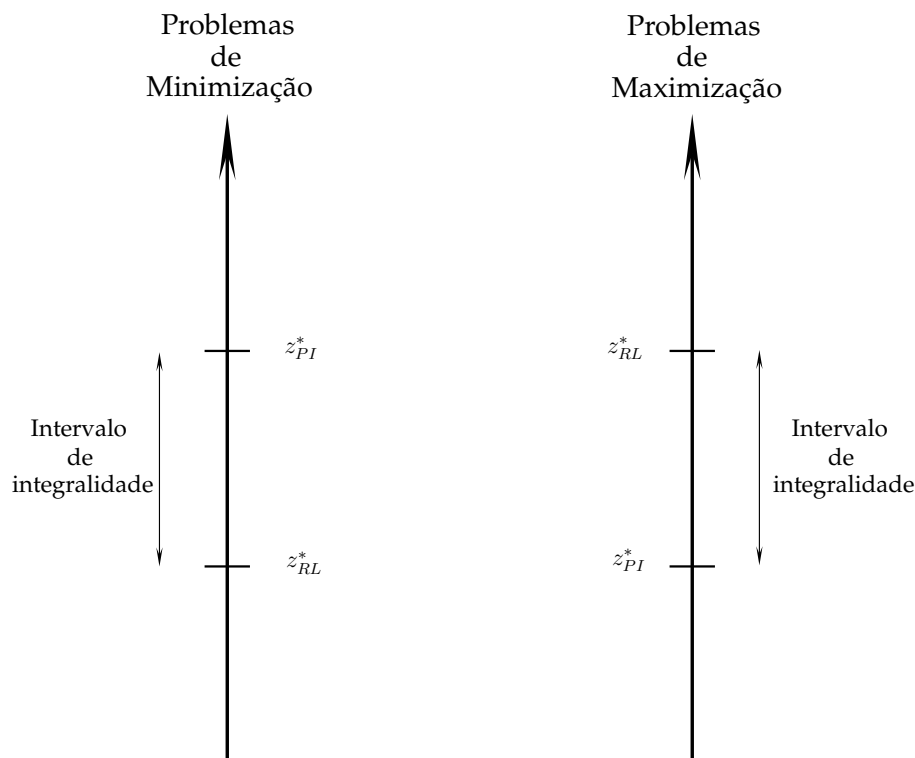


Figura 3.4: Relação entre os valores da solução óptima inteira e da relaxação linear

O intervalo de integralidade corresponde precisamente ao tamanho do intervalo que separa o valor óptimo da relaxação linear do óptimo do respectivo modelo de Programação Inteira. O valor absoluto desse intervalo é igual a

$$|z_{PI}^* - z_{RL}^*|,$$

enquanto o intervalo relativo de integralidade é dado por

$$\frac{|z_{PI}^* - z_{RL}^*|}{z_{PI}^*}.$$

Uma formulação será tanto melhor quanto menor for o seu intervalo de integralidade. No melhor caso, em que se verifica a propriedade da integralidade, o intervalo de integralidade é nulo uma vez que $z_{RL}^* = z_{PI}^*$.

O intervalo de integralidade mede a qualidade da aproximação que é conseguida através da relaxação linear. A sua importância deve-se ao facto de alguns dos melhores métodos genéricos de resolução de modelos de Programação Inteira basearem-se na resolução das respectivas relaxações lineares. É o caso do método de partição e avaliação e do método de planos de corte. Ambos os métodos procuram a solução óptima inteira dentro do intervalo de integralidade. Quanto maior for esse intervalo, maior será geralmente o tempo de convergência dos métodos. A situação ideal acontece quando o intervalo de integralidade é nulo. Nesse caso, a solução é obtida logo após a resolução da relaxação linear. Na secção seguinte, caracterizam-se os modelos em que essa situação ocorre de forma sistemática.

3.3.2 Propriedade da Total Unimodularidade

Num modelo de Programação Linear, quando a matriz dos coeficientes a_{ij} ($i = 1, \dots, m$, $j = 1, \dots, n$) do lado esquerdo das restrições possui a chamada *propriedade da total unimodularidade*, e os coeficientes b_i ($i = 1, \dots, m$) do termo independente das restrições são inteiros, os pontos extremos do espaço de soluções válidas são todos pontos inteiros. A solução óptima desses modelos é sempre uma solução inteira qualquer que sejam os coeficientes da função objectivo.

Uma matriz é totalmente unimodular se todas as suas sub-matrizes quadradas tiverem um determinante igual a 0, -1 ou 1 . Em alguns casos, é possível verificar que uma matriz possui a propriedade da total unimodularidade sem ter de calcular todos os determinantes das suas sub-matrizes quadradas. Com efeito, uma matriz possui a propriedade da total unimodularidade se verificar as seguintes condições:

- 1) todos os seus coeficientes deverão ser iguais a 0, -1 ou 1 ;

- 2) as suas colunas não poderão ter mais de 2 coeficientes não nulos;
- 3) deverá ser possível dividir as linhas da matriz em dois conjuntos tais que se uma coluna tiver dois elementos de mesmo sinal, eles não deverão estar no mesmo conjunto, enquanto que se uma coluna tiver dois elementos de sinal oposto eles deverão estar no mesmo conjunto.

O Exemplo seguinte ilustra um caso em que essas condições se verificam.

Exemplo 3.4 Considere o modelo de Programação Linear seguinte:

$$\begin{aligned}
 \min \quad & z = 2x_1 + 6x_2 + x_3 + 6x_4 + 8x_5 + 9x_6 + 4x_7 + 3x_8 + 2x_9 \\
 \text{s. a} \quad & \\
 R_1 : \quad & -x_1 \qquad \qquad \qquad + x_7 \qquad \qquad \qquad \geq 19 \\
 R_2 : \quad & x_1 \qquad \qquad \qquad + x_6 \qquad \qquad \qquad \geq 41 \\
 R_3 : \quad & \qquad \qquad \qquad \qquad \qquad \qquad \qquad x_8 - x_9 \geq 23 \\
 R_4 : \quad & x_2 \qquad \qquad \qquad + x_6 \qquad \qquad \qquad \geq 57 \\
 R_5 : \quad & \qquad -x_3 + x_4 \qquad \qquad \qquad + x_7 \qquad \qquad \geq 120 \\
 R_6 : \quad & \qquad \qquad x_3 - x_4 + x_5 \qquad \qquad \qquad + x_8 \qquad \geq 34 \\
 & x_1 \quad , \quad x_2 \quad , \quad x_3 \quad , \quad x_4 \quad , \quad x_5 \quad , \quad x_6 \quad , \quad x_7 \quad , \quad x_8 \quad , \quad x_9 \geq 0
 \end{aligned}$$

A propriedade da total unimodularidade diz respeito à matriz de coeficientes do lado esquerdo das restrições. Para essa matriz, é fácil confirmar que se verificam as propriedades 1) e 2) enunciadas acima. Se dividirmos o conjunto de restrições do modelo nos subconjuntos $\{R_1, R_2, R_5, R_6\}$ e $\{R_3, R_4\}$, confirmamos que a propriedade 3) se verifica também. A matriz de coeficientes possui por isso a propriedade da total unimodularidade. Desde que os termos independentes das restrições sejam inteiros, a solução óptima será sempre inteira independentemente dos valores dos coeficientes da função objectivo. $\square$

Essas condições são úteis uma vez que podem facilmente verificadas. No entanto, são apenas condições suficientes: uma matriz que não verifique essas condições poderá mesmo assim possuir a propriedade da total unimodularidade.

3.4 Melhoria dos Modelos

Usando algumas regras simples, é possível melhorar a qualidade dos limites obtidos através da relaxação linear de um modelo de Programação Inteira. O objectivo é reduzir tanto quanto possível o intervalo de integralidade reformulando algumas componentes

do modelo, ou derivando inequações válidas a partir dos dados do modelo original. Nesta secção, descrevemos algumas técnicas de reformulação e de cálculo de inequações válidas que podem ser aplicadas numa fase de pré-processamento.

3.4.1 Limites nas Variáveis

Uma forma de melhorar a qualidade de um modelo de Programação Inteira consiste em derivar limites para o valor das variáveis de um modelo usando argumentos baseados na integralidade das variáveis. O Exemplo seguinte ilustra esse procedimento.

Exemplo 3.5 Considere o modelo de Programação Inteira seguinte:

$$\begin{aligned} \max z_{PI} = & x_1 + 4x_2 + 3x_3 + 5x_4 \\ \text{s. a} & \\ & 5x_1 + 4x_2 + 8x_3 + 4x_4 \leq 17 \\ & 2x_1 + x_2 + 7x_4 \leq 25 \\ & 2x_1 + 4x_2 + x_3 \leq 7 \\ & x_1, x_2, x_3, x_4 \geq 0 \text{ e inteiros.} \end{aligned}$$

O valor da solução óptima z_{PI}^* desse modelo é igual a 19. A relaxação linear do modelo tem uma solução óptima z_{RL}^* cujo valor é aproximadamente igual a 20.46. Esse óptimo pode ser obtido através dos seguintes valores das variáveis de decisão:

$$(x_1, x_2, x_3, x_4) = (0, 0.79, 0, 3.46).$$

Partindo da segunda restrição do modelo, podemos definir o seguinte limite superior para a variável x_4 :

$$x_4 \leq \frac{25}{7} - \frac{2}{7}x_1 - \frac{1}{7}x_2.$$

Dado que x_1 e x_2 são variáveis não-negativas, é válida a seguinte restrição:

$$x_4 \leq \frac{25}{7} \approx 3.57.$$

Dado que x_4 é uma variável inteira, por definição, ela não poderá tomar valores no intervalo $[3, \frac{25}{7}]$. Por essa razão, é válido o seguinte limite superior para a variável x_4 :

$$x_4 \leq 3.$$

Se esse limite superior tivesse sido imposto logo à partida, a solução da relaxação linear passaria a ter um valor igual a 20. Os seguintes valores para as variáveis de decisão resultam nesse valor óptimo:

$$(x_1, x_2, x_3, x_4) = (0, 1.25, 0, 3).$$

Usando um argumento semelhante podemos derivar um limite superior para a variável x_2 partindo da terceira restrição do modelo: a inequação

$$x_2 \leq \frac{7}{4} - \frac{2}{4}x_1 - \frac{1}{4}x_3$$

conduz-nos ao limite superior seguinte

$$x_2 \leq \frac{7}{4},$$

que por sua vez, e devido à integralidade da variável x_2 , nos leva à restrição seguinte

$$x_2 \leq 1.$$

Com essa nova restrição, o valor da relaxação linear passa a ser igual a 19.375. $\square$

O procedimento usado no Exemplo 3.5 é facilmente generalizado. Partindo de uma restrição com a forma seguinte

$$a_1x_1 + a_2x_2 + a_3x_3 + \dots + a_nx_n \leq b,$$

e assumindo que são conhecidos limites superiores U_j e limites inferiores L_j para todas as variáveis ($L_j \leq x_j \leq U_j$, $j = 1, \dots, n$), é possível definir novos limites superior ou inferior para a variável x_j dependendo do seu coeficiente a_j na restrição ser respectivamente positivo ou negativo:

- se $a_j > 0$:

$$x_j \leq \left\lceil \frac{b - \sum_{i:a_i>0} a_i L_i - \sum_{i:a_i<0} a_i U_i}{a_j} \right\rceil.$$

Se o valor desse limite superior for inferior a U_j , esse limite passará a ser o novo limite superior para a variável x_j ;

- se $a_j < 0$:

$$x_j \geq \left\lfloor \frac{b - \sum_{i:a_i>0} a_i L_i - \sum_{i:a_i<0} a_i U_i}{a_j} \right\rfloor.$$

Se o valor desse limite inferior for superior a L_j , esse limite passará a ser o novo limite inferior para a variável x_j .

As funções $\lceil z \rceil$ e $\lfloor z \rfloor$ usadas nas expressões acima são funções de arredondamento. A primeira retorna o menor valor inteiro superior ou igual a z , enquanto a segunda retorna o maior valor inteiro inferior ou igual a z .

3.4.2 Análise de Coeficientes

Outra forma de reforçar um modelo consiste em reformular algumas das suas restrições, melhorando os valores dos coeficientes das variáveis e do termo independente. Em alguns casos, é fácil melhorar os coeficientes de uma restrição. Isso acontece, por exemplo, quando numa restrição de tipo *menor ou igual* composta apenas por variáveis binárias, existem variáveis com coeficientes que são superiores ao termo independente da restrição. Na restrição

$$a_1x_1 + a_2x_2 + a_3x_3 + \dots + a_nx_n \leq b,$$

as variáveis x_j cujo coeficiente a_j é estritamente superior a b ($a_j > b$) serão forçosamente iguais a 0, e podem por isso ser retiradas do modelo.

A redução dos coeficientes é também possível noutros casos. O exemplo seguinte ilustra um procedimento de redução que pode ser aplicado a restrições com variáveis binárias e do tipo menor ou igual.

Exemplo 3.6 Considere a restrição seguinte:

$$8x_1 + 4x_2 + 3x_3 \leq 13,$$

$$x_1, x_2, x_3 \in \{0, 1\}.$$

Se tivermos em atenção a variável x_1 , observamos que duas situações poderão ocorrer:

- $x_1 = 0$: nesse caso, a restrição reduz-se à inequação seguinte

$$4x_2 + 3x_3 \leq 13,$$

que, por sua vez, pode ser simplificada como segue

$$4x_2 + 3x_3 \leq 7,$$

uma vez que a soma dos coeficientes das variáveis que ainda fazem parte da restrição é estritamente menor que o termo independente dessa restrição. Temos assim que $4 + 3 = 7 < 13$, o que pode ser também escrito na forma $(8+4+3)-8=7<13$, se nos basearmos na soma de todos os coeficientes das variáveis que fazem parte originalmente da restrição, e lhe subtrairmos o coeficiente da variável que é eliminada dessa restrição.

- $x_1 = 1$: nesse caso, a restrição de partida reduz-se à forma seguinte

$$4x_2 + 3x_3 \leq 5.$$

A restrição inicial pode ser reescrita como segue

$$4x_2 + 3x_3 \leq 13 - 8x_1.$$

Se somarmos o valor $(15 - 13)x_1$ a cada um dos termos dessa inequação (o valor 15 foi escolhido porque é a soma dos coeficientes das variáveis na restrição original; o valor 13 corresponde ao termo independente da restrição original), obtemos a seguinte restrição

$$(15 - 13)x_1 + 4x_2 + 3x_3 \leq 13 - 8x_1 + (15 - 13)x_1,$$

que pode ser simplificada da forma seguinte

$$2x_1 + 4x_2 + 3x_3 \leq 13 - 13x_1 + (15 - 8)x_1.$$

Uma restrição é tanto mais forte, quanto mais baixo for o valor do seu termo independente (e mais elevados forem os coeficientes das variáveis do termo do lado esquerdo da restrição). No caso da última restrição, o menor valor possível é igual a 7, e corresponde a fazermos $x_1 = 1$ no termo do lado direito. A restrição resultante é a seguinte:

$$2x_1 + 4x_2 + 3x_3 \leq 7.$$

Por inspecção dos valores de x_1 , observamos facilmente que esta última restrição é equivalente à restrição original:

- com $x_1 = 0$, obtemos a restrição $4x_2 + 3x_3 \leq 7$;
- com $x_1 = 1$, obtemos a restrição $4x_2 + 3x_3 \leq 5$.

Vamos designar por S a soma dos coeficientes a_j , $j = 1, \dots, n$, das variáveis na restrição, e por b o valor do termo independente. Na prática, o processo pode ser aplicado enquanto $S - \max_j \{a_j\} < b$.

Na última restrição, temos que $S = 9$, $b = 7$ e $\max_j \{a_j\} = 4$. Uma vez que $9 - 4 < 7$, podemos repetir o processo usando como referência a variável x_2 . A restrição resultante é a seguinte:

$$2x_1 + 2x_2 + 3x_3 \leq 5.$$

O valor de S é agora igual a 7, enquanto $\max_j \{a_j\} = 3$. Dado que $7 - 3 < 5$, repetimos o processo com base na variável x_3 , do qual resulta a restrição:

$$2x_1 + 2x_2 + 2x_3 \leq 4,$$

e conseqüentemente

$$x_1 + x_2 + x_3 \leq 2.$$

Nesse ponto, e uma vez que a condição $S - \max_j \{a_j\} < b$ já não se verifica, o processo não pode ser mais repetido. $\square$

O exemplo anterior sugere um procedimento geral de redução de coeficientes aplicável a restrições lineares do tipo *menor ou igual*, com variáveis binárias e coeficientes positivos nas variáveis de decisão. Vamos usar a notação que introduzimos no Exemplo 3.6. Para uma dada restrição, o procedimento é aplicável enquanto $S - \max_j \{a_j\} < b$. Uma restrição nessas condições poderá ser substituída por outra definida da forma seguinte (sem perda de generalidade, vamos assumir que a variável de maior coeficiente é a variável de índice $j = 1$):

$$(S - b)x_1 + a_2x_2 + a_3x_3 + \dots + a_nx_n \leq S - a_1,$$

$$x_j \in \{0, 1\}, \quad j = 1, \dots, n.$$

Para aplicar este procedimento a restrições com variáveis inteiras gerais, teremos que exprimir essas últimas em ordem a variáveis binárias. Assim, se tivermos uma variável inteira geral x_0 positiva e tal que $x_0 \leq U_0$, essa variável poderá ser substituída pela expressão seguinte:

$$x_1 + 2x_2 + 4x_3 + \dots + 2^{m-1}x_m,$$

em que $x_j \in \{0, 1\}$, $j = 1, \dots, m$, e $m = \lceil \log_2(U_0) \rceil$.

Por outro lado, podemos facilmente transformar variáveis binárias cujos coeficientes nas restrições são negativos em variáveis com coeficientes positivos. Por exemplo, na restrição seguinte

$$3x_1 - 2x_2 + 4x_3 \leq 5,$$

$$x_1, x_2, x_3 \in \{0, 1\},$$

se introduzirmos uma nova variável binária x'_2 tal que $x'_2 = 1 - x_2$, obtemos a seguinte restrição equivalente

$$3x_1 + 2x'_2 + 4x_3 \leq 7,$$

na qual todas as variáveis binárias têm coeficientes positivos.

3.4.3 Restrições de Cobertura

As restrições de cobertura permitem reforçar modelos em que existem restrições do tipo *menor ou igual*, formuladas na maior parte dos casos em ordem a variáveis binárias. O princípio dessas restrições consiste em impedir que alguns subconjuntos de variáveis tomem simultaneamente valores positivos, quando é possível antecipar que em nenhuma solução válida inteira, essas variáveis poderão ser todas positivas. Formalmente, para uma restrição

$$a_1x_1 + a_2x_2 + a_3x_3 + \dots + a_nx_n \leq b,$$

com $x_j \in \{0, 1\}$, $j = 1, \dots, n$, se C for um subconjunto de índices tal que

$$\sum_{i \in C} a_i > b,$$

então a seguinte restrição é uma restrição válida no sentido em que não exclui nenhuma solução inteira válida:

$$\sum_{i \in C} x_i \leq |C| - 1.$$

Essa restrição é designada por *restrição de cobertura*, e o conjunto C por *cobertura*. Se o conjunto C for tal que

$$\forall C' \subset C : \sum_{i \in C'} a_i \leq b,$$

então a restrição de cobertura baseada nesse conjunto de variáveis será designada por *restrição de cobertura mínima*. Da mesma forma, o conjunto C é designado por *cobertura mínima*.

Exemplo 3.7 Considere a restrição seguinte:

$$7x_1 + 4x_2 + 2x_3 \leq 10,$$

em que $x_i \in \{0, 1\}$, $i = 1, \dots, 3$. Para essa restrição, é fácil antecipar que as variáveis x_1 e x_2 nunca tomarão simultaneamente valores positivos qualquer que seja a solução inteira válida considerada. No entanto, é importante observar que se relaxarmos a restrição de integralidade das variáveis de decisão, soluções como, por exemplo, a seguinte

$$(x_1, x_2, x_3, x_4) = (1, 0.75, 0, 0),$$

em que essas duas variáveis tomam de facto valores positivos, passam a ser soluções válidas (se não existir nenhuma outra restrição que as exclua).

Um conjunto C constituído pelos índices 1 e 2 é uma cobertura, uma vez que

$$\sum_{i \in C} a_i = 7 + 4 > 10.$$

A seguinte restrição é por isso uma restrição de cobertura:

$$x_1 + x_2 \leq 1.$$

Um conjunto C formado pelos índices 1, 2 e 3 é também uma cobertura dado que

$$\sum_{i \in C} a_i = 7 + 4 + 2 > 10.$$

A restrição de cobertura baseada nesse conjunto é a seguinte:

$$x_1 + x_2 + x_3 \leq 2.$$

Note que a primeira restrição de cobertura que derivámos é mais forte que esta última: soluções que satisfazem a segunda restrição de cobertura não satisfazem necessariamente a primeira; o contrário já não se verifica. É fácil observar que a primeira restrição de cobertura é de facto uma restrição de cobertura mínima. $\square$

3.4.4 Teste e Fixação do Valor das Variáveis

É possível identificar restrições redundantes num modelo testando o valor das variáveis de decisão. Para isso, deve-se ter em atenção o caso extremo, *i.e.* aquele que ocorre quando fixamos as variáveis num dos seus limites inferior ou superior. Essa identificação pode ser feita numa fase de pré-processamento conforme ilustrado no exemplo seguinte.

Exemplo 3.8 Considere o modelo de Programação Inteira seguinte:

$$\begin{aligned} \max \quad & z_{PI} = 4x_1 + 2x_2 + 7x_3 + x_4 \\ \text{s. a} \quad & \\ R_1 : \quad & x_1 - 2x_2 + 4x_3 + x_4 \leq 13 \\ R_2 : \quad & -x_1 + 3x_2 \geq 5 \\ R_3 : \quad & 2x_1 + 4x_2 + 4x_3 + 3x_4 \leq 10 \\ & x_1, x_2, x_3, x_4 \geq 0 \text{ e inteiros.} \end{aligned}$$

A solução óptima z_{PI}^* tem um valor igual a 8. Tendo em conta a restrição R_3 , assim como a integralidade das variáveis, é possível derivar os seguintes limites superiores para as variáveis do modelo:

$$x_1 \leq 5, \quad x_2 \leq 2, \quad x_3 \leq 2, \quad x_4 \leq 3.$$

De igual forma, dado que as variáveis são não-negativas, podemos deduzir a partir da restrição R_2 o seguinte limite inferior para a variável x_2 :

$$x_2 \geq 2.$$

O termo do lado esquerdo da restrição R_1 atinge o seu maior valor quando as variáveis x_1 , x_3 e x_4 estão no seu nível máximo, e a variável x_2 no seu nível mínimo. Nesse caso extremo, o termo do lado esquerdo terá um valor igual a $5 - 2 \times 2 + 4 \times 2 + 2 = 11 < 13$. Claramente, a restrição R_1 é redundante uma vez que não exclui nenhuma solução que não tenha já sido excluída pelas outras restrições. $\square$

Eliminar restrições redundantes traz vantagens ao permitir, por exemplo, reduzir a dimensão dos modelos. No entanto, é importante notar que essa operação não afecta a qualidade do limite (inferior ou superior) dado pela relaxação linear. Todas as soluções fraccionárias que eram válidas no modelo com essas restrições continuarão a sê-lo no modelo simplificado.

Usando restrições como as restrições de cobertura que introduzimos na Secção 3.4.3, e explorando algumas combinações dessas restrições, é muitas vezes possível fixar o valor de algumas variáveis. O exemplo seguinte ilustra um desses casos.

Exemplo 3.9 As restrições seguintes definem parte de um modelo de Programação Inteira:

$$\begin{aligned} R_1 : \quad & 3x_1 + 5x_2 + 4x_3 \leq 7, \\ R_2 : \quad & x_1 \quad \quad + x_3 \geq 1, \\ & x_1, \quad x_2, \quad x_3 \in \{0, 1\}. \end{aligned}$$

Com base na restrição R_1 , podemos derivar as seguintes restrições de cobertura:

$$\begin{aligned} x_1 + x_2 &\leq 1, \\ x_2 + x_3 &\leq 1. \end{aligned}$$

Se combinarmos a restrição R_2 com a primeira restrição de cobertura, obtemos a seguinte inequação:

$$x_2 - x_3 \leq 0.$$

Combinando agora essa restrição com a segunda restrição de cobertura, chegamos à seguinte inequação:

$$2x_2 \leq 1.$$

Dado que x_2 é uma variável binária, temos então que $x_2 = 0$. O valor da variável é fixado a 0, e a variável é eliminada do modelo. $\square$

3.4.5 Restrições Desagregadas

Em modelos de Programação Inteira, é frequente existirem variáveis cujo valor depende de uma determinada condição (como acontecia, por exemplo, no problema dos custos fixos que vimos atrás). Nas formulações, essa dependência dá origem a restrições que envolvem essas variáveis e outras variáveis binárias. Tipicamente, quanto mais desagregadas forem essas restrições mais restritos serão os domínios das variáveis e, consequentemente, mais forte serão os modelos. O exemplo seguinte ilustra um desses casos.

Exemplo 3.10 A restrição seguinte:

$$x_1 + x_2 + x_3 + x_4 + x_5 \leq My,$$

em que $x_i \in \mathbb{N}$, $i = 1, \dots, 5$, e $y \in \{0, 1\}$, faz com que as variáveis x_i possam tomar valores não nulos apenas se a variável binária y estiver activa ($y = 1$). Se a variável y for de facto igual a 1, as outras variáveis poderão tomar qualquer valor positivo (M é um número positivo muito elevado).

Vamos assumir que é possível derivar um limite superior para as variáveis x_i , e que esse limite é igual a 5 para todas as variáveis. A restrição acima pode-ser reforçada como segue:

$$x_i \leq 5y, \quad i = 1, \dots, 5.$$

Mantém-se a condição segundo a qual as variáveis só podem tomar valores positivos se $y = 1$, sendo que o domínio das variáveis passa agora a ser contido no intervalo mais restrito $[0, 5]$. □

3.5 Planos de Corte

Uma forma de reforçar os modelos consiste em adicionar restrições lineares que consigam excluir soluções fraccionárias não admissíveis. As restrições de cobertura que analisámos na secção anterior são um exemplo. Nesta Secção, analisamos outras famílias de restrições que têm em comum o facto de poderem ser derivadas com base em funções particulares (funções superaditivas).

Introduzir planos de corte na relaxação linear de um modelo de Programação Inteira permite reduzir o intervalo de integralidade do modelo ao excluir soluções que são fraccionárias e que estejam fora do espaço convexo de soluções definido por pontos extremos inteiros. A Figura 3.5 ilustra essa abordagem. O espaço de soluções da relaxação linear é representado a cinzento, enquanto o espaço convexo de soluções definido por pontos extremos inteiros corresponde à área quadriculada. As restrições representadas pelas rectas a tracejado permitem excluir soluções que são válidas para a relaxação linear original, mas que não o são para o respectivo modelo de Programação Inteira. Essas restrições são também designadas por *planos de corte*.

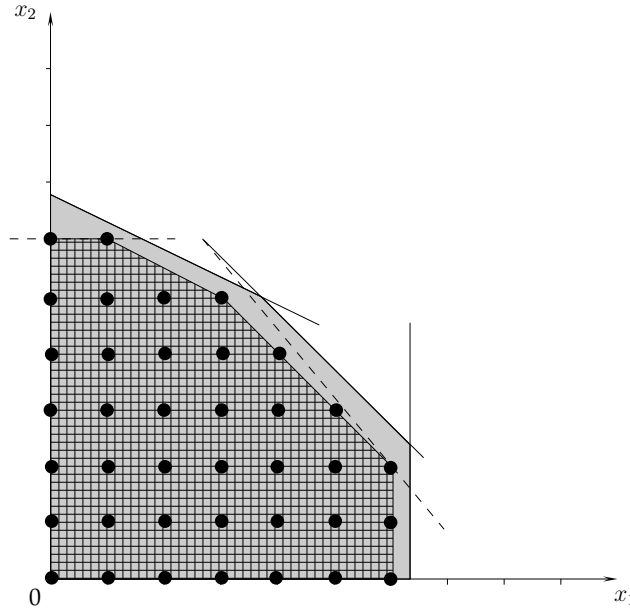


Figura 3.5: Reforço de um modelo através da adição de restrições (planos de corte)

3.5.1 Funções Superaditivas

Uma função $f : \mathbb{R} \rightarrow \mathbb{R}$ é superaditiva se, para qualquer $x, y \in \mathbb{R}$,

$$f(x) + f(y) \leq f(x + y).$$

A mesma função f será não decrescente se, para dois valores $x, y \in \mathbb{R}$ tal que $x \leq y$, se verificar a seguinte relação

$$f(x) \leq f(y).$$

As funções superaditivas e não decrescentes são importantes na medida em que permitem derivar planos de corte válidos para modelos de Programação Inteira.

Vamos considerar o modelo de Programação Inteira seguinte:

$$\begin{aligned} \max \quad & z_{PI} = \sum_{j=1}^n c_j x_j, \\ \text{s. a} \quad & \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad i = 1, \dots, m, \\ & x_j \geq 0 \text{ e inteiro, } j = 1, \dots, n, \end{aligned}$$

Proposição 3.1 Se $f : \mathbb{R} \rightarrow \mathbb{R}$ for uma função superaditiva, não decrescente e tal que $f(0) = 0$, então para qualquer $i = 1, \dots, m$, a restrição seguinte

$$\sum_{j=1}^n f(a_{ij})x_j \leq f(b_i),$$

é um plano de corte válido para esse modelo de Programação Inteira.

Prova. Para um dado i , a expressão $\sum_{j=1}^n a_{ij}x_j$ pode ser reescrita da forma seguinte:

$$\sum_{j=1}^n \sum_{k=1}^{x_j} a_{ij}.$$

Dado que f é uma função superaditiva, temos que:

$$\sum_{j=1}^n \sum_{k=1}^{x_j} f(a_{ij}) \leq f\left(\sum_{j=1}^n \sum_{k=1}^{x_j} a_{ij}\right).$$

No caso em que $x_j = 0$, $j = 1, \dots, n$, a inequação anterior reduz-se à forma seguinte:

$$0 \leq f(0).$$

É possível mostrar que a definição de uma função superaditiva obriga a que $f(0) \leq 0$, o que nos leva a concluir neste caso que $f(0) = 0$.

Dado que f é uma função não decrescente, para um dado i , temos que:

$$f\left(\sum_{j=1}^n a_{ij}x_j\right) = f\left(\sum_{j=1}^n \sum_{k=1}^{x_j} a_{ij}\right) \leq f(b_i). \quad (3.1)$$

Como f é superaditiva, é válida também a inequação seguinte:

$$\sum_{j=1}^n \sum_{k=1}^{x_j} f(a_{ij}) \leq f\left(\sum_{j=1}^n \sum_{k=1}^{x_j} a_{ij}\right). \quad (3.2)$$

A partir de (3.1) e (3.2), concluímos que a inequação

$$\sum_{j=1}^n \sum_{k=1}^{x_j} f(a_{ij}) = \sum_{j=1}^n f(a_{ij})x_j \leq f(b_i),$$

é válida. ■

3.5.2 Inequações de Chvátal-Gomory

As inequações de Chvátal-Gomory são obtidas com base na função de arredondamento seguinte:

$$f(x) = \lfloor x \rfloor.$$

A Proposição seguinte mostra que essa função pode de facto ser usada para derivar inequações válidas (planos de corte) para modelos de Programação Inteira.

Proposição 3.2 A função $f(x)$ é superaditiva e não decrescente.

Prova. A parte fraccionária do resultado da soma de dois valores x_1 e x_2 é igual a:

$$r_{x_1+x_2} = x_1 - \lfloor x_1 \rfloor + x_2 - \lfloor x_2 \rfloor.$$

Podemos distinguir dois casos:

a) $r_{x_1+x_2} \geq 1$: nesse caso, $\lfloor x_1 + x_2 \rfloor = \lfloor x_1 \rfloor + \lfloor x_2 \rfloor + 1$;

b) $r_{x_1+x_2} < 1$: e $\lfloor x_1 + x_2 \rfloor = \lfloor x_1 \rfloor + \lfloor x_2 \rfloor$.

Em ambos os casos verifica-se a inequação seguinte:

$$\lfloor x_1 \rfloor + \lfloor x_2 \rfloor \leq \lfloor x_1 + x_2 \rfloor,$$

o que prova que $f(x)$ é de facto superaditiva.

Por outro lado, se x_1 e x_2 forem tais que $x_1 \leq x_2$, a inequação

$$\lfloor x_1 \rfloor \leq \lfloor x_2 \rfloor,$$

é imediata. ■

Vamos considerar o modelo de Programação Inteira com n variáveis e m restrições expresso como segue:

$$\begin{aligned} \max \quad & z = cx, \\ \text{s. a} \quad & \\ & Ax \leq b, \\ & x \geq 0, \text{ e inteiros,} \end{aligned}$$

com $A = (a_1, a_2, \dots, a_n)$. As inequações de Chvátal-Gomory (*Chvatal-Gomory rank 1 inequalities*) são definidas da forma seguinte:

$$\sum_{i=1}^n \lfloor ua_i \rfloor x_i \leq \lfloor ub \rfloor,$$

com $u \in \mathbb{R}_+^m$. Outras inequações do mesmo tipo podem ser derivadas combinando essas inequações e restrições do modelo original. Esse processo de geração de cortes acaba por produzir todas as inequações do espaço convexo de soluções definido por pontos extremos inteiros.

Exemplo 3.11 Considere o modelo de Programação Inteira seguinte:

$$\begin{aligned}
\max \quad & z = 11x_1 + 9x_2 \\
s. \quad & a \\
& 7x_1 + 10x_2 \leq 70 \\
& 10x_1 + 3x_2 \leq 65 \\
& x_2 \leq 5 \\
& x_1, x_2 \geq 0 \text{ e inteiros.}
\end{aligned}$$

A solução da relaxação linear tem um valor igual a $\frac{7045}{79}$, e corresponde a

$$(x_1, x_2) = \left(\frac{440}{79}, \frac{245}{79} \right).$$

Vamos designar por π a solução dual da relaxação linear. O valor óptimo dessa solução é o seguinte:

$$(\pi_1, \pi_2, \pi_3) = \left(\frac{57}{79}, \frac{47}{79}, 0 \right).$$

Fazendo $u = \pi$, derivamos a seguinte inequação de Chvátal-Gomory:

$$11x_1 + 9x_2 \leq \left\lfloor \frac{7045}{79} \right\rfloor = 89.$$

Se fizermos $u = \left(\frac{54}{79}, \frac{81}{79}, \frac{7}{79} \right)$, obtemos a inequação de Chvátal-Gomory seguinte:

$$15x_1 + 10x_2 \leq 114.$$

que também corta a solução óptima da relaxação linear. □

3.5.3 Corte Fraccionário de Gomory

Os cortes fraccionários de Gomory estão relacionados com as inequações de Chvátal-Gomory, e podem também ser usados para reforçar modelos de Programação Inteira. Partindo da equação seguinte

$$\sum_{i=1}^n a_i x_i = b$$

em que $a_i \in \mathbb{R}, i = 1, \dots, n$, e $b \in \mathbb{R}$, vamos assumir que todas as variáveis devem ser inteiras e não-negativas ($x_i \geq 0$, e inteiros, $i = 1, \dots, n$). Uma vez que todas as variáveis são não-negativas, temos que:

$$\sum_{i=1}^n \lfloor a_i \rfloor x_i \leq b.$$

O facto de todas as variáveis serem inteiras conduz ao resultado seguinte:

$$\sum_{i=1}^n \lfloor a_i \rfloor x_i \leq \lfloor b \rfloor.$$

Subtraindo essa expressão à restrição de partida, obtemos a inequação seguinte:

$$\sum_{i=1}^n (a_i - \lfloor a_i \rfloor) x_i \geq b - \lfloor b \rfloor,$$

que corresponde a um corte fraccionário de Gomory.

As soluções básicas da relaxação linear de um modelo de Programação Inteira podem ser usadas para derivar cortes fraccionários de Gomory. O exemplo seguinte ilustra esse procedimento.

Exemplo 3.12 Vamos admitir que na relaxação linear de um modelo de Programação Inteira, a variável x_1 era uma variável básica expressa em ordem às variáveis não-básicas da forma seguinte:

$$x_1 - \frac{3}{4}x_2 + \frac{79}{2}x_3 - \frac{14}{5}x_4 = \frac{27}{5}.$$

O corte fraccionário de Gomory associado a essa equação é o seguinte:

$$\frac{1}{4}x_2 + \frac{1}{2}x_3 + \frac{1}{5}x_4 \geq \frac{2}{5}.$$

□

3.5.4 Corte Inteiro Misto de Gomory

Os cortes inteiros mistos de Gomory permitem reforçar modelos de Programação Inteira mista onde existam simultaneamente variáveis inteiras e contínuas. Os cortes que descrevemos nas secções anteriores não podem ser usados com esse tipo de modelos.

Seja S o espaço definido pelas variáveis inteiras e não-negativas x_i , $i \in N_1$, pelas variáveis contínuas e não-negativas y_i , $i \in N_2$, e pela restrição seguinte:

$$\sum_{i \in N_1} a_i x_i + \sum_{i \in N_2} d_i y_i \leq b,$$

em que todos os parâmetros são números reais. Seja ainda $r_i = a_i - \lfloor a_i \rfloor$, $i \in N_1$, e $r_0 = b - \lfloor b \rfloor$, com $1 > r_0 > 0$. A expressão geral do corte inteiro misto de Gomory é a seguinte:

$$\sum_{i \in N_1: r_i \leq r_0} r_j x_j + \frac{r_0}{1 - r_0} \sum_{i \in N_1: r_i > r_0} (1 - r_i) x_i + \sum_{i \in N_2: d_i > 0} d_i y_i - \frac{r_0}{1 - r_0} \sum_{i \in N_2: d_i < 0} d_i y_i \geq r_0.$$

Exemplo 3.13 Vamos considerar um espaço S definido como segue:

$$S = \{x_1, x_2 \in \mathbb{N}, y_1, y_2 \in \mathbb{R} : \frac{15}{4}x_1 + \frac{7}{3}x_2 - \frac{1}{2}y_1 + \frac{7}{4}y_2 \leq \frac{12}{5}\}.$$

Temos que:

$$\cdot r_0 = \frac{12}{5} - \left\lfloor \frac{12}{5} \right\rfloor = \frac{2}{5};$$

$$\cdot r_1 = \frac{15}{4} - \left\lfloor \frac{15}{4} \right\rfloor = \frac{3}{4};$$

$$\cdot r_2 = \frac{7}{3} - \left\lfloor \frac{7}{3} \right\rfloor = \frac{1}{3};$$

$$\cdot \frac{r_0}{1-r_0} = \frac{2}{3}.$$

O corte inteiro misto de Gomory associado ao espaço S é obtido da forma seguinte:

$$\begin{aligned} \frac{1}{3}x_2 + \frac{2}{3} \left(\frac{1}{4}x_1 \right) + \frac{7}{4}y_2 - \frac{2}{3} \left(-\frac{1}{2}y_1 \right) &\geq \frac{2}{5}, \\ \frac{1}{6}x_1 + \frac{1}{3}x_2 + \frac{1}{3}y_1 + \frac{7}{4}y_2 &\geq \frac{2}{5}. \end{aligned}$$

□

3.6 Resolução de Modelos de Programação Inteira

Actualmente, não existe nenhum algoritmo com o qual seja possível resolver de forma eficiente qualquer modelo de Programação Inteira, e tudo indica para que tal algoritmo não exista. A abordagem de resolução mais óbvia consiste em enumerar todas as soluções válidas do modelo, e escolher aquela que resulta no melhor valor da função objectivo, uma vez que o espaço de soluções de um modelo de Programação Inteira (pura) é finito. Contudo, mesmo sendo finito, o número de soluções de um modelo de Programação Inteira é normalmente muito elevado, o que torna essa abordagem claramente ineficiente na maior parte dos casos. Por outro lado, o próprio processo de geração de soluções válidas pode também ser um processo complexo.

Os métodos de resolução que abordamos nesta secção são métodos de resolução de modelos gerais de Programação Inteira, *i.e.* que não tiram partido da estrutura dos modelos. Os métodos baseiam-se em Programação Linear: todos partem da relaxação linear do modelo de Programação Inteira inicial, e resolvem sucessivamente diferentes relaxações lineares até obter finalmente a solução óptima inteira do problema.

3.6.1 Método de Partição e Avaliação

O método de partição e avaliação é designado na literatura anglo-saxónica por método de *branch-and-bound*. Trata-se de um método geral de enumeração que permite explorar de forma eficiente as soluções de um modelo de Programação Inteira. O método baseia-se na partição sucessiva do espaço original de soluções, e na avaliação de cada um dos subespaços resultantes. Essa avaliação é importante na medida em que pode permitir excluir conjuntos consideráveis de soluções, sem ter de as enumerar explicitamente. A avaliação baseia-se na comparação entre um limite para o valor do óptimo do problema original e um limite para o valor da melhor solução que é possível obter no subespaço de soluções em análise. As soluções contidas num subespaço de soluções são excluídas quando é possível afirmar que nenhuma delas será jamais a solução óptima do problema original. Isso acontece, por exemplo, quando a melhor solução do subespaço em análise é pior que uma solução válida já conhecida para o problema original.

Na resolução de modelos gerais de Programação Inteira, a relaxação linear do modelo original pode ser usada para derivar esses limites. Nesse contexto, a partição do espaço de soluções é feita impondo sucessivamente restrições adicionais (lineares) à relaxação linear do modelo. Essas restrições são designadas por *restrições de partição*. O método de partição e avaliação constroi uma árvore de pesquisa em que os nodos são subproblemas do problema original, e os ramos estão associados a restrições de partição. O subproblema de um nodo da árvore corresponde à relaxação linear do modelo original ao qual foram adicionadas todas as restrições de partição associadas aos ramos que conduzem a esse nodo. Na Figura 3.6, o subproblema 3, por exemplo, é um problema de Programação Linear definido a partir da relaxação linear do modelo de Programação Inteira original e das restrições de partição RP1 e RP3.

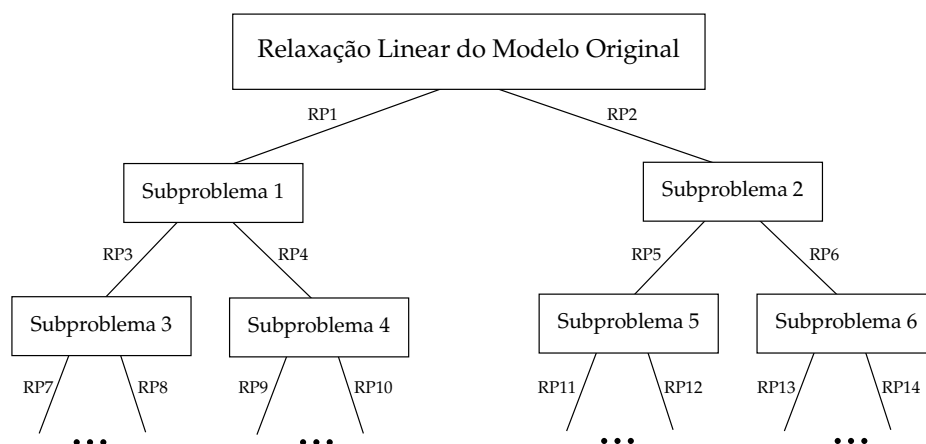


Figura 3.6: Árvore de pesquisa

Vamos assumir que o modelo de Programação Inteira original corresponde a um problema de maximização. Quando o método de partição e avaliação é usado para resolver modelos gerais de Programação Inteira baseado na resolução da relaxação linear do modelo, as operações fundamentais do método podem ser descritas como segue:

- **A partição:** vamos designar por SP_q o subproblema de Programação Linear que é resolvido num nodo q da árvore de pesquisa. Na raiz da árvore, esse subproblema corresponde à relaxação linear do modelo de Programação Inteira original. Num nodo q da árvore de pesquisa, se a solução que é obtida não satisfaz as restrições de integralidade do modelo original (algumas variáveis que deveriam ser inteiras são fraccionárias nessa solução), e se não for possível rejeitar esse nodo baseado na avaliação que é feita do nodo, são criados vários subproblemas. Esses subproblemas são problemas de Programação Linear definidos a partir de SP_q e de uma ou mais restrições de partição.

As restrições de partição são definidas de forma a que a solução fraccionária de SP_q não seja válida em nenhum dos novos subproblemas, e a solução ótima inteira de SP_q seja válida em pelo menos um desses subproblemas. Idealmente, os espaços de soluções válidas dos novos subproblemas deverão ser disjuntos para evitar que as mesmas soluções sejam exploradas mais do que uma vez em ramos diferentes da árvore.

As restrições de partição baseiam-se em variáveis que são fraccionárias na solução de SP_q . O modo como são seleccionadas as variáveis que irão fazer parte das restrições de partição, e mais geralmente, a forma como são definidas as restrições de partição constituem aquilo que é designado por *regra de partição*. A regra mais simples consiste em escolher uma variável fraccionária (x_i), e criar dois subproblemas baseados numa das duas restrições de partição seguintes:

$$x_i \leq \lfloor x_i \rfloor \quad \text{e} \quad x_i \geq \lceil x_i \rceil.$$

- **A avaliação:** num nodo q da árvore de pesquisa, a solução de SP_q constitui um limite superior para o valor da solução ótima inteira que é possível obter nesse nodo, e em qualquer um dos nodos descendentes. Se a solução de SP_q satisfizer todas as restrições de integralidade do modelo original, uma de duas situações poderá ocorrer:
 - o valor da solução de SP_q é menor ou igual do que o valor da melhor solução válida inteira obtida até ao momento (a *solução incumbente*), e nesse caso, o nodo é simplesmente abandonado;

- o valor da solução de SP_q é maior do que o valor da solução incumbente: a solução incumbente passa a ser a solução de SP_q , e o nodo é também abandonado.

Em qualquer um dos casos, não há lugar a partição uma vez que nenhuma das soluções em nodos descendentes será melhor que a solução de SP_q . Um nodo da árvore de pesquisa é abandonado sempre que exista a garantia que o seu espaço de soluções não contém a solução ótima do modelo de Programação Inteira original.

Quando a solução de SP_q não satisfaz as restrições de integralidade originais, o nodo pode ser abandonado apenas se o limite superior associado à solução de SP_q for inferior ao valor da solução incumbente. À semelhança do que acontecia anteriormente, nenhuma das soluções nos subproblemas gerados a partir de q poderá jamais ser melhor do que essa solução incumbente.

Finalmente, quando num nodo da árvore de pesquisa, o subproblema correspondente é impossível, obviamente, o nodo é abandonado.

A solução incumbente é necessariamente uma solução válida para o modelo de Programação Inteira original. O valor dessa solução constitui um limite inferior para o valor da solução ótima desse modelo. Só é possível afirmar que a solução incumbente é ótima para o modelo original se o seu valor coincidir com um limite superior para o valor do ótimo. É apenas quando isso acontece que o método de partição e avaliação termina.

É possível determinar soluções válidas para o modelo original usando diferentes abordagens. Nos nodos da árvore de pesquisa, por exemplo, as soluções fraccionárias das relaxações lineares podem ser arredondadas de modo a que nenhuma das restrições originais seja violada. Uma solução válida pode também ser construída usando um processo heurístico. Em todo o caso, é importante notar que a qualidade das soluções incumbentes pode ter um impacto significativo na convergência do método, assim como têm os limites superiores gerados a partir dos subproblemas, dado que é com base nesses valores que é possível rejeitar nodos, e assim evitar a enumeração de muitas soluções.

O exemplo seguinte ilustra o funcionamento do método de partição e avaliação aplicado a um pequeno modelo de Programação Inteira pura.

Exemplo 3.14 Considere novamente o modelo de Programação Inteira que usámos no Exemplo 3.11:

$$\begin{aligned}
\max \quad & z = 11x_1 + 9x_2 \\
s. \quad & a \\
& 7x_1 + 10x_2 \leq 70 \\
& 10x_1 + 3x_2 \leq 65 \\
& x_2 \leq 5 \\
& x_1, x_2 \geq 0 \text{ e inteiros.}
\end{aligned}$$

O espaço de soluções válidas desse modelo está representado na Figura 3.7. Os pontos a negro representam as soluções válidas do modelo de Programação Inteira. Essas soluções estão contidas no espaço de soluções da relaxação linear do modelo. A solução ótima da relaxação linear corresponde ao ponto branco assinalado na Figura 3.7. Neste caso, para obter a solução ótima inteira (ponto a negro cercado por outro círculo na Figura 3.7), bastaria arredondar para baixo o valor das variáveis fracionárias.

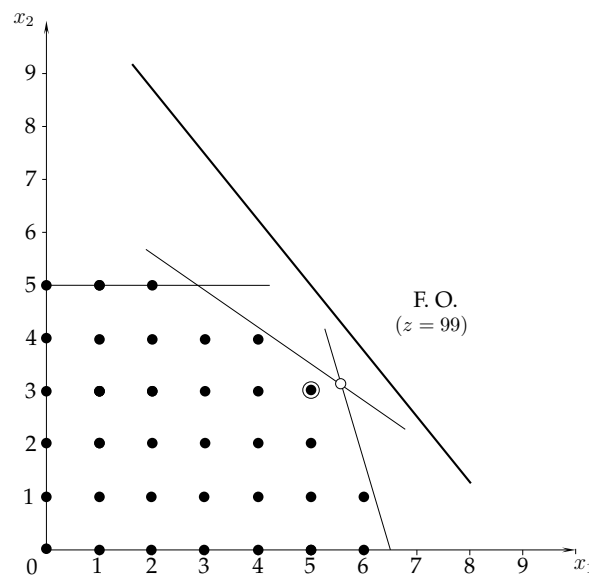


Figura 3.7: Representação gráfica (modelo do Exemplo 3.14)

Um limite inferior para o valor da solução ótima deste modelo pode ser obtido através da solução trivial seguinte:

$$(x_1, x_2) = (0, 0).$$

Vamos designar por z^{inc} o valor da solução incumbente. Neste caso, temos que $z^{inc} = 0$. Qualquer outro ponto do espaço de soluções inteiras válidas seria melhor do que o ponto $(0, 0)$. Quando é difícil determinar uma solução válida para um modelo de Programação

Inteira, podemos inicializar o valor de z^{inc} fazendo $z^{inc} = -\infty$ ($z^{inc} = +\infty$ para problemas de minimização). Vamos designar por U o valor do limite superior para o valor da solução óptima inteira. Neste momento, temos que $U = +\infty$.

A solução óptima da relaxação linear tem um valor igual a 89.18. Esse valor permite-nos actualizar o limite superior para o valor do óptimo inteiro. Dado que todas as variáveis do modelo original são inteiras, assim como os coeficientes do modelo, temos que $U = 89$, *i.e.* a solução óptima inteira está contida no intervalo $[0, 89]$. A solução da relaxação linear consiste nos seguintes valores das variáveis de decisão:

$$(x_1, x_2) \approx (5.57, 3.1).$$

Claramente, a solução não é válida para o modelo original uma vez que é composta por variáveis fraccionárias. O nodo também não pode ser rejeitado com base no valor do seu limite superior.

Vamos usar a seguinte regra de partição:

- as restrições de partição baseiam-se numa única variável;
- a variável que é escolhida é aquela cuja parte fraccionária é a mais elevada;
- se x_i for a variável escolhida, são criados dois subproblemas: um em que é imposta a restrição adicional $x_i \leq \lfloor x_i \rfloor$, e outro com a restrição $x_i \geq \lceil x_i \rceil$.

Essa regra de partição dá origem aos nodos ilustrados na Figura 3.8.

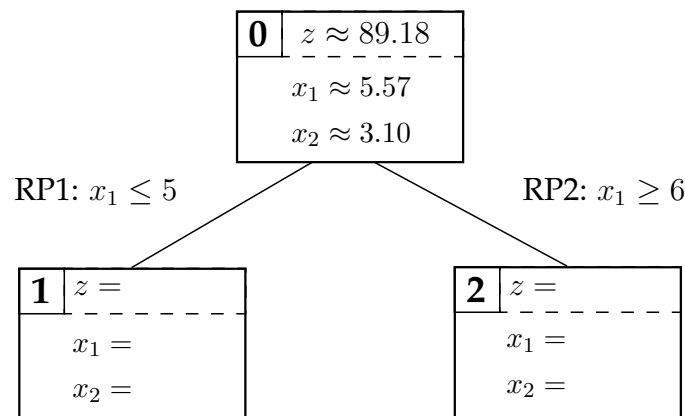


Figura 3.8: Árvore de pesquisa (Exemplo 3.14)

Cada uma das restrições de partição divide o espaço de soluções válidas da relaxação linear do modelo original conforme ilustrado na Figura 3.9. Note que o espaço de soluções do subproblema 2 (à direita da recta RP2) é bem mais pequeno do que o

espaço de soluções do subproblema 1, o que pode dar origem a uma árvore de pesquisa desequilibrada em que um dos ramos contém muitos mais nodos do que o outro. Esse fenómeno pode ter consequências negativas na convergência do método. Em alguns casos, ao escolher o ramo com mais nodos, podemos ser obrigados a resolver um número considerável de problemas de Programação Linear.

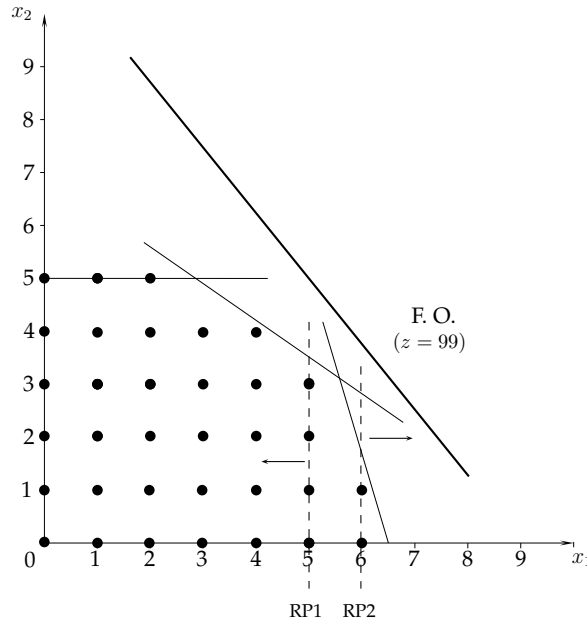


Figura 3.9: Representação gráfica (modelo do Exemplo 3.14)

Uma das questões que se levanta no método de partição e avaliação é a forma como devem ser percorridos os nodos da árvore. No nosso caso, vamos optar por escolher sempre o nodo que esteja mais à esquerda e no maior nível de profundidade. O próximo subproblema a resolver corresponde assim ao nodo 1. A solução desse subproblema é a seguinte:

$$(x_1, x_2) = (5, 3.5).$$

O valor dessa solução é igual a 86.5. Note que, neste caso, não é possível actualizar o valor de U dado que não conhecemos ainda o valor do limite superior associado ao subproblema 2. Também não é possível rejeitar o nodo dado que

$$86.5 > z^{inc} = 0.$$

Por essas razões, prosseguimos com a partição que resulta nos nodos ilustrados na Figura 3.10. A partição é feita com base na única variável que é fraccionária (x_2).

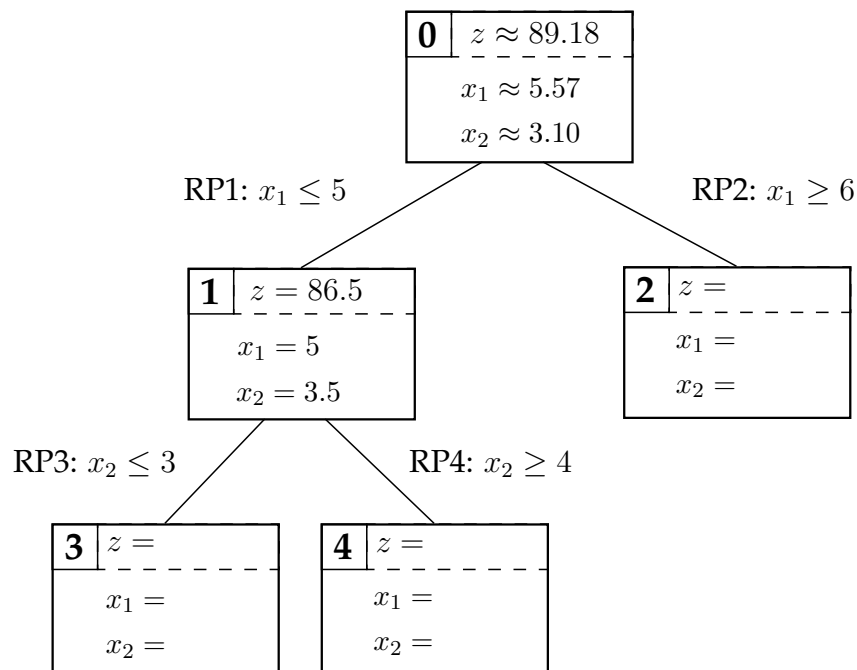


Figura 3.10: Árvore de pesquisa (Exemplo 3.14)

As restrições de partição RP3 e RP4 dividem o espaço de soluções conforme indicado na Figura 3.11. Note que os pontos extremos do subproblema 3 são todos pontos inteiros. A solução ótima desse subproblema é necessariamente uma solução inteira.

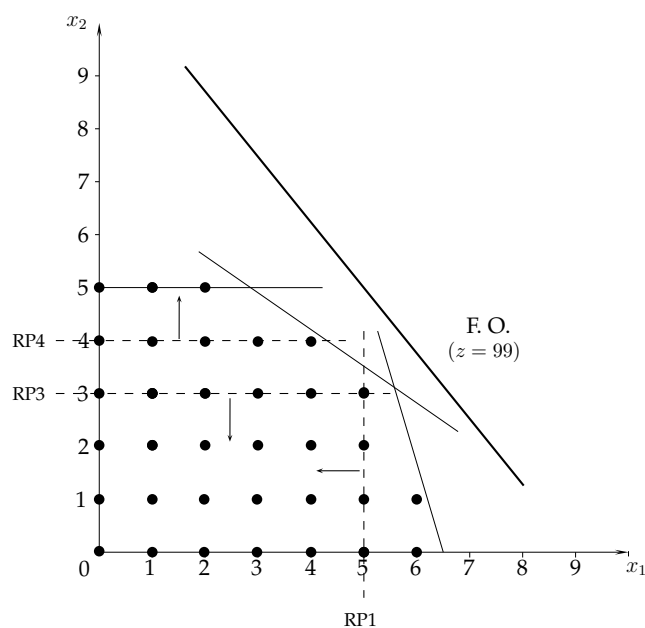


Figura 3.11: Representação gráfica (modelo do Exemplo 3.14)

Atendendo à regra de pesquisa que definámos atrás, o próximo nodo a ser explorado corresponde ao subproblema 3. A solução óptima desse subproblema é de facto inteira, e corresponde aos seguintes valores das variáveis de decisão:

$$(x_1, x_2) = (5, 3).$$

Essa solução satisfaz portanto todas as restrições do modelo de Programação Inteira original. O valor dessa solução é igual a 82. Uma vez que

$$82 > z^{inc} = 0,$$

e a respectiva solução é válida, podemos actualizar a solução incumbente que passa a ter o valor $z^{inc} = 82$. Neste momento, podemos então afirmar que a solução óptima do modelo original se encontra no intervalo $[82, 89]$.

O próximo nodo a ser explorado é corresponde ao subproblema 4. A solução desse subproblema é a seguinte

$$(x_1, x_2) \approx (4.29, 4),$$

e tem um valor aproximadamente igual a 83.14. O nodo não pode ser abandonado, dado que ainda pode ser possível encontrar uma solução válida melhor do que a solução incumbente. A partição é feita na variável x_1 , resultando na árvore ilustrada na Figura 3.12.

No subproblema 6, o valor da variável x_1 é fixado a 5 (RP1 e RP6). Na representação gráfica da Figura 3.13, podemos observar que a restrição RP6 não é compatível com a restrição RP4. O subproblema 6 é impossível.

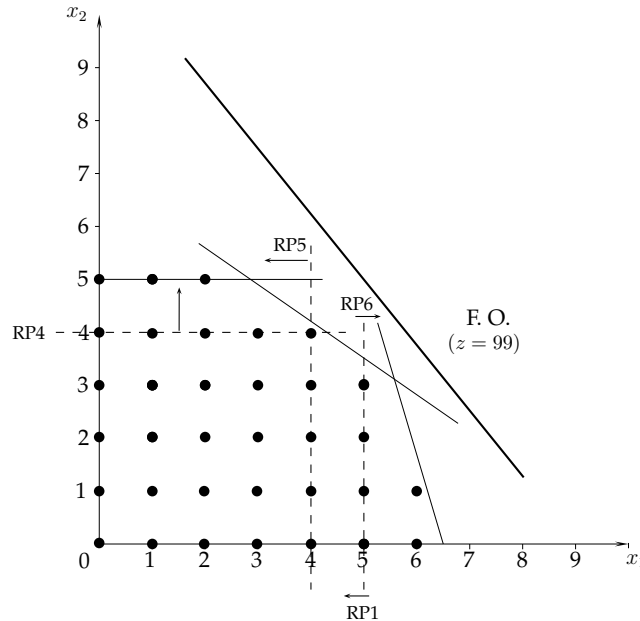


Figura 3.13: Representação gráfica (modelo do Exemplo 3.14)

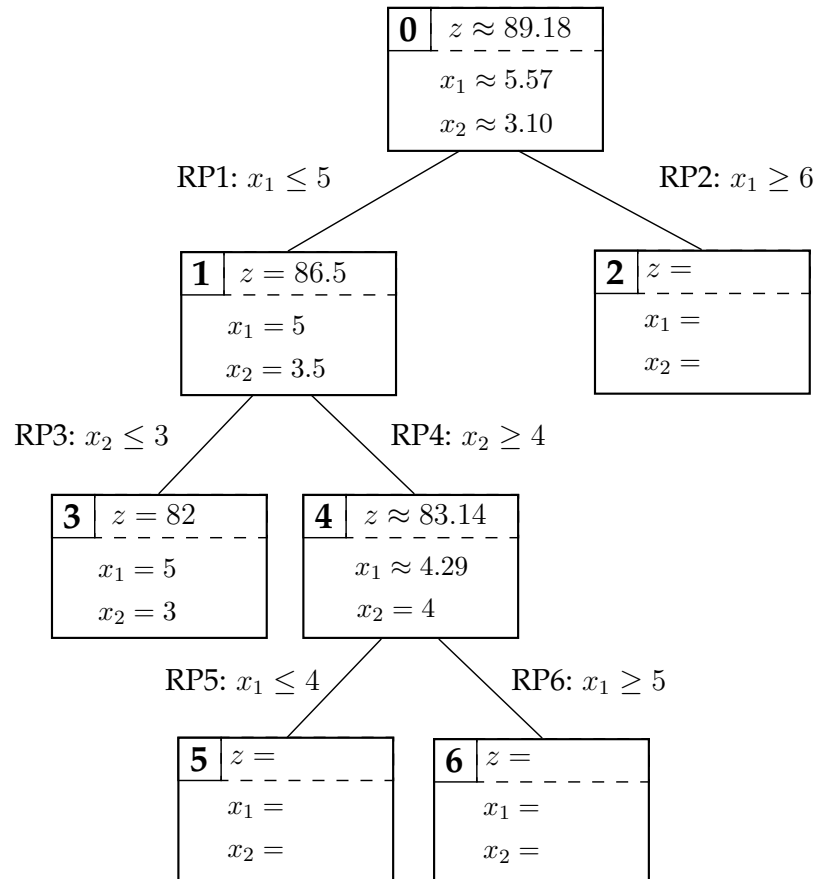


Figura 3.12: Árvore de pesquisa (Exemplo 3.14)

O valor da solução do subproblema 5 é igual a 81.8, sendo inferior ao valor da solução incumbente actual. Claramente, não existe nenhuma solução nesse nodo ou em qualquer um dos seus descendentes que possa ser melhor do que a actual solução incumbente. O nodo é por isso abandonado. Note que, nesta fase do método, o valor de U continua a ser igual a 89, uma vez que o subproblema 2 ainda pode gerar soluções de valor igual a 89.

O próximo nodo a ser explorado é o nodo 2. A solução do subproblema é a seguinte:

$$(x_1, x_2) \approx (6, 1.67),$$

e tem um valor igual a 81. Qualquer solução num nodo abaixo do nodo 2 terá sempre um valor inferior ou igual a 81. O valor de U pode ser por isso actualizado, passando a ser igual a 82, o valor da actual incumbente. Nesta fase, temos a prova que a solução incumbente é óptima uma vez que é impossível obter uma solução válida cujo valor seja superior a 82. O método de partição e avaliação termina nesse ponto. A Figura 3.14 completa a árvore de pesquisa.

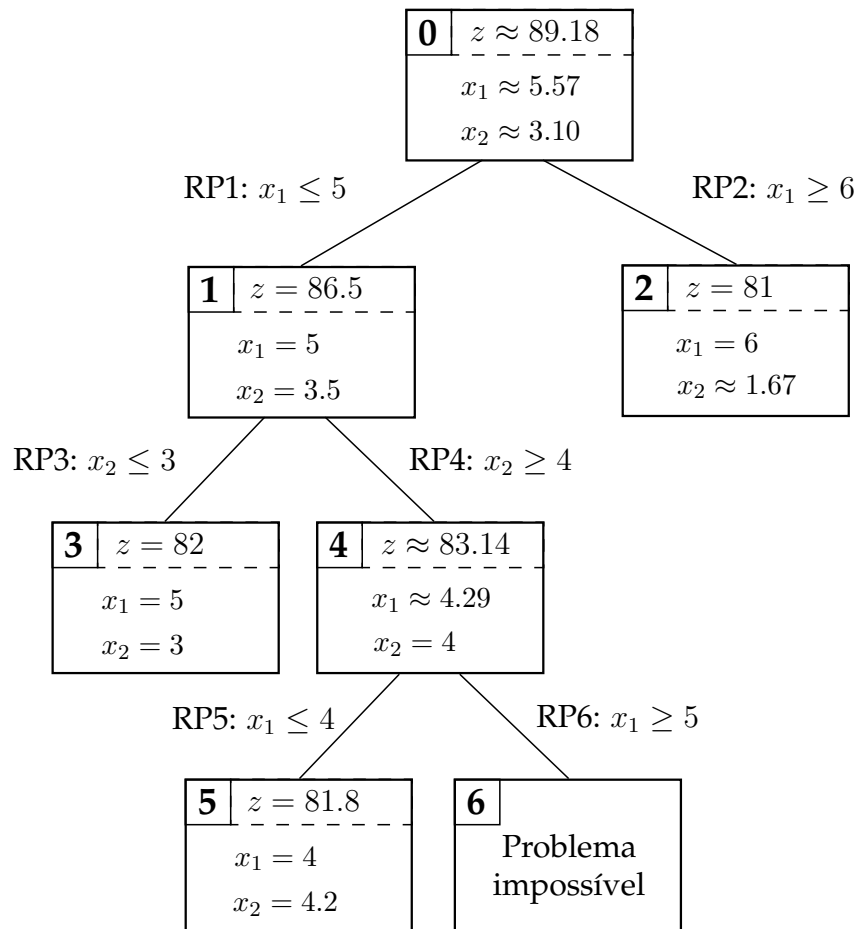


Figura 3.14: Árvore de pesquisa (Exemplo 3.14)

Nos nodos da árvore de pesquisa, os subproblemas são problemas de Programação Linear que podem ser resolvidos usando qualquer um dos métodos de Programação Linear que analisámos no Capítulo 2. De seguida, apresentamos os quadros Simplex associados à relaxação linear e à solução óptima do modelo de Programação Inteira que usámos neste Exemplo.

	z	x_1	x_2	F_1	F_2	F_3	
F_1	0	7	10	1	0	0	70
F_2	0	10	3	0	1	0	65
F_3	0	0	1	0	0	1	5
z	1	-11	-9	0	0	0	0

	z	x_1	x_2	F_1	F_2	F_3	
x_2	0	0	1	$\frac{10}{79}$	$-\frac{7}{79}$	0	$\frac{245}{79}$
x_1	0	1	0	$-\frac{3}{79}$	$\frac{10}{79}$	0	$\frac{440}{79}$
F_3	0	0	0	$-\frac{10}{79}$	$\frac{7}{79}$	1	$\frac{150}{79}$
z	1	0	0	$\frac{57}{79}$	$\frac{47}{79}$	0	$\frac{7045}{79}$

A restrição de partição RP1 a partir da qual é definido o subproblema 1 pode ser introduzida no quadro Simplex inicial fazendo :

$$x_1 + F_4 = 5.$$

	z	x_1	x_2	F_1	F_2	F_3	F_4	
x_2	0	0	1	$\frac{10}{79}$	$-\frac{7}{79}$	0	0	$\frac{245}{79}$
x_1	0	1	0	$-\frac{3}{79}$	$\frac{10}{79}$	0	0	$\frac{440}{79}$
F_3	0	0	0	$-\frac{10}{79}$	$\frac{7}{79}$	1	0	$\frac{150}{79}$
F_4	0	1	0	0	0	0	1	5
z	1	0	0	$\frac{57}{79}$	$\frac{47}{79}$	0	0	$\frac{7045}{79}$

Efectuando operações elementares sobre linhas do quadro, chegamos ao seguinte quadro válido.

	z	x_1	x_2	F_1	F_2	F_3	F_4	
x_2	0	0	1	$\frac{10}{79}$	$-\frac{7}{79}$	0	0	$\frac{245}{79}$
x_1	0	1	0	$-\frac{3}{79}$	$\frac{10}{79}$	0	0	$\frac{440}{79}$
F_3	0	0	0	$-\frac{10}{79}$	$\frac{7}{79}$	1	0	$\frac{150}{79}$
F_4	0	0	0	$\frac{3}{79}$	$-\frac{10}{79}$	0	1	$-\frac{45}{79}$
z	1	0	0	$\frac{57}{79}$	$\frac{47}{79}$	0	0	$\frac{7045}{79}$

Finalmente, o método Simplex Dual permite nos obter a solução óptima representada no quadro abaixo.

	z	x_1	x_2	F_1	F_2	F_3	F_4	
x_2	0	0	1	$\frac{1}{10}$	0	0	$\frac{7}{79}$	$\frac{7}{2}$
x_1	0	1	0	0	0	0	1	5
F_3	0	0	0	$-\frac{1}{10}$	0	1	$\frac{7}{10}$	$\frac{3}{2}$
F_2	0	0	0	$-\frac{3}{10}$	1	0	$-\frac{79}{10}$	$\frac{45}{10}$
z	1	0	0	9	0	0	$\frac{47}{10}$	$\frac{13667}{158}$

□

O método de partição e avaliação aplicado a modelos de Programação Inteira com variáveis binárias não difere daquele que acabamos de apresentar. Na relaxação linear de um modelo desse tipo, as variáveis binárias tomam sempre valores compreendidos entre 0 e 1. Quando uma restrição de partição é definida com base numa dessas variáveis, essa restrição acaba por reduzir-se à fixação do valor da variável. Suponha, por exemplo, que uma variável binária x_i tem um valor fraccionário na solução da relaxação linear, e que as restrições de partição são definidas com base em apenas uma variável. As restrições de partição baseadas nessa variável serão as seguintes:

$$x_1 \leq 0 \quad \text{e} \quad x_1 \geq 1,$$

que por sua vez são equivalentes a

$$x_1 = 0 \quad \text{e} \quad x_1 = 1.$$

O método de partição e avaliação é um método geral de optimização, em que alguns elementos podem ser ajustados. Um desses aspectos prende-se com a regra que é usada para escolher o próximo nodo a explorar. No Exemplo 3.14, os nodos foram escolhidos segundo o seu nível de profundidade. Uma pesquisa em profundidade privilegia a obtenção de soluções válidas que possam melhorar a solução incumbente. Quanto mais profundo estiver um nodo, maior será o número de restrições de partição que lhe terão sido impostas, e maior será a probabilidade da solução respeitar as restrições de integralidade do modelo original.

A árvore de pesquisa também pode ser percorrida em largura: os subproblemas de um nível inferior só são resolvidos depois de terem sido resolvidos todos os subproblemas do nível superior. Se no Exemplo 3.14 tivesse sido usada uma pesquisa em largura, os nodos teriam sido visitados pela ordem seguinte: 0 – 1 – 2 – 3 – 4 – 5 – 6 (em vez de 0 – 1 – 3 – 4 – 5 – 6 – 2). Uma pesquisa em largura privilegia a melhoria do limite superior para o valor do óptimo do modelo de Programação Inteira original: uma vez resolvidos todos os subproblemas de um determinado nível, o limite superior é igual ao maior valor obtido nos subproblemas desse nível. Uma vez que vão sendo impostas restrições de partição à medida que se avança para níveis inferiores, é natural que o limite superior diminua gradualmente.

O próximo nodo a ser explorado pode também ser escolhido tendo em atenção o valor do seu limite. Em problemas de maximização, escolher-se-á o nodo com o maior limite superior. No Exemplo 3.14, essa regra faria com que os nodos fossem visitados pela ordem seguinte: 0 – 1 – 2 – 3 – 4 – 5 – 6. Com essa regra de pesquisa, procura-se determinar soluções válidas cujo valor seja o mais próximo do limite superior.

No Exemplo 3.14, vimos que a regra de partição era outro dos elementos que podia afectar a convergência do método de partição e avaliação. Actualmente, não é conhecida nenhuma regra que garante a resolução eficiente de qualquer modelo de Programação Inteira. Uma abordagem pode passar por atribuir prioridades às variáveis. Essas prioridades são usadas para seleccionar a variável de partição. As variáveis com maior prioridade são aquelas que potencialmente maior impacto têm na integralidade da solução final. Outra abordagem consiste em tentar estimar o impacto que uma variável de partição pode ter no valor da função objectivo dos nodos descendentes. Quanto maior for esse impacto, maior será a probabilidade de encontrar uma solução inteira rapidamente. Na prática, o esforço computacional que é necessário para medir esse impacto torna essa abordagem pouco atractiva.

3.6.2 Métodos Baseados em Planos de Corte

Os cortes de Gomory que introduzimos na Secção 3.5 podem ser usados num algoritmo de resolução para modelos de Programação Inteira. O procedimento inicia-se com a resolução da relaxação linear do modelo, e o cálculo de planos de corte a partir da solução válida de base óptima dessa relaxação. Os cortes são adicionados à relaxação linear que é novamente resolvida. O processo é repetido até ser encontrada uma solução que cumpra todas as restrições de integralidade.

No caso dos cortes fraccionários de Gomory, é possível provar que o processo converge após um número finito de iterações. Já no caso dos cortes inteiros mistos de Gomory, o mesmo não sucede no caso geral. O Algoritmo 4.1 resume os passos gerais do método no caso em que são usados cortes fraccionários de Gomory. O Exemplo 3.15 ilustra o funcionamento do método.

Exemplo 3.15 O modelo do Exemplo 3.14 pode ser resolvido usando um método de planos de corte baseado nos cortes fraccionários de Gomory. Partindo do quadro óptimo da relaxação linear desse modelo, é possível derivar o seguinte corte fraccionário usando a linha do quadro associada à variável x_1 :

$$\frac{76}{79}F_1 + \frac{10}{79}F_2 \geq \frac{45}{79}.$$

Se adicionarmos o corte ao quadro óptimo da relaxação linear, obtemos o quadro Simplex seguinte:

Algoritmo 4.1: Método baseado em Cortes Fraccionários de Gomory

```

1 Input:  $P$  (modelo de Programação Inteira pura) ;
2 Resolver a relaxação linear de  $P$  ;
3 se não existir nenhuma solução válida de base então
4   | Output: “Modelo não tem soluções válidas” ;
5 senão
6   | Seja  $x_{RL}^*$  a solução ótima da relaxação linear, e  $z_{RL}^*$  o valor dessa solução ;
7   |  $s:=1$ , continuar:=verdadeiro ;
8   | Enquanto continuar fazer
9     | se  $x^*$  é inteiro então
10      |    $x^* := x_{RL}^*$ ,  $z^* := z_{RL}^*$  ;
11      |   continuar:=falso;
12      | senão
13        | Escolher uma variável básica  $x_i$  cujo valor seja fraccionário em  $x^*$ ;
14        | Seja  $l$  o índice da linha correspondente (quadro Simplex);
15        | Seja  $x_i + \sum_{j \in NB} \bar{a}_{lj} x_j = \bar{b}_l$  a expressão dessa variável em ordem às
16        | variáveis não-básicas ( $NB$ : conjunto dos índices dessas variáveis) ;
17        | Adicionar o corte fraccionário de Gomory
18        |  $\sum_{j \in NB} (\bar{a}_{lj} - [\bar{a}_{lj}]) x_j - E_s = \bar{b}_l - [\bar{b}_l]$  ao conjunto de restrições da
19        | última relaxação linear ;
20        | Resolver a relaxação linear com esse novo corte (solução:  $x_{RL}^*$ ,  $z_{RL}^*$ );
21        | se não existir nenhuma solução válida de base então
22          |   continuar:=falso ;
23          |    $z^* := -\infty$  ;
24          | senão
25            |    $s:=s+1$  ;
26          |
27 se  $z^* = -\infty$  então
28   | Output: “Modelo não tem soluções válidas” ;
29 senão
30   | Output: “Solução ótima:”  $x^*$  “com valor igual a”  $z^*$  ;
31 Stop ;

```

	z	x_1	x_2	F_1	F_2	F_3	E_1	
x_2	0	0	1	$\frac{10}{79}$	$-\frac{7}{79}$	0	0	$\frac{245}{79}$
x_1	0	1	0	$-\frac{3}{79}$	$\frac{10}{79}$	0	0	$\frac{440}{79}$
F_3	0	0	0	$-\frac{10}{79}$	$\frac{7}{79}$	1	0	$\frac{150}{79}$
E_1	0	0	0	$-\frac{76}{79}$	$-\frac{10}{79}$	0	1	$-\frac{45}{79}$
z	1	0	0	$\frac{57}{79}$	$\frac{47}{79}$	0	1	$\frac{7045}{79}$

A solução ótima é obtida usando o método Simplex dual.

	z	x_1	x_2	F_1	F_2	F_3	E_1	
x_2	0	0	1	0	$-\frac{4}{38}$	0	$\frac{5}{38}$	$\frac{115}{38}$
x_1	0	1	0	0	$\frac{5}{38}$	0	$-\frac{3}{76}$	$\frac{425}{76}$
F_3	0	0	0	0	$\frac{2}{19}$	1	$-\frac{5}{38}$	$\frac{75}{38}$
F_1	0	0	0	0	$\frac{5}{38}$	0	$-\frac{79}{76}$	$\frac{45}{76}$
z	1	0	0	0	$\frac{1}{2}$	0	$\frac{57}{76}$	$\frac{355}{4}$

Esse corte é representado na Figura 3.15. Como se pode observar, o corte mesmo não sendo muito forte, é suficiente para excluir a solução da relaxação linear.

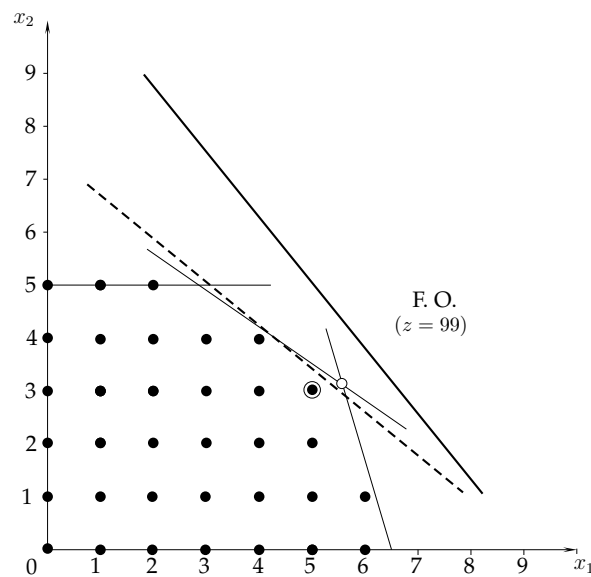


Figura 3.15: Corte fraccionário de Gomory

Repetindo o processo (derivar um novo corte fraccionário, adicioná-lo à relaxação linear, e resolver de novo essa relaxação), a solução óptima inteira acabaria por ser encontrada após um número finito de iterações. $\square$

3.7 Conclusões

A Programação Inteira permite formular e resolver muitos problemas reais de optimização. Na prática, muitos desses problemas possuem incógnitas que são quantidades inteiras, ou restrições que não podem ser formuladas apenas com variáveis contínuas. Neste capítulo, analisámos os fundamentos das técnicas de modelação e resolução de Programação Inteira. Abordámos a questão da qualidade dos modelos, e vimos como era possível melhorar a qualidade de um modelo reformulando algumas das suas partes, ou introduzindo restrições adicionais. A qualidade dos modelos é importante uma vez que condiciona a eficiência dos métodos de resolução.

A Programação Inteira continua a ser uma área activa de investigação. Um dos desenvolvimentos mais recentes e promissores consiste em integrar métodos de Programação Inteira e de Programação por Restrições (Inteligência Artificial). As duas abordagens possuem características complementares. Como vimos, os primeiros assentam na formulação de modelos fortes obtidos através de técnicas sofisticadas de modelação e em técnicas de reforço de modelos. O inconveniente desses métodos é o facto da sua eficiência depender em grande parte da estrutura dos problemas. Quando os problemas têm restrições adicionais que lhes fazem perder a sua estrutura, os métodos de Programação Inteira podem ter mais dificuldades em resolvê-los. A Programação por Restrições baseia-se em regras de inferência lógica e na propagação de restrições para determinar soluções que satisfaçam um conjunto de restrições. A Programação por Restrições permite determinar de forma eficiente soluções válidas para problemas cujas restrições não são fáceis de formular num modelo de Programação Inteira.

3.8 Exercícios

1. A companhia ACOMPANHIA SA dedica-se ao fabrico de dois artigos: o A e o B. Para produzir esses artigos, é necessário executar um conjunto de operações em máquinas diferentes. O tempo (em horas) necessário em cada máquina para produzir uma unidade de cada artigo é indicado na tabela seguinte:

	Máquina			
	1	2	3	4
Artigo A	2	2	3	0
Artigo B	0	7	5	3

As máquinas são alugadas mensalmente pela companhia. O custo mensal do aluguer (em euros), e o número máximo de horas em que uma máquina está disponível num mês, estão indicados na tabela seguinte:

	Máquina			
	1	2	3	4
Custo do aluguer	350	450	400	100
Disponibilidade mensal	100	150	220	110

O preço unitário de venda dos artigos A e B é respectivamente de 65 e 85 euros.

- a) Formule este problema usando um modelo de Programação Inteira.
 - b) Determine a solução óptima usando um *software* de optimização.
2. Uma companhia pretende recrutar 3 novos colaboradores. Entre as várias candidaturas que recebeu, 6 foram consideradas aceitáveis. No formulário de candidatura, a companhia pedia aos concorrentes que indicassem o salário anual que pretendiam auferir. Nas respostas que recebeu, os candidatos com um maior nível de qualificações e maior experiência tendiam a pedir os salários mais elevados. A tabela seguinte indica o montante dos salários pedidos (em milhares de euros/ano):

	Salário anual
Candidato 1	42
Candidato 2	35
Candidato 3	40
Candidato 4	45
Candidato 5	30
Candidato 6	50

Após ter realizado várias entrevistas e uma série de testes psico-técnicos, a companhia concluiu que deveriam ser seguidas algumas regras na contratação dos novos colaboradores de modo a garantir a maior coesão possível da futura equipa. Essas regras traduzem-se nas seguintes restrições:

- i. se os candidatos 2 e 5 forem contratados, o candidato 6 não poderá ser contratado;
- ii. se o candidato 2 for contratado, o candidato 3 também deverá ser contratado;
- iii. o candidato 1 está em conflito com os candidatos 3 e 5, pelo que não deverá ser contratado se alguns destes últimos forem contratados.

A companhia pretende ainda garantir um nível mínimo de competências para a futura equipa, e assim:

- i. dos candidatos 1, 4 e 6, pelo menos um deverá ser contratado;
- ii. se o candidato 5 for contratado, o candidato 6 também deverá ser contratado.

Formule um modelo de Programação Inteira que permita determinar o plano óptimo de contratações.

3. A famosa companhia de táxis ANDACA pretende redistribuir a sua rede de centrais pela cidade de Braga para melhorar o serviço que presta às populações de algumas freguesias dessa cidade. As freguesias em questão são S. Vítor, S. Lázaro, Gualtar, Fraião, Nogueira e Nogueiró. Cada uma dessas freguesias é também candidata à instalação de uma central de táxis. Para aumentar a visibilidade da companhia, a ANDACA quer evitar que haja um número excessivo de centrais abertas em freguesias limítrofes, e impõe assim as seguintes restrições:

- i. as freguesias de S. Vítor e S. Lázaro não poderão ser escolhidas em simultâneo;
- ii. se for aberta uma central em Nogueiró, não poderá ser aberta outra central em Gualtar.

Para conter os custos de arrendamento dos espaços, a ANDACA não quer abrir mais de 3 centrais.

A ANDACA considera que está a prestar um bom serviço às populações se for capaz de colocar um dos seus táxis em menos de 8 minutos no local onde se encontra o cliente. A tabela seguinte indica os tempos de deslocamento (em minutos) nas horas de ponta entre as freguesias de Braga.

	S. Vítor	S. Lázaro	Gualtar	Fraião	Nogueira	Nogueiró
S. Vítor	0	5	10	5	10	7
S. Lázaro		0	10	7	7	10
Gualtar			0	10	10	7
Fraião				0	5	7
Nogueira					0	7
Nogueiró						0

Assuma que existe sempre pelo menos um táxi disponível em qualquer uma das centrais abertas.

Formule este problema usando um modelo de Programação Inteira.

4. O ano lectivo está prestes a começar, e o Pedro, um estudante universitário do 1º ano de Mestrado em Engenharia, quer escolher os turnos práticos aos quais irá assitir. Motivado com tudo o que aprendeu no ano anterior na disciplina de Investigação Operacional, o Pedro quer agora aplicar esses conhecimentos para tomar as melhores decisões.

Este semestre, o Pedro irá ter 4 unidades curriculares: Simulação, Sistemas de Apoio à Decisão, Gestão da Produção e Logística. Face ao número de alunos inscritos, foram abertos 4 turnos práticos para cada uma dessas UCs. Os horários desses turnos estão indicados na tabela seguinte:

	Simulação	Sistemas de Apoio à Decisão	Gestão da Produção	Logística
Turno 1	2ªf, 9h-11h	3ªf, 9h-11h	2ªf, 9h-11h	2ªf, 14h-16h
Turno 2	2ªf, 11h-13h	3ªf, 11h-13h	2ªf, 11h-13h	3ªf, 11h-13h
Turno 3	2ªf, 16h-18h	3ªf, 14h-16h	2ªf, 14h-16h	3ªf, 14h-16h
Turno 4	3ªf, 9h-11h	3ªf, 16h-18h	2ªf, 16h-18h	3ªf, 16h-18h

O Pedro prefere alguns turnos a outros. Para poder ter em conta esse factor na tomada de decisão, o Pedro classificou de 0 a 20 cada um dos turnos disponíveis conforme ilustrado na tabela seguinte:

	Simulação	Sistemas de Apoio à Decisão	Gestão da Produção	Logística
Turno 1	7	15	20	10
Turno 2	12	20	20	15
Turno 3	20	10	10	15
Turno 4	15	7	20	12

O Pedro, grande apreciador de música erudita, frequenta também as aulas de canto medieval do Conservatório. No próximo semestre, essas aulas serão à 2^af e à 3^af das 17h às 18h. O Pedro quer assistir a pelo menos uma aula por semana.

O Pedro acredita que muitas aulas no mesmo dia irão certamente prejudicar a assimilação das matérias. Como tal, não quer assistir a mais de duas unidades curriculares no mesmo dia.

Formule o problema de selecção de horários do Pedro usando um modelo de Programação Inteira.

5. Uma companhia produz 4 artigos diferentes, e dispõe para esse efeito de 7 máquinas. Cada máquina pode produzir apenas um subconjunto de artigos. A tabela seguinte identifica os artigos que podem ser produzidos por cada uma das máquinas:

	Máquina						
	1	2	3	4	5	6	7
Artigo 1	×		×		×		
Artigo 2	×	×		×	×	×	
Artigo 3		×		×			×
Artigo 4		×	×		×	×	×

As máquinas têm todas a mesma capacidade produtiva. A capacidade das máquinas expressa em termos do número máximo de artigos de cada tipo que podem ser produzidos diariamente é indicada na tabela seguinte:

	Capacidade das máquinas (unidades/dia)
Artigo 1	18
Artigo 2	9
Artigo 3	12
Artigo 4	6

As máquinas podem produzir vários artigos no mesmo dia. Assim, se a máquina 3, por exemplo, produzir 9 unidades do artigo 1 num determinado dia, essa máquina poderá ainda produzir até 3 unidades do artigo 4.

Assuma que a procura de cada um dos artigos para o dia seguinte é igual respectivamente a 5, 15, 9 e 12 unidades, e que o objectivo da companhia é de minimizar o número de máquinas que são efectivamente usadas.

- a) Formule um modelo de Programação Inteira que permita determinar as máquinas que devem ser usadas e o número de artigos que deve ser produzido em cada máquina.
 - b) Determine a solução óptima usando um *software* de optimização.
6. Considere o caso de uma companhia que pretende determinar os locais onde instalar os seus armazéns. Existem 4 locais candidatos designados por A, B, C e D. Os custos fixos associados aos locais variam conforme indicado na tabela seguinte:

	Custo fixo (em milhares de euros/mês)
Local A	17
Local B	23
Local C	12
Local D	30

A companhia tem actualmente 5 clientes em vários pontos do país. O custo de servir um cliente a partir de um armazém depende da distância que separa o armazém do cliente, e da quantidade que é fornecida. A tabela seguinte indica o custo de satisfazer toda a procura de um cliente a partir de cada um locais candidatos:

	Cliente				
	1	2	3	4	5
Local A	25	20	22	12	14
Local B	18	18	25	15	10
Local C	30	15	30	22	12
Local D	12	14	20	10	15

Assuma que a procura de um cliente pode ser satisfeita a partir de vários armazéns.

- a) Formule este problema usando um modelo de Programação Inteira.
 - b) Determine a solução óptima usando um *software* de optimização.
7. A SOBRACEL é uma empresa nacional que opera no sector da indústria do papel. A empresa produz rolos de dimensões standard com 2 metros de largura e 100 metros de comprimento. Essas dimensões são suficientes para satisfazer a procura específica de todos os seus clientes. Quando um cliente pede um rolo com uma largura inferior à do rolo standard, a SOBRACEL satisfaz o seu pedido cortando o rolo standard de forma adequada.

A empresa acaba de receber uma encomenda de 4 clientes diferentes. A largura dos rolos que foram pedidos pelos clientes, e o número de rolos pedidos, são indicados na tabela seguinte:

	Largura do rolo (em metros)	Quantidade encomendada
Cliente 1	120	25
Cliente 2	80	150
Cliente 3	70	90
Cliente 4	50	130

A SOBRACEL pretende determinar a forma como deve cortar os rolos de modo a minimizar o desperdício total que é gerado.

- a) Formule este problema usando um modelo de Programação Inteira.
- b) Determine a solução óptima usando um *software* de optimização.

8. Considere o caso de uma companhia que, para produzir os seus artigos, tem de executar 4 tarefas distintas. Uma tarefa é constituída por um conjunto de subtarefas que correspondem a vários processamentos nas máquinas disponíveis. A companhia dispõe de 5 máquinas. Uma tarefa é completada apenas quando a sua última subtarefa é realizada. O processamento das subtarefas nas máquinas é feito por ordem crescente do índice das máquinas, *i.e.* uma subtarefa só pode ser iniciada numa máquina j quando todas as restantes subtarefas (da mesma tarefa) tiverem sido completadas nas máquinas $i < j$.

O tempo de processamento (em minutos) das subtarefas nas máquinas é indicado na tabela seguinte:

	Máquina				
	1	2	3	4	5
Tarefa 1	15	—	10	20	10
Tarefa 2	10	25	—	5	30
Tarefa 3	12	5	10	—	20
Tarefa 4	5	20	10	20	15

Sabendo que uma máquina só pode processar uma subtarefa de cada vez, e que, uma vez iniciada, uma subtarefa não pode ser interrompida, pretende-se determinar a forma como devem ser processadas as subtarefas nas máquinas de modo a minimizar o instante de conclusão de todas as tarefas.

- Formule este problema usando um modelo de Programação Inteira.
 - Determine a solução óptima usando um *software* de optimização.
9. A estação de rádio Ondas no Neiva quer começar a emitir as suas emissões até ao final do mês. Neste momento, falta-lhe apenas decidir em que localidades irá colocar os seus emissores. A estação quer que as suas emissões sejam ouvidas em perfeitas condições em 6 grandes centros urbanos da região Norte ($L_1, L_2, \dots, L_6$). Para isso, identificou 5 locais potenciais onde poderão ser colocados os emissores ($C_1, C_2, \dots, C_5$). Cada um desses locais abrange apenas um subconjunto dos centros urbanos conforme indicado na tabela seguinte:

	Centros urbanos abrangidos
C_1	L_1, L_4, L_5
C_2	L_1, L_2, L_3, L_6
C_3	L_1, L_4, L_5
C_4	L_2, L_5
C_5	L_1, L_3, L_6

Sabendo que a estação pretende colocar o menor número possível de emissores, formule o problema usando um modelo de Programação Inteira.

10. Formule as expressões lógicas seguintes usando variáveis binárias:

a) $(R_1 \vee R_2) \wedge (\neg R_3 \vee \neg R_4)$

b) $R_1 \vee R_2 \Rightarrow \neg R_3$

c) $R_1 \wedge R_2 \Rightarrow R_3 \vee R_4$

11. Considere o modelo de Programação Inteira seguinte:

$$\begin{aligned}
 \max \quad & z = 15x_1 + 25x_2 + 10x_3 + 30x_4 \\
 \text{s. a} \quad & \\
 & 9x_1 + 2x_2 + 5x_3 + 3x_4 \leq 15 \\
 & \qquad \qquad \qquad 6x_3 - 2x_4 \geq 11 \\
 & x_1 + 3x_2 \qquad \qquad + 4x_4 \leq 17 \\
 & \qquad \qquad x_2 + 3x_3 + 6x_4 \leq 23 \\
 & x_1, x_2, x_3, x_4 \geq 0 \text{ e inteiros.}
 \end{aligned}$$

a) Usando um *software* de optimização, determine

- i. a solução óptima da relaxação linear;
- ii. a solução óptima inteira.

Indique o valor do intervalo de integralidade.

- b) Derive limites superiores e inferiores para cada uma das variáveis. Analise o impacto dessas restrições no modelo.
- c) Reforce os limites que derivou em b) testando o valor das variáveis.

12. Considere o modelo de Programação Inteira com variáveis binárias seguinte:

$$\begin{aligned}
\max \quad & z = 7x_1 + 3x_2 + 4x_3 + 8x_4 \\
s. \quad & a \\
& 4x_1 + x_2 + 3x_3 + 2x_4 \leq 5 \\
& 3x_2 + 3x_3 + 2x_4 \leq 7 \\
& 2x_1 + 3x_2 + x_3 + 2x_4 \leq 9 \\
& x_1 + 2x_3 + 2x_4 \leq 3 \\
& x_1, x_2, x_3, x_4 \in \{0, 1\}.
\end{aligned}$$

Use um *software* de optimização para responder às alíneas seguintes.

- a) Analise a qualidade do modelo.
 - b) Sugira formas de reforçar o modelo. Analise o impacto de cada uma das suas propostas no modelo original.
- 13.** Considere novamente o problema de localização do exercício 6.
- a) Analise a qualidade do modelo que formulou para esse problema.
 - b) Caso tenha optado por um modelo com restrições agregadas, desagregue essas restrições tanto quanto possível. Caso contrário, agregue as restrições. Analise o impacto dessa operação na qualidade do modelo.
- 14.** Considere o modelo de Programação Inteira seguinte:

$$\begin{aligned}
\max \quad & z = 3x_1 + 5x_2 \\
s. \quad & a \\
& x_1 \geq 1 \\
& 3x_1 + 4x_2 \leq 13 \\
& x_2 \leq 3 \\
& x_1, x_2 \geq 0 \text{ e inteiros.}
\end{aligned}$$

- a) Represente graficamente o espaço de soluções válidas deste modelo.
- b) Determine a solução óptima da relaxação linear.
- c) Usando a representação gráfica, obtenha a solução óptima inteira através do método de partição e avaliação. Assuma que a partição é feita usando a variável com a maior parte fraccionária e que a pesquisa é feita em profundidade. Escolha sempre em primeiro lugar o nodo que corresponde à adição de uma restrição de partição do tipo *menor ou igual*. Represente a árvore de pesquisa.

15. Considere o modelo de Programação Inteira seguinte:

$$\begin{aligned}
 \max \quad & z = 4x_1 + 8x_2 + 3x_3 \\
 \text{s. a} \quad & \\
 & 3x_2 + x_3 \leq 7 \\
 & 2x_1 + x_2 + 3x_3 \leq 15 \\
 & x_1 + 2x_2 \leq 6 \\
 & x_1, x_2, x_3 \geq 0 \text{ e inteiros.}
 \end{aligned}$$

O quadro óptimo associado à relaxação linear deste modelo é o seguinte:

	z	x_1	x_2	x_3	F_1	F_2	F_3	
x_2	0	0	1	0	$\frac{1}{4}$	$-\frac{1}{12}$	$\frac{1}{6}$	$\frac{3}{2}$
x_3	0	0	0	1	$\frac{1}{4}$	$\frac{1}{4}$	$-\frac{1}{2}$	$\frac{5}{2}$
x_1	0	1	0	0	$-\frac{1}{2}$	$\frac{1}{6}$	$\frac{2}{3}$	3
z	1	0	0	0	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{5}{2}$	$\frac{63}{2}$

Determine a solução óptima inteira usando o método de partição e avaliação. Resolva os subproblemas usando o método Simplex. Para efectuar a partição, escolha sempre a variável fraccionária de menor índice.

Nota: a solução óptima inteira é obtida após a resolução de apenas 2 subproblemas.

16. A resolução de um problema de Programação Inteira de minimização através do método de partição e avaliação produziu a árvore de pesquisa ilustrada na Figura 3.16. Assuma que a pesquisa é feita em profundidade, e que é sempre explorado em primeiro lugar o nodo associado à restrição de partição do tipo *maior ou igual*.

- Apesar da solução do nodo com $z = 211.67$ não ser inteira, o nodo foi abandonado. Porquê?
- Mostre como evoluiu o limite inferior e superior para o valor da solução óptima inteira ao longo da árvore.
- Para este problema, qual teria sido a melhor estratégia de pesquisa? Quantos problemas teriam sido resolvidos nesse caso?

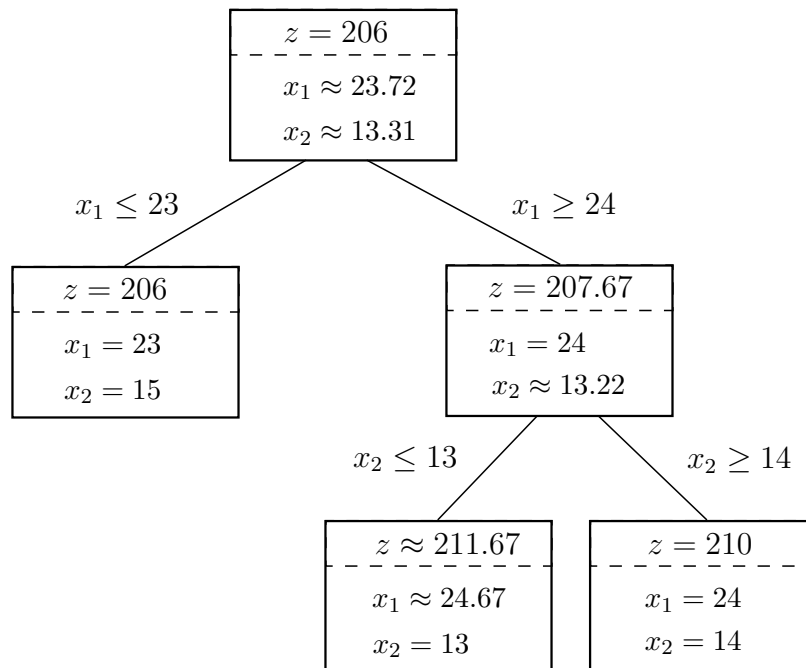


Figura 3.16: Árvore de pesquisa

17. Responda às perguntas seguintes:

- de que forma a qualidade de um modelo de Programação Inteira condiciona a convergência do método de partição e avaliação?
- uma árvore de pesquisa diz-se balanceada se a regra de partição que for usada tender a dividir o espaço de soluções em subespaços de dimensão aproximadamente idêntica. Qual é a vantagem em ter uma árvore de pesquisa balanceada?
- em que circunstâncias é que um problema de Programação Inteira pode ser resolvido como um problema de Programação Linear?

18. Considere o modelo de Programação Inteira seguinte:

$$\max z = 5x_1 + 3x_2$$

s. a

$$5x_1 + 2x_2 \leq 31$$

$$4x_1 - 3x_2 \leq 12$$

$$-x_1 + x_2 \leq 2$$

$$4x_1 + 6x_2 \leq 47$$

$$x_1, x_2 \geq 0 \text{ e inteiros.}$$

Mostre que as inequações seguintes são inequações válidas de Chvátal-Gomory:

a) $x_1 \leq 5$;

b) $x_2 \leq 5$.

19. Considere o modelo de Programação Inteira seguinte:

$$\begin{aligned} \max \quad & z = 5x_1 + 2x_2 + 4x_3 \\ \text{s. a} \quad & 5x_1 + x_2 + 2x_3 \leq 15 \\ & x_2 + 3x_3 \leq 9 \\ & 2x_1 + x_2 \leq 3 \\ & x_1, x_2, x_3 \geq 0 \text{ e inteiros.} \end{aligned}$$

O quadro óptimo associado à relaxação linear desse modelo é o seguinte:

	z	x_1	x_2	x_3	F_1	F_2	F_3	
F_1	0	0	$-\frac{13}{6}$	0	1	$-\frac{2}{3}$	$-\frac{5}{2}$	$\frac{3}{2}$
x_3	0	0	$\frac{1}{3}$	1	0	$\frac{1}{3}$	0	3
x_1	0	1	$\frac{1}{2}$	0	0	0	$\frac{1}{2}$	$\frac{3}{2}$
z	1	0	$\frac{3}{2}$	0	0	1	$\frac{5}{2}$	$\frac{33}{2}$

Aplique o método de planos de corte baseado em cortes fraccionários de Gomory a este problema. Efectue no máximo 2 iterações.

20. Considere o modelo de Programação Inteira mista seguinte:

$$\begin{aligned} \max \quad & z = 7x_1 + 4x_2 + 5x_3 \\ \text{s. a} \quad & 2x_1 + 2x_2 + 3x_3 \leq 11 \\ & 3x_1 + x_2 \leq 8 \\ & 2x_2 + 5x_3 \leq 12 \\ & x_1, x_2 \geq 0 \\ & x_3 \geq 0 \text{ e inteiro.} \end{aligned}$$

O quadro óptimo associado à relaxação linear desse modelo é o seguinte:

	z	x_1	x_2	x_3	F_1	F_2	F_3	
x_3	0	0	$-\frac{2}{3}$	1	$\frac{1}{3}$	$-\frac{1}{3}$	0	$\frac{4}{3}$
x_1	0	1	2	0	0	$\frac{1}{2}$	0	$\frac{7}{2}$
F_3	0	0	$\frac{19}{3}$	0	$-\frac{8}{3}$	$\frac{8}{3}$	1	$\frac{4}{3}$
z	1	0	$\frac{16}{3}$	0	$\frac{5}{3}$	$\frac{7}{3}$	0	$\frac{104}{3}$

Aplique o método de planos de corte a este problema usando cortes inteiros mistos de Gomory. Efectue apenas uma iteração.

Capítulo 4

Fluxos em Rede

4.1 Introdução

No contexto da Investigação Operacional, a teoria dos fluxos em rede é impulsionada pelos trabalhos de Lester Ford e Ray Fulkerson em meados dos anos 50. Em 1962, esses autores publicam um livro de referência nesse domínio intitulado *Flows in Networks*. Os modelos de fluxos em rede são definidos a partir de grafos, e podem ser usados para formular problemas relevantes de optimização. Além de algumas aplicações óbvias, os modelos de fluxos em rede permitem formular problemas de gestão de projectos, de planeamento da produção, ou mesmo de processamento de sinal.

A importância dos modelos de fluxos em rede deve-se principalmente a dois factores. Em primeiro lugar, o conceito de fluxos em rede constitui uma ferramenta rica e intuitiva de modelação a partir da qual é possível representar diferentes problemas de optimização. O segundo factor prende-se com a estrutura de muitos desses modelos que possibilita a sua resolução através de métodos eficientes. Como iremos ver neste capítulo, muitos problemas de fluxos em rede podem ser formulados através de modelos de Programação Linear ou Programação Inteira. No entanto, a estrutura particular desses modelos faz com que eles possam ser resolvidos de forma bem mais eficiente. Existe assim uma vantagem clara em reduzir um problema de optimização a um problema particular de fluxos em rede (caminho mais curto, por exemplo), uma vez que isso pode ser a garantia da obtenção da solução óptima em tempos computacionais reduzidos. Sabendo que muitos problemas de optimização têm como subproblema um problema de fluxos em rede, esse aspecto ganha um carácter ainda mais relevante.

Neste capítulo, analisamos alguns problemas de fluxos em rede nomeadamente o problema de transportes, o problema de afectação e o problema de caminho mais curto. Esses problemas são casos especiais do problema de fluxo de custo mínimo que é também descrito neste capítulo. Os respectivos modelos de Programação Linear são

apresentados e discutidos, e exploramos os algoritmos especializados de resolução que existem para cada um destes problemas. No final do capítulo, são descritas algumas aplicações.

4.2 Problema de Transportes

4.2.1 Definição do Problema

O problema de transportes consiste em determinar a forma como devem ser distribuídos produtos, pessoas ou outras entidades desde locais de origem até determinados destinos de modo a minimizar ou maximizar uma função objectivo linear. O problema de transportes é definido num grafo bipartido: os vértices do grafo podem ser divididos em dois conjuntos de tal forma que qualquer arco tem sempre uma das suas extremidades num dos conjuntos de vértices, e a outra extremidade no outro conjunto.

O problema de transportes é definido pelos seguintes elementos:

- um conjunto de n origens em que cada origem pode enviar até d_i unidades, $i = 1, \dots, n$, até um ou mais destinos da rede; a quantidade d_i representa a oferta da origem i ;
- um conjunto de m destinos, cada um com uma procura de p_j unidades, $j = 1, \dots, m$;
- custos unitários de transporte c_{ij} , $i = 1, \dots, n$, $j = 1, \dots, m$, (esse valor está associado ao envio de uma unidade desde uma origem i até um destino j , e poderá representar outra quantidade diferente de um custo).

O problema consiste em determinar as quantidades que devem ser enviadas nos arcos entre as origens e os destinos, de modo a minimizar o custo total de transporte. Esse custo total é directamente proporcional às quantidades enviadas. Note que um problema de transportes poderá também ser um problema de maximização, desde de que possua todas as outras características que enunciámos acima.

A Figura 4.1 ilustra a rede que representa o problema geral de transportes. Conforme se pode observar, o grafo é bipartido: o primeiro conjunto de vértices é formado pelos pontos de origem, enquanto que o segundo conjunto é constituído pelos destinos. A cada um dos arcos (i, j) do grafo está associado um custo unitário de transporte c_{ij} . O Exemplo 4.1 apresenta um problema de transportes de pequenas dimensões.

Exemplo 4.1 Uma empresa possui 3 fábricas distribuídas por todo o país. A capacidade de produção de cada uma das fábricas é respectivamente de 20, 40 e 30 unidades.

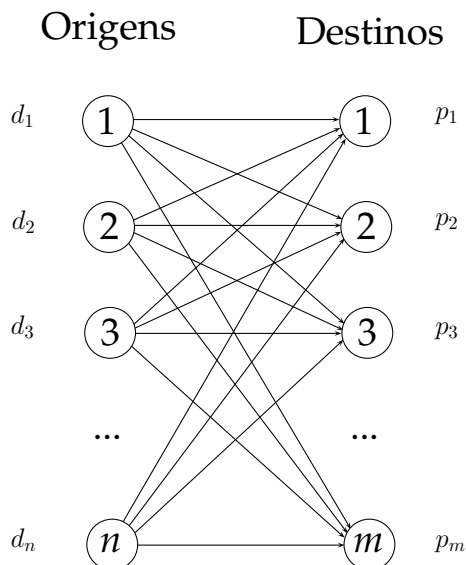


Figura 4.1: Representação em rede de um problema de transportes

A empresa acaba de assinar um contrato de fornecimento com 3 clientes, no qual se compromete a entregar 10 unidades do seu produto ao 1º cliente, 50 ao 2º cliente, e 30 ao 3º cliente.

A empresa pretende determinar quais devem ser as fábricas que irão fornecer os produtos aos clientes, e em que quantidades, de modo a que o custo total de transporte seja o menor possível. A empresa estimou os custos unitários de transporte (em Unidades Monetárias/artigo), e determinou os valores indicados na tabela seguinte.

	Cliente 1	Cliente 2	Cliente 3
Fábrica 1	3	5	2
Fábrica 2	1	7	4
Fábrica 3	2	3	8

Este problema pode ser formulado como um problema de transportes com 3 origens (as fábricas) e 3 destinos (os clientes). A Figura 4.2 representa o problema numa rede.

As quantidades enviadas entre uma fábrica i e um cliente j correspondem ao fluxo no respectivo arco (i, j) dessa rede. O objectivo do problema está em determinar os fluxos que minimizem o custo total de transporte. $\square$

Para que exista uma solução válida para o problema de transportes, a oferta total nas origens deve ser igual à procura total nos destinos:

$$\sum_{i=1}^n d_i = \sum_{j=1}^m p_j.$$

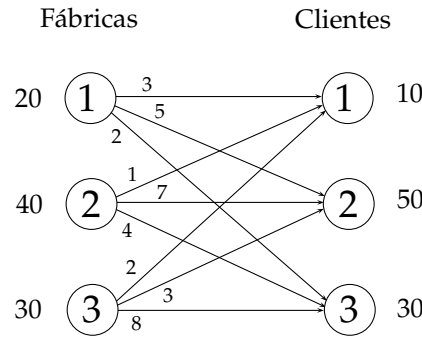


Figura 4.2: Rede do problema de transportes do Exemplo 4.1

Um problema em que a oferta total é diferente da procura total é facilmente convertido num problema que cumpra esse requisito:

- quando $\sum_{i=1}^n d_i > \sum_{j=1}^m p_j$: adiciona-se um destino fictício ao problema com uma procura igual ao excesso de oferta ($\sum_{i=1}^n d_i - \sum_{j=1}^m p_j$);
- quando $\sum_{i=1}^n d_i < \sum_{j=1}^m p_j$: adiciona-se uma origem fictícia ao problema com uma oferta igual ao excesso de procura ($\sum_{j=1}^m p_j - \sum_{i=1}^n d_i$).

O Exemplo seguinte ilustra o procedimento.

Exemplo 4.2 No Exemplo 4.1, se a capacidade de produção da fábrica 1 fosse de 10 unidades em vez de 20, a procura total dos clientes excederia a oferta em 10 unidades. Para que o problema de transportes associado tenha uma solução válida, é necessário adicionar uma origem fictícia com uma oferta de 10 unidades, conforme ilustrado na Figura 4.3 (a origem fictícia corresponde ao vértice OF). Os produtos que são enviados através dos arcos (OF, j) representam procura não satisfeita no cliente j .

Vamos agora admitir que a procura do cliente 2 passava para 40 unidades, mantendo-se todos os outros dados do problema original. O excesso de oferta obrigaria a que fosse criado um destino fictício que pudesse absorver as 10 unidades excedentárias. A Figura 4.4 ilustra a rede deste novo problema de transportes. O vértice DF representa o destino fictício. As quantidades que são enviadas pelos arcos que conduzem a esse vértice representam a parcela de capacidade produtiva que não foi usada pelas respectivas fábricas. Se o fluxo no arco $(2, DF)$ fosse igual a 5, por exemplo, a fábrica 2 estaria apenas a produzir 35 unidades em vez das 40 que a sua capacidade lhe permite produzir.

□

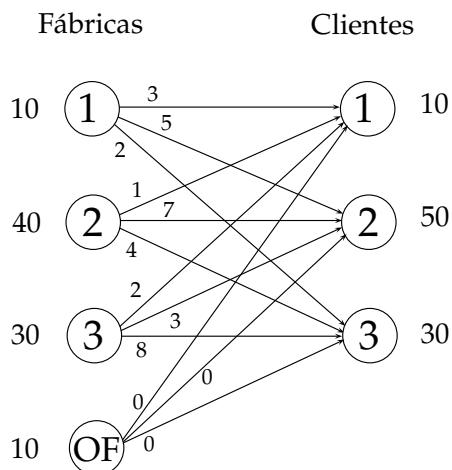


Figura 4.3: Rede com uma origem fictícia (Exemplo 4.2)

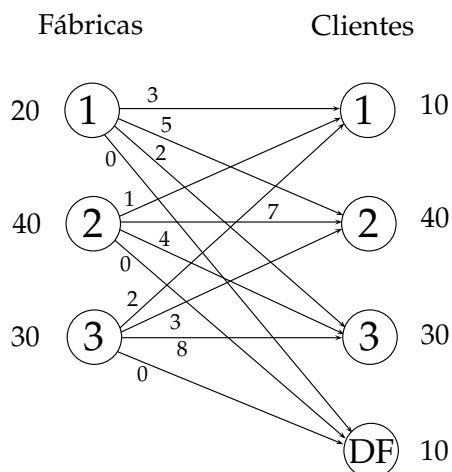


Figura 4.4: Rede com um destino fictício (Exemplo 4.2)

4.2.2 Formulação de Programação Linear

O problema de transportes pode ser formulado através de um modelo de Programação Linear em que as variáveis de decisão x_{ij} representam as quantidades que são transportadas entre uma origem i e um destino j , $i = 1, \dots, n$, $j = 1, \dots, m$. O modelo é composto por dois grupos de restrições, um associado às origens do problema, e outro aos destinos. O primeiro conjunto de restrições obriga a que as quantidades que saem de cada origem seja exactamente igual à oferta nessa origem. O segundo conjunto de restrições força as quantidades transportadas até um destino a serem iguais à procura nesse destino. O modelo é definido formalmente como segue.

$$\begin{aligned}
\min \quad & z = \sum_{i=1}^n \sum_{j=1}^m c_{ij} x_{ij}, \\
s. \ a \quad & \sum_{j=1}^m x_{ij} = d_i, \ i = 1, \dots, n, \\
& \sum_{i=1}^n x_{ij} = p_j, \ j = 1, \dots, m, \\
& x_{ij} \geq 0, \ i = 1, \dots, n, \ j = 1, \dots, m.
\end{aligned}$$

Uma vez que a oferta total deve ser igual à procura total, temos que uma das restrições do modelo é necessariamente redundante, podendo ser obtida por combinação linear das restantes. Por outro lado, se não for possível transportar mercadorias desde uma origem i até um determinado destino j , a variável x_{ij} associada ao arco poderá ser simplesmente eliminada do modelo. Uma alternativa consistiria em atribuir um custo muito elevado (M) ao respectivo arco.

A tabela seguinte ilustra a estrutura matricial do modelo de Programação Linear associado ao problema de transportes.

	x_{11}	x_{12}	x_{13}	$\dots$	x_{1n}	x_{21}	x_{22}	x_{23}	$\dots$	x_{2n}	$\dots$	x_{m1}	x_{m2}	x_{m3}	$\dots$	x_{mn}		
	1	1	1	$\dots$	1												=	d_1
						1	1	1	$\dots$	1							=	d_2
											$\dots$							
												1	1	1	$\dots$	1	=	d_m
	1					1						1					=	p_1
		1					1						1				=	p_2
			1	$\dots$				1	$\dots$		$\dots$			1				
					1					1						1	=	p_n
	c_{11}	c_{12}	c_{13}	$\dots$	c_{1n}	c_{21}	c_{22}	c_{23}	$\dots$	c_{2n}	$\dots$	c_{m1}	c_{m2}	c_{m3}	$\dots$	c_{mn}		

A matriz de coeficientes do modelo associado ao problema de transportes possui a propriedade da total unimodularidade. Consequentemente, e desde que o termo independente das restrições seja inteiro, a solução do modelo de Programação Linear que apresentámos acima será sempre uma solução inteira.

Exemplo 4.3 O modelo de Programação Linear associado ao problema de transportes do Exemplo 4.1 é o seguinte:

$$\begin{aligned}
 \min \quad & z = 3x_{11} + 5x_{12} + 2x_{13} + x_{21} + 7x_{22} + 4x_{23} + 2x_{31} + 3x_{32} + 8x_{33} \\
 \text{s. a} \quad & \\
 & x_{11} + x_{12} + x_{13} = 20 \\
 & \quad \quad \quad x_{21} + x_{22} + x_{23} = 40 \\
 & \quad \quad \quad \quad \quad x_{31} + x_{32} + x_{33} = 30 \\
 & x_{11} \quad \quad \quad + x_{21} \quad \quad \quad + x_{31} = 10 \\
 & \quad \quad x_{12} \quad \quad \quad + x_{22} \quad \quad \quad + x_{32} = 50 \\
 & \quad \quad \quad x_{13} \quad \quad \quad + x_{23} \quad \quad \quad + x_{33} = 30 \\
 & x_{11} \quad , \quad x_{12} \quad , \quad x_{13} \quad , \quad x_{21} \quad , \quad x_{22} \quad , \quad x_{23} \quad , \quad x_{31} \quad , \quad x_{32} \quad , \quad x_{33} \geq 0
 \end{aligned}$$

□

4.2.3 Método Simplex para Problemas de Transportes

Como vimos, o problema de transportes pode ser formulado através de um modelo de Programação Linear e, como tal, pode ser resolvido usando o método Simplex. No entanto, a estrutura particular do modelo faz com que ele possa ser resolvido de forma muito eficiente com uma versão adaptada desse método. O método resultante é também designado na literatura por *método Simplex para transportes*. Antes de introduzirmos os passos que o definem, vamos rever os fundamentos no qual ele assenta.

Considere novamente o modelo de Programação Linear para o problema de transportes que introduzimos na secção anterior. Se associarmos uma variável u_i , $i = 1, \dots, n$, a cada uma das restrições ligadas às origens, e uma variável v_j , $j = 1, \dots, m$, às restrições ligadas aos destinos, chegamos à seguinte definição do respectivo modelo dual:

$$\begin{aligned}
 \max \quad & \sum_{i=1}^n d_i u_i + \sum_{j=1}^m p_j v_j, \\
 \text{s. a} \quad & \\
 & u_i + v_j \leq c_{ij}, \quad i = 1, \dots, n, \quad j = 1, \dots, m, \\
 & u_i, v_j \in \mathbb{R}, \quad i = 1, \dots, n, \quad j = 1, \dots, m.
 \end{aligned}$$

A cada uma das restrições desse modelo dual, está associada uma variável x_{ij} . Note que as variáveis do modelo dual não têm restrições de sinal uma vez que as restrições do modelo primal são todas restrições de *igualdade*.

No Capítulo 3, vimos que o método Simplex, em cada uma das suas iterações, caminha para uma solução primal que melhora o valor da função objectivo, e, ao mesmo tempo, determina uma solução para o problema dual que é complementar à solução primal (propriedade da folga complementar). De acordo com essa propriedade, se tivermos uma solução válida de base na qual a variável x_{ij} é básica, a solução dual associada a essa solução será tal que

$$u_i + v_j = c_{ij}. \quad (4.1)$$

Por outro lado, se para as restantes variáveis não-básicas x_{ij} tivermos

$$u_i + v_j \leq c_{ij},$$

então a solução dual definida pelas variáveis u_i e v_j será uma solução válida para o modelo dual. Nesse caso, e pela propriedade da dualidade forte, podemos afirmar que a solução válida de base para o modelo primal é óptima para o problema. Quando a solução dual não é válida para o respectivo modelo, temos que

$$u_i + v_j > c_{ij}.$$

Nesses casos, existem variáveis não-básicas com potencial para melhorar o valor da função objectivo. A taxa de variação do valor da função objectivo quando uma variável não-básica x_{ij} varia de uma unidade é igual a

$$c_{ij} - u_i - v_j.$$

Esse valor corresponde ao custo reduzido da variável não-básica (primal).

A partir das variáveis básicas de uma solução válida de base para o modelo primal, é possível determinar o valor das variáveis duais u_i e v_j com base nas equações (4.1). Dado que o modelo de Programação Linear associado a um problema de transportes com n origens e m destinos tem uma restrição redundante, podemos redefini-lo usando apenas $n + m - 1$ restrições. Uma solução válida de base para o modelo resultante terá também $n + m - 1$ variáveis básicas. O sistema de equações definido a partir de (4.1) terá assim $n + m$ variáveis e $n + m - 1$ equações. Arbitrando o valor de uma das variáveis, determinamos facilmente o valor das restantes variáveis. O valor dessas variáveis duais é calculado no método Simplex para problemas de transportes para poder determinar se a solução válida de base actual é ou não óptima, e para identificar variáveis não-básicas que possam potencialmente melhorar o valor da função objectivo.

O método Simplex para transportes não envolve as operações algébricas que são próprias do método Simplex para modelos de Programação Linear. O funcionamento

do método pode ser visualizado recorrendo a um *quadro de transportes* que representa o problema. As linhas e colunas desse quadro correspondem às origens e destinos do problema, respectivamente. Uma célula na linha i e coluna j do quadro corresponde à variável de decisão x_{ij} . Os custos unitários de transporte são indicados no canto superior direito de cada célula. Uma origem fictícia é representada no quadro através de uma linha adicional, e um destino fictício através de uma coluna adicional. A forma genérica de um quadro de transportes é ilustrada na Figura 4.5.

		Destinos					
		1	2	3	...	m	
Origens	1	c_{11}	c_{12}	c_{13}	...	c_{1m}	d_1
	2	c_{21}	c_{22}	c_{23}	...	c_{2m}	d_2
	3	c_{31}	c_{32}	c_{33}	...	c_{3m}	d_3
	...	...	...	...	...	...	...
	n	c_{n1}	c_{n2}	c_{n3}	...	c_{nm}	d_n
		p_1	p_2	p_3	...	p_m	

Figura 4.5: Quadro de transportes

Tal como acontece com os modelos de Programação Linear, o primeiro passo do método Simplex para problemas de transportes consiste em determinar uma primeira solução válida de base. Essa solução inicial é construída passo a passo: uma das células não preenchida do quadro é escolhida (a variável correspondente passa a ser uma variável básica), e é-lhe atribuído o maior valor possível tendo em conta a oferta ainda disponível na respectiva origem, e a procura que ainda falta satisfazer no respectivo destino. Uma vez atribuído o valor, a oferta e a procura da respectiva linha e coluna da célula são actualizadas. Claramente, um desses valores passa a ser nulo, e, consequentemente, é eliminada a linha ou a coluna correspondente uma vez que não

será possível atribuir-lhe qualquer outro valor positivo. Quando após a atribuição do valor à célula, a oferta e a procura passam ambos a 0, eliminamos a linha ou a coluna correspondente, mas nunca as duas ao mesmo tempo. Essa situação ocorre quando a solução que está a ser construída é uma solução degenerada (com variáveis básicas ao nível 0). No final, a solução válida de base é composta por exactamente $n + m - 1$ variáveis básicas.

A selecção da próxima célula a preencher pode ser feito usando regras diferentes:

- canto noroeste: seleccionar a célula não preenchida do quadro que esteja mais acima e à esquerda; a vantagem desse método é a sua simplicidade (se as variáveis estiverem ordenadas por ordem crescente do seu primeiro e segundo índice, esta regra de selecção corresponderá a escolher sempre a primeira variável da lista); o inconveniente é a qualidade da solução encontrada que em alguns casos poderá ser arbitrariamente má (função objectivo de valor muito elevado num problema de minimização);
- custo mínimo: seleccionar a célula não preenchida do quadro cujo custo seja o mais pequeno (se o problema for do tipo *maximização*, a célula escolhida é aquela que tem o maior coeficiente); essa regra obriga a inspeccionar todos os custos do problema pelo menos uma vez, mas gera soluções que são normalmente melhores do que as que são obtidas com a regra do canto noroeste.

O exemplo seguinte ilustra o processo de cálculo de uma primeira solução válida de base para o pequeno problema do Exemplo 4.1.

Exemplo 4.4 O quadro de transportes para o problema do Exemplo 4.1 é representado abaixo.

		Clientes			
		1	2	3	
Fábricas	1	3	5	2	20
	2	1	7	4	40
	3	2	3	8	30
		10	50	30	

Uma solução válida de base para este problema é composta por 5 variáveis básicas. Se usarmos a regra do canto noroeste, a primeira célula a ser preenchida é a célula (1,1). O maior valor que pode ser atribuído à respectiva variável é igual a

$$\min\{d_1, p_1\} = \min\{10, 20\} = 10.$$

Ao atribuírmos o valor 10 à variável x_{11} , passamos a satisfazer toda a procura do cliente 1. A fábrica 1 passa a ter uma capacidade residual igual a 10 unidades. A coluna 1 é eliminada uma vez que a procura associada é igual a 0, e a oferta da fábrica 1 é actualizada.

		Clientes				
		1	2	3		
Fábricas	1	3 10	5	2	20	10
	2	1	7	4	40	
	3	2	3	8	30	
		10	50	30		
		0				

A próxima célula a ser preenchida é a célula (1,2). A respectiva variável passa a ter um valor igual a 10.

		Clientes					
		1	2	3			
Fábricas	1	310	510	2 —	20 10 0		
	2	1	7	4			40
	3	2	3	8			30
		10 0	50 40	30			

Nesta fase, a próxima célula a preencher é a célula (2,2). O maior valor que a respectiva variável x_{22} pode tomar é igual a $\min\{d_2, p_2\} = \min\{40, 40\} = 40$. Ao atribuímos esse valor a x_{22} , esgotamos a capacidade da fábrica 2, e ao mesmo tempo passamos a satisfazer toda a procura do cliente 2. A respectiva linha e coluna passam a ter ambas valor 0. No entanto, apenas eliminamos uma delas, e mantemos a outra aberta. No nosso caso, optámos por fechar a linha.

		Clientes			
		1	2	3	
Fábricas	1	3 10	5 10	2 —	20 10 0
	2	1	7 40	4 —	40 0
	3	2	3	8	30
		10 0	50 40 0	30	

Na iteração seguinte, é preenchida a célula (2,3). O valor que é atribuído à respectiva variável x_{23} é 0. A variável x_{23} é básica, mas ao nível 0. A solução é por isso degenerada. O quadro final obtido após preencher a última célula (3,3) é ilustrado abaixo.

		Clientes			
		1	2	3	
Fábricas	1	3 10	5 10	2 —	20 10 0
	2	1	7 40	4 —	40 0
	3	2	3 0	8 30	30 0
		10 0	50 40 0	30 0	

A solução representada no quadro corresponde a fornecer 10 unidades do produto ao cliente 1 através da fábrica 1, 10 e 40 unidades ao cliente 2 através da fábrica 1 e 2, respectivamente, e 30 unidades ao cliente 3 através da fábrica 3. O custo dessa solução é de 600 U.M.

Se usássemos o regra do custo mínimo, a primeira célula a preencher seria a célula (2, 1). O quadro seguinte ilustra a solução válida de base que é obtida quando é usado essa regra.

		Clientes			
		1	2	3	
Fábricas	1	3 	5 — 	2 20	20 0
	2	1 10	7 20	4 10	40 30 20 0
	3	2 	3 30	8 — 	30 0
		10 0	50 20 0	30 10 0	

□

O próximo passo do método consiste em determinar se a solução válida de base corrente é ou não ótima. Como vimos, é possível verificar a optimalidade de uma solução primal calculando o valor das variáveis u_i e v_j da solução dual complementar. Num problema de minimização, para todas as variáveis não-básicas x_{ij} , se o custo reduzido $c_{ij} - u_i - v_j$ for não negativo, a solução válida de base é ótima para o problema. Se o problema for de maximização, a solução será ótima se esses valores forem todos menores ou iguais a 0.

Caso a solução corrente não seja ótima, o método determina uma nova solução válida de base trocando uma variável básica por outra, à semelhança do que acontece com um modelo de Programação Linear. Essa troca é feita passando o valor de uma variável básica da solução a 0, e aumentando o valor de uma variável não-básica. As restantes variáveis básicas permanecem básicas, mas o seu valor é alterado para compensar a entrada para a base de uma nova variável. O exemplo seguinte ilustra esse passo do método.

Exemplo 4.5 Vamos partir da solução válida de base obtida no Exemplo 4.4 usando a regra do canto noroeste. Para essa solução, podemos calcular os valores das variáveis u_i e v_j do modelo dual usando as equações (4.1) associadas às variáveis básicas:

$$u_1 + v_1 = 3$$

$$u_1 + v_2 = 5$$

$$u_2 + v_2 = 7$$

$$u_3 + v_2 = 3$$

$$u_3 + v_3 = 8$$

O sistema tem 6 variáveis e 5 equações. Para podermos determinar o valor de uma solução, atribuímos arbitrariamente o valor a uma das variáveis. Vamos fazer por exemplo $u_1 = 0$. O valor das outras variáveis é facilmente calculado. Para este caso, temos que

$$(u_1, u_2, u_3, v_1, v_2, v_3) = (0, 2, -2, 3, 5, 10).$$

O valor do custo reduzido $c_{ij} - u_i - v_j$ das variáveis não-básicas x_{ij} da solução válida de base permite-nos verificar a optimalidade da solução:

$$\begin{array}{ll} c_{13} - u_1 - v_3 = 2 - 0 - 10 = -8 & [x_{13}]; \\ c_{21} - u_2 - v_1 = 1 - 2 - 3 = -4 & [x_{21}]; \\ c_{23} - u_2 - v_3 = 4 - 2 - 10 = -8 & [x_{23}]; \\ c_{31} - u_3 - v_1 = 2 + 2 - 3 = 1 & [x_{31}]. \end{array}$$

Uma vez que existem valores negativos, a solução válida de base não é ótima. As variáveis não-básicas x_{ij} cujo custo reduzido $c_{ij} - u_i - v_j$ é negativo podem potencialmente melhorar a função objectivo se o seu valor for aumentado, *i.e.* se passarem a variáveis básicas. Vamos escolher a variável x_{13} dado que tem o custo reduzido mais negativo. Na próxima solução válida de base, essa variável será uma variável básica. Nesta fase, falta-nos determinar a variável que irá sair da base, e o valor que a nova variável da base irá tomar. Se aumentarmos o valor da variável x_{13} , é necessário compensar a alteração que é produzida na respectiva linha e coluna do quadro aumentando ou diminuindo o valor de outras variáveis básicas. Se isso não fosse feito, a nova solução deixaria de respeitar as restrições do problema. Esse procedimento equivale a determinar um ciclo no quadro que inclua a célula (1,3) e algumas células básicas. O quadro abaixo ilustra esse procedimento.

		Clientes			
		1	2	3	
Fábricas	1	³ 10	⁵ 10 _{$-\theta$}	² $+\theta$	20
	2	¹ 	⁷ 40	⁴ 	40
	3	² 	³ 0 _{$+\theta$}	⁸ 30 _{$-\theta$}	30
		10	50	30	

O valor de θ determina o valor da variável x_{13} . O maior valor que θ pode tomar é igual $\min\{10, 30\}$ uma vez que nenhuma das outras variáveis poderá tomar valores negativos. O quadro pode ser actualizado com a solução válida de base resultante.

		Clientes			
		1	2	3	
Fábricas	1	³ 10	⁵ 	² 10	20
	2	¹ 	⁷ 40	⁴ 	40
	3	² 	³ 10	⁸ 20	30
		10	50	30	

A variável que sai da base é a variável x_{12} . De forma genérica, a variável que entra para a base numa dada iteração toma sempre o valor da variável que sai da base. Note que o valor da função objectivo passou de 600 U.M. para 520 U.M.

O método prossegue com a verificação da optimalidade da solução válida de base. As variáveis duais u_i e v_j são calculadas de novo com base nas equações (4.1) definidas a partir das variáveis básicas da solução corrente:

$$u_1 + v_1 = 3$$

$$u_1 + v_3 = 2$$

$$u_2 + v_2 = 7$$

$$u_3 + v_2 = 3$$

$$u_3 + v_3 = 8$$

Fazendo $u_1 = 0$, obtemos a solução dual seguinte:

$$(u_1, u_2, u_3, v_1, v_2, v_3) = (0, 10, 6, 3, -3, 2).$$

Os custos reduzidos das variáveis não-básicas passam a ter os seguintes valores:

$$\begin{array}{ll} c_{12} - u_1 - v_2 = 5 - 0 + 3 = 8 & [x_{12}]; \\ c_{21} - u_2 - v_1 = 1 - 10 - 3 = -12 & [x_{21}]; \\ c_{23} - u_2 - v_3 = 4 - 10 - 2 = -8 & [x_{23}]; \\ c_{31} - u_3 - v_1 = 2 - 6 - 3 = -7 & [x_{31}]. \end{array}$$

A célula mais atractiva é a $(2, 1)$. O valor da variável correspondente é determinado calculando o valor de θ no quadro seguinte.

		Clientes			
		1	2	3	
Fábricas	1	$\begin{array}{c} 3 \\ 10 \\ -\theta \end{array}$	$\begin{array}{c} 5 \end{array}$	$\begin{array}{c} 2 \\ 10 \\ +\theta \end{array}$	20
	2	$\begin{array}{c} 1 \\ +\theta \end{array}$	$\begin{array}{c} 7 \\ 40 \\ -\theta \end{array}$	$\begin{array}{c} 4 \end{array}$	40
	3	$\begin{array}{c} 2 \end{array}$	$\begin{array}{c} 3 \\ 10 \\ +\theta \end{array}$	$\begin{array}{c} 8 \\ 20 \\ -\theta \end{array}$	30
		10	50	30	

Neste caso, temos que $\theta = \min\{10, 40, 20\} = 10$, o que resulta na nova solução válida de base representada no quadro seguinte.

		Clientes				
		1	2	3		
Fábricas	1	3	5	2	20	20
	2	1	7	4	40	40
	3	2	3	8	30	30
		10	50	30		

O método prosseguiria com o teste de optimalidade e, no caso da solução ainda não ser óptima, determinaria uma nova solução válida de base usando o mesmo procedimento do que aquele que usámos acima. A solução óptima seria encontrada após mais uma iteração. O quadro seguinte representa a solução óptima final. O valor das variáveis duais, e os custos reduzidos das variáveis não-básicas, estão indicados directamente no quadro (no caso dos custos reduzidos, o valor encontra-se no canto superior esquerdo da células).

		$v_1 = -1$	$v_2 = 5$	$v_3 = 2$		
		1	2	3		
Fábricas	$u_1 = 0$ 1	4	0	2	20	20
	$u_2 = 2$ 2	1	7	4	10	40
	$u_3 = -2$ 3	5	3	8	30	30
		10	50	30		
		Clientes				

Conforme se pode observar, todos os custos reduzidos das variáveis não-básicas são positivos, logo, a solução é optima. O custo dessa solução é de 320 U.M. $\square$

O Algoritmo 6.1 resume os passos associados ao método Simplex para problemas de transportes.

Algoritmo 6.1: Método Simplex para problemas de transportes

```

1 Input: problema de transportes definido num grafo bipartido com  $n$  origens e
   $m$  destinos, custos  $c_{ij}$  em cada arco  $(i, j)$ , ofertas  $d_i$  nas origens, e procuras  $p_j$ 
  nos destinos,  $i = 1, \dots, n, j = 1, \dots, m$ , tal que  $\sum_{i=1}^n d_i = \sum_{j=1}^m p_j$  (vamos
  assumir que o problema é de minimização) ;
2 Determinar uma primeira solução válida de base ;
3  $solucaoOptima := falso$  ;
4 Enquanto  $solucaoOptima=falso$  fazer
5   Calcular as variáveis duais  $u_i$  e  $v_j$  da solução complementar ;
6   /* Para as variáveis básicas  $x_{ij}$ :  $u_i + v_j = c_{ij}$  */
7    $solucaoOptima := verdadeiro$  ;
8   Para todas as variáveis  $x_{ij}$  não-básicas fazer
9     Calcular  $c_{ij} - u_i - v_j$  ;
10    se  $c_{ij} - u_i - v_j < 0$  então
11       $solucaoOptima := falso$  ;
12  se  $solucaoOptima=falso$  então
13    Escolher a variável não-básica com o custo reduzido  $c_{ij} - u_i - v_j$  mais
    negativo ;
14    Construir um circuito usando a célula não-básica escolhida e células
    básicas da solução corrente, e subtrair e somar alternadamente um valor
     $\theta$  a cada uma delas ;
15     $\theta$  é igual ao menor valor da célula básica onde  $\theta$  é subtraído ;
16    Actualizar a solução válida de base ;
17 Output: “A solução válida de base corrente é óptima para o problema” ;
18 Stop ;

```

4.2.4 Problema de Transbordo

No problema de transportes, aquilo que é transportado na rede vai directamente das origens aos destinos. No problema de transbordo, para além dos elementos que definem um problema de transportes, existem ainda pontos intermédios na rede para os quais as origens podem enviar a sua mercadoria (ou qualquer outra entidade), e a partir dos quais a mercadoria pode seguir para os destinos. Esses pontos são também designados por pontos de transbordo. Apesar de não ser exactamente um problema de transportes, o problema de transbordo pode ser formulado e resolvido usando um modelo de transportes.

Vamos considerar o problema genérico de transbordo ilustrado na Figura 4.6. O problema é composto por 3 origens (vértices 1, 2 e 3), 3 destinos (vértices 6, 7, e 8) e 2 pontos de transbordo (vértices 4 e 5). Vamos assumir que os pontos de transbordo são apenas pontos de passagem que não possuem inicialmente nenhuma mercadoria. A soma das ofertas nas origens vai ser designada por t ($t = d_1 + d_2 + d_3$).

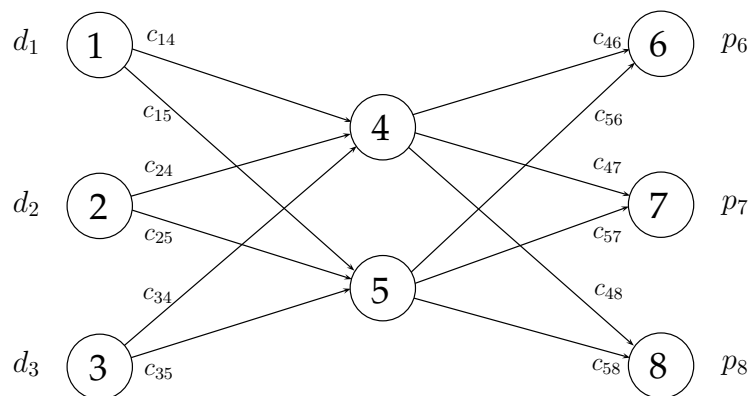


Figura 4.6: Problema de transbordo

O quadro de transportes seguinte ilustra o modelo de transportes associado a este problema de transbordo.

	4	5	6	7	8	
1	c_{14}	c_{15}	M	M	M	d_1
2	c_{21}	c_{22}	M	M	M	d_2
3	c_{31}	c_{32}	M	M	M	d_3
4	0	M	c_{46}	c_{47}	c_{48}	t
5	M	0	c_{56}	c_{57}	c_{58}	t
	t	t	p_6	p_7	p_8	

Os pontos de transbordo são simultaneamente pontos de destino e pontos de origem.

Por esse motivo, cada um desses pontos tem associado uma linha e uma coluna no quadro. A procura associada aos pontos de transbordo (enquanto destinos) deve ser pelo menos igual à soma das ofertas das origens 1, 2 e 3. Claramente, os dois pontos de transbordo não podem receber simultaneamente o total dessas quantidades. A diferença entre esse total e a quantidade efectivamente recebida por um ponto de transbordo é representada na célula na intersecção da linha e da coluna associada a um ponto de transbordo. Essa célula representa a quantidade de mercadoria que o ponto de transbordo “se envia a ele próprio”.

Por outro lado, a oferta dos pontos de transbordo (enquanto origens) deve ser também pelo menos igual a t . Uma vez que as origens 1, 2 e 3 não podem enviar mercadoria directamente para os destinos finais, atribuímos um custo elevado (M) às respectivas células. O mesmo acontece com as células que representam quantidades transportadas entre pontos de transbordo.

Uma vez definido o modelo de transportes para o problema de transbordo, a solução óptima é obtida usando o método Simplex para transportes que descrevemos na secção anterior.

4.3 Problema de Afectação

4.3.1 Definição do Problema

O problema de afectação é um caso particular do problema de transportes. O problema é caracterizado pelo facto de todas as origens e destinos terem ofertas e procuras respectivamente iguais à unidade. O problema consiste assim em formar pares (afecções) compostos por um elemento de cada um dos conjuntos. O problema de afectação é muitas vezes apresentado como o problema de alocar agentes a tarefas. Contudo, esse problema não se limita apenas a essa aplicação. A representação em rede desse problema é ilustrada na Figura 4.7.

Tal como o problema de transportes, o problema de afectação só tem soluções válidas se a soma das ofertas for igual à soma das procuras. No caso particular do problema de afectação, esse equilíbrio verifica-se quando o número de agentes é igual ao número de tarefas. Quando isso não acontece, o problema pode ser equilibrado adicionando agentes ou tarefas fictícias.

O modelo de Programação Linear associado ao problema de afectação é semelhante ao do problema de transportes, excepto para o valor dos termos independentes das restrições que passa a ser igual a 1:

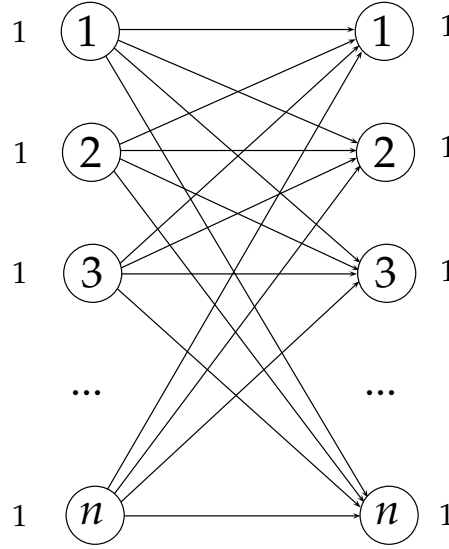


Figura 4.7: Representação em rede de um problema de afectação

$$\begin{aligned}
 \min \quad & z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}, \\
 \text{s. a} \quad & \sum_{j=1}^n x_{ij} = 1, \quad i = 1, \dots, n, \\
 & \sum_{i=1}^n x_{ij} = 1, \quad j = 1, \dots, n, \\
 & x_{ij} \geq 0, \quad i, j = 1, \dots, n.
 \end{aligned}$$

O custo de um agente i realizar uma tarefa j é designado por c_{ij} . Uma vez que a solução óptima do modelo é sempre uma solução inteira (propriedade da total unimodularidade), as variáveis de decisão apenas irão tomar o valor 0 ou 1. Uma variável x_{ij} é igual a 1 se o agente i realizar a tarefa j , e 0 caso contrário.

O exemplo seguinte descreve um problema de pequenas dimensões que pode ser formulado como um problema de afectação.

Exemplo 4.6 Considere o caso de uma empresa que pretende contratar 3 novos colaboradores para 3 dos seus departamentos (um por departamento). Após a análise dos currículos de cada um dos candidatos, a empresa seleccionou 3, e pretende agora determinar a forma como devem ser distribuídos pelos departamentos. Os 3 candidatos concorreram simultaneamente aos 3 departamentos. Possuem as mesmas competências, e podem por isso assumir funções com o mesmo desempenho em qualquer um dos departamentos.

A empresa pediu aos candidatos para que ordenassem os departamentos por ordem decrescente de preferência (o 1º departamento da lista é o preferido pelo candidato). As respostas que recebeu são descritas na tabela seguinte.

	Dept. 1	Dept. 2	Dept. 3
Candidato 1	1	2	3
Candidato 2	1	3	2
Candidato 3	2	3	1

O problema desta empresa pode ser formulado como um problema de afectação em que um dos conjuntos é composto pelos candidatos, e o outro conjunto pelos departamentos. O problema consiste em associar cada um dos candidatos a um departamento de modo a minimizar o peso total da afectação. A rede da Figura 4.8 representa esquematicamente o problema de afectação correspondente. $\square$

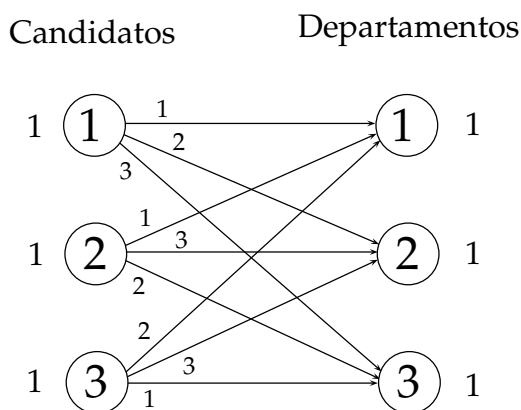


Figura 4.8: Problema de afectação (Exemplo 4.6)

Apesar de ser um caso especial do problema de transportes, o método Simplex para transportes resolve o problema de afectação de forma muito ineficiente. O problema reside no facto de existirem muitas soluções degeneradas que vão sendo exploradas pelo método, em iterações em que o valor da função objectivo não é melhorado. É possível resolver problemas de afectação recorrendo a estratégias alternativas. Na secção seguinte, exploramos o algoritmo Húngaro que permite resolver estes problemas de forma eficiente.

4.3.2 Algoritmo Húngaro

O algoritmo húngaro é um método especializado de resolução de problemas de afectação proposto por Kuhn em 1955¹. O método tem uma interpretação dual baseada no modelo dual associado ao modelo de Programação Linear que introduzimos na secção anterior. Nesta Secção, apresentamos o método de uma forma alternativa. A versão do método que apresentamos baseia-se em dois princípios simples: a redução da matriz de custos do problema, e a determinação de uma solução de custo nulo.

Uma vez que as variáveis associadas a um problema de afectação têm sempre valor 0 ou 1, o problema é normalmente representado usando apenas a sua matriz de custos, num quadro semelhante ao quadro de transportes. O problema consiste assim em escolher uma célula por linha e uma por coluna. O quadro seguinte representa um problema genérico de afectação com n agentes e n tarefas.

		Tarefas				
		1	2	3	...	n
Agentes	1	c_{11}	c_{12}	c_{13}	...	c_{1n}
	2	c_{21}	c_{22}	c_{23}	...	c_{2n}
	3	c_{31}	c_{32}	c_{33}	...	c_{3n}
	...	...	...	...	...	...
	n	c_{n1}	c_{n2}	c_{n3}	...	c_{nn}

Figura 4.9: Problema de afectação

Note que a solução de um problema de afectação não é alterada se for somada ou subtraída uma determinada quantidade a todos os custos de uma linha ou de uma coluna. Podemos assim assumir sem perda de generalidade que todos os custos de um problema são não negativos. Quando todos os custos de um problema de afectação são não negativos, um limite inferior trivial para o valor da solução óptima é 0. Assim, se existir uma afectação de valor 0, essa afectação será necessariamente óptima. O

¹H. Kuhn. *The Hungarian Method for the Assignment Problem*. Naval Research Logistics Quarterly, 2:83-97, 1955.

algoritmo húngaro procura construir soluções desse tipo, alterando se necessário os custos do problema.

O algoritmo começa por reduzir os custos das linhas e das colunas do quadro associado ao problema: o menor custo de uma linha é subtraído a todos os custos dessa linha; o mesmo processo é aplicado a cada uma das colunas do quadro. Depois dessa redução, verifica se existe uma afectação de custo nulo. É possível provar que essa afectação existe se o menor número de traços (horizontais ou verticais) necessários para cobrir todos os 0 do quadro for igual à dimensão do problema. Se esse não for o caso, o algoritmo prossegue tentando gerar novos custos nulos no quadro. Para isso, o menor custo das células não traçadas do quadro é subtraído a todos os custos do quadro. Para que não sejam criadas células com custo negativo, e para que as células que tinham anteriormente um custo nulo continuem com um custo nulo, esse custo é somado a todas as linhas e colunas traçadas do quadro. Esse processo é repetido até que o número de traços necessários para cobrir os custos nulos do quadro seja igual à dimensão do problema. Quando isso acontece, existe uma afectação de custo nulo (óptima). Essa afectação é construída passo a passo seleccionando preferencialmente as linhas e colunas que têm apenas uma célula (disponível) de custo nulo. Sempre que uma célula é escolhida, a linha e a coluna respectiva são fechadas. O Exemplo seguinte ilustra o funcionamento do algoritmo húngaro.

Exemplo 4.7 Considere o problema de afectação definido através do quadro seguinte:

	1	2	3	4
1	17	34	22	19
2	14	19	27	16
3	14	12	15	17
4	22	16	23	14

O primeiro passo do método consiste em subtrair o menor custo de cada linha aos restantes custos dessa linha, e repetir o processo para todas as linhas e colunas. O resultado dessas operações é ilustrado nos quadros seguintes.

	1	2	3	4
1	0	17	5	2
2	0	5	13	2
3	2	0	3	5
4	8	2	9	0

	1	2	3	4
1	0	17	2	2
2	0	5	10	2
3	2	0	0	5
4	8	2	6	0

São necessários apenas 3 traços para cobrir todos os custos nulos do último quadro, o que prova que ainda não é possível construir uma afectação de custo nulo. O menor custo das células não traçadas é 2. Esse custo é subtraído a todos os custos do quadro, e somado aos custos das linhas e colunas traçadas. Essa operação é equivalente a subtrair o valor 2 aos custos não traçados, e somá-lo aos custos das células traçadas duas vezes. O quadro resultante é o seguinte.

	1	2	3	4
1	0	15	0	0
2	0	3	8	0
3	4	0	0	5
4	10	2	6	0

Nesse quadro, já não é possível cobrir todos os 0 com menos de 4 traços. Logo, é possível construir uma afectação de custo nulo. Essa afectação corresponde às células a cinzento no quadro acima. O custo original dessa afectação é igual a $22 + 14 + 12 + 14 = 62$. $\square$

O Algoritmo 6.2 resume os passos do algoritmo húngaro para problemas de afectação.

Algoritmo 6.2: Algoritmo Húngaro

- 1 **Input:** problema de afectação com n agentes e n tarefas definido num grafo bipartido com custos nos arcos (custos de afectação por agente e tarefa) ;
 - 2 Inicializar o quadro do problema de afectação com a matriz de custos do problema ;
 - 3 **Para todas** as linhas i do quadro, $i = 1, \dots, n$ **fazer**
 - 4 Subtrair o menor custo da linha i a todos os custos dessa linha ;
 - 5 **Para todas** as colunas j do quadro, $j = 1, \dots, n$ **fazer**
 - 6 Subtrair o menor custo da coluna j a todos os custos dessa coluna ;
 - 7 Seja n' o menor número de traços que cobrem todos os custos nulos do quadro ;
 - 8 **Enquanto** $n' < n$ **fazer**
 - 9 Seja c' o menor custo das células não traçadas ;
 - 10 Subtrair c' ao custo de todas as células não traçadas ;
 - 11 Somar c' ao custo de todas as células traçadas duas vezes ;
 - 12 Seja n' o menor número de traços necessários para cobrir todos os custos nulos do quadro ;
 - 13 Construir uma afectação de custo nulo a partir do último quadro escolhendo em primeiro lugar pelas linhas e colunas com apenas um custo nulo ;
 - 14 **Stop** ;
-

4.4 Problema de Caminho Mais Curto

4.4.1 Definição do Problema

O problema de caminho mais curto é definido num grafo orientado $G = (V, A)$ em que a cada arco $(i, j) \in A$ está associado um custo ou comprimento c_{ij} . O problema consiste em determinar o caminho de menor custo desde uma origem s até um destino t do grafo G . O problema é bem definido se não existirem circuitos de comprimento negativo. Se um grafo tiver circuitos de comprimento negativo, pode ser possível construir um caminho de custo arbitrariamente pequeno entre dois vértices percorrendo esse circuito um dado número de vezes.

Se um caminho é um caminho mais curto entre dois vértices de um grafo, então qualquer um dos seus subcaminhos é também ele um caminho mais curto. Se assim não fosse, deveria ser possível substituir esse subcaminho por outro caminho de menor custo, o que contradiria a hipótese inicial segundo a qual o caminho original é um caminho mais curto.

O problema de caminho mais curto entre uma origem s e um destino t pode ser

formulado da forma seguinte:

$$\begin{aligned}
 \min \quad & z = \sum_{(i,j) \in A} c_{ij} x_{ij}, \\
 \text{s. a} \quad & - \sum_{(j,i) \in A} x_{ji} + \sum_{(i,j) \in A} x_{ij} = \begin{cases} 1, & \text{se } i = s, \\ 0, & \text{se } i \neq s \text{ e } i \neq t, \\ -1, & \text{se } i = t, \end{cases} \\
 & x_{ij} \in \{0, 1\}, \quad \forall (i, j) \in A.
 \end{aligned}$$

As variáveis binárias x_{ij} tomam o valor 1 se o arco (i, j) fizer parte do caminho, e 0 caso contrário. As restrições do modelo são restrições de conservação de fluxo. Uma unidade de fluxo sai da origem para chegar finalmente ao destino. Nos vértices intermédios do grafo, o fluxo que entra deve ser igual ao fluxo que sai.

Existem variações do problema de caminho mais curto entre uma origem e um destino num grafo. Esses problemas consistem em determinar o caminho mais curto entre uma origem s de V e todos os outros vértices do grafo, ou entre cada par de vértices do grafo. Vamos designar por n o número de vértices em V ($n = |V|$). O problema de caminho mais curto entre um vértice e todos os outros vértices do grafo pode ser formulado como segue:

$$\begin{aligned}
 \min \quad & z = \sum_{(i,j) \in A} c_{ij} x_{ij}, \\
 \text{s. a} \quad & - \sum_{(j,i) \in A} x_{ji} + \sum_{(i,j) \in A} x_{ij} = \begin{cases} n - 1, & \text{se } i = s, \\ -1, & \text{se } i \neq s, \end{cases} \\
 & x_{ij} \geq 0, \text{ e inteiros, } \quad \forall (i, j) \in A.
 \end{aligned}$$

Os dois modelos possuem a propriedade da integralidade: as matrizes de coeficientes das restrições são totalmente unimodulares. Mesmo que seja relaxada a restrição que obriga as variáveis a serem inteiras, a solução desses modelos será sempre uma solução inteira.

4.4.2 Algoritmo de Dijkstra

O algoritmo de Dijkstra determina o caminho mais curto entre uma origem e todos os outros vértices, num grafo em que os arcos têm todos custos não negativos. O algoritmo baseia-se na atribuição de rótulos aos vértices. O rótulo de um vértice i identifica o predecessor de i num caminho que parte da origem e termina em i , assim como o comprimento desse caminho (ou distância mais curta da origem até i). Vamos

designar por p_i o predecessor do vértice i , e por d_i a distância do caminho até i . O rótulo de um vértice i fica assim definido pelo par (p_i, d_i) . O algoritmo divide os vértices em dois conjuntos: um composto por vértices com rótulos temporários, e outro composto por vértices com rótulos permanentes. Em cada iteração, um dos rótulos temporários é convertido num rótulo permanente. Quando um rótulo passa a ser permanente, ele continua a sê-lo até ao fim do algoritmo. O algoritmo termina quando todos os vértices têm rótulos permanentes.

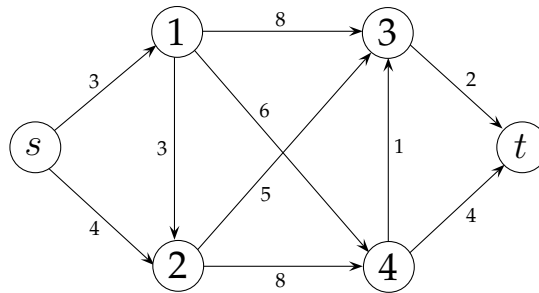
Seja N o conjunto dos vértices com um rótulo permanente, e $\overline{N}$ o conjunto dos vértices com rótulo temporário. Inicialmente, apenas a origem tem um rótulo permanente ($N = \{s\}$, $\overline{N} = V \setminus \{s\}$). Sendo a origem o primeiro vértice de todos os caminhos, o seu rótulo é inicializado com uma distância igual a 0, e com um predecessor indefinido.

Os rótulos dos vértices i ligados à origem s através de um arco (s, i) são inicializados com uma distância igual ao comprimento do arco (s, i) , e com um predecessor que corresponde à origem s . Os rótulos de todos os outros vértices são inicializados de forma temporária com uma distância muito elevada, e um predecessor indefinido.

Em cada iteração, selecciona-se o vértice i com rótulo temporário cuja distância seja a menor entre todos os rótulos temporários. O rótulo de i passa a ser permanente ($N = N \cup \{i\}$ e $\overline{N} = \overline{N} \setminus \{i\}$). Para todos os vértices j com um rótulo temporário e que estejam ligados ao vértice i através de um arco $(i, j) \in A$, se a soma da distância do rótulo de i com o comprimento do arco (i, j) for menor que a distância do rótulo de j , então o rótulo de j é actualizado. O predecessor de j passa a ser i , e a distância passa a ser igual ao valor dessa soma.

No final do algoritmo, quando todos os vértices têm rótulos permanentes, os caminhos mais curtos entre a origem s e qualquer vértice do grafo são construídos do fim para o início usando a informação dos rótulos. O exemplo seguinte ilustra o funcionamento do algoritmo de Dijkstra.

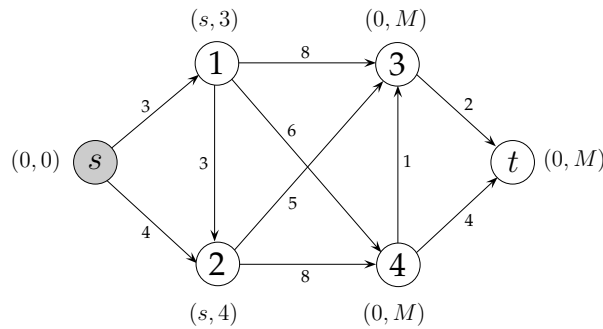
Exemplo 4.8 Considere o problema de caminho mais curto desde um vértice s definido no grafo representado abaixo.



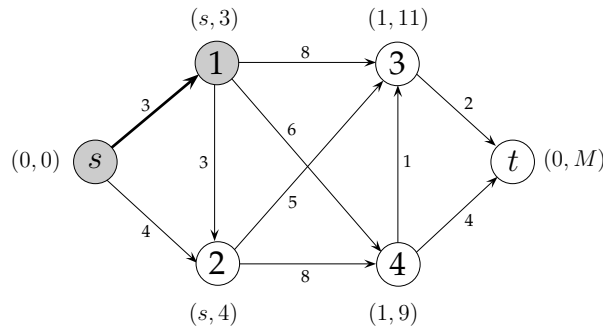
No algoritmo de Dijkstra, os conjuntos de vértices com rótulos permanentes e tem-

porários (N e $\overline{N}$, respectivamente) são inicializados da seguinte forma: $N = \{s\}$ e $\overline{N} = \{1, 2, 3, 4, t\}$. Quando o rótulo de um vértice i passa a ser permanente, o caminho mais curto desde a origem s até i fica completamente definido. O predecessor de i nesse caminho mais curto é p_i , e o comprimento do caminho é d_i .

Inicialmente, temos que $(p_s, d_s) = (0, 0)$. Os rótulos dos dois vértices adjacentes a s são inicializados com o rótulo $(p_1, d_1) = (s, 3)$ (vértice 1) e $(p_2, d_2) = (s, 4)$ (vértice 2). Os rótulos dos restantes vértices j são inicializados com os valores $(p_j, d_j) = (0, M)$. Os rótulos estão indicados junto aos vértices no grafo abaixo. Os vértices com rótulo permanente estão representados a cinzento.

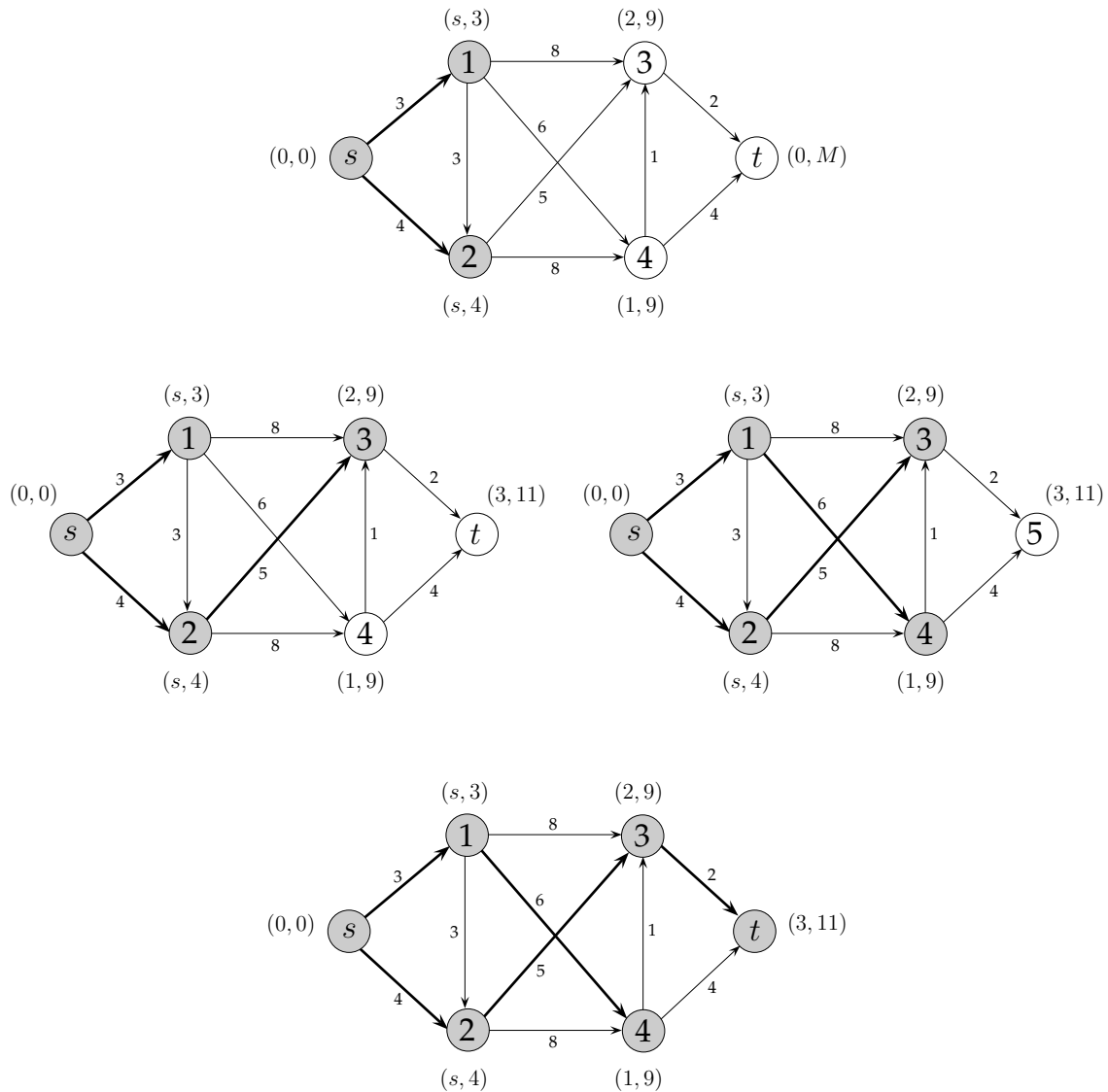


O vértice 1 é o primeiro a ser escolhido uma vez que a distância do seu rótulo é a menor entre todos os rótulos temporários. O rótulo desse vértice torna-se permanente ($N = \{s, 1\}$, $\overline{N} = \{2, 3, 4, t\}$). Nesta fase da execução do algoritmo, o caminho mais curto até ao vértice 1 é conhecido. O rótulo dos vértices 3 e 4 é actualizado dado que $d_1 + 8 < d_3 = M$ e $d_1 + 6 < d_4 = M$. O predecessor desses dois vértices passa a ser o vértice 1. O resultado dessa iteração é representado no grafo seguinte.



O próximo vértice a ser escolhido é o vértice 2. O rótulo desse vértice passa a ser permanente. Temos assim que $N = \{s, 1, 2\}$ e $\overline{N} = \{3, 4, t\}$. O rótulo temporário do vértice 3 que é adjacente ao vértice 2 é actualizado.

O resultado obtido nas iterações seguintes é ilustrado nos grafos seguintes.



O caminho mais curto entre a origem s e o vértice t corresponde à sequência de arcos $(s, 2)$, $(2, 3)$ e $(3, t)$. O custo desse caminho é igual a 11. O algoritmo determina também o caminho mais curto entre a origem e todos os outros vértices do grafo. Esses caminhos são compostos pelos arcos a negro. Os caminhos podem ser reconstruídos partindo do último vértice, e seguindo para o vértice predecessor que é indicado no rótulo. $\square$

Os passos que definem o algoritmo de Dijkstra são descritos no Algoritmo 6.3.

4.4.3 Algoritmo de Bellman-Ford

O algoritmo de Bellman-Ford é usado para determinar o caminho mais curto entre um vértice s e todos os outros vértices num grafo onde existem arcos de custo negativo. A

Algoritmo 6.3: Algoritmo de Dijkstra

```

1 Input: problema de caminho mais curto definido num grafo  $G = (V, A)$  com
   custos não negativos ( $s \in V$  é o vértice de origem);
2  $N := \{s\}$ ,  $\overline{N} = V \setminus N$ ;
3  $p_s := 0$ ,  $d_s := 0$  ;
4 Para todos os  $i \in \overline{N}$  fazer
5   se  $(s, i) \in A$  então
6      $p_i := s$ ,  $d_i := c_{si}$  ;
7   senão
8      $p_i := 0$ ,  $d_i := M$  ;
9 Enquanto  $N \neq V$  fazer
10   Escolher um vértice  $i$  de  $\overline{N}$  tal que  $d_i = \min_{j \in \overline{N}} \{d_j\}$  ;
11    $N := N \cup \{i\}$ ,  $\overline{N} := \overline{N} \setminus \{i\}$  ;
12   Para todos os  $j \in \overline{N}$  fazer
13     se  $d_i + c_{ij} < d_j$  então
14        $p_j := i$ ,  $d_j := d_i + c_{ij}$  ;
15 Output: Conjunto de rótulos permanentes que identificam os caminhos mais
   curtos ;
16 Stop ;

```

principal diferença com o algoritmo de Dijkstra reside no facto do algoritmo de Bellman-Ford poder voltar a analisar um vértice que já tenha sido analisado. No algoritmo de Dijkstra, ao tornar permanente o rótulo de um vértice i , assumimos que o caminho mais curto até esse vértice foi encontrado. Essa constatação assenta no argumento simples de que partindo de um vértice j com rótulo temporário (p_j, d_j) tal que $d_j \geq d_i$, e percorrendo arcos cujo custo é não negativo, nunca irá ser possível melhorar o rótulo de i . Quando existem custos negativos, esse argumento deixa de ser válido.

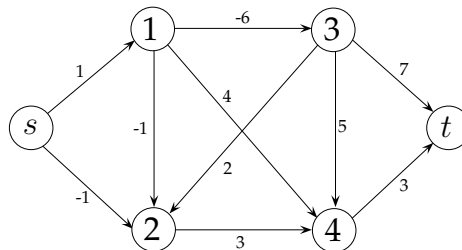
Vamos designar por N o conjunto dos vértices que devem ser analisados, e por $\overline{N}$ o conjunto dos vértices do grafo original que não pertencem a N . Inicialmente, temos que $N = \{s\}$. Sempre que o rótulo de um vértice é actualizado (porque foi encontrado um caminho de menor custo), e caso o vértice não pertença já a N , o vértice é adicionado ao conjunto de vértices por analisar. A actualização dos rótulos é feita da mesma forma do que no algoritmo de Dijkstra. O algoritmo de Bellman-Ford termina quando já não existem vértices por analisar.

A existência de arcos de custo negativo torna possível a existência de circuitos de

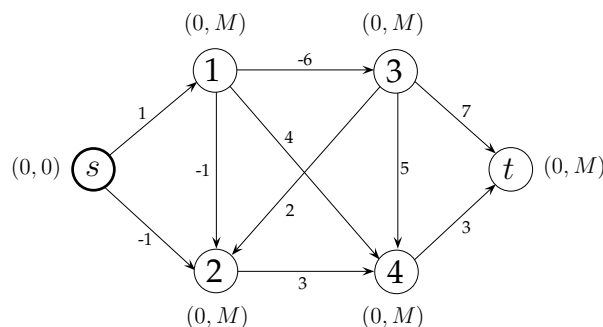
comprimento negativo. Para garantir que um algoritmo de resolução aplicado a esses casos termine após um número finito de passos, é necessário que o algoritmo inclua formas de detectar esses circuitos. Se um grafo não possuir circuitos de comprimento negativo, o menor custo de um caminho será sempre superior ou igual à soma dos custos negativos dos arcos desse grafo. O valor dessa soma pode ser usado para detectar circuitos de comprimento negativo. Vamos designar esse valor por l . Como vimos atrás, quando existe um circuito desse tipo, é sempre possível reduzir o custo de um caminho percorrendo mais uma vez o circuito. Se, numa dada iteração do algoritmo, o rótulo de um vértice i passar a ter um valor d_i inferior a l , fica então provada a existência de um circuito de comprimento negativo. Nessa situação, a execução do algoritmo deve ser terminada.

O algoritmo de Bellman-Ford é descrito no Algoritmo 6.4. O Exemplo ilustra o seu funcionamento.

Exemplo 4.9 Considere o problema de caminho mais curto desde um vértice s definido no grafo representado abaixo.



O conjunto N de vértices a analisar é inicializado com a origem s . Os vértices de N são assinalados no grafos abaixo com um contorno a negro.



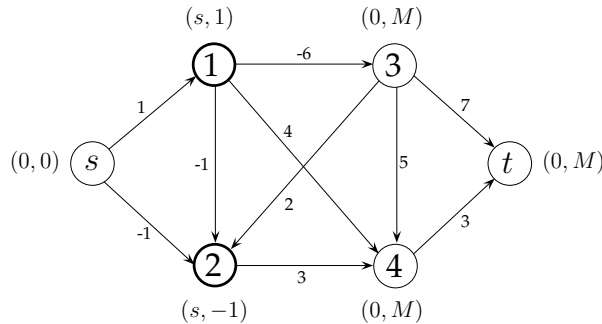
O rótulo dos dois vértices adjacentes a s é actualizado. Os vértices passam assim a pertencer ao conjunto N de vértices a analisar, enquanto que a origem s deixa de fazer parte desse conjunto.

Algoritmo 6.4: Algoritmo de Bellman-Ford

```

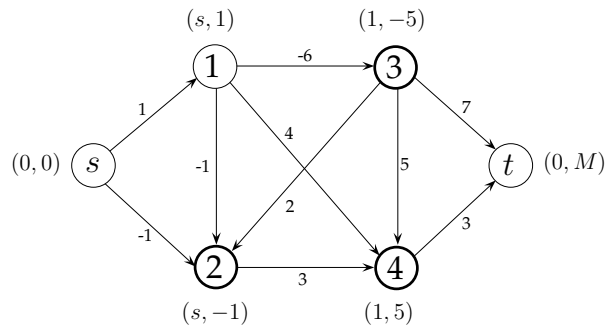
1 Input: problema de caminho mais curto definido num grafo  $G = (V, A)$  ( $s \in V$ 
   é o vértice de origem);
2  $N := \{s\}$ ,  $\bar{N} = V \setminus N$ ;
3  $p_s := 0$ ,  $d_s := 0$  ;
4 Para todos os  $i \in \bar{N}$  fazer
5    $p_i := 0$ ,  $d_i := M$  ;
6 circuitoNegativo:=falso ;
7 Enquanto  $N \neq \emptyset$  e circuitoNegativo=falso fazer
8   Escolher um vértice  $i$  de  $N$  ;
9    $N := N \setminus \{i\}$ ,  $\bar{N} := \bar{N} \cup \{i\}$  ;
10  Para todos os  $(i, j) \in A$  fazer
11    se  $d_i + c_{ij} < d_j$  então
12       $p_j := i$ ,  $d_j := d_i + c_{ij}$  ;
13      se  $d_j < l$  então
14        circuitoNegativo:=verdadeiro ;
15      senão
16        se  $j \in \bar{N}$  então
17           $N := N \cup \{j\}$ ,  $\bar{N} := \bar{N} \setminus \{j\}$  ;
18 se circuitoNegativo=verdadeiro então
19   Output: “ $G$  tem circuitos de comprimento negativo” ;
20 senão
21   Output: Conjunto de rótulos que identificam os caminhos mais curtos ;
22 Stop ;

```

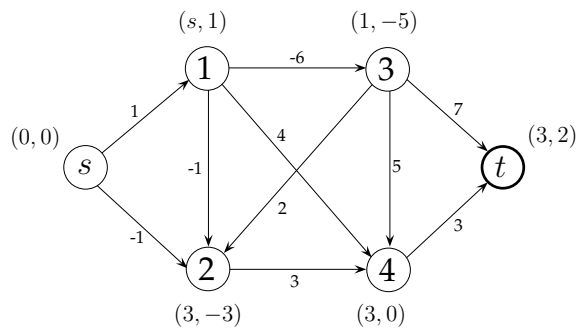
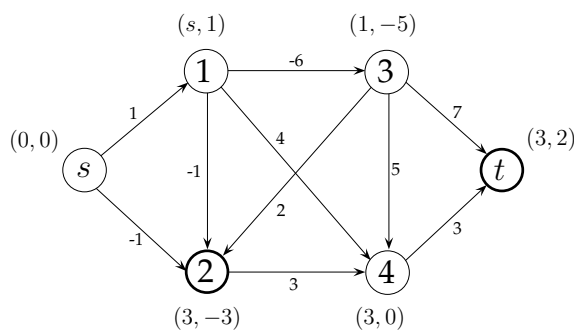
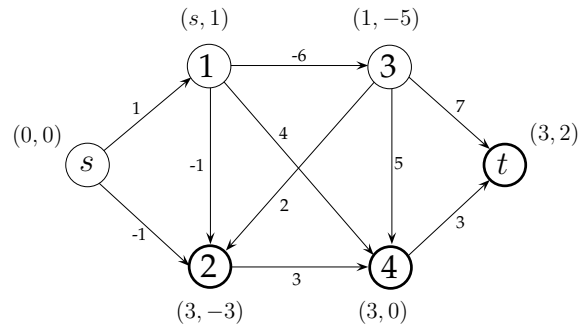
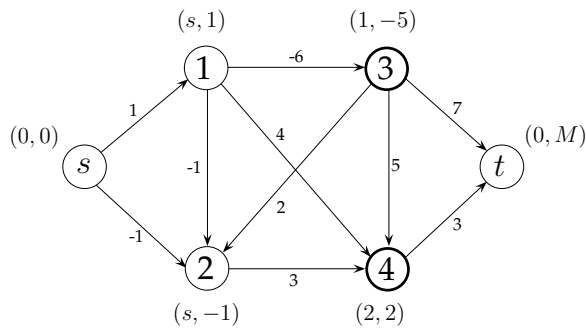


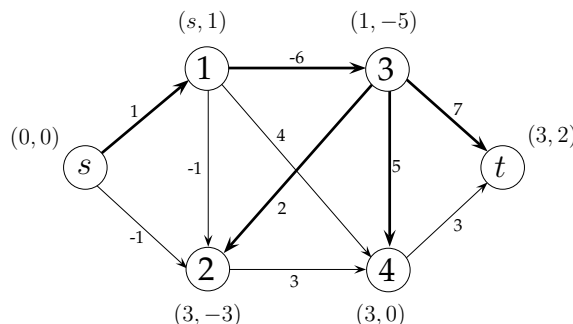
O próximo vértice a ser analisado é necessariamente um vértice de N . Optámos por escolher o primeiro vértice a ter sido inserido nesse conjunto, assumindo que, em

cada iteração do algoritmo, os vértices são adicionados a N por ordem crescente do seu índice. Nesta iteração, escolhemos assim o vértice 1. O vértice 1 é removido do conjunto N . Os rótulos dos vértices 3 e 4 são actualizados, e os dois vértices são adicionados a N .



O próximo vértice que é escolhido é o vértice 2. A inspecção desse vértice produz os resultados ilustrados no primeiro grafo abaixo. O rótulo do vértice 4 (que já pertence a N) é actualizado. As restantes iterações do algoritmo de Bellman-Ford resultam na actualização de rótulos ilustrada nos grafos seguintes.





O caminho mais curto entre s e t é constituído pelos arcos $(s, 1)$, $(1, 3)$ e $(3, t)$, e tem um custo igual a 2. Tal como acontece com o algoritmo de Dijkstra, o algoritmo de Bellman-Ford permite determinar os caminhos mais curtos desde a origem s até qualquer vértice do grafo. Esses caminhos estão assinalados a negro no último grafo. Todos os os caminhos podem ser reconstruídos com base na informação dos rótulos. $\square$

4.5 Problema de Fluxo de Custo Mínimo

4.5.1 Definição do Problema

O problema de fluxo de custo mínimo é o problema geral de fluxos em rede do qual todos os outros problemas que abordámos neste capítulo são casos especiais. O problema é definido num grafo $G = (V, A)$ em que V representa o conjunto de vértices e A o conjunto de arcos. Os vértices representam locais, armazéns, fábricas ou clientes, por exemplo, e podem ser de três tipos: podem ser pontos onde existe uma oferta (de mercadoria, por exemplo), podem ser pontos de procura para onde a mercadoria disponível deve seguir, ou podem representar pontos de transbordo onde não existe nem oferta nem procura.

O grafo G é também orientado: os arcos são percorridos num determinado sentido. Cada arco $(i, j) \in A$ tem associado um custo por cada unidade de fluxo que segue por (i, j) , assim como uma capacidade que limita o valor do fluxo que pode ser enviado por esse arco.

O problema de fluxo de custo mínimo consiste assim em determinar o fluxo que deve seguir por cada um dos arcos do grafo de modo a garantir que a procura é satisfeita a partir da oferta disponível, sem exceder a capacidade dos arcos e minimizando o custo total.

Os parâmetros do problema de fluxo de custo mínimo podem ser resumidos da forma seguinte:

- uma quantidade b_i , $i = 1, \dots, |V|$, associada a cada um dos vértices do grafo, e tal que
 - se $b_i > 0$: o vértice i é uma origem ;
 - se $b_i < 0$: o vértice i é um destino;
 - se $b_i = 0$: o vértice i é um ponto de transbordo;
- um custo unitário c_{ij} por cada unidade de fluxo que é enviada por um arco $(i, j) \in A$;
- uma capacidade u_{ij} que limita o fluxo máximo que pode ser enviado por um arco $(i, j) \in A$;
- um limite inferior l_{ij} para o valor do fluxo num arco $(i, j) \in A$.

Um problema de fluxo de custo mínimo só tem uma solução válida se o total da procura for igual ao total da oferta:

$$\sum_{i=1}^{|V|} b_i = 0. \quad (4.2)$$

Mesmo quando essa condição se verifica, um problema de fluxo de custo mínimo poderá não ter nenhuma solução válida devido às restrições de capacidade nos arcos. Quando (4.2) não se verifica, o grafo original é alterado adicionando um vértice fictício de modo a compensar o excesso de oferta ou de procura. Se o total da oferta for superior ao total da procura, por exemplo, adiciona-se um destino fictício cuja procura é igual ao excesso de oferta, e criam-se arcos que ligam todas as origens do grafo a esse destino fictício. O procedimento é semelhante ao que usámos para os problemas de transportes e de afectação.

Exemplo 4.10 Considere o problema de distribuição representado no grafo abaixo. Os vértices 1 e 2 representam unidades de produção onde são produzidos respectivamente 30 e 60 unidades de um determinado produto. Os vértices 5 e 6 representam clientes cuja procura é de 70 e 20 unidades, respectivamente. Os vértices 2 e 4 representam centros de distribuição. Os produtos que são enviados para esses centros são depois redistribuídos para os clientes finais. Os valores junto aos arcos definem o custo unitário. Os arcos $(1, 4)$, $(4, 5)$ e $(5, 6)$ são os únicos arcos com capacidade limitada. Assumimos ainda que $l_{ij} = 0$, $\forall (i, j) \in A$.

O total da oferta para este caso é de 90 unidades, o que corresponde exactamente ao valor da procura. O problema é equilibrado. Logo, não é necessário adicionar vértices ao grafo.

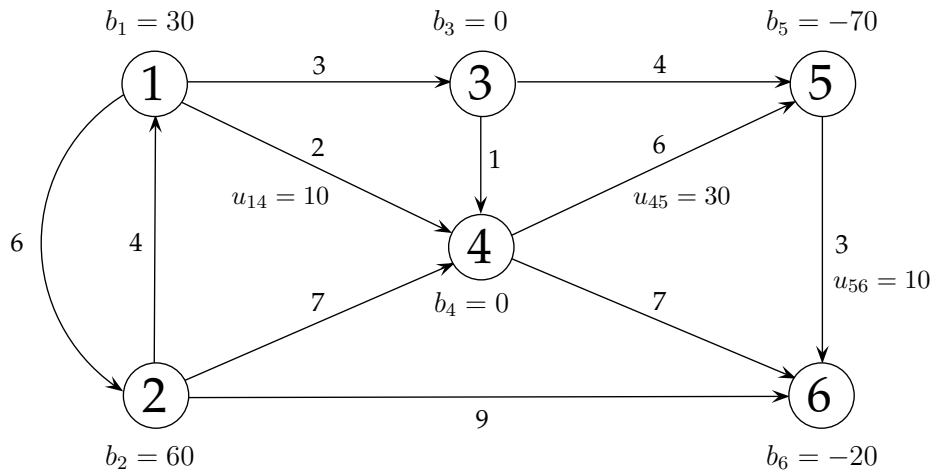


Figura 4.10: Rede do problema de fluxo de custo mínimo do Exemplo 4.10

Vamos supor que a procura no vértice 5 passava para 60 unidades (em vez das actuais 70). O total da procura passa a exceder em 10 unidades o valor da oferta total. Nesse caso, adicionamos um destino fictício ao grafo com uma procura igual a 10 unidades, e ligamos os vértices 1 e 2 a esse destino fictício através de um arco.

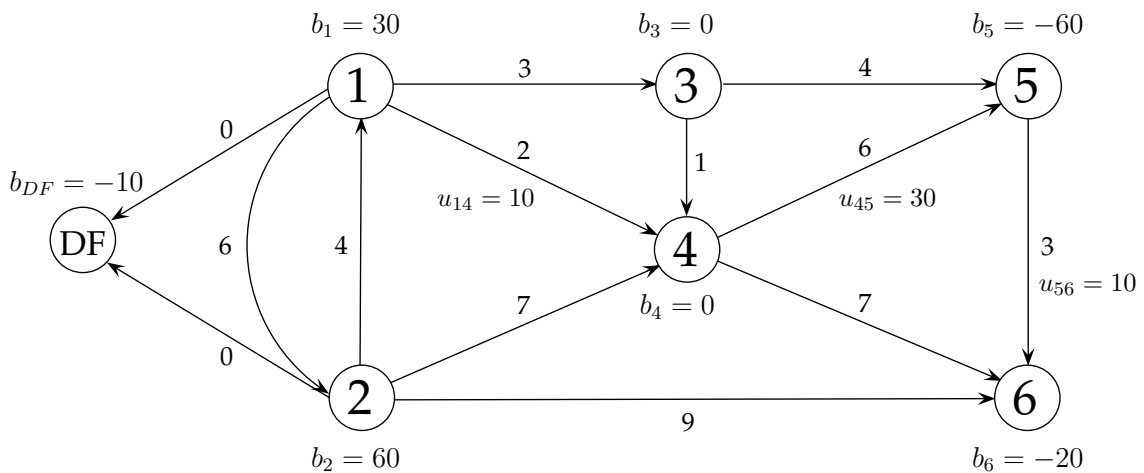


Figura 4.11: Rede com um destino fictício (Exemplo 4.10)

O fluxo nos arcos que foram adicionados ao grafo representam produtos que não chegam a sair das origens. O custo associado a esses arcos depende sempre da interpretação que é feita do fluxo que por eles passa.

Vamos agora supor que a oferta no vértice 2 passava para 40 unidades, mantendo-se todos os outros dados do problema. A procura total passa a exceder em 20 unidades o total da oferta. Nesse caso, adicionamos uma origem fictícia cuja oferta é de 20

unidades, e ligamos essa origem a todos os destinos do grafo. A Figura 4.12 ilustra esse procedimento.

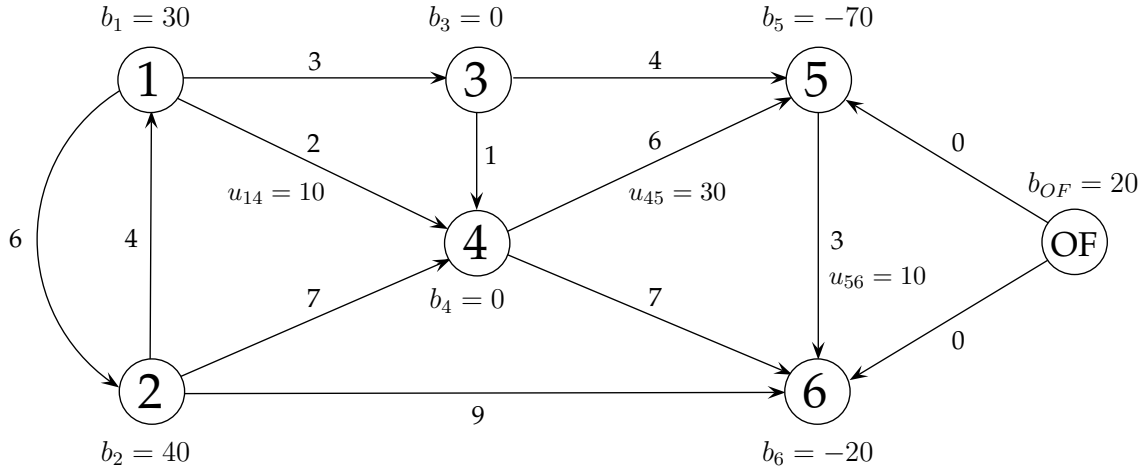


Figura 4.12: Rede com uma origem fictícia (Exemplo 4.10)

□

4.5.2 Formulação de Programação Linear

Num problema de fluxo de custo mínimo, as incógnitas correspondem ao valor do fluxo que deve ser enviado em cada um dos arcos do grafo. Vamos designar por x_{ij} a variável que representa esse fluxo para um arco $(i, j) \in A$. O fluxo na rede é sujeito a restrições de capacidade nos arcos, e a restrições associadas aos níveis de oferta e procura nos vértices. O custo total de uma solução é proporcional ao valor das variáveis x_{ij} . O problema de fluxo de custo mínimo pode ser formulado através do modelo de Programação Linear seguinte.

$$\begin{aligned}
 \min \quad & z = \sum_{(i,j) \in A} c_{ij} x_{ij}, \\
 \text{s. a} \quad & - \sum_{(j,i) \in A} x_{ji} + \sum_{(i,j) \in A} x_{ij} = b_i, \quad \forall i \in V, \\
 & x_{ij} \leq u_{ij}, \quad \forall (i, j) \in A, \\
 & x_{ij} \geq l_{ij}, \quad \forall (i, j) \in A.
 \end{aligned}$$

O primeiro conjunto de restrições traduz a conservação do fluxo em cada um dos vértices do grafo. Num vértice onde exista oferta, a diferença entre o fluxo que sai do

vértice e aquele que entra no vértice deve ser igual ao valor da oferta. Num vértice onde exista procura, o fluxo que entra no vértice deve ser superior àquele que sai. A diferença é exactamente igual ao valor da procura. Finalmente, nos vértices de transbordo, o fluxo que entra deve ser igual ao fluxo que sai. O segundo conjunto de restrições diz respeito às capacidades nos arcos, enquanto o último grupo de restrições define o limite inferior para o valor de um fluxo num arco (i, j) .

Exemplo 4.11 O problema de fluxo de custo mínimo do Exemplo 4.10 pode ser formulado através do modelo de Programação Linear seguinte:

$$\begin{aligned}
 \min \quad & z = 6x_{12} + 3x_{13} + 2x_{14} + 4x_{21} + 7x_{24} + 9x_{26} + x_{34} + 4x_{35} + 6x_{45} + 7x_{46} + 3x_{56} \\
 \text{s. a} \quad & \\
 & x_{12} + x_{13} + x_{14} - x_{21} = 30 \\
 & -x_{12} \quad \quad \quad x_{21} + x_{24} + x_{26} = 60 \\
 & \quad -x_{13} \quad \quad \quad \quad + x_{34} + x_{35} = 0 \\
 & \quad \quad -x_{14} \quad \quad -x_{24} \quad -x_{34} \quad + x_{45} + x_{46} = 0 \\
 & \quad \quad \quad \quad -x_{35} - x_{45} \quad + x_{56} = -70 \\
 & \quad \quad \quad \quad \quad -x_{26} \quad \quad -x_{46} - x_{56} = -20 \\
 & \quad \quad \quad \quad \quad \quad x_{14} \leq 10 \\
 & \quad \quad \quad \quad \quad \quad \quad x_{45} \leq 30 \\
 & \quad \quad \quad \quad \quad \quad \quad \quad x_{56} \leq 10 \\
 & x_{12}, x_{13}, x_{14}, x_{21}, x_{24}, x_{26}, x_{34}, x_{35}, x_{45}, x_{46}, x_{56} \geq 0
 \end{aligned}$$

□

O problema de fluxo de custo mínimo possui a propriedade da integralidade: desde que o valor das ofertas, das procuras e das capacidades sejam inteiros, a solução ótima do modelo de Programação Linear será uma solução inteira. Podemos facilmente verificar essa propriedade à luz dos critérios enunciados na Secção 3.3.2 adicionando variáveis de folga não negativas nas restrições de capacidade do modelo. Para todos arcos $(i, j) \in A$ tal que $u_{ij} > 0$, se fizermos

$$x_{ij} + x'_{ij} = u_{ij},$$

com $x'_{ij} \geq 0$, e introduzirmos essas variáveis no modelo, as restrições de capacidade são eliminadas. O novo modelo passa a ter apenas restrições de conservação de fluxo e restrições de não negatividade. Dado que $x'_{ij} \geq 0$ implica que $x_{ij} \leq u_{ij}$, o novo modelo é equivalente ao original. A introdução de uma variável de folga x'_{ij} corresponde a adicionar um novo vértice ligado aos vértices i e j do grafo original, e com uma oferta

igual a u_{ij} . A oferta ou procura dos vértices i e j é alterada de uma quantidade igual a u_{ij} . A Figura 4.13 ilustra o resultado dessa transformação.

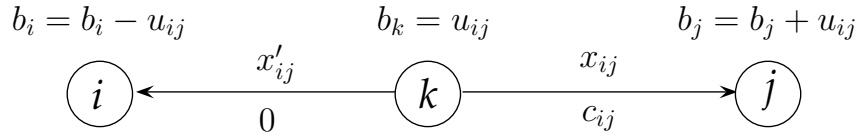


Figura 4.13: Eliminação das restrições de capacidade

Usando um procedimento semelhante, é possível converter um problema de fluxo de custo mínimo com custos negativos nos arcos num problema equivalente com custos não negativos. Os arcos $(i, j) \in A$ tal que $c_{ij} < 0$ são substituídos por arcos (j, i) com capacidade u_{ij} , e um custo igual a $-c_{ij}$. O fluxo nesses arcos é igual a $u_{ij} - x_{ij}$ (que corresponde à variável de folga x'_{ij} da transformação anterior). A disponibilidade nos vértices i e j passa a ser igual respectivamente a $b_i - u_{ij}$ e $b_j + u_{ij}$.

Da mesma forma, um limite inferior num arco $(i, j) \in V$ pode ser facilmente eliminado do modelo substituindo a variável x_{ij} por uma variável x''_{ij} definida como segue:

$$x''_{ij} = x_{ij} - l_{ij},$$

com $x''_{ij} \geq 0$.

Assim, e sem perda de generalidade, vamos assumir a partir deste ponto que o problema não tem restrições de capacidade e de limite inferior nos arcos, e que todos os custos são não negativos.

4.5.3 Método Simplex de Rede

O método Simplex de rede é uma generalização do método Simplex para problemas de transportes. Ambos os métodos tiram partido da estrutura do problema para realizar de forma eficiente algumas das operações do método Simplex para modelos gerais de Programação Linear.

À semelhança do que acontece com o problema de transportes, uma solução válida de base para o problema de fluxo de custo mínimo é composta por $|V| - 1$ variáveis básicas. O equilíbrio entre o total da oferta e o total da procura faz com que uma das restrições do modelo de Programação Linear seja sempre redundante. Os arcos associados às variáveis básicas de uma solução válida de base formam ainda aquilo que é designado por uma *árvore de suporte*. Uma árvore de suporte é um conjunto de arcos que liga todos os vértices de um grafo e que não possui nenhum ciclo. Uma árvore de

suporte é válida se for possível construir com base nela uma solução que satisfaça todas as restrições do problema.

Em cada iteração do método Simplex de rede, é gerada uma solução válida de base definida a partir de uma árvore de suporte válida. Para determinar se essa solução é óptima ou não, o método gera a solução dual associada à solução primal recorrendo à propriedade da folga complementar. Se essa solução dual é válida para o modelo dual, então, pela propriedade da dualidade forte, a solução primal é ótima para o problema de fluxo de custo mínimo. Vamos designar por π_i a variável dual associada à restrição do vértice i do modelo primal. O modelo dual do problema de fluxo de custo mínimo é definido da forma seguinte:

$$\begin{aligned} & \max \sum_{i=1}^{|V|} b_i \pi_i, \\ & s. \ a \\ & \pi_i - \pi_j \leq c_{ij}, \forall (i, j) \in A, \\ & \pi_i \in \mathbb{R}, \ i = 1, \dots, |V|. \end{aligned}$$

Se, numa solução válida de base, x_{ij} é superior a 0 (variável básica), então

$$\pi_i - \pi_j = c_{ij}.$$

Essas expressões definem um sistema com $|V|$ variáveis e $|V| - 1$ equações. Arbitrando o valor de uma das variáveis, podemos facilmente construir uma solução dual válida. Essa solução é válida para o modelo dual se, para todas as variáveis não-básicas x_{ij} , for satisfeita a restrição seguinte

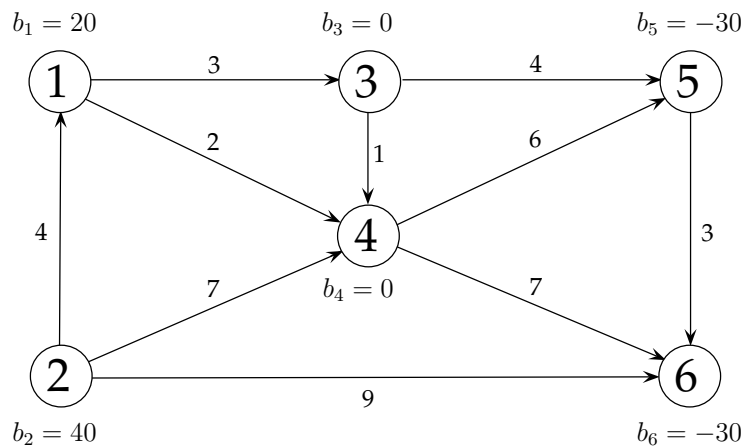
$$\pi_i - \pi_j \leq c_{ij}.$$

Nesse caso, a solução válida de base corrente é ótima, e o método Simplex de rede termina. Caso contrário, é gerada uma nova solução válida de base.

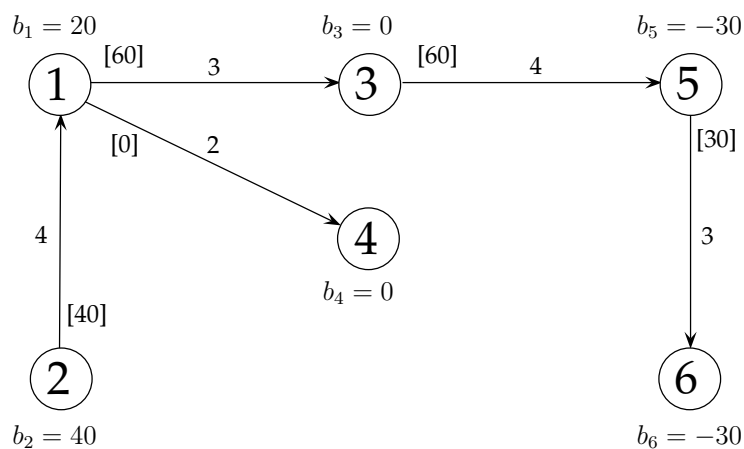
A variável que entra para a base é escolhida entre as variáveis não-básicas x_{ij} com potencial para melhorar o valor da função objectivo, *i.e.* aquelas cujo custo reduzido $c_{ij} - \pi_i + \pi_j$ é negativo. Seja x_{ab} a variável não-básica escolhida (custo reduzido mais negativo). Ao adicionar o arco (a, b) à árvore de suporte válida corrente, é gerado um ciclo. O valor de x_{ab} é calculado com base nos arcos que estão nesse ciclo. Seja θ o valor que irá ser atribuído a x_{ab} . Os arcos do ciclo que têm o mesmo sentido do que (a, b) terão o seu fluxo aumentado em θ unidades, enquanto o fluxo nos arcos em sentido inverso diminuirá de θ unidades. O valor de θ é igual ao menor fluxo dos arcos que estão no ciclo, e cujo sentido é contrário ao do arco (a, b) . O valor dos fluxos é actualizado, gerando-se assim uma nova solução válida de base. A variável que saiu

da base foi aquela cujo valor passou a ser nulo. Este processo é repetido até que seja provada a optimalidade da solução válida de base corrente. O exemplo seguinte ilustra o funcionamento do método Simplex de rede.

Exemplo 4.12 Considere o problema de fluxo de custo mínimo definido a partir do grafo seguinte. Assuma que a capacidade dos arcos é infinita, e que o fluxo em cada um dos arcos deve ser não negativo.



Uma possível árvore de suporte para esse grafo é indicada abaixo. A árvore é válida uma vez que é possível determinar uma solução que satisfaça todas as restrições do problema. O custo total dessa solução é igual a 670. O valor do fluxo é indicado entre parênteses rectos junto ao respectivo arco.



A solução dual associada a essa solução válida de base é calculada através das

equações seguintes:

$$\pi_1 - \pi_3 = 3$$

$$\pi_1 - \pi_4 = 2$$

$$\pi_2 - \pi_1 = 4$$

$$\pi_3 - \pi_5 = 4$$

$$\pi_5 - \pi_6 = 3$$

Fazendo $\pi_1 = 0$, obtemos a solução dual seguinte:

$$(\pi_1, \pi_2, \pi_3, \pi_4, \pi_5, \pi_6) = (0, 4, -3, -2, -7, -10).$$

A partir dessa solução dual, calculamos os custos reduzidos $(c_{ij} - \pi_i + \pi_j)$ de todas as variáveis não-básicas:

$$c_{24} - \pi_2 + \pi_4 = 7 - 4 - 2 = 1 \quad [x_{24}];$$

$$c_{26} - \pi_2 + \pi_6 = 9 - 4 - 10 = -5 \quad [x_{26}];$$

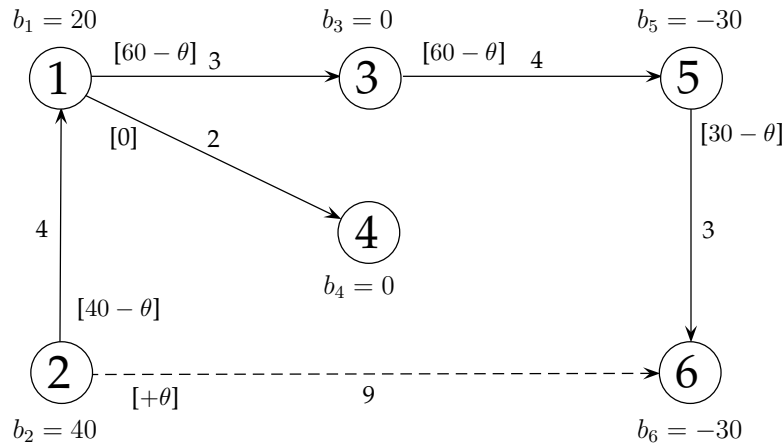
$$c_{34} - \pi_3 + \pi_4 = 1 + 3 - 2 = 2 \quad [x_{34}];$$

$$c_{35} - \pi_3 + \pi_5 = 4 + 3 - 7 = 0 \quad [x_{35}];$$

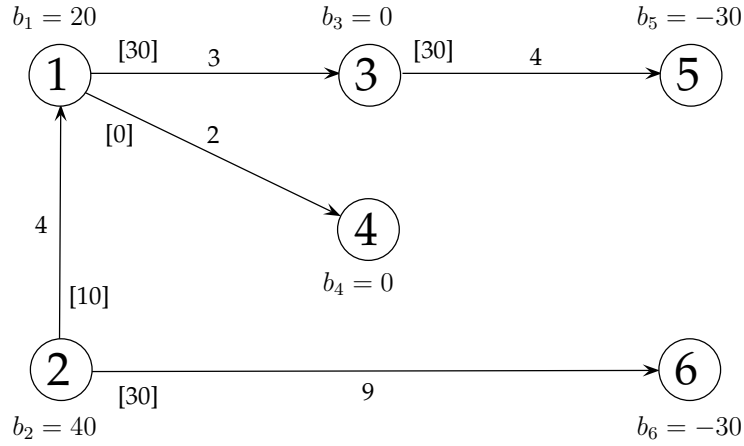
$$c_{45} - \pi_4 + \pi_5 = 6 + 2 - 7 = 1 \quad [x_{45}];$$

$$c_{46} - \pi_4 + \pi_6 = 7 + 2 - 10 = -1 \quad [x_{46}].$$

A solução dual não é válida dado que alguns desses valores são negativos. Logo, a solução válida de base corrente não é ótima. A variável não-básica mais atractiva é a variável x_{26} . Ao adicionarmos o respectivo arco à árvore de suporte, é gerado um ciclo (não orientado) composto pelos arcos $(2, 6)$, $(5, 6)$, $(3, 5)$, $(1, 3)$ e $(2, 1)$. Dado que esses arcos têm um sentido contrário ao do arco $(2, 6)$, o fluxo θ que irá ser adicionado ao arco $(2, 6)$ será subtraído ao fluxo desses arcos, conforme ilustrado abaixo.



O maior valor que θ pode tomar é 30. Acima desse valor, a variável x_{56} viola a sua restrição de não negatividade. A solução válida de base é actualizada. A variável x_{56} sai da base. O resultado é indicado na figura seguinte.



A solução dual associada a essa nova solução válida de base é calculada através das equações seguintes:

$$\pi_1 - \pi_3 = 3$$

$$\pi_1 - \pi_4 = 2$$

$$\pi_2 - \pi_1 = 4$$

$$\pi_3 - \pi_5 = 4$$

$$\pi_2 - \pi_6 = 9$$

Fazendo novamente $\pi_1 = 0$, obtemos a solução seguinte:

$$(\pi_1, \pi_2, \pi_3, \pi_4, \pi_5, \pi_6) = (0, 4, -3, -2, -7, -5).$$

Para cada uma das variáveis não-básicas, temos que:

$$c_{24} - \pi_2 + \pi_4 = 7 - 4 - 2 = 1 \quad [x_{24}];$$

$$c_{34} - \pi_3 + \pi_4 = 1 + 3 - 2 = 2 \quad [x_{34}];$$

$$c_{35} - \pi_3 + \pi_5 = 4 + 3 - 7 = 0 \quad [x_{35}];$$

$$c_{45} - \pi_4 + \pi_5 = 6 + 2 - 7 = 1 \quad [x_{45}];$$

$$c_{46} - \pi_4 + \pi_6 = 7 + 2 - 5 = 4 \quad [x_{46}];$$

$$c_{56} - \pi_5 + \pi_6 = 3 + 7 - 5 = 5 \quad [x_{56}].$$

Todos os valores são não negativos. A solução dual é válida para o modelo dual. Logo, a solução válida de base actual é óptima para este problema de fluxo de custo mínimo. O custo dessa solução é igual a 520. $\square$

O Algoritmo 6.5 resume os passos associados ao método Simplex de rede.

Algoritmo 6.5: Método Simplex de rede

```

1 Input: problema de fluxo de custo mínimo definido num grafo  $G = (V, A)$  com
   custos não negativos, e tal que  $l_{ij} = 0$  e  $u_{ij} = +\infty, \forall (i, j) \in A$  ;
2 Construir uma árvore de suporte válida ;
3 Determinar a solução válida de base associada a essa árvore ;
4  $solucaoOptima := falso$  ;
5 Enquanto  $solucaoOptima=falso$  fazer
6   Calcular as variáveis duais  $\pi_i, i = 1, \dots, |V|$ , da solução complementar ;
   /* Para as variáveis básicas  $x_{ij}$ :  $\pi_i - \pi_j = c_{ij}$  */
7    $solucaoOptima := verdadeiro$  ;
8   Para todas as variáveis  $x_{ij}$  não-básicas fazer
9     Calcular  $c_{ij} - \pi_i + \pi_j$  ;
10    se  $c_{ij} - \pi_i + \pi_j < 0$  então
11       $solucaoOptima := falso$  ;
12  se  $solucaoOptima=falso$  então
13    Seja  $x_{ab}$  a variável não-básica com o custo reduzido mais negativo
      ( $c_{ab} - \pi_a + \pi_b = \min_{(i,j) \in A} \{c_{ij} - \pi_i + \pi_j\}$ ) ;
      /* A adição de  $(a, b)$  à árvore de suporte gera um ciclo. */
14    Seja  $\theta$  o menor fluxo nos arcos do ciclo com sentido contrário a  $(a, b)$  ;
15     $x_{ab} := \theta$  ;
16    Somar  $\theta$  ao fluxo dos arcos do ciclo que têm o mesmo sentido que  $(a, b)$ ;
17    Subtrair  $\theta$  a todos os arcos do ciclo que têm sentido contrário a  $(a, b)$ ;
18    A variável do arco cujo fluxo é anulado torna-se variável não-básica ;
      /* Se mais do que uma variável for anulada, só uma sairá da
         base. */
19 Output: “A solução válida de base corrente é ótima para o problema” ;
20 Stop ;

```

4.6 Aplicações

4.6.1 Gestão de Projectos

Os projectos podem muitas vezes ser descritos através de sequências de actividades definidas por relações de precedência. Uma actividade é predecessora de outra actividade quando esta última não pode ser iniciada sem que a primeira tenha sido concluída. De forma equivalente, considera-se que esta última actividade é sucessora da primeira.

Quando não existem relações de precedência (directas ou indirectas) entre duas actividades, elas podem tipicamente ser executadas em paralelo (admitindo que não existem restrições ao nível dos recursos). Em todo o caso, um projecto é dado por terminado apenas quando todas as suas actividades forem concluídas. Nesse contexto, uma das questões importantes que se levanta consiste em determinar qual irá ser a duração de um projecto atendendo à duração de cada actividade e a todas as relações de precedência entre elas.

A duração de um projecto depende invariavelmente da sequência de actividades ligadas por relações de precedência, e cuja duração total é a mais longa. Essa sequência de actividades é designada por *caminho crítico*. Se uma das actividades do caminho crítico demorar mais tempo que o previsto, ou se não for iniciada logo que todas as suas actividades predecessoras (se existirem) terminarem, o projecto atrasará automaticamente. Vamos assumir que a duração das actividades é perfeitamente conhecida. O método do caminho crítico (*Critical Path Method*, ou CPM, na terminologia anglo-saxónica) permite identificar esse caminho, e calcular a duração de um projecto nessas condições.

A expressão *caminho crítico* tem origem na representação em rede de um projecto. Existem duas formas de representar um projecto na forma de uma rede (ou grafo). A primeira consiste em associar as actividades aos vértices do grafo. A segunda (a que iremos considerar nesta secção) associa as actividades aos arcos do grafo. Em qualquer uma delas, o grafo é definido a partir das relações de precedência do projecto. Na segunda forma de representação (actividades nos arcos), os vértices representam o fim de uma ou mais actividades. O custo de cada arco corresponde à duração da respectiva actividade. O início do projecto é representado por um vértice único do qual partem todos os arcos associados a actividades que não tenham predecessores. O fim do projecto é também representado por um vértice. Claramente, o grafo não pode ter ciclos, senão estaria a representar um projecto que não é possível terminar devido às relações de precedência entre actividades.

Como vimos, determinar o caminho crítico no grafo que representa um projecto é equivalente a determinar o caminho mais longo nesse grafo. O comprimento desse caminho corresponde à duração do projecto. Se existirem caminhos de igual comprimento, eles serão também caminhos críticos. Se multiplicarmos as durações das actividades (custo dos arcos) por -1 , o problema reduz-se a um problema de caminho mais curto num grafo com custos negativos. O facto de não existirem ciclos no grafo é uma garantia que o problema tem uma solução finita.

O dual do problema do caminho mais longo usado na análise do caminho crítico tem uma interpretação interessante. As variáveis do modelo dual representam os instantes

de tempo em que as actividades são iniciadas. O modelo dual tem uma restrição por cada um dos arcos (actividades) do grafo. As restrições traduzem as relações de precedência entre actividades. Uma actividade associada a um arco (i, j) do grafo poderá ser atrasada se, mesmo com esse atraso, ainda for possível terminar essa actividade até ao instante de tempo em que são iniciadas todas as actividades que partem do vértice j . Nesse caso, a actividade não faz parte do caminho crítico. O Exemplo seguinte ilustra a correspondência entre os dois modelos para um caso de pequena dimensão.

Exemplo 4.13 Considere um projecto de desenvolvimento de software composto por 8 actividades cujas durações são indicadas abaixo.

A. Análise de requisitos	2 meses
B. Prototipagem	3 meses
C. Implementação dos módulos funcionais	3 meses
D. Implementação dos módulos de gestão de dados	2 meses
E. Implementação do interface gráfico	6 meses
F. Interligação dos módulos	2 meses
G. Testes	3 meses
H. Produção de documentação	4 meses

A lista de precedências do projecto é indicada na tabela seguinte.

Actividades	Actividades imediatamente predecessoras
A	—
B	A
C	B
D	C
E	B
F	D,E
G	F
H	D,E

O projecto pode ser representado através do grafo representado na Figura 4.14. Os vértices 1 e 7 representam respectivamente o início e o fim do projecto.

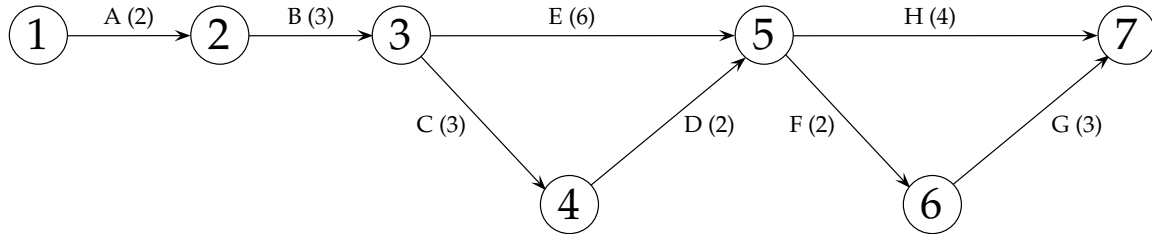


Figura 4.14: Rede do projecto do Exemplo 4.13

O problema que consiste em determinar o caminho mais longo nesse grafo pode ser formulado através de um modelo de Programação Linear semelhante ao do problema de caminho mais curto. As variáveis de decisão x_{ij} representam a inclusão ou não do arco (i, j) no caminho mais longo. O modelo é definido da forma seguinte:

$$\max z = 2x_{12} + 3x_{23} + 3x_{34} + 6x_{35} + 2x_{45} + 2x_{56} + 4x_{57} + 3x_{67}$$

s. a

$$\begin{array}{rcl}
 x_{12} & & = 1 \\
 -x_{12} + x_{23} & & = 0 \\
 -x_{23} + x_{34} + x_{35} & & = 0 \\
 -x_{34} & + x_{45} & = 0 \\
 -x_{35} - x_{45} + x_{56} + x_{57} & & = 0 \\
 -x_{56} & + x_{67} & = 0 \\
 -x_{57} - x_{67} & & = -1 \\
 x_{12} , x_{23} , x_{34} , x_{35} , x_{45} , x_{56} , x_{57} , x_{67} & \in & \{0, 1\}
 \end{array}$$

Obviamente, a variável x_{12} e x_{23} poderiam ser retiradas do modelo dado que o seu valor é conhecido à partida. Na solução óptima desse modelo, as variáveis x_{12} , x_{23} , x_{35} , x_{56} e x_{67} têm todas o valor 1, enquanto as restantes variáveis são iguais a 0. Os arcos correspondentes fazem parte do caminho mais longo cuja duração total é igual a 16 meses. Consequentemente, as actividades A-B-E-F-G formam o caminho crítico.

No modelo acima, existe uma restrição por cada vértice do grafo. Por razões de clareza, vamos assumir que os termos de todas as restrições foram multiplicadas por -1 . O modelo resultante é equivalente ao original. Se associarmos uma variável dual π_i à restrição associada ao vértice i , obtemos o modelo dual seguinte:

$$\begin{array}{rcll}
\min z' = & -\pi_1 & & + \pi_7 \\
s. a & & & \\
& -\pi_1 & + \pi_2 & \geq 2 \\
& & -\pi_2 & + \pi_3 \geq 3 \\
& & & -\pi_3 & + \pi_4 \geq 3 \\
& & & -\pi_3 & & + \pi_5 \geq 6 \\
& & & & -\pi_4 & + \pi_5 \geq 2 \\
& & & & & -\pi_5 & + \pi_6 \geq 2 \\
& & & & & -\pi_5 & & + \pi_7 \geq 4 \\
& & & & & & -\pi_6 & + \pi_7 \geq 3 \\
& \pi_1 & , & \pi_2 & , & \pi_3 & , & \pi_4 & , & \pi_5 & , & \pi_6 & , & \pi_7 \in \mathbb{R}
\end{array}$$

As restrições do modelo dual estão associados aos arcos do grafo. As variáveis π_i representam o instante de tempo em que as actividades que partem do vértice i são iniciadas. As restrições definem as relações de precedência do projecto. A primeira restrição, por exemplo, obriga a que o instante de tempo em que é iniciada a actividade B (que parte do vértice 2) seja posterior à conclusão da actividade A (superior a $\pi_1 + 2$). Uma solução para este modelo é a seguinte:

$$(\pi_1, \pi_2, \pi_3, \pi_4, \pi_5, \pi_6, \pi_7) = (0, 2, 5, 9, 11, 13, 16).$$

Note que a actividade E não pode ser atrasada: essa actividade é iniciada no instante de tempo 5 (início do 6º mês) e tem de estar terminada no instante de tempo 11. Já a actividade C pode ser atrasada de 1 mês. Essa actividade demora 3 meses a ser realizada, é iniciada no instante de tempo 5, e tem de estar terminada no instante de tempo 9. O fim do projecto é dado pelo valor de π_7 . $\square$

4.6.2 Planeamento de Circuitos de Recolha e Distribuição

Um problema que ocorre frequentemente na prática é aquele que consiste em recolher ou distribuir mercadorias, em vários pontos distribuídos ao longo de vias (ruas ou estradas, por exemplo). Tipicamente, essas operações iniciam-se num determinado ponto de uma rede, e terminam nesse mesmo ponto. O problema é conhecido na literatura por problema do carteiro chinês¹. Com efeito, uma das suas aplicações mais evidentes é no campo da distribuição do correio. Outras aplicações são, por exemplo, a recolha do lixo, o planeamento de rotas de autocarros, ou ainda o planeamento de operações de inspecção e manutenção.

¹O problema foi estudado pela primeira vez pelo matemático chinês Kwan Mei-Ko no artigo: Graphic Programming Using Odd or Even Points, *Chinese Mathematics*, 1:273-277, 1962.

O problema pode ser representado num grafo em que os arcos representam as vias que devem ser atravessadas, e os vértices cruzamentos entre essas vias. Se existirem restrições relativamente ao sentido em que as vias podem ser percorridas, então o grafo terá necessariamente arcos orientados. Nesta secção, consideramos o caso em que o grafo a partir do qual é definido o problema é um grafo orientado.

O problema do carteiro chinês consiste assim em determinar o circuito de menor custo que atravesse todos os arcos do grafo. Num grafo orientado, o problema tem solução se e só se existir um caminho entre cada par de vértices. O problema pode ser formulado como um problema de fluxo de custo mínimo. As variáveis x_{ij} do modelo, associado ao arco (i, j) do grafo, representam o número de vezes que um arco é atravessado. Uma vez que procuramos um circuito no grafo, as restrições do modelo reduzem-se a restrições de conservação de fluxo para pontos de transbordo: o fluxo que entra num vértice deve ser igual ao fluxo que sai desse vértice, *i.e.* deve ser escolhido o mesmo número de arcos à entrada e à saída de um vértice. Além disso, um segundo conjunto de restrições obriga a que as variáveis sejam todas maiores ou iguais 1, uma vez que todos os arcos devem ser percorridos pelo menos uma vez.

O problema do carteiro chinês num grafo orientado $G = (V, A)$ com custos c_{ij} nos arcos, $(i, j) \in A$, pode ser formulado como segue:

$$\begin{aligned} \min \quad & z = \sum_{(i,j) \in A} c_{ij} x_{ij}, \\ \text{s. a} \quad & - \sum_{(j,i) \in A} x_{ji} + \sum_{(i,j) \in A} x_{ij} = 0, \quad \forall i \in V, \\ & x_{ij} \geq 1, \quad \forall (i, j) \in A. \end{aligned}$$

A solução deste modelo não determina imediatamente a solução do respectivo problema do carteiro chinês. A sequência pela qual devem ser visitados os arcos pode não ser evidente. Contudo, a solução é facilmente reconstruída a partir da solução do problema de fluxo de custo mínimo. O processo passa por decompôr a solução do modelo em circuitos independentes, e percorrer sucessivamente cada um deles até reconstruir um circuito global. Esse processo é ilustrado em detalhe no exemplo seguinte.

Exemplo 4.14 Considere o problema do carteiro chinês definido através do grafo da Figura 4.15. Os valores junto aos arcos representam o custo de atravessar o respectivo arco.

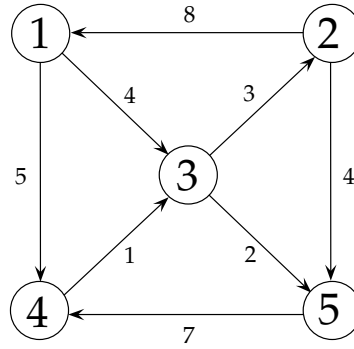


Figura 4.15: Problema do carteiro chinês (Exemplo 4.14)

O problema é formulado como um problema de fluxo de custo mínimo usando variáveis x_{ij} , para todos os arcos (i, j) do grafo, que representam o número de vezes que o arco é atravessado. Dado que todos eles devem ser atravessados pelo menos uma vez, o limite inferior dessas variáveis é igual a 1. O modelo de Programação Linear para este problema é o seguinte.

$$\begin{aligned}
 \min z &= 4x_{13} + 5x_{14} + 8x_{21} + 4x_{25} + 3x_{32} + 2x_{35} + x_{43} + 7x_{54} \\
 \text{s. a} \\
 x_{13} + x_{14} - x_{21} &= 0 \\
 x_{21} + x_{25} - x_{32} &= 0 \\
 -x_{13} + x_{32} + x_{35} - x_{43} &= 0 \\
 -x_{14} + x_{43} - x_{54} &= 0 \\
 -x_{25} - x_{35} + x_{54} &= 0 \\
 x_{13}, x_{14}, x_{21}, x_{25}, x_{32}, x_{35}, x_{43}, x_{54} &\geq 1
 \end{aligned}$$

A solução ótima do problema tem um custo igual a 57, e corresponde a atravessar uma vez os arcos $(1, 3)$, $(1, 4)$, $(2, 5)$ e $(3, 5)$, duas vezes os arcos $(2, 1)$ e $(5, 4)$, e três vezes os arcos $(3, 2)$ e $(4, 3)$. Essa solução é representada na Figura 4.16.

A solução do problema do carteiro chinês é o circuito global que atravessa uma vez todos os arcos da solução ilustrada na Figura 4.16. Para determinar esse circuito, a solução da Figura 4.16 é decomposta em subcircuitos independentes conforme ilustrado na Figura 4.17. O circuito global é reconstruído partindo de um vértice escolhido arbitrariamente, e percorrendo sucessivamente os subcircuitos usando a regra seguinte: sempre que seja atravessado um vértice i incluído noutro subcircuito que não tenha sido já percorrido, esse subcircuito é percorrido desde o vértice i até ele próprio. Percorrer um único subcircuito pode assim envolver o percurso de vários outros subcircuitos. No

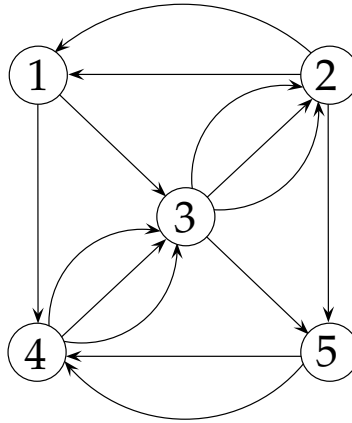


Figura 4.16: Representação da solução do Exemplo 4.14

final, a aplicação desta regra garante que todos os subcircuitos tenham sido percorridos uma (única) vez, determinando assim a solução do problema do carteiro chinês.

Neste caso, escolhemos, por exemplo, o vértice 1 do circuito 1 (Figura 4.17). Caminhamos para o vértice 3 usando o arco $(1, 3)$ desse circuito. O vértice 3 pertence ao circuito 2 que ainda não foi percorrido. Por essa razão, passamos a percorrer o circuito 2 desde o vértice 3, e caminhamos no sentido do arco $(3, 2)$ em direcção ao vértice 2. Por sua vez, o vértice 2 pertence ao circuito 3 que também não foi ainda percorrido. Transitamos assim para o circuito 3, e caminhamos desde o vértice 2 até ao vértice 5 usando o arco $(2, 5)$ desse circuito. Uma vez que o vértice 5 pertence ao circuito 4 que ainda não foi percorrido, passamos então a percorrer o circuito 4 desde o vértice 5. O próximo arco a ser percorrido é o arco $(5, 4)$. O vértice 4 pertence a vários outros circuitos. No entanto, como todos eles estão actualmente a ser percorridos, não transitamos para nenhum deles, e continuamos a percorrer o circuito 4. O mesmo se aplica ao vértice 3, o próximo vértice a ser visitado no circuito 4. Do vértice 3, passamos para o vértice 5, e completamos assim o percurso do circuito 4. Quando um circuito é completamente percorrido, regressamos ao circuito anterior. Neste caso, o circuito anterior é o circuito 3, que continuamos a percorrer desde o vértice 5. O processo continua da mesma forma até que seja completado o percurso do circuito que começámos a percorrer (neste caso o circuito 1). Ao atingir esse ponto, o circuito global foi completamente reconstruído. Neste caso, esse circuito corresponde a atravessar os vértices do grafo pela ordem seguinte:

$$1 - 3 - 2 - 5 - 4 - 3 - 5 - 4 - 3 - 2 - 1 - 4 - 3 - 2 - 1.$$

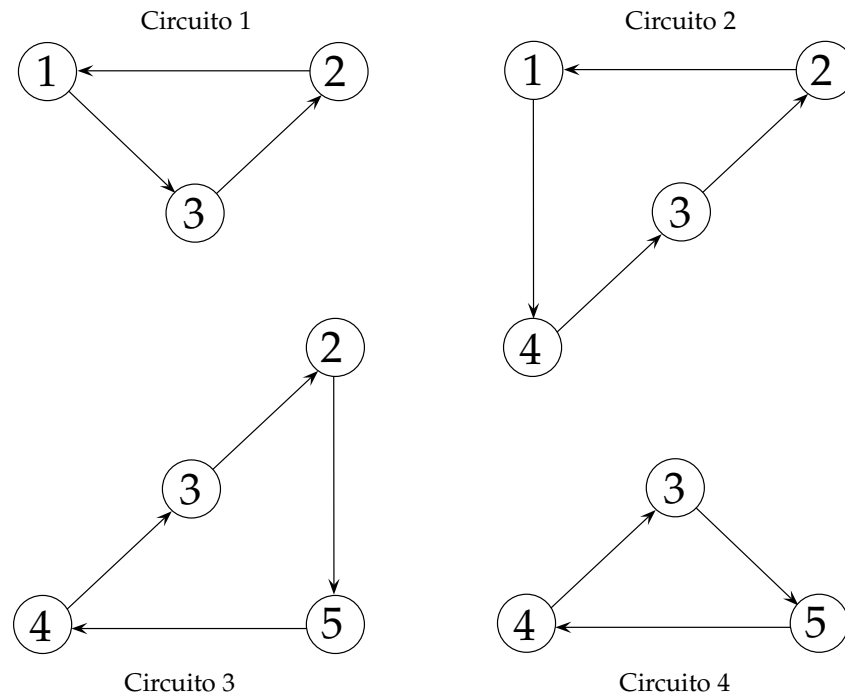


Figura 4.17: Decomposição em circuitos da solução do Exemplo 4.14

□

4.7 Conclusões

Neste capítulo, analisámos vários problemas clássicos de fluxos em rede. Todos os problemas que abordámos são casos especiais do problema de fluxo de custo mínimo. Estes problemas podem ser formulados usando modelos de Programação Linear. Esses modelos têm a propriedade da integralidade (as matrizes de coeficientes dos modelos são totalmente unimodulares): desde que os coeficientes dos termos independentes das restrições sejam inteiros, a solução óptima do modelo será sempre uma solução inteira. Consequentemente, os modelos de Programação Linear podem ser usados mesmo quando as variáveis do problema só podem tomar valores inteiros.

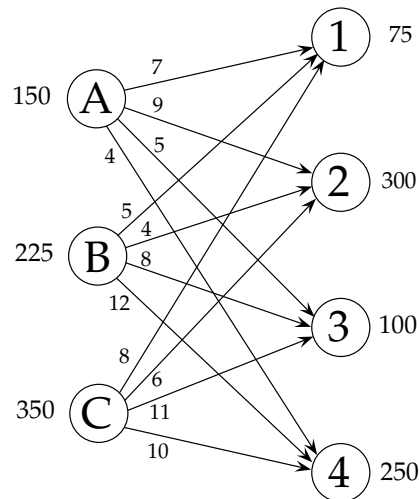
A estrutura particular destes problemas faz com que seja possível recorrer a técnicas eficientes de resolução, muitas delas baseadas na teoria da Programação Linear. O resultado são variantes do método Simplex como o métodos Simplex para transportes, ou o método Simplex de rede.

Por esse motivo, existe todo o interesse em reduzir um problema real de optimização a um problema de fluxos em rede, sempre que isso é possível. Neste texto, descrevemos duas aplicações dos modelos de fluxos em rede. O espectro de aplicação é contudo bem

mais vasto, indo muito além das aplicações imediatas de transporte de bens e pessoas.

4.8 Exercícios

1. Considere o problema de transportes definido a partir do grafo representado abaixo.



- a) Formule o problema usando um modelo de Programação Linear.
 - b) Represente o quadro de transportes deste problema. Determine uma solução válida usando
 - i. a regra do canto noroeste
 - ii. a regra do custo mínimo
 - c) Determine a solução ótima do problema. Existem soluções ótimas alternativas? Em caso afirmativo, indique quais são essas soluções.
 - d) Assuma que a procura no destino 1 passou a ser de 20 unidades. Determine a nova solução ótima para o problema.
2. Um alto comando militar pretende distribuir os seus efectivos por 3 cenários de operações, um localizado no Médio-Oriente, outro no sul de África, e um último situado na Ásia. Os militares disponíveis encontram-se actualmente nos quartéis de Braga, Porto e Lisboa. A experiência dos militares depende das missões em que estiveram envolvidos no passado. O alto comando observou que, em cada quartel, a experiência dos militares era relativamente homogênea. Com vista

a avaliar a adequação dos militares às novas missões, e depois de uma análise exaustiva às folhas de serviço, o alto comando atribuiu uma classificação média (de 0 a 20) por quartel e por cada uma das missões. Os resultados constam da tabela seguinte.

	Médio-Oriente	África	Ásia
Braga	11	15	13
Porto	12	14	16
Lisboa	13	16	12

Neste momento, o quartel de Braga tem 10 militares prontos para serem enviados em missão. Nos quartéis do Porto e Lisboa, existem respectivamente 100 e 200 militares nessas condições. O alto comando estimou que eram necessários 60 militares para o Médio-Oriente, 85 para o sul de África e 115 para a Ásia.

Usando um modelo de transportes, determine a forma como devem ser distribuídos os militares pelas missões.

3. A NESTLU é uma pequena empresa familiar de transformação de produtos alimentares. Actualmente, a empresa dedica-se exclusivamente ao condicionamento e transformação do produto *xyz*. Os fornecedores da empresa são 4 agricultores locais que lhe entregam diariamente várias paletes com o produto *xyz*.

As paletes são processadas por operadores. Cada operador demora 1 hora e meia a processar uma paleta inteira. A partir do momento em que são recebidas, e a cada hora que passa, os produtos armazenados nas paletes deterioram-se. A NESTLU observou que a taxa de deterioração dos produtos dependia da sua proveniência. Essa taxa é indicada na tabela seguinte (em unidades/hora) para cada um dos fornecedores.

Fornecedor A	Fornecedor B	Fornecedor C	Fornecedor D
24	14	18	26

Neste momento, a empresa labora com 3 operadores. A NESTLU acaba de receber 2 paletes do fornecedor A e B, 4 paletes do fornecedor C, e 1 paleta do fornecedor D, e pretende determinar a ordem pela qual cada paleta deve ser processada.

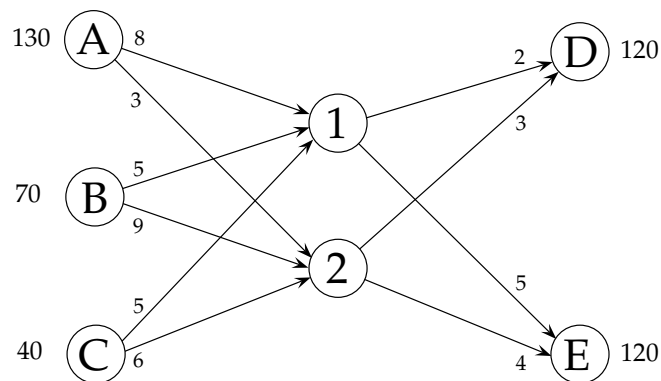
- Formule este problema usando um modelo de transportes.
- Determine a solução óptima do problema.

4. Considere o caso de uma companhia que se dedica ao fabrico de um determinado tipo de artigos. A companhia possui uma única fábrica com capacidade para produzir 500 artigos por mês. A companhia dispõe também de um armazém para o qual encaminha todos os artigos produzidos em excesso em cada mês. A companhia usa o stock para satisfazer a procura dos seus clientes nos meses seguintes. Nos próximos 4 meses, a companhia deverá entregar respectivamente 110, 470, 550, 390 e 480 unidades do seu artigo a um dos seus clientes.

Os custos de produção do artigo variam ao longo do tempo. No próximo mês, a companhia estima que esse custo será de 17 euros por artigo, passando para 19, 22 e 24 euros nos meses seguintes. O custo de posse de stock depende do número de artigos que são armazenados, e do tempo que cada artigo passa em armazém. Por cada artigo que permaneça um mês em armazém, incorre-se num custo de 3 euros. Esse custo é igual a 4 e 5 euros se o artigo ficar 2 e 3 meses em armazém, respectivamente.

Assuma que o stock inicial é nulo, e que as entregas são feitas no final de cada mês.

- a) Formule este problema usando um modelo de transportes.
 - b) Determine a solução óptima do problema.
5. Considere o problema de transbordo definido a partir do grafo seguinte:



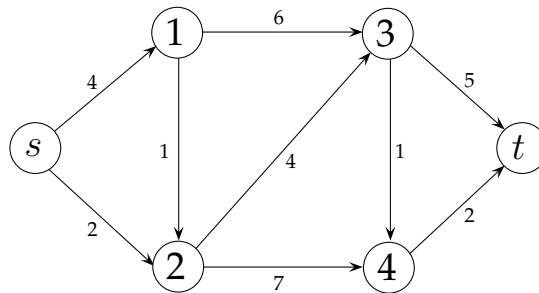
- a) Formule o problema usando um modelo de Programação Linear.
 - b) Determine a solução óptima do problema.
6. Considere o problema de afectação definido através da matriz de custos seguinte.

	1	2	3	4	5
A	7	5	3	2	6
B	9	6	7	3	4
C	9	5	4	1	7
D	8	2	2	1	3
E	4	3	8	2	1

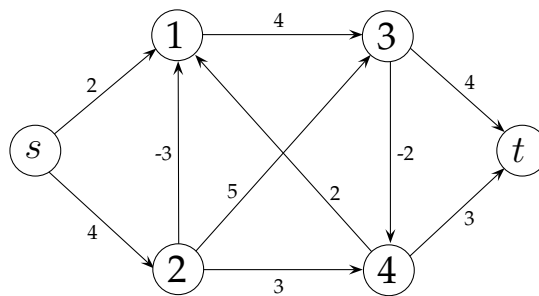
- a) Determine a solução óptima do problema assumindo que se trata de um problema de minimização.
- b) Assuma agora que o problema é de maximização.
- Diga que alterações devem ser introduzidas no problema para que este possa ser resolvido através do algoritmo Húngaro.
 - Determine a solução óptima do problema.
7. Quatro alunos (A, B, C, D) devem escolher a opção cultural que irão frequentar no próximo semestre. Actualmente, existe uma vaga em 5 disciplinas diferentes que iremos designar por “a, b, c, d, e”. A preferência dos alunos está indicada na tabela seguinte (quanto maior for o coeficiente, menor será a preferência do aluno pela disciplina):

	a	b	c	d	e
A	1	2	3	4	5
B	5	3	2	4	1
C	2	1	4	3	5
D	1	2	4	3	5

- a) Determine a forma como devem ser distribuídos os alunos pelas disciplinas de modo a maximizar globalmente a sua satisfação.
- b) Se o aluno B desistir das disciplinas “c” e “e”, qual será o novo plano óptimo de afectação?
8. Considere o problema de mais curto entre o vértice s e t do grafo representado abaixo.



- Formule o problema usando um modelo de Programação Linear.
 - Usando o algoritmo de Dijkstra, determine o caminho mais curto entre s e t .
 - Formule através de um modelo de Programação Linear o problema que consiste em determinar o caminho mais curto entre s e todos os outros vértices do grafo.
 - Use a solução gerada pelo algoritmo de Dijkstra na alínea c) para determinar os caminhos mais curtos de s para todos os outros vértices do grafo.
9. Considere o problema de mais curto entre o vértice s e t do grafo com custos negativos representado abaixo.

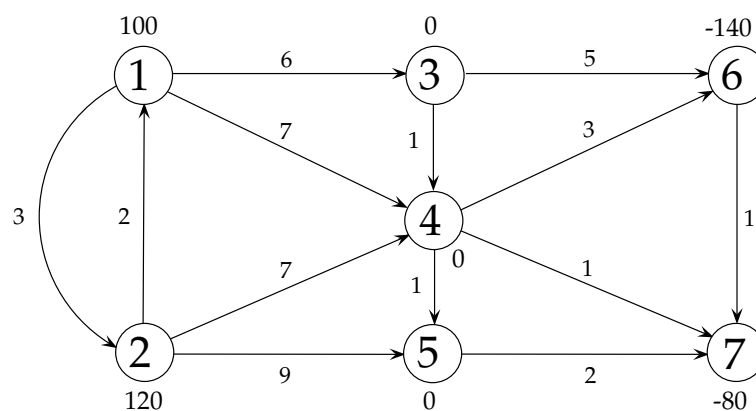


Usando o algoritmo de Bellman-Ford, determine o caminho mais curto entre s e t .

10. Considere um projecto composto por 10 actividades cujas durações e relações de precedência estão indicadas na tabela seguinte:

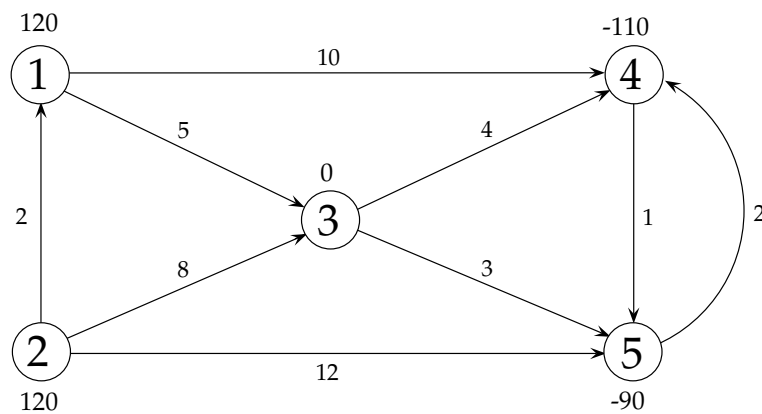
Actividades	Durações	Actividades imediatamente predecessoras
A	2	—
B	3	—
C	5	—
D	1	A,B
E	2	C
F	3	E
G	7	D
H	2	F,G
I	5	H
J	4	F,G

- a) Represente o projecto através de uma rede de actividades nos arcos.
- b) Formule o problema que consiste em determinar o caminho crítico deste projecto como um problema de caminho mais curto.
- c) Determine o caminho crítico do projecto resolvendo o problema que formulou na alínea anterior.
11. Considere o problema de fluxo de custo mínimo definido a partir do grafo seguinte:



- a) Formule o problema usando um modelo de Programação Linear.

- b) Determine uma árvore de suporte válida para o problema, e a solução válida de base que lhe está associada.
- c) Determine a solução ótima do problema usando o método Simplex de rede.
- d) Existem soluções ótimas alternativas? Em caso afirmativo, indique uma dessas soluções.
- 12.** Considere o problema de fluxo de custo mínimo definido a partir do grafo seguinte:



Usando o método Simplex de rede, determine a solução ótima deste problema.

Capítulo 5

Programação Dinâmica

5.1 Introdução

O advento da Programação Dinâmica data de meados do século passado e deve-se fundamentalmente a Richard Bellman, que na altura era investigador na RAND Corporation. Como acontece em muitos outros casos, os princípios da técnica foram esboçados bem antes por outros investigadores. No caso da Programação Dinâmica, os trabalhos de Pierre de Fermat (1601-1665) são apontados como pioneiros. Para a pequena história ficará o facto de Bellman ter escolhido o nome de *programação dinâmica* para designar aquilo que são processos de decisão com vários estágios devido à aversão do Secretário da Defesa norte-americano (que tutela o RAND)¹ por termos como *investigação* ou *matemática*.

A Programação Dinâmica é usada para resolver problemas em que as decisões são tomadas de forma sequencial, e que são susceptíveis de serem decompostos em subproblemas interrelacionados. Permite resolver em particular problemas que verificam o seguinte princípio de optimalidade: dada uma sequência óptima de decisões para um determinado problema, qualquer subsequência dessa sequência é também óptima para o respectivo subproblema. O problema do caminho mais curto ilustra bem esse princípio. Se soubermos, por exemplo, que o caminho mais curto entre duas cidades A e C passa pela cidade B , então o subcaminho que conduz de B a C será também o caminho mais curto entre essas duas cidades. O problema de caminho mais curto verifica por isso o princípio de optimalidade, e pode ser resolvido usando Programação Dinâmica. É importante salientar que esse princípio não se verifica em todos os processos de decisão sequenciais.

A solução de um modelo de Programação Dinâmica obtém-se de forma recursiva

¹S. Dreyfus, Richard Bellman on the Birth of Dynamic Programming, *Operations Research*, 50(1):48–51, 2002.

através da resolução de subproblemas de menor dimensão. A solução óptima do problema original é construída gradualmente com base na solução desses subproblemas. Desta forma, a Programação Dinâmica permite explorar o espaço de soluções de forma eficiente. Evita a enumeração de muitas soluções que não são interessantes do ponto de vista do critério que se pretende otimizar, minorando assim o carácter combinatório dos problemas.

Os modelos de Programação Dinâmica assentam num formalismo diferente de outros modelos de Programação Matemática. Recorrem, por exemplo, a conceitos como o estado do sistema ou a fase (ou estágio) do processo de decisão. Esses conceitos são usados para definir as relações de recorrência a partir das quais são definidas as políticas óptimas dos problemas. Esses conceitos serão explorados em detalhe ao longo deste Capítulo.

5.2 Exemplo

Para introduzirmos os elementos que caracterizam um modelo de Programação Dinâmica, vamos usar o pequeno exemplo seguinte.

Exemplo 5.1 Considere o caso específico de um investidor que dispõe de 10000 euros para investir em 4 planos diferentes. O investidor só pode investir uma vez em cada plano e, se o fizer, terá de investir o montante contratualizado. Esses montantes assim como os respectivos retornos líquidos (em milhares de euros) estão indicados na tabela seguinte:

Plano de Investimento	Montante a Investir	Retorno Líquido
1	6	10
2	5	8
3	3	6
4	1	3

Este problema pode ser formulado como um problema de mochila em que as variáveis de decisão (binárias) estão associadas a cada um dos planos de investimento. Conforme vimos no Capítulo 3 (Exemplo 3.1), o problema pode ser resolvido usando Programação Inteira. Neste Exemplo, mostramos como resolver este problema usando um modelo de Programação Dinâmica.

A solução do problema pode ser decomposta numa sequência de 4 decisões tomadas em estágios diferentes do processo de decisão. Vamos usar a letra n para indexar esses estágios. O 1º estágio ($n = 1$) corresponde ao processo de decisão associado ao plano 1. Nesse estágio, as decisões possíveis correspondem a *investir* ou *não investir* no plano 1. Vamos designar por x_n a variável de decisão associada ao estágio n , e assumir que a decisão de investir no plano i , $i \in \{1, 2, 3, 4\}$, é representada fazendo $x_i = 1$ (e $x_i = 0$ no caso de não investir no plano i).

Num determinado estágio, as decisões que podem ser tomadas dependem do estado do sistema. Neste Exemplo, os estados podem ser associados à quantidade de dinheiro disponível para investir. A decisão que é tomada num estágio define uma transição entre estados do sistema. Uma decisão de investimento reduz o montante de dinheiro disponível para investir nos planos seguintes. Vamos designar por s_n o estado em que se encontra o sistema no estágio n do processo de decisão.

A Figura 5.1 representa a rede completa de políticas válidas para este problema de investimento. Uma política consiste numa sequência de decisões que conduz de um estado inicial no primeiro estágio até outro estado no estágio final. Os estados e os estágios são elementos fundamentais de um modelo de Programação Dinâmica. Na Figura 5.1, os estados são representados na horizontal, enquanto que os estágios são representados na vertical. Os arcos representam decisões. O arco no estágio $n = 1$ que conduz do estado $s_1 = 10$ ao estado $s_2 = 4$ corresponde, por exemplo, a investir (6000 euros) no plano 1 ($x_1 = 1$). A contribuição de cada decisão está representada junto ao respectivo arco. O problema consiste assim em determinar o melhor caminho entre dois estados possíveis do sistema desde o estágio inicial $n = 1$ e até ao estágio final $n = 5$.

Para determinar a solução óptima do modelo, vamos usar um procedimento que começa no estágio final e termina no estágio inicial $n = 1$. Para cada estágio n , procuramos determinar o valor da melhor subpolítica desde todos os estados possíveis nesse estágio até ao estágio final $n = 5$. Depois de concluir a inspeção de um estágio n , passamos ao estágio $n - 1$. A solução óptima do problema global é obtida após a inspeção do estágio $n = 1$. Esta abordagem sugere um procedimento recursivo em que as soluções para um estágio n são obtidas com base nas soluções para o estágio $n - 1$. Os passos deste procedimento para este Exemplo são descritos a seguir.

No estágio final $n = 5$, não havendo mais planos de investimento, não haverá lugar a mais decisões. O valor das subpolíticas óptimas desde todos os estados possíveis nesse estágio é igual a 0. Esses valores são valores iniciais para o nosso procedimento de resolução, e baseiam-se na assunção de que o investidor não lucra nada com o dinheiro não investido. Em alternativa, poderíamos ter considerado uma transição desde todos

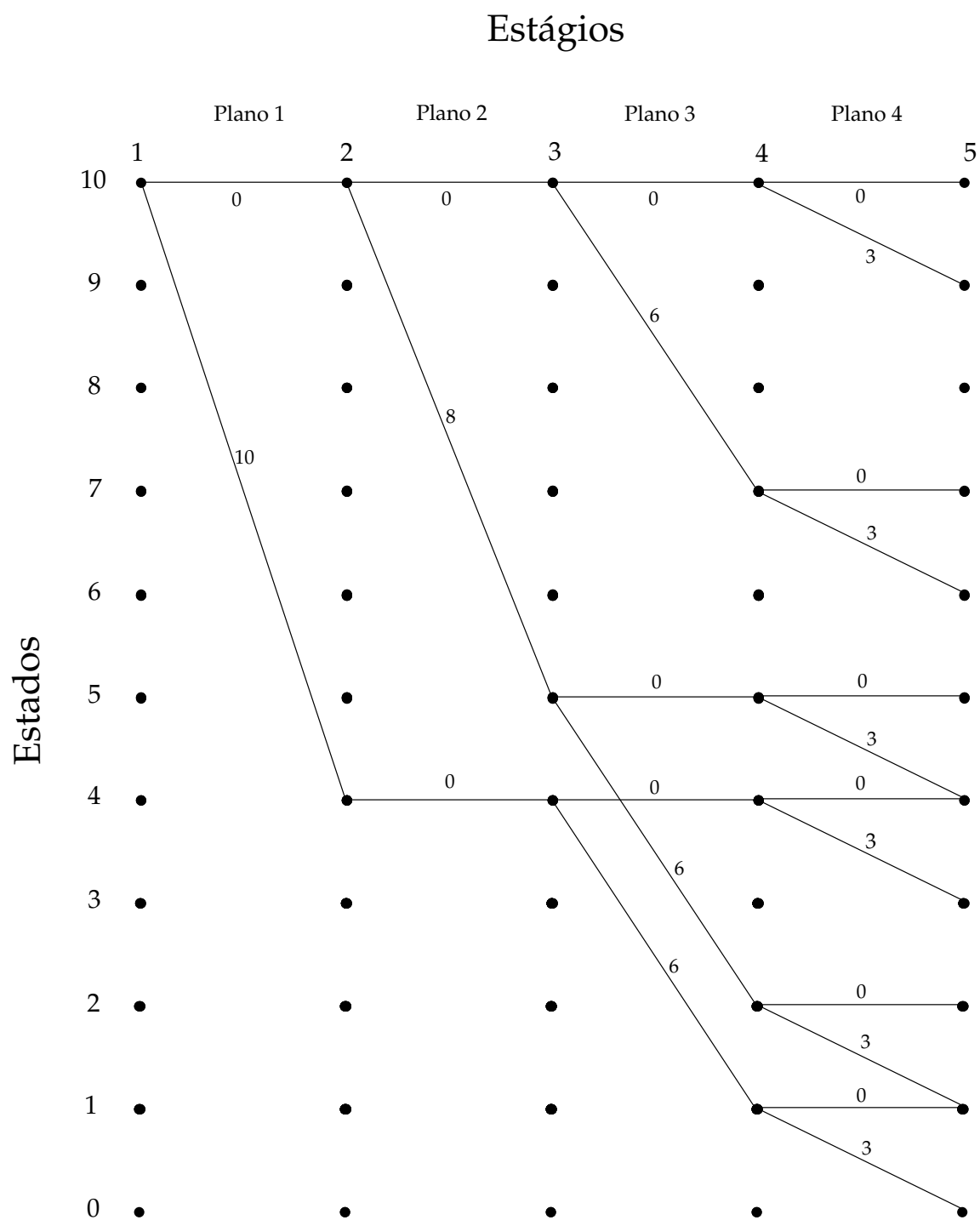


Figura 5.1: Modelo de Programação Dinâmica - Rede de Políticas (Exemplo 5.1)

os estados do estágio $n = 5$ até um estado terminal único num estágio $n = 6$. Nesse caso, a contribuição das respectivas decisões seria nula, e o valor da subpolítica óptima desde esse estado terminal seria também igual a 0.

No estágio $n = 4$, existem 6 estados possíveis ($s_4 = \{10, 7, 5, 4, 2, 1\}$). Se partirmos do estado $s_4 = 10$, a melhor subpolítica consistirá em investir no plano 4 ($x_4 = 1$). O retorno associado será de 3000 euros. Esse retorno corresponde ao valor da contribuição associada à decisão de investimento somado ao valor da subpolítica óptima desde o estado $s_5 = 9$ (estágio $n = 5$). A mesma conclusão aplica-se aos restantes estados do estágio $n = 4$. A Figura 5.2 ilustra o resultado deste procedimento para os diversos estágios. Os arcos a negrito identificam as subpolíticas óptimas. Os valores junto aos estados identificam o valor da função objectivo associado a cada uma dessas subpolíticas.

Depois de ter atribuído um valor a cada um dos estados do estágio $n = 4$, recuamos de um estágio, e procuramos determinar a subpolítica óptima desde todos os estados possíveis do estágio $n = 3$. Vamos usar como exemplo o estado $s_3 = 5$. Na Figura 5.2, podemos observar que existem dois caminhos (políticas) possíveis. O primeiro passa por não investir no plano 3 ($x_3 = 0$), transitar para o estado $s_4 = 5$, e escolher a subpolítica óptima desde o estado $s_4 = 5$. O valor associado a essa política é de 3000 euros, e é obtido somando a contribuição dessa decisão com o valor da subpolítica óptima desde o estado $s_4 = 5$. O segundo caminho tem um valor associado de 9000 euros, e corresponde assim à subpolítica óptima desde o estado $s_3 = 5$.

Este procedimento é repetido até atingirmos o estágio inicial $n = 1$. Nesse estágio, a melhor política desde o único estado possível $s_1 = 10$ tem um valor associado de 19000 euros. Essa política corresponde a investir no plano 1 ($x_1 = 1$), e escolher a subpolítica óptima desde o estado $s_2 = 4$ que corresponde a não investir no plano 2, e investir nos planos 3 e 4. Esta política considera todos os planos de investimento, e é por isso, a solução óptima do problema. $\square$

Os elementos introduzidos neste Exemplo são formalizados na Secção seguinte.

5.3 Terminologia e Definições

Os modelos de Programação Dinâmica assentam num formalismo diferente daquele que é usado noutras técnicas de Programação Matemática. Os elementos que caracterizam esses modelos podem ser descritos formalmente conforme segue:

- um modelo de Programação Dinâmica divide-se numa sequência de estágios que correspondem às várias fases do processo de decisão. Em cada estágio, é tomada

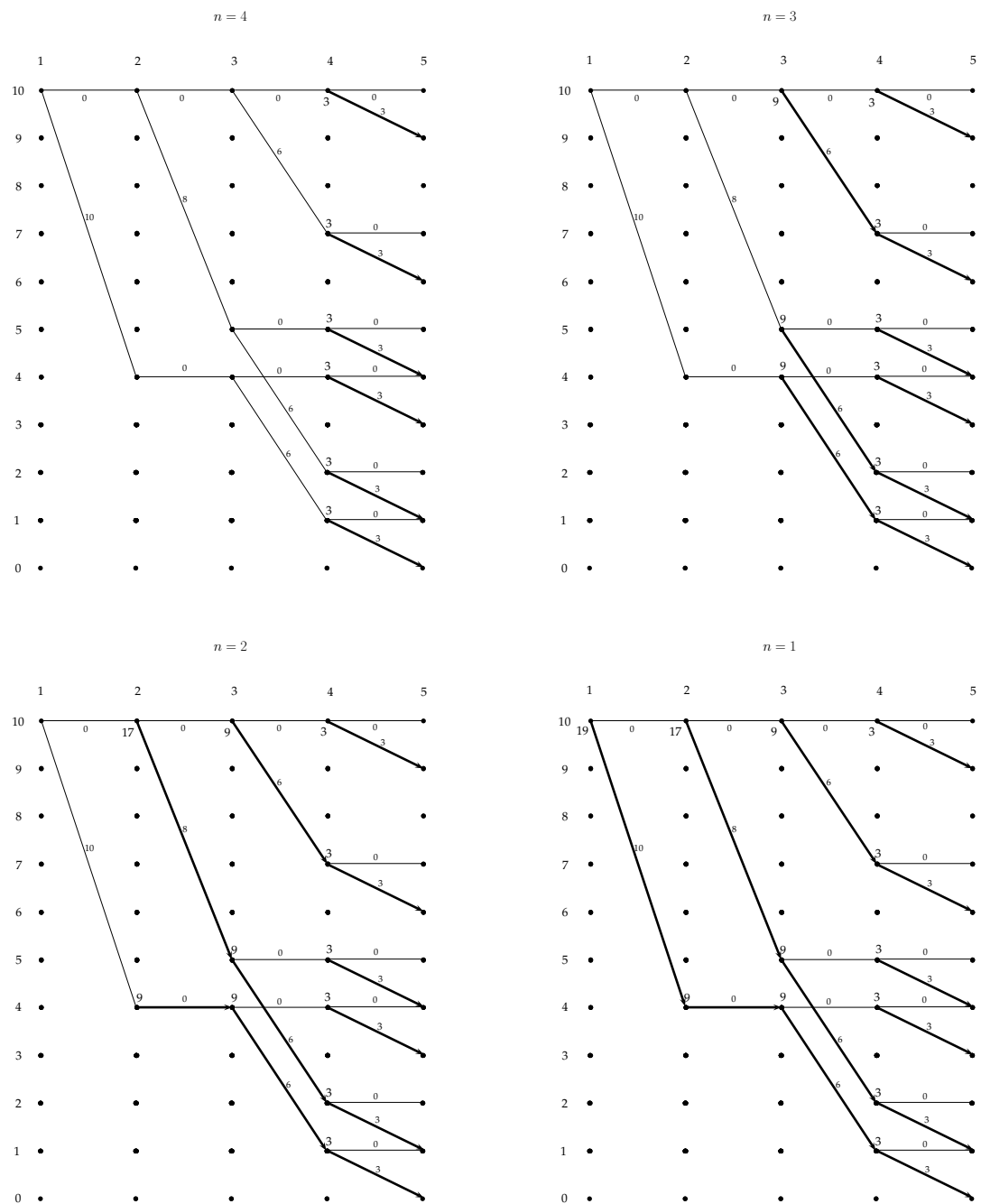


Figura 5.2: Subpolíticas óptimas desde o estágio n (Exemplo 5.1)

uma decisão (montante a investir num plano de investimento, próxima cidade a visitar, quantidade de recursos a gastar numa actividade, por exemplo). Essa decisão é escolhida dentro de um conjunto de alternativas, e permite transitar para o estágio seguinte;

- os estados definem as condições em que se encontra o sistema. As decisões que podem ser tomadas em cada estágio dependem do estado de partida, e afectam directamente o estado para o qual transita o sistema: uma decisão leva assim a uma transição entre estados de um estágio ao estágio seguinte do processo de decisão (tal como fizemos acima, iremos designar por s_n o estado em que se encontra o sistema no estágio n);
- as sequências de decisões são designadas por *políticas*; esse termo é usado devido ao facto da sequência óptima de decisões depender do estado de partida. Vamos designar por x_n a variável que identifica a decisão tomada no estágio n . Em Programação Dinâmica, a política óptima para o problema global é determinada de forma recursiva, calculando passo a passo as políticas óptimas associadas a subproblemas de menor dimensão para qualquer estado de partida possível;
- o valor da função objectivo associado a uma política que parte de um estado s_n no estágio n depende da decisão x_n que é tomada nesse estágio, e do valor da política óptima desde o estado s_{n+1} (o estado para o qual se transita partindo de s_n e tomando a decisão associada a x_n). Vamos designar esse valor da função objectivo por $f_n(s_n, x_n)$ ¹. O valor da política óptima desde o estado s_n do estágio n é definido através de uma relação de recorrência que designaremos por $f_n^*(s_n)$. Para um problema de maximização temos que:

$$f_n^*(s_n) = \max_{x_n} f_n(s_n, x_n).$$

No Exemplo 5.1, se designarmos por $p_n(x_n)$ a contribuição associada à variável de decisão x_n , e por $q_n(x_n)$ o respectivo montante a investir, teremos que:

$$f_n(s_n, x_n) = p_n(x_n) + f_{n+1}^*(s_n - q_n(x_n)).$$

Partindo de um estado s_n , o estado para o qual se transita quando é tomada a decisão associada a x_n é o estado $s_{n+1} = s_n - q_n(x_n)$ (por exemplo, para $s_1 = 10$ e $x_1 = 1$, temos que $s_2 = 10 - 6 = 4$).

¹Ao usarmos os valores para os estágios seguintes ($n + 1$), estamos a assumir que o problema é resolvido como no Exemplo 5.1 do fim para o início.

Existem vários tipos de relações de recorrência: minimização, maximização, com contribuições aditivas ou multiplicativas, do tipo MinMax, por exemplo. Mais adiante, exploraremos alguns casos.

- o princípio de optimalidade (que tem vários enunciados, entre os quais o de Bellman) no qual assenta a Programação Dinâmica, verifica-se quando qualquer subpolítica da política ótima, associada a estágios consecutivos do problema, é também ótima para o subproblema associado a esses estágios. Os problemas que verificam esse princípio podem assim ser resolvidos de forma recursiva, resolvendo passo a passo subproblemas de menor dimensão (com menos estágios). Um problema que não verifique o princípio de optimalidade não pode ser resolvido usando Programação Dinâmica.

5.4 Princípio de Optimalidade

O princípio de optimalidade é fundamental em Programação Dinâmica. Apesar desse princípio ser descrito de formas diferentes na literatura, as propriedades que destaca são essencialmente as mesmas. O princípio de optimalidade determina que, para alguns problemas de decisão sequencial, a política ótima é composta por subpolíticas que são também elas ótimas para os respectivos subproblemas, podendo assim ser construída com base nestas últimas. Nos problemas em que esse princípio se aplica, a política ótima desde um estágio n até ao final depende apenas do estado s_n no qual se encontra o sistema no momento da tomada de decisão, e é independente da política que conduziu a esse estado. De forma equivalente, a política ótima que conduz do estado inicial até um determinado estado s_n é independente da política que possa ser seguida do estágio n em diante.

O princípio de optimalidade é expresso de forma algébrica através das funções $f_n^*(s_n)$ e $f_n(s_n, x_n)$ que determinam respectivamente o valor da política ótima desde um estado s_n e o valor da política associada a uma decisão particular x_n no estágio n . No Exemplo 5.1, ao definirmos a primeira dessas funções da forma seguinte

$$f_n^*(s_n) = \max_{x_n} f_n(s_n, x_n) = \max_{x_n} \{p_n(x_n) + f_{n+1}^*(s_n - q_n(x_n))\},$$

assumimos que a política ótima desde o estado s_n do estágio n até ao último estágio do problema dependia da política ótima para os estágios seguintes partindo do estado s_{n+1} (que é alcançado quando é tomada a decisão x_n no estágio n).

As relações de recorrência dos modelos de Programação Dinâmica, que traduzem o princípio de optimalidade, sugerem também um procedimento de resolução para estes

modelos. No caso da função $f_n^*(s_n)$ definida acima, esse procedimento inicia-se no último estágio do modelo: nesse estágio, consideram-se valores iniciais para o valor a atribuir aos estados válidos do sistema conforme vimos no Exemplo da Secção anterior. O passo seguinte consiste em decrementar de uma unidade o índice do estágio, e determinar o valor de $f_n^*(s_n)$ para todos os estados s_n possíveis desse estágio. Esses valores dependem das contribuições de estágio associadas a cada decisão possível, e do valor das políticas óptimas $f_{n+1}^*(s_{n+1})$ para os estágios seguintes. Esse passo corresponde à resolução de um subproblema de menor dimensão onde só se consideram os últimos estágios do problema desde o estágio n . O processo termina com o cálculo de $f_1^*(s_1)$. Este procedimento recursivo resolve o problema de trás para a frente, desde o último estágio até ao estágio inicial.

Em alternativa, as relações de recorrência podem ser expressas de forma a traduzir a evolução natural do processo de decisão (do início para o fim). No caso do Exemplo 5.1, a relação de recorrência poderia ser expressa da forma seguinte:

$$f_n^*(s_n) = \max_{x_n} \{p_n(x_n) + f_{n-1}^*(s_n + q_n(x_n))\}.$$

O Exemplo seguinte ilustra a aplicação deste procedimento na resolução de um problema de caminho mais curto.

Exemplo 5.2 Vamos considerar o problema que consiste em determinar o caminho mais curto entre os vértices A e K da rede ilustrada na Figura 5.3. A distância d_{ij} entre dois vértices i e j da rede está indicada junto ao arco que liga esses vértices.

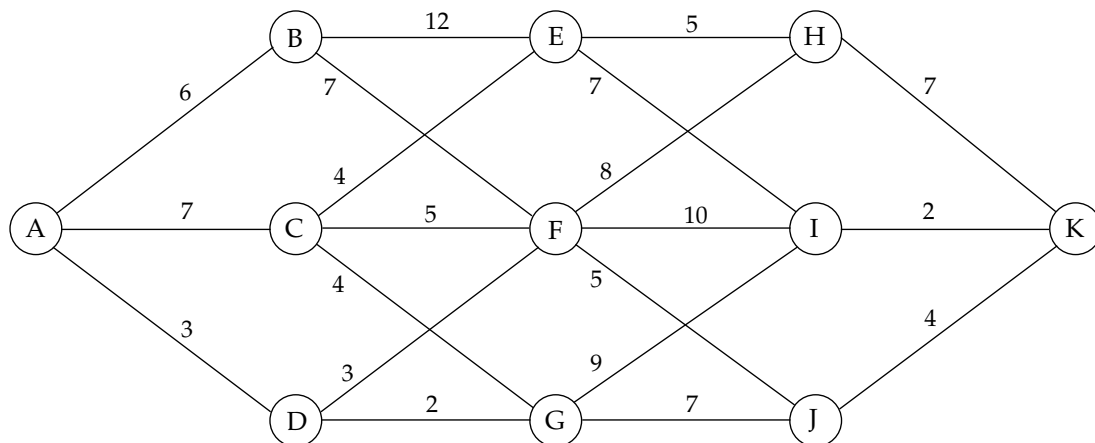


Figura 5.3: Problema de caminho mais curto (Exemplo 5.2)

O problema pode ser representado através de um modelo de Programação Dinâmica em que os estados s_n estão associados aos vértices da rede, e os estágios correspondem

aos passos que são dados em direcção ao vértice K . As variáveis de decisão do modelo determinam os passos dados em cada estágio, e podem ser definidos da forma seguinte:

$$x_n = \text{próximo vértice a visitar no passo } n, n = 1, \dots, 4.$$

O valor da política desde um estado s_n do estágio n , quando é tomada a decisão associada à variável x_n , é dado pela expressão seguinte:

$$f_n(s_n, x_n) = d_{s_n x_n} + f_{n+1}^*(s_{n+1}).$$

O valor da política óptima desde um estado s_n corresponde assim ao menor desses valores tendo em conta todas as decisões que podem ser tomadas nesse estado e estágio:

$$f_n^*(s_n) = \min_{x_n} \{d_{s_n x_n} + f_{n+1}^*(s_{n+1})\}.$$

A relação de recorrência é definida de trás para a frente. O procedimento de resolução associado inicia-se por isso no último estágio $n = 5$. Nesse estágio, o único estado possível corresponde ao vértice K ($s_5 = K$), o destino do caminho que se pretende determinar. Não havendo lugar a decisões nesse estágio, é atribuído o valor inicial 0 à política óptima desde esse estado:

$$f_5^*(K) = 0.$$

Recuando de um estágio, procuramos determinar o valor das políticas óptimas desde todos os estados possíveis do estágio $n = 4$:

$n = 4$

$s_4 = H$

$$f_4(H, K) = d_{HK} + f_5^*(K) = 7 + 0 = 7$$

$$f_4^*(H) = \min_{x_4} \{f_4(H, x_4)\} = \min\{7\} = 7$$

$s_4 = I$

$$f_4(I, K) = d_{IK} + f_5^*(K) = 2 + 0 = 2$$

$$f_4^*(I) = \min_{x_4} \{f_4(I, x_4)\} = \min\{2\} = 2$$

$s_4 = J$

$$f_4(J, K) = d_{JK} + f_5^*(K) = 4 + 0 = 4$$

$$f_4^*(J) = \min_{x_4} \{f_4(J, x_4)\} = \min\{4\} = 4$$

Determinar o valor das políticas óptimas desde o estágio $n = 4$ é trivial, uma vez que a única decisão possível consiste em caminhar para o vértice K . Nos estágios seguintes, o facto de existirem decisões alternativas para os diferentes estados do sistema, obriga a um maior esforço computacional. Contudo, deve-se ter em atenção o facto de só serem

exploradas as decisões individuais relativas a um estágio de cada vez, sendo o valor das políticas óptimas desde os estados que são atingidos usados como referência para avaliar a política para os estágios seguintes.

O valor das políticas desde o estágio $n = 3$ é obtido com base nos valores das políticas óptimas desde o estágio $n = 4$ (relação de recorrência).

$n = 3$

$s_3 = E$

$$\begin{aligned} f_3(E, H) &= d_{EH} + f_4^*(H) &= 5 + 7 &= 12 \\ f_3(E, I) &= d_{EI} + f_4^*(I) &= 7 + 2 &= 9 \\ f_3^*(E) &= \min_{x_3} \{f_3(E, x_3)\} &= \min\{12, 9\} &= 9 \end{aligned}$$

$s_3 = F$

$$\begin{aligned} f_3(F, H) &= d_{FH} + f_4^*(H) &= 8 + 7 &= 15 \\ f_3(F, I) &= d_{FI} + f_4^*(I) &= 10 + 2 &= 12 \\ f_3(F, J) &= d_{FJ} + f_4^*(J) &= 5 + 4 &= 9 \\ f_3^*(F) &= \min_{x_3} \{f_3(F, x_3)\} &= \min\{15, 12, 9\} &= 9 \end{aligned}$$

$s_3 = G$

$$\begin{aligned} f_3(G, I) &= d_{GI} + f_4^*(I) &= 9 + 2 &= 11 \\ f_3(G, J) &= d_{GJ} + f_4^*(J) &= 7 + 4 &= 11 \\ f_3^*(G) &= \min_{x_3} \{f_3(G, x_3)\} &= \min\{11, 11\} &= 11 \end{aligned}$$

Os valores para os estágios $n = 2$ e $n = 1$ são obtidos da mesma forma por aplicação da relação de recorrência. A resolução do problema termina no estágio $n = 1$ com a política ótima para o problema global.

$n = 2$

$s_2 = B$

$$\begin{aligned} f_2(B, E) &= d_{BE} + f_3^*(E) &= 12 + 9 &= 21 \\ f_2(B, F) &= d_{BF} + f_3^*(F) &= 7 + 9 &= 16 \\ f_2^*(B) &= \min_{x_2} \{f_2(B, x_2)\} &= \min\{21, 16\} &= 16 \end{aligned}$$

$s_2 = C$

$$\begin{aligned} f_2(C, E) &= d_{CE} + f_3^*(E) &= 4 + 9 &= 13 \\ f_2(C, F) &= d_{CF} + f_3^*(F) &= 5 + 9 &= 14 \\ f_2(C, G) &= d_{CG} + f_3^*(G) &= 4 + 11 &= 15 \\ f_2^*(C) &= \min_{x_2} \{f_2(C, x_2)\} &= \min\{13, 14, 15\} &= 13 \end{aligned}$$

$s_2 = D$

$$\begin{aligned} f_2(D, F) &= d_{DF} + f_3^*(F) &= 3 + 9 &= 12 \\ f_2(D, G) &= d_{DG} + f_3^*(G) &= 2 + 11 &= 13 \\ f_2^*(D) &= \min_{x_2} \{f_2(D, x_2)\} &= \min\{12, 13\} &= 12 \end{aligned}$$

$$n = 1$$

$$s_1 = A$$

$$\begin{aligned} f_1(A, B) &= d_{AB} + f_2^*(B) &= 6 + 16 &= 22 \\ f_1(A, C) &= d_{AC} + f_2^*(C) &= 7 + 13 &= 20 \\ f_1(A, D) &= d_{AD} + f_2^*(D) &= 3 + 12 &= 15 \\ f_1^*(A) &= \min_{x_1} \{f_2(C, x_1)\} &= \min\{22, 20, 15\} &= 15 \end{aligned}$$

O valor da política óptima (caminho mais curto) para este problema é igual a 15. As decisões individuais que fazem parte dessa política são determinadas usando os resultados dos cálculos efectuados partindo do estágio $n = 1$. Nesse estágio, a melhor decisão consiste em caminhar para o vértice D ($d_{AD} = 3$), e seguir então pelo caminho mais curto entre D e K . Do estado $s_2 = D$, caminha-se para o vértice F ($d_{DF} = 3$), e segue-se novamente pelo caminho mais curto entre F e K . De F , seguimos para J ($d_{FJ} = 5$). O caminho mais curto de J para K é a ligação directa entre os dois vértices ($d_{JK} = 4$). O caminho mais curto entre A e K corresponde assim ao percurso $A - D - F - J - K$. $\square$

5.5 Algoritmo de Programação Dinâmica

O Algoritmo 5.1 sintetiza os passos do procedimento de resolução de modelos de Programação Dinâmica que usámos nos exemplos das secções anteriores. Na descrição que é feita, assumimos que o problema é de minimização, que as contribuições nos estágios são aditivas e que a relação de recorrência é expressa de trás para a frente. Este procedimento de resolução é também designado por *algoritmo de recorrência para trás*.

O *algoritmo de recorrência para diante* é usado para resolver problemas cujas relações de recorrência são expressas da frente para trás. Esse algoritmo constroi de forma recursiva a política óptima do problema adicionando gradualmente estágios que estão à frente do estágio inicial. As características dos dois métodos são essencialmente as mesmas.

5.6 Outras Relações de Recorrência

As relações de recorrência que usámos até agora baseiam-se todas em contribuições aditivas: o valor de uma política corresponde à soma da contribuição num estágio e do valor de uma política óptima para os estágios seguintes. Em muitos casos, o valor de uma política é obtida a partir de relações onde as contribuições não são aditivas. Nesta Secção, analisamos o caso particular das contribuições multiplicativas, e das relações

Algoritmo 5.1: Algoritmo de Programação Dinâmica

- 1 **Input:** um processo de decisão sequencial separável em N estágios;
- 2 Inicializar o valor da política óptima desde o estágio $N + 1$ fazendo

$$f_{N+1}^*(s_{N+1}) = \text{valor associado ao estado final } s_{N+1};$$

/* Um modelo com vários estados finais pode ser transformado noutra modelo equivalente com apenas um estado final s_{N+1} , adicionando um estágio e ligando todos os estados s_N ao estado s_{N+1} */

- 3 **Para** $n = N, \dots, 1$ **fazer**

- 4 **Para todos** os estados possíveis s_n no estágio n **fazer**

- 5 **Para todos** os valores possíveis de x_n a partir do estado s_n **fazer**

- 6 Calcular $f_n(s_n, x_n) = c_{s_n s_{n+1}} + f_{n+1}^*(s_{n+1})$;

/* s_{n+1} corresponde ao estado alcançado a partir de s_n quando é tomada a decisão representada por x_n ; $c_{s_n s_{n+1}}$ representa a contribuição associada a essa decisão */

- 7 Calcular $f_n^*(s_n) = \min_{x_n} f_n(s_n, x_n)$;

- 8 Seja x_n^* o valor óptimo de x_n tal que $f_n(s_n, x_n^*) = \min_{x_n} f_n(s_n, x_n)$;

- 9 **Output:** Política óptima $x_1^*, x_2^*, \dots, x_N^*$ com valor $f_1^*(s_1)$ (estado inicial s_1);

- 10 **Stop** ;
-

de recorrência do tipo *MinMax* e *MaxMin*. Começamos por considerar um caso que pode ser aplicado a diferentes tipos de relações de recorrência, e que consiste em usar factores de actualização no cálculo do valor das políticas. Essa variante é apresentada tendo como base uma relação de recorrência com contribuições aditivas.

É importante notar que algumas relações de recorrência podem não verificar o princípio de optimalidade. É o caso, por exemplo, das relações de recorrência multiplicativas com contribuições negativas nos estágios.

5.6.1 Relações de Recorrência com Contribuições Descontadas

Em problemas em que são tomadas decisões em diferentes períodos de tempo, e que envolvem factores de carácter financeiro, é importante traduzir nos respectivos modelos a evolução natural do valor do dinheiro ao longo do tempo. As relações de recorrência

que temos vindo a usar não consideram esse aspecto. O peso das contribuições nos estágios é idêntico para todos os estágios do modelo.

A actualização do valor do dinheiro pode ser feita introduzindo nas relações de recorrência um factor de desconto β . No caso de uma relação de recorrência aditiva do tipo *maximização*, a função que define o valor de uma política desde um estado s_n quando é tomada a decisão x_n passa a ser expressa da forma seguinte:

$$f_n(s_n, x_n) = c_{s_n s_{n+1}} + \beta f_{n+1}^*(s_{n+1}).$$

Nos casos em que se considera uma taxa de juro sobre o capital de $\rho\%$, o factor de desconto a aplicar será de $\beta = \frac{1}{1-\rho}$. Temos que $\beta \in]0, 1[$, razão pela qual esse factor de actualização é designado por *factor de desconto*. Esse factor de desconto traduz o valor presente do dinheiro em períodos futuros.

A tabela seguinte descreve o efeito da conversão em valores presentes dos valores das subpolíticas óptimas em períodos futuros. Nessa tabela, designamos por s_n^* o estado em que se encontra sistema no estágio n quando é usada a política óptima para o problema. O valor $c_{s_n s_{n+1}}$ designa a contribuição no estágio n associada à transição entre o estado s_n e o estado s_{n+1} .

Estágio n	Valor da política óptima desde o estágio n sem factor de actualização	Valor presente
N	$c_{s_N^* s_{N+1}^*}$	$\beta^{N-1} c_{s_N^* s_{N+1}^*}$
$N-1$	$c_{s_{N-1}^* s_N^*} + c_{s_N^* s_{N+1}^*}$	$\beta^{N-2} c_{s_{N-1}^* s_N^*} + \beta^{N-1} c_{s_N^* s_{N+1}^*}$
$\dots$	$\dots$	$\dots$
2	$c_{s_2^* s_3^*} + c_{s_3^* s_4^*} + \dots + c_{s_N^* s_{N+1}^*}$	$\beta c_{s_2^* s_3^*} + \beta^2 c_{s_3^* s_4^*} + \dots + \beta^{N-1} c_{s_N^* s_{N+1}^*}$
1	$c_{s_1^* s_2^*} + c_{s_2^* s_3^*} + \dots + c_{s_N^* s_{N+1}^*}$	$c_{s_1^* s_2^*} + \beta c_{s_2^* s_3^*} + \dots + \beta^{N-1} c_{s_N^* s_{N+1}^*}$

O factor de desconto pode ser usado para formular situações de depreciação menos explícitas do que aquelas que se verificam em transacções simples de dinheiro em que a taxa de juro o factor de actualização de referência. Ilustramos um caso desse tipo através do Exemplo seguinte.

Exemplo 5.3 Vamos considerar o caso de um agricultor proprietário de uma exploração de ovelhas que se quer desfazer de todas as suas ovelhas no espaço de 3 anos.

Actualmente, a exploração é constituída por 300 ovelhas. As ovelhas são vendidas em grupos de 100. O agricultor estima que os lucros das vendas realizadas nos próximos 3 anos será de 0.5, 0.6 e 0.75 unidades monetárias por ovelha. O agricultor prevê também que todos os anos 5% das ovelhas morrerão em média de doença. Vamos assumir que as ovelhas da exploração não se reproduzem. O agricultor pretende determinar o número de ovelhas que terá de vender anualmente para maximizar os seus lucros.

O problema desse agricultor pode ser formulado usando um modelo de Programação Dinâmica em que os estados representam o número de ovelhas que ainda não foram vendidas, e os estágios correspondem aos períodos (anos) em que são tomadas as decisões de venda. As variáveis de decisão x_n , $n = 1, \dots, 3$, do modelo correspondem ao número de ovelhas vendidas. Temos assim que:

$$x_n \in \{0, 100, 200, 300\}, \quad n = 1, \dots, 3.$$

A redução anual da exploração causada pelas mortes das ovelhas pode ser modelada de duas formas. A primeira consiste em considerar explicitamente a redução de 5% do número de ovelhas no final de cada ano. Por exemplo, se no 1º ano não for vendida nenhuma ovelha, o número de ovelhas no início do 2º ano passa para 285 em vez de 300. A segunda abordagem consiste em imputar a taxa de mortalidade directamente aos lucros das vendas realizadas em cada ano. Por exemplo, se forem vendidas 100 ovelhas no 2º ano, o lucro dessa venda será apenas de 57 unidades monetárias ($0.95 \times 100 \times 0.6$) em vez de 60 unidades monetárias (100×0.6). Por seu lado, o número de ovelhas disponíveis mantém-se inalterado. O factor β de desconto a aplicar neste caso é de 95%. A rede de políticas relativa a esta segunda abordagem é ilustrada na Figura 5.4.

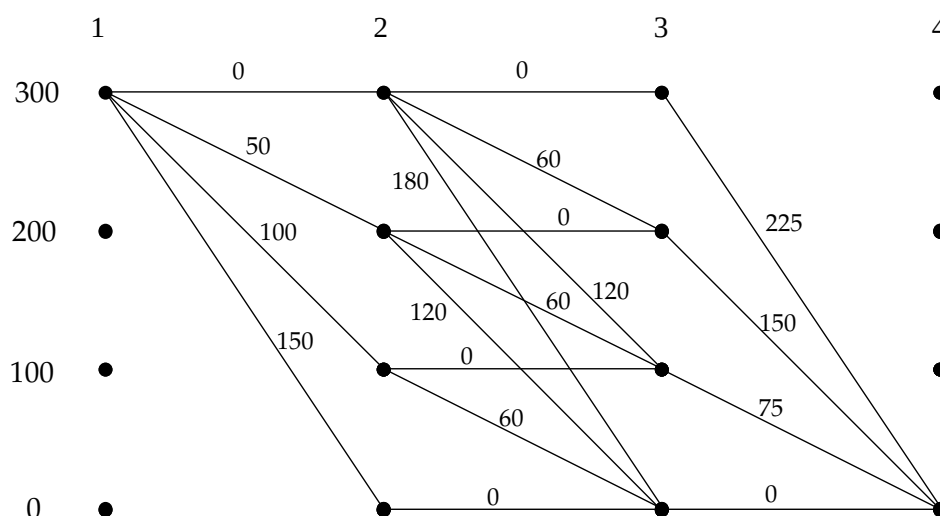


Figura 5.4: Modelo de Programação Dinâmica - Rede de Políticas (Exemplo 5.3)

O factor de desconto é usado na relação de recorrência da forma seguinte:

$$f_n^*(s_n) = \max_{x_n} f_n(s_n, x_n) = \max_{x_n} \{p_n(x_n) + 0.95f_{n+1}^*(s_n - x_n)\},$$

em que $p_n(x_n)$ representa o lucro associado à venda de x_n ovelhas no ano n .

A aplicação do algoritmo de Programação Dinâmica a este modelo resulta nos valores de $f_n(s_n, x_n)$ e $f_n^*(s_n)$ que indicamos a seguir. O algoritmo é inicializado com $f_4^*(0) = 0$.

$n = 3$

$s_3 = 300$

$$f_3(300, 0) = p_3(300) + 0.95f_4^*(0) = 225 + 0.95 \times 0 = 225$$

$$f_3^*(300) = \max_{x_3} \{f_3(300, x_3)\} = 225$$

$s_3 = 200$

$$f_3(200, 0) = p_3(200) + 0.95f_4^*(0) = 150 + 0.95 \times 0 = 150$$

$$f_3^*(200) = \max_{x_3} \{f_3(200, x_3)\} = 150$$

$s_3 = 100$

$$f_3(100, 0) = p_3(100) + 0.95f_4^*(0) = 75 + 0.95 \times 0 = 75$$

$$f_3^*(100) = \max_{x_3} \{f_3(100, x_3)\} = 75$$

$s_3 = 0$

$$f_3(0, 0) = p_3(0) + 0.95f_4^*(0) = 0 + 0.95 \times 0 = 0$$

$$f_3^*(0) = \max_{x_3} \{f_3(0, x_3)\} = 0$$

$n = 2$

$s_2 = 300$

$$f_2(300, 300) = p_2(0) + 0.95f_3^*(300) = 0 + 0.95 \times 225 = 213.75$$

$$f_2(300, 200) = p_2(100) + 0.95f_3^*(200) = 60 + 0.95 \times 150 = 202.5$$

$$f_2(300, 100) = p_2(200) + 0.95f_3^*(100) = 120 + 0.95 \times 75 = 191.25$$

$$f_2(300, 0) = p_2(300) + 0.95f_3^*(0) = 180 + 0.95 \times 0 = 180$$

$$f_2^*(300) = \max_{x_2} \{f_2(300, x_2)\} = 213.75$$

$s_2 = 200$

$$f_2(200, 200) = p_2(0) + 0.95f_3^*(200) = 0 + 0.95 \times 150 = 142.5$$

$$f_2(200, 100) = p_2(100) + 0.95f_3^*(100) = 60 + 0.95 \times 75 = 131.25$$

$$f_2(200, 0) = p_2(200) + 0.95f_3^*(0) = 120 + 0.95 \times 0 = 120$$

$$f_2^*(200) = \max_{x_2} \{f_2(200, x_2)\} = 142.5$$

$s_2 = 100$

$$f_2(100, 100) = p_2(0) + 0.95f_3^*(100) = 0 + 0.95 \times 75 = 71.25$$

$$f_2(100, 0) = p_2(100) + 0.95f_3^*(0) = 60 + 0.95 \times 0 = 60$$

$$f_2^*(100) = \max_{x_2} \{f_2(100, x_2)\} = 71.25$$

$$s_2 = 0$$

$$\begin{aligned} f_2(0, 0) &= p_2(0) + 0.95f_3^*(0) = 0 + 0.95 \times 0 = 0 \\ f_2^*(0) &= \max_{x_2} \{f_2(0, x_2)\} = 0 \end{aligned}$$

$$n = 1$$

$$s_1 = 300$$

$$\begin{aligned} f_1(300, 300) &= p_1(0) + 0.95f_2^*(300) = 0 + 0.95 \times 213.75 = 203.06 \\ f_1(300, 200) &= p_1(100) + 0.95f_2^*(200) = 50 + 0.95 \times 142.5 = 185.38 \\ f_1(300, 100) &= p_1(200) + 0.95f_2^*(100) = 100 + 0.95 \times 71.25 = 167.69 \\ f_1(300, 0) &= p_1(300) + 0.95f_2^*(0) = 150 + 0.95 \times 0 = 150 \\ f_1^*(300) &= \max_{x_1} \{f_1(300, x_1)\} = 203.06 \end{aligned}$$

A política ótima tem um lucro associado de 203.06 unidades monetárias, e corresponde a vender todas as ovelhas no 3º ano. $\square$

5.6.2 Contribuições Multiplicativas

Em alguns processos de decisão, o valor das políticas é obtido multiplicando as contribuições de cada estágio:

$$f_n^*(s_n) = \min_{x_n} c_{s_n s_{n+1}} f_{n+1}^*(s_{n+1}).$$

O princípio de optimalidade não se verifica caso algumas contribuições nos estágios sejam negativas. O Exemplo seguinte ilustra um caso em que a relação de recorrência baseia-se em contribuições multiplicativas.

Exemplo 5.4 Considere o problema de uma empresa comercial que pretende distribuir 5 vendedores pela região norte, centro e sul do país, para ajudar as equipas já instaladas nessas regiões. A probabilidade das vendas serem bem sucedidas depende do número de vendedores adicionais presentes numa região. A tabela seguinte fornece-nos essas probabilidades.

N.º de vendedores adicionais	Região norte	Região centro	Região sul
0	0.3	0.4	0.2
1	0.4	0.5	0.3
2	0.5	0.5	0.5
3	0.6	0.55	0.6
4	0.7	0.6	0.65
5	0.8	0.7	0.7

A empresa considera que tem sucesso apenas se as vendas forem bem sucedidas em todas as regiões.

O problema pode ser formulado através de um modelo de Programação Dinâmica em que os estados representam o número de vendedores disponíveis, e os estágios representam cada uma das regiões. As variáveis de decisão x_n , $n = 1, \dots, 3$, representam o número de vendedores adicionais alocados à região norte, centro e sul para $n = 1$, $n = 2$ e $n = 3$, respectivamente.

O valor total de uma política é obtido multiplicando as contribuições associadas às decisões tomadas em cada um dos estágios. A política definida da forma seguinte:

$$(x_1, x_2, x_3) = (1, 2, 2)$$

tem um valor associado de $0.4 \times 0.5 \times 0.5 = 0.1$.

O algoritmo de Programação Dinâmica prossegue conforme descrito no Algoritmo 5.1. A resolução deste Exemplo é deixada como exercício. $\square$

5.6.3 Funções MinMax e MaxMin

Em alguns casos, o valor das políticas corresponde ao valor da maior ou da menor contribuição nos vários estágios do problema. Tipicamente, no primeiro caso, procura-se determinar a política cuja maior contribuição de uma decisão num estágio seja a menor possível, o que corresponde a uma função objectivo do tipo *MinMax*. No segundo caso, procura-se a política que maximize a menor contribuição na sequência de decisões: a função objectivo é do tipo *MaxMin*.

No caso da relação do tipo *MinMax*, a função que define o valor de uma política desde um estado s_n até ao final quando é tomada a decisão x_n no estágio n é expressa da forma seguinte:

$$f_n(s_n, x_n) = \max\{c_{s_n s_{n+1}}, f_{n+1}^*(s_{n+1})\}.$$

O valor da política óptima desde o estágio n e partindo do estado s_n é dado pela expressão seguinte:

$$f_n^*(s_n) = \min_{x_n} \{\max\{c_{s_n s_{n+1}}, f_{n+1}^*(s_{n+1})\}\}.$$

O objectivo consiste em determinar a política cuja maior contribuição num estágio seja a menor entre todas as políticas válidas.

Exemplo 5.5 Vamos considerar novamente a rede representada na Figura 5.3, e vamos assumir que os valores d_{ij} junto aos arcos representam agora a altitude (em centenas

de metro) do pico mais elevado no percurso entre i e j . Suponha que se pretende determinar se existe algum caminho entre A e K que não passe por nenhum pico de altura superior a 500 metros. Este problema pode ser resolvido como um problema de optimização, determinando o valor do melhor caminho, e comparando esse valor com a altura limite de 500 metros. Nesse caso, procura-se o caminho cuja altitude do maior pico seja a menor possível.

O modelo de Programação Dinâmica que usamos neste caso é idêntico ao modelo que definimos no Exemplo 5.2, excepto no que toca à relação de recorrência que passa a ser do tipo *MinMax*. Os passos do algoritmo de Programação Dinâmica são descritos a seguir. O algoritmo é inicializado com $f_5^*(K) = 0$.

$n = 4$

$s_4 = H$

$$\begin{aligned} f_4(H, K) &= \max\{d_{HK}, f_5^*(K)\} = \max\{7, 0\} = 7 \\ f_4^*(H) &= \min_{x_4}\{f_4(H, x_4)\} = \min\{7\} = 7 \end{aligned}$$

$s_4 = I$

$$\begin{aligned} f_4(I, K) &= \max\{d_{IK}, f_5^*(K)\} = \max\{2, 0\} = 2 \\ f_4^*(I) &= \min_{x_4}\{f_4(I, x_4)\} = \min\{2\} = 2 \end{aligned}$$

$s_4 = J$

$$\begin{aligned} f_4(J, K) &= \max\{d_{JK}, f_5^*(K)\} = \max\{4, 0\} = 4 \\ f_4^*(J) &= \min_{x_4}\{f_4(J, x_4)\} = \min\{4\} = 4 \end{aligned}$$

$n = 3$

$s_3 = E$

$$\begin{aligned} f_3(E, H) &= \max\{d_{EH}, f_4^*(H)\} = \max\{5, 7\} = 7 \\ f_3(E, I) &= \max\{d_{EI}, f_4^*(I)\} = \max\{10, 2\} = 10 \\ f_3^*(E) &= \min_{x_3}\{f_3(E, x_3)\} = \min\{7, 10\} = 7 \end{aligned}$$

$s_3 = F$

$$\begin{aligned} f_3(F, H) &= \max\{d_{FH}, f_4^*(H)\} = \max\{8, 7\} = 8 \\ f_3(F, I) &= \max\{d_{FI}, f_4^*(I)\} = \max\{10, 2\} = 10 \\ f_3(F, J) &= \max\{d_{FJ}, f_4^*(J)\} = \max\{5, 4\} = 5 \\ f_3^*(F) &= \min_{x_3}\{f_3(F, x_3)\} = \min\{8, 10, 5\} = 5 \end{aligned}$$

$s_3 = G$

$$\begin{aligned} f_3(G, I) &= \max\{d_{GI}, f_4^*(I)\} = \max\{9, 2\} = 9 \\ f_3(G, J) &= \max\{d_{GJ}, f_4^*(J)\} = \max\{7, 4\} = 7 \\ f_3^*(G) &= \min_{x_3}\{f_3(G, x_3)\} = \min\{9, 7\} = 7 \end{aligned}$$

$n = 2$

$s_2 = B$

$$f_2(B, E) = \max\{d_{BE}, f_3^*(E)\} = \max\{12, 7\} = 12$$

$$\begin{aligned}
f_2(B, F) &= \max\{d_{BF}, f_3^*(F)\} = \max\{7, 5\} = 7 \\
f_2^*(B) &= \min_{x_2}\{f_2(B, x_2)\} = \min\{12, 7\} = 7 \\
s_2 &= C \\
f_2(C, E) &= \max\{d_{CE}, f_3^*(E)\} = \max\{4, 7\} = 7 \\
f_2(C, F) &= \max\{d_{CF}, f_3^*(F)\} = \max\{5, 5\} = 5 \\
f_2(C, G) &= \max\{d_{CG}, f_3^*(G)\} = \max\{4, 7\} = 7 \\
f_2^*(C) &= \min_{x_2}\{f_2(C, x_2)\} = \min\{7, 5, 7\} = 5 \\
s_2 &= D \\
f_2(D, F) &= \max\{d_{DF}, f_3^*(F)\} = \max\{3, 5\} = 5 \\
f_2(D, G) &= \max\{d_{DG}, f_3^*(G)\} = \max\{2, 7\} = 7 \\
f_2^*(D) &= \min_{x_2}\{f_2(D, x_2)\} = \min\{5, 7\} = 5 \\
n &= 1 \\
s_1 &= A \\
f_1(A, B) &= \max\{d_{AB}, f_2^*(B)\} = \max\{6, 7\} = 7 \\
f_1(A, C) &= \max\{d_{AC}, f_2^*(C)\} = \max\{7, 5\} = 7 \\
f_1(A, D) &= \max\{d_{AD}, f_2^*(D)\} = \max\{3, 5\} = 5 \\
f_1^*(A) &= \min_{x_1}\{f_1(A, x_1)\} = \min\{7, 7, 5\} = 5
\end{aligned}$$

O caminho cujo pico mais elevado tem a menor altitude corresponde ao percurso $A - D - F - J - K$. O pico mais elevado desse caminho tem uma altitude de 500 metros, logo, é possível caminhar entre A e K sem nunca atravessar um pico com uma altitude maior do que 500 metros. $\square$

5.7 Variantes

5.7.1 Espaço de Estados com Várias Dimensões

Nos exemplos que abordámos até agora, os estados do sistema s_n eram definidos por uma única variável (quantidade de dinheiro disponível ou cidade visitada, por exemplo). Em muitos casos, as decisões que são tomadas em processos de decisão sequencial afectam vários elementos do sistema. A formulação de um modelo de Programação Dinâmica para estes problemas obriga à definição de estados definidos com base em diferentes variáveis (estados com várias dimensões). No Exemplo que descrevemos a seguir, consideramos o caso de um problema de produção no qual se pretende determinar o número de artigos que devem ser produzidos a partir de diferentes matérias-primas cuja disponibilidade é limitada. As decisões que podem ser tomadas para cada artigo dependem do nível de cada uma das matérias-primas, e afectam directamente os níveis

remanescentes que poderão ser usados na produção dos outros artigos. Os estados do sistema consistem assim em conjuntos de valores que correspondem ao nível de cada matéria-prima.

Esta variante não introduz nenhuma alteração de fundo quer nos modelos quer no procedimento de resolução. Uma decisão continua a definir uma transição entre dois estados do sistema, afectando agora as várias componentes que definem o estado do sistema em vez de uma simples componente. A principal questão que levanta (e que não se limita apenas a este caso) prende-se com a eficiência com que estes modelos podem ser resolvidos. Note que para estes casos o espaço de estados pode crescer muito rapidamente, implicando um maior número de operações na execução do algoritmo de Programação Dinâmica. Voltaremos a esta questão mais adiante neste Capítulo.

Exemplo 5.6 Considere o caso de uma empresa que se dedica ao fabrico de 3 artigos a partir de duas matérias-primas distintas. Os artigos são designados por A, B e C, e as matérias-primas por MP1 e MP2. Assuma que existem actualmente em stock 3 unidades de cada uma dessas matérias-primas. A tabela seguinte indica a quantidade de matéria-prima necessária à produção de uma unidade de cada um dos artigos.

Artigos produzidos	Artigo A		Artigo B		Artigo C	
	MP1	MP2	MP1	MP2	MP1	MP2
1	0	1	2	1	1	0
2	1	1	2	2	2	1
3	2	2	3	3	3	1

Os artigos A, B e C são vendidos ao preço unitário de 400, 500 e 450 euros, respectivamente. Pretende-se determinar em que quantidades os artigos devem ser produzidos de modo a maximizar o lucro total.

O problema é formulado através de um modelo de Programação Dinâmica em que os estágios estão associados aos artigos, e as variáveis de decisão representam o número de artigos que cada tipo que são produzidos. Uma decisão de produção afecta os níveis de matéria-prima disponível, e por outro lado, são esses níveis que condicionam as decisões que podem ser tomadas. Em consequência, os estados do sistema irão corresponder às quantidades disponíveis de cada uma das matérias-primas.

A relação de recorrência para este problema é de maximização, e baseia-se em contribuições aditivas. A rede de políticas é ilustrada na Figura 5.5. Os lucros estão indicados em centenas de euros.

A resolução do modelo segue os passos gerais do algoritmo de Programação Dinâmica.

□

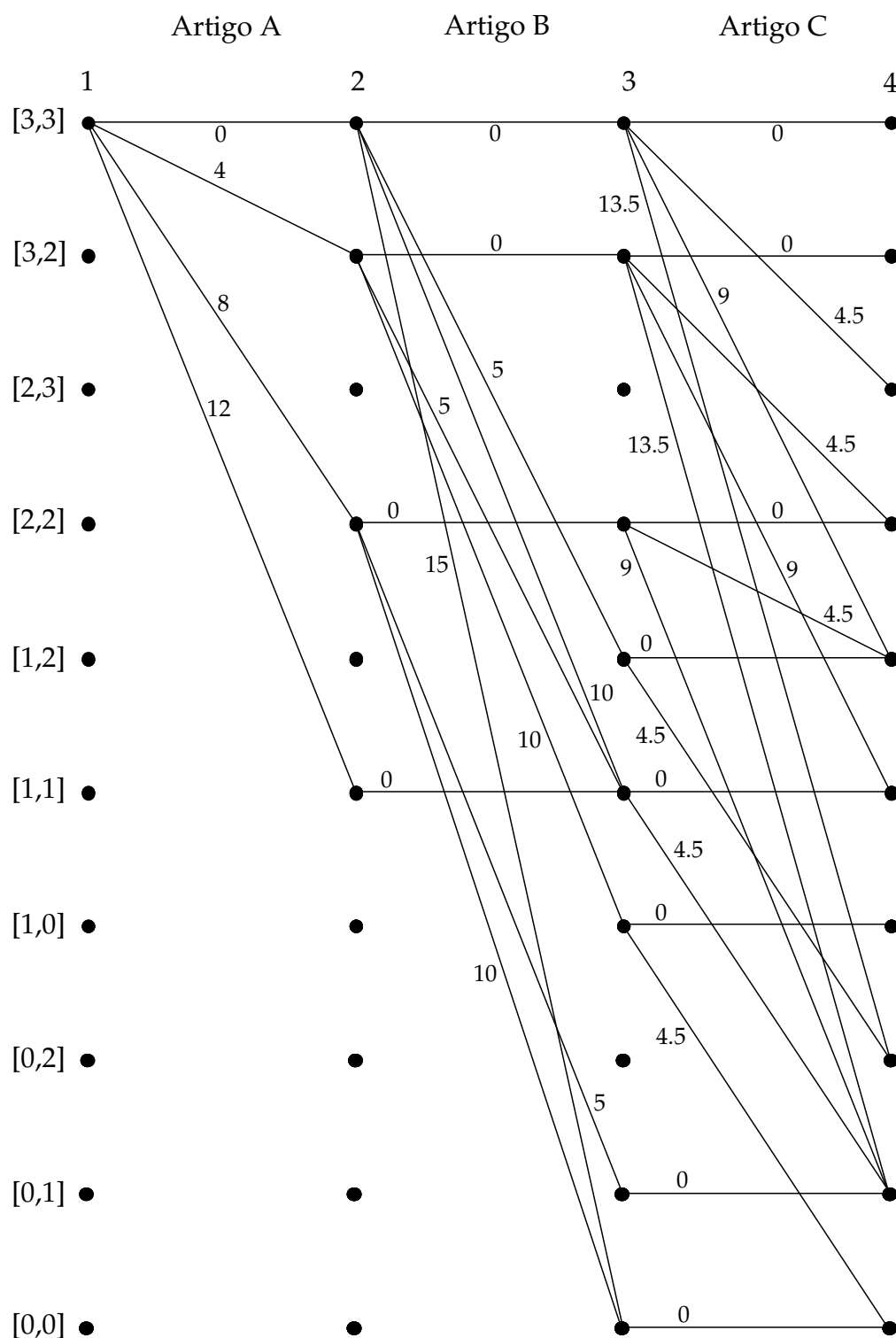


Figura 5.5: Modelo de Programação Dinâmica - Rede de Políticas (Exemplo 5.6)

5.7.2 Espaço Contínuo de Estados

A Programação Dinâmica pode ser usada também nos casos em que os estados s_n do sistema não podem ser descritos por variáveis inteiras (e limitadas), podendo tomar um número infinito de valores num determinado intervalo. Claramente, não é possível para esses casos enumerar todos os valores de s_n tal como fizemos nos exemplos que analisámos na secções anteriores. Em alternativa, o valor da política óptima partindo de um estado s_n deve ser expresso em função desse estado s_n . O Exemplo seguinte ilustra um caso desse género.

Exemplo 5.7 Considere o problema de Programação Linear seguinte, que usámos já em várias ocasiões em exemplos do Capítulo 2:

$$\begin{aligned} \max z = & x_1 + 2x_2 \\ \text{sujeito a} & \\ & x_1 + 4x_2 \leq 16 \\ & x_1 + x_2 \leq 7 \\ & x_1 \leq 6 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Este problema pode ser resolvido usando um modelo de Programação Dinâmica em que os estágios estão associados às variáveis x_1 e x_2 , e os estados correspondem à folga em cada uma das restrições do problema. Os estados são compostos assim por 3 variáveis:

$$s_n = (F_1, F_2, F_3).$$

As variáveis de estado F_1 , F_2 e F_3 podem tomar qualquer valor real no espaço de soluções válidas definido pelas restrições do problema de Programação Linear. O estado inicial corresponde a:

$$s_1 = (16, 7, 6).$$

Para o estágio $n = 2$, as funções que definem o valor de uma política para um dado valor de x_2 e o valor da política óptima são respectivamente as seguintes:

$$\begin{aligned} f_2(F_1, F_2, F_3, x_2) &= 2x_2, \\ f_2^*(F_1, F_2, F_3) &= \max_{\substack{4x_2 \leq F_1 \\ x_2 \leq F_2 \\ x_2 \geq 0}} 2x_2 = \max_{\substack{4x_2 \leq 16 - x_1 \\ x_2 \leq 7 - x_1 \\ x_2 \geq 0}} 2x_2 = f_2^*(16 - x_1, 7 - x_1, 6 - x_1). \end{aligned}$$

Dado x_1 , e partindo do estado inicial $s_1 = (16, 7, 6)$, o estado de partida para o estágio $n = 2$ corresponde a:

$$s_2 = (16 - x_1, 7 - x_1, 6 - x_1).$$

Para o estágio inicial $n = 1$, as funções referidas acima dependem do valor da política ótima desde o estágio $n = 2$, e correspondem respectivamente a:

$$f_1(16, 7, 6, x_1) = x_1 + f_2^*(16 - x_1, 7 - x_1, 6 - x_1),$$

$$f_1^*(16, 7, 6) = \max_{\substack{x_1 \leq 6 \\ x_1 \geq 0}} x_1 + f_2^*(16 - x_1, 7 - x_1, 6 - x_1).$$

Começando a resolução no estágio $n = 2$, temos que o valor ótimo da contribuição nesse estágio é alcançada no ponto x_2^* seguinte:

$$x_2^* = \min \left\{ \frac{16 - x_1}{4}, 7 - x_1 \right\},$$

e é respectivamente igual a

$$f_2^*(F_1, F_2, F_3) = f_2^*(16 - x_1, 7 - x_1, 6 - x_1) = 2 \min \left\{ \frac{16 - x_1}{4}, 7 - x_1 \right\}.$$

Substituindo esse valor na função que define o valor da política ótima no estágio $n = 1$, obtemos:

$$f_1^*(16, 7, 6) = \max_{\substack{x_1 \leq 6 \\ x_1 \geq 0}} x_1 + 2 \min \left\{ \frac{16 - x_1}{4}, 7 - x_1 \right\}.$$

No domínio válido da variável x_1 , o mínimo na expressão anterior corresponde aos valores seguintes:

$$\min \left\{ \frac{16 - x_1}{4}, 7 - x_1 \right\} = \begin{cases} \frac{16 - x_1}{4}, & x_1 \in [0, 4] \\ 7 - x_1, & x_1 \in [4, 6]. \end{cases}$$

Reportando esse resultado à expressão de $f_1(16, 7, 6, x_1)$, obtemos o seguinte resultado:

$$\begin{aligned} x_1 + 2 \min \left\{ \frac{16 - x_1}{4}, 7 - x_1 \right\} &= \begin{cases} x_1 + 2 \frac{16 - x_1}{4}, & x_1 \in [0, 4] \\ x_1 + 2(7 - x_1), & x_1 \in [4, 6] \end{cases} \\ &= \begin{cases} 8 + \frac{1}{2}x_1, & x_1 \in [0, 4] \\ 14 - x_1, & x_1 \in [4, 6] \end{cases} \end{aligned}$$

O ótimo é obtido no ponto $x_1^* = 4$, e corresponde a $f_1^*(16, 7, 6) = 10$. O valor de x_1^* permite-nos calcular o valor de x_2^* :

$$x_2^* = \min \left\{ \frac{16 - 4}{4}, 7 - 4 \right\} = 3.$$

A solução ótima do problema corresponde assim a $x_1^* = 4$ e $x_2^* = 3$, e tem valor igual a 10. □

5.8 Aspectos Computacionais

Do ponto de vista computacional, as abordagens baseadas em Programação Dinâmica distinguem-se pelo facto de permitirem pesquisar o espaço de soluções válidas de forma eficiente. Para pormos em evidência essa característica, vamos considerar o enunciado genérico de um problema de optimização formulado através de um modelo de Programação Dinâmica com 5 estados e 7 estágios, e uma relação de recorrência aditiva e de *minimização*. A Figura 5.6 ilustra a rede de políticas desse modelo. Consideramos o caso limite em que, em todos os estágios intermédios, é possível transitar entre qualquer par de estados. O estado nos estágios inicial e final é único.

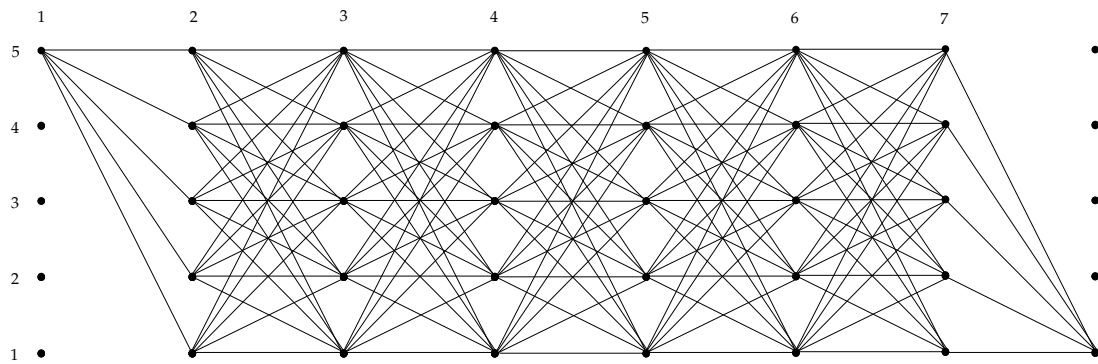


Figura 5.6: Rede de políticas

Uma forma de resolver este problema consiste em enumerar todas as suas soluções válidas, e escolher aquela para a qual o valor da função objectivo é o menor. Neste caso, existem no total $5^6 = 15625$ soluções diferentes. A carga computacional associada à enumeração completa das soluções, medida em termos do número de operações realizadas, é descrita na tabela seguinte.

Tipo de operação	N.º de operações
Adição	$5^6 \times 6 = 93750$
Comparação	$5^6 - 1 = 15624$
TOTAL	109374

No modelo de Programação Dinâmica, a função que define o valor da política óptima desde um estágio n partindo de um estado s_n é dada pela expressão seguinte:

$$f_n^*(s_n) = \min c_{s_n s_{n+1}} + f_n^*(s_{n+1}),$$

em que $c_{s_n s_{n+1}}$ representa a contribuição no estágio n de uma decisão que conduz do estado s_n ao estado s_{n+1} . Essa função é calculada 5 vezes em cada um dos estágios intermédios desde $n = 2$ até $n = 6$, e envolve 5 adições e $5 - 1 = 4$ comparações para cada valor de s_n .

No estágio final e no estágio inicial, são efectuadas no total $5 \times 2 = 10$ adições. Assumimos o caso genérico em que $f_8^*(5)$ pode tomar qualquer valor (diferente de 0). Para calcularmos o valor de uma política, o valor de $f_8^*(5)$ deve ser adicionado ao valor das contribuições nos estágios anteriores. No estágio final, não há lugar a comparações dado que só existe uma transição possível a partir de cada estado. No estágio inicial, o cálculo de $f_1^*(1)$ envolve 4 comparações. A tabela seguinte indica o número de operações efectuadas na resolução do modelo de Programação Dinâmica.

Tipo de operação	N.º de operações
Adição	$5 \times 25 + 10 = 135$
Comparação	$5 \times 20 + 4 = 104$
TOTAL	239

A diferença entre as duas abordagens é clara. O modelo de Programação Dinâmica envolve apenas 0.21% das operações necessárias para resolver o problema por enumeração completa. A resolução de problemas de optimização por enumeração completa das soluções é viável apenas para problemas de pequena dimensão. A Programação Dinâmica é na maior parte dos casos uma alternativa muito mais eficiente.

A aplicabilidade da Programação Dinâmica também tem os seus limites. A carga computacional envolvida na resolução de um modelo de Programação Dinâmica pode aumentar de forma considerável com a dimensão do problema. A maior dificuldade prende-se com o espaço de estados dos modelos de Programação Dinâmica. Tipicamente, quanto maior é o número de estados, maior é o número de operações associadas à resolução dos modelos. Voltando ao Exemplo 5.6, se considerarmos instâncias mais próximas da realidade, o modelo torna-se rapidamente intratável. Se a disponibilidade de cada uma das matérias-primas fosse igual a 1000 unidades, por exemplo, o número de estados possíveis em cada estágio seria (no pior caso) igual a $1001 \times 1001 = 1002001$. Mesmo não envolvendo transições entre todos os estados em todos os estágios (como no exemplo acima), o modelo muito dificilmente seria resolvido em tempo útil. Veja-se então o que acontece se adicionarmos ao problema uma terceira matéria-prima também ela com disponibilidade igual a 1000 unidades. Esta tendência dos modelos de Programação Dinâmica em tornarem-se cada vez difíceis de resolução à medida que a dimensão dos problemas aumenta é conhecido por problema da dimensionalidade.

5.9 Aplicações

5.9.1 Problemas de Substituição de Equipamentos

A Programação Dinâmica pode ser usada para definir políticas de substituição de equipamentos num determinado horizonte de tempo. O processo de decisão consiste em determinar no final de cada período se o equipamento em uso deve ser vendido e substituído por outro novo, ou, pelo contrário, se deve ser mantido por mais um período. O uso de equipamentos envolve diferentes custos desde um custo fixo de aquisição a custos de manutenção ao longo do período de operação do equipamento. A venda de um equipamento permite ao seu proprietário recuperar o valor residual que depende obviamente da idade do equipamento. O Exemplo seguinte ilustra a aplicação da Programação Dinâmica a problemas de substituição de equipamentos.

Exemplo 5.8 Vamos considerar o caso de uma companhia que usa uma determinada máquina na sua actividade produtiva, e que pretende planear a sua política de substituição para os próximos 4 anos. A máquina é adquirida no início do período de planeamento por 500 euros. Não se prevê que esse custo de aquisição venha a sofrer alterações nos próximos 4 anos. Os custos anuais de manutenção da máquina, assim como o valor residual da máquina no final de cada ano de vida são indicados na tabela seguinte (em euros):

Idade da máquina	Custo de manutenção	Valor residual
1º Ano	25	350
2º Ano	30	300
3º Ano	45	200
4º Ano	70	150

A rede da Figura 5.7 representa o conjunto de alternativas para este problema. A origem dos arcos representa o momento em que a máquina é adquirida, enquanto que o destino representa o momento em que ela é vendida.

O problema pode ser formulado usando dois modelos diferentes de Programação Dinâmica. O primeiro consiste em associar os estados do sistema à idade da máquina, e os estágios ao horizonte de planeamento. Nesse caso, as variáveis de decisão estão associadas às decisões de compra ou venda da máquina: Vamos assumir os seguintes valores para as variáveis x_n ($n = 1, \dots, 4$):

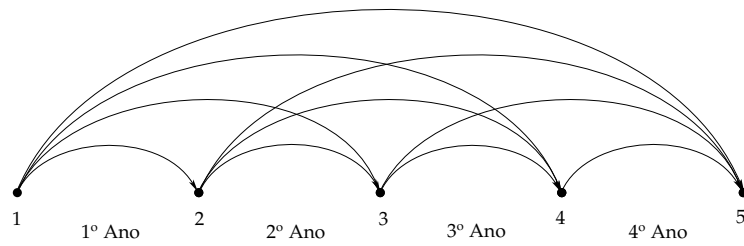


Figura 5.7: Substituição de equipamentos (Exemplo 5.8)

$$x_n = \begin{cases} 0, & \text{se a máquina for mantida mais um ano,} \\ 1, & \text{se a máquina for vendida e substituída por uma máquina nova.} \end{cases}$$

A Figura 5.8 ilustra a rede de políticas. No 1º ano, a única decisão possível consiste em manter a máquina que acabou de ser adquirida. O custo total corresponde à soma do custo de aquisição com o custo de manutenção no primeiro ano ($500 + 25 = 525$). Já no 2º ano, pode-se optar por vender a máquina a um preço de 350 euros, e comprar uma nova por 500 euros mais o custo de manutenção de 25 euros no primeiro, ou, em alternativa, pode-se optar por manter a máquina por mais um ano. Neste último caso, o custo corresponde ao custo de manutenção de uma máquina no seu 2º ano de vida, *i.e.*, 30 euros. No final do 4º ano, a máquina é vendida pelo seu valor residual.

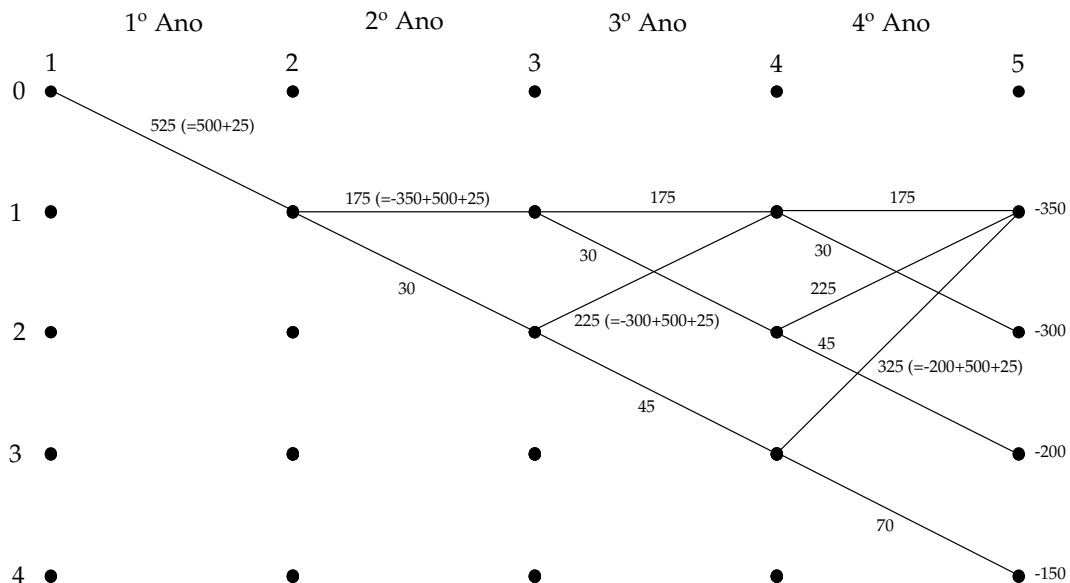


Figura 5.8: Modelo I - Rede de Políticas (Exemplo 5.8)

No estágio $n = 5$, os valores iniciais atribuídos aos estados são os seguintes:

$$f_5^*(1) = -350, f_5^*(2) = -300, f_5^*(3) = -200, f_5^*(4) = -150.$$

A aplicação do algoritmo de Programação Dinâmica resulta nos valores das políticas óptimas indicados a seguir.

$n = 4$

$s_4 = 1$

$$\begin{aligned} f_4(1, 1) &= 175 + f_5^*(1) &= 175 - 350 &= -175 \\ f_4(1, 0) &= 30 + f_5^*(2) &= 30 - 300 &= -270 \\ f_4^*(1) &= \min_{x_4} \{f_4(1, x_4)\} &= \min\{-175, -270\} &= -270 \end{aligned}$$

$s_4 = 2$

$$\begin{aligned} f_4(2, 1) &= 225 + f_5^*(1) &= 225 - 350 &= -125 \\ f_4(2, 0) &= 45 + f_5^*(3) &= 45 - 200 &= -155 \\ f_4^*(2) &= \min_{x_4} \{f_4(2, x_4)\} &= \min\{-125, -155\} &= -155 \end{aligned}$$

$s_4 = 3$

$$\begin{aligned} f_4(3, 1) &= 325 + f_5^*(1) &= 325 - 350 &= -25 \\ f_4(3, 0) &= 70 + f_5^*(4) &= 70 - 150 &= -80 \\ f_4^*(3) &= \min_{x_4} \{f_4(3, x_4)\} &= \min\{-25, -80\} &= -80 \end{aligned}$$

$n = 3$

$s_3 = 1$

$$\begin{aligned} f_3(1, 1) &= 175 + f_4^*(1) &= 175 - 270 &= -95 \\ f_3(1, 0) &= 30 + f_4^*(2) &= 30 - 155 &= -125 \\ f_3^*(1) &= \min_{x_3} \{f_3(1, x_3)\} &= \min\{-95, -125\} &= -125 \end{aligned}$$

$s_3 = 2$

$$\begin{aligned} f_3(2, 1) &= 225 + f_4^*(1) &= 225 - 270 &= -45 \\ f_3(2, 0) &= 45 + f_4^*(3) &= 45 - 80 &= -10 \\ f_3^*(2) &= \min_{x_3} \{f_3(2, x_3)\} &= \min\{-45, -10\} &= -10 \end{aligned}$$

$n = 2$

$s_2 = 1$

$$\begin{aligned} f_2(1, 1) &= 175 + f_3^*(1) &= 175 - 125 &= 50 \\ f_2(1, 0) &= 30 + f_3^*(2) &= 30 - 45 &= -15 \\ f_2^*(1) &= \min_{x_2} \{f_2(1, x_2)\} &= \min\{50, -15\} &= -15 \end{aligned}$$

$n = 1$

$s_1 = 0$

$$\begin{aligned} f_1(0, 0) &= 525 + f_2^*(1) &= 525 - 15 &= 510 \\ f_1^*(0) &= \min_{x_1} \{f_1(0, x_1)\} &= \min\{510\} &= 510 \end{aligned}$$

O custo mínimo é de 510 euros, e corresponde à sequência de decisões indicadas na tabela seguinte:

n	ANO	Decisão	x_n^*	Contribuição
1	1º Ano	Manter a máquina	0	525
2	2º Ano	Manter a máquina	0	30
3	3º Ano	Substituir a máquina	1	225
4	4º Ano	Manter a máquina	0	30

Valor residual	-300
TOTAL	510

O 2º modelo de Programação Dinâmica baseia-se em estados que representam agora o ano em que é efectuada uma substituição da máquina. Os estágios continuam associados a cada um dos anos do horizonte de planeamento. As variáveis de decisão x_n representam o número de anos que uma máquina é mantida sem ser substituída. A transição entre dois estados s_n e s_{n+1} deste modelo implica a aquisição de uma máquina, o pagamento dos custos de manutenção dos primeiros $s_{n+1} - s_n$ anos, e a venda da máquina pelo valor residual associado a uma máquina com $s_{n+1} - s_n$ anos. A rede de políticas para este modelo é ilustrada na Figura 5.9.

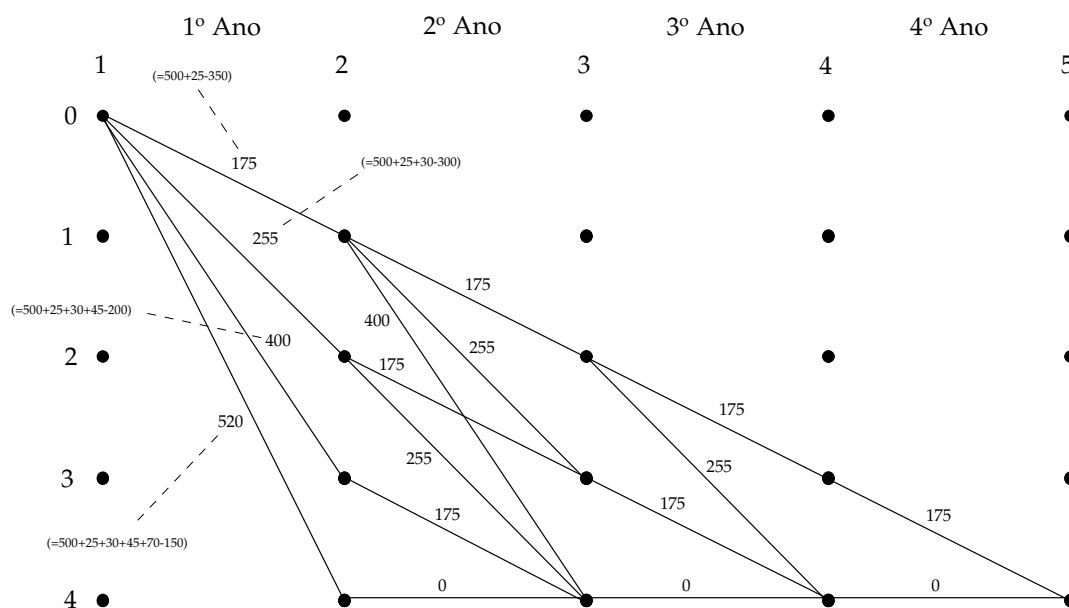


Figura 5.9: Modelo II - Rede de Políticas (Exemplo 5.8)

O valor inicial $f_5^*(4)$ é nulo. O algoritmo de Programação Dinâmica conduz aos resultados indicados a seguir.

$n = 4$

$s_4 = 3$

$$f_4(3, 1) = 175 + f_5^*(4) = 175 + 0 = 175$$

$$f_4^*(3) = \min_{x_4} \{f_4(3, x_4)\} = \min\{175\} = 175$$

$s_4 = 4$

$$f_4(4, 0) = 0 + f_5^*(4) = 0 + 0 = 0$$

$$f_4^*(4) = \min_{x_4} \{f_4(4, x_4)\} = \min\{0\} = 0$$

$n = 3$

$s_3 = 2$

$$f_3(2, 1) = 175 + f_4^*(3) = 175 + 175 = 350$$

$$f_3(2, 2) = 255 + f_4^*(4) = 255 + 0 = 255$$

$$f_3^*(2) = \min_{x_3} \{f_3(2, x_3)\} = \min\{350, 255\} = 255$$

$s_3 = 3$

$$f_3(3, 1) = 175 + f_4^*(4) = 175 + 0 = 175$$

$$f_3^*(3) = \min_{x_3} \{f_3(3, x_3)\} = \min\{175\} = 175$$

$s_3 = 4$

$$f_3(4, 0) = 0 + f_4^*(4) = 0 + 0 = 0$$

$$f_3^*(4) = \min_{x_3} \{f_3(4, x_3)\} = \min\{0\} = 0$$

$n = 2$

$s_2 = 1$

$$f_2(1, 1) = 175 + f_3^*(2) = 175 + 255 = 430$$

$$f_2(1, 2) = 255 + f_3^*(3) = 255 + 175 = 430$$

$$f_2(1, 3) = 400 + f_3^*(4) = 400 + 0 = 400$$

$$f_2^*(1) = \min_{x_2} \{f_2(1, x_2)\} = \min\{430, 400\} = 400$$

$s_2 = 2$

$$f_2(2, 1) = 175 + f_3^*(3) = 175 + 175 = 350$$

$$f_2(2, 2) = 255 + f_3^*(4) = 255 + 0 = 255$$

$$f_2^*(2) = \min_{x_2} \{f_2(2, x_2)\} = \min\{350, 255\} = 255$$

$s_2 = 3$

$$f_2(3, 1) = 175 + f_3^*(4) = 175 + 0 = 175$$

$$f_2^*(3) = \min_{x_2} \{f_2(3, x_2)\} = \min\{175\} = 175$$

$s_2 = 4$

$$f_2(4, 0) = 0 + f_3^*(4) = 0 + 0 = 0$$

$$f_2^*(4) = \min_{x_2} \{f_2(4, x_2)\} = \min\{0\} = 0$$

$$n = 1$$

$$s_1 = 0$$

$$\begin{aligned} f_1(0, 1) &= 175 + f_2^*(1) &= 175 + 400 &= 575 \\ f_1(0, 2) &= 255 + f_2^*(2) &= 255 + 255 &= 510 \\ f_1(0, 3) &= 400 + f_2^*(3) &= 400 + 175 &= 575 \\ f_1(0, 4) &= 520 + f_2^*(4) &= 520 + 0 &= 520 \\ f_1^*(0) &= \min_{x_1} \{f_1(0, x_1)\} &= \min\{575, 510, 520\} &= 510 \end{aligned}$$

A política óptima consiste em manter a primeira máquina durante 2 anos, adquirir uma nova no final desses 2 anos e mantê-la durante mais 2 anos até ao final do período de planeamento. $\square$

5.9.2 Problemas de Comparação de Sequências

A comparação entre sequências é um problema que ocorre em várias áreas como as ciências da computação (processamento de texto, comparação entre ficheiros) e a bioinformática, por exemplo. Estes problemas consistem em verificar as semelhanças que existem entre sequências de elementos (caracteres, dígitos, sequências de ADN). Um dos problemas mais comuns nesse domínio é o chamado *problema da maior subsequência comum*, que consiste em determinar o comprimento da maior subsequência de elementos presente em todas as sequências analisadas. Nesta Secção, consideramos o caso em que são comparadas apenas duas sequências.

Considere duas sequências $A = a_1 \dots a_m$ e $B = b_1 \dots b_n$, ambas compostas por um número finito de elementos. Uma sequência C é subsequência de A se for composta por elementos que pertencem a A , e se a ordem destes elementos em C for idêntica à ordem pela qual aparecem em A . A sequência $C = cfh$, por exemplo, é uma subsequência da sequência $A = abcdefgh$. A sequência C é subsequência comum de A e B se for subsequência de A e também de B .

O problema da maior subsequência comum pode ser resolvido usando um modelo de Programação Dinâmica em que os estados estão associados a prefixos de cada uma das sequências. Um prefixo de comprimento i de uma sequência $A = a_1 a_2 \dots a_m$ é uma subsequência de A constituída pelos primeiros i elementos de A . Os estados podem ser representados através de um par de índices que determinam o comprimento de cada prefixo. Para duas sequências $A = a_1 \dots a_m$ e $B = b_1 \dots b_n$, a definição formal de um estado é a seguinte:

$$s_n = (i, j) = (\text{prefixo de } A \text{ de comprimento } i, \text{ prefixo de } B \text{ de comprimento } j).$$

A relação de recorrência, as contribuições nos estágios e as decisões associadas a este

modelo procuram contabilizar passo a passo os elementos coincidentes das duas sequências. Essas decisões correspondem a encurtar os prefixos tendo em conta o facto do último elemento de ambos ser ou não o mesmo. As respectivas variáveis de decisão podem ser definidas da forma seguinte:

$$x_n = \begin{cases} 1, & \text{eliminar o último elemento do 1º prefixo,} \\ & \text{(transição de um estado } (i, j) \text{ para um estado } (i-1, j)); \\ 2, & \text{eliminar o último elemento do 2º prefixo,} \\ & \text{(transição de um estado } (i, j) \text{ para um estado } (i, j-1)); \\ 3, & \text{eliminar o último elemento de ambos os prefixos,} \\ & \text{(transição de um estado } (i, j) \text{ para um estado } (i-1, j-1)); \\ 4, & \text{não eliminar nenhum elemento,} \\ & \text{(transição de um estado } (i, 0) \text{ ou } (0, j) \text{ para o mesmo estado).} \end{cases}$$

As decisões (e o respectivo valor das variáveis de decisão) que podem ser tomadas em cada estágio depende do estado do sistema. Para um estado (i, j) num estágio n , a decisão de eliminar o último elemento de ambos os prefixos só é tomada se o último elemento dos dois prefixos for o mesmo. Nesse caso, essa é a única decisão possível. Se o último elemento dos dois prefixos for diferente, deve ser eliminado o último elemento de um dos prefixos ($x_n = 1$ ou $x_n = 2$). Num estado em que pelo menos um dos prefixos tenha comprimento nulo, e dado que não há lugar a comparações no respectivo estágio (nem nos seguintes), não é eliminado nenhum elemento e a transição é feita para o mesmo estado.

Para este problema, consideramos uma relação de recorrência expressa de trás para a frente. O valor da política óptima e das políticas quando é tomada uma decisão x_n no estágio n são definidos da forma seguinte:

$$f_n^*(s_n) = \max_{x_n} \{f_n(s_n, x_n)\},$$

$$f_n((i-1, j), 1) = f_{n-1}^*((i, j)), \quad \text{com } i > 0, j > 0, \text{ e } x_i \neq y_j,$$

$$f_n((i, j-1), 2) = f_{n-1}^*((i, j)), \quad \text{com } i > 0, j > 0, \text{ e } x_i \neq y_j,$$

$$f_n((i-1, j-1), 3) = 1 + f_{n-1}^*((i, j)), \quad \text{com } i > 0, j > 0, \text{ e } x_i = y_j,$$

$$f_n((i, 0), 4) = f_{n-1}^*((i, 0)),$$

$$f_n((0, j), 4) = f_{n-1}^*((0, j)).$$

Quando os últimos elementos dos prefixos coincidem, a única decisão que pode ser tomada ($x_n = 3$) tem associada uma contribuição no estágio igual à unidade. As restantes alternativas têm todas contribuições nulas.

O Exemplo seguinte ilustra a aplicação da Programação Dinâmica ao problema da maior subsequência comum.

Exemplo 5.9 Considere o problema que consiste em determinar a maior subsequência comum às sequências $A = abdf$ e $B = bfg$. Essa subsequência tem comprimento igual a 2, e corresponde à sequência bf .

A rede associada ao modelo de Programação Dinâmica que descrevemos nesta Secção é representada na Figura 5.10. Neste caso, iniciamos a resolução do problema com o seguinte valor inicial para o estado de partida $(4, 3)$:

$$f_1^*((4, 3)) = 0.$$

O algoritmo de Programação Dinâmica (recorrência para diante) aplicada aos três primeiros estágios do modelo conduz aos resultados que são indicados a seguir:

$n = 2$

$$s_2 = (4, 2)$$

$$f_2((4, 2), 2) = 0 + f_1^*((4, 3)) = 0$$

$$f_2^*((4, 2)) = \max_{x_2} \{f_2((4, 2), x_2)\} = 0$$

$$s_2 = (3, 3)$$

$$f_2((3, 3), 1) = 0 + f_1^*((4, 3)) = 0$$

$$f_2^*((3, 3)) = \max_{x_2} \{f_2((3, 3), x_2)\} = 0$$

$n = 3$

$$s_3 = (3, 2)$$

$$f_3((3, 2), 2) = 0 + f_2^*((3, 3)) = 0$$

$$f_3^*((3, 2)) = \max_{x_3} \{f_3((3, 2), x_3)\} = 0$$

$$s_3 = (3, 1)$$

$$f_3((3, 1), 3) = 1 + f_2^*((4, 2)) = 1$$

$$f_3^*((3, 1)) = \max_{x_3} \{f_3((3, 1), x_3)\} = 1$$

$$s_3 = (2, 3)$$

$$f_3((2, 3), 1) = 0 + f_2^*((3, 3)) = 0$$

$$f_3^*((2, 3)) = \max_{x_3} \{f_3((2, 3), x_3)\} = 0$$

O resto da resolução é deixada como exercício. □

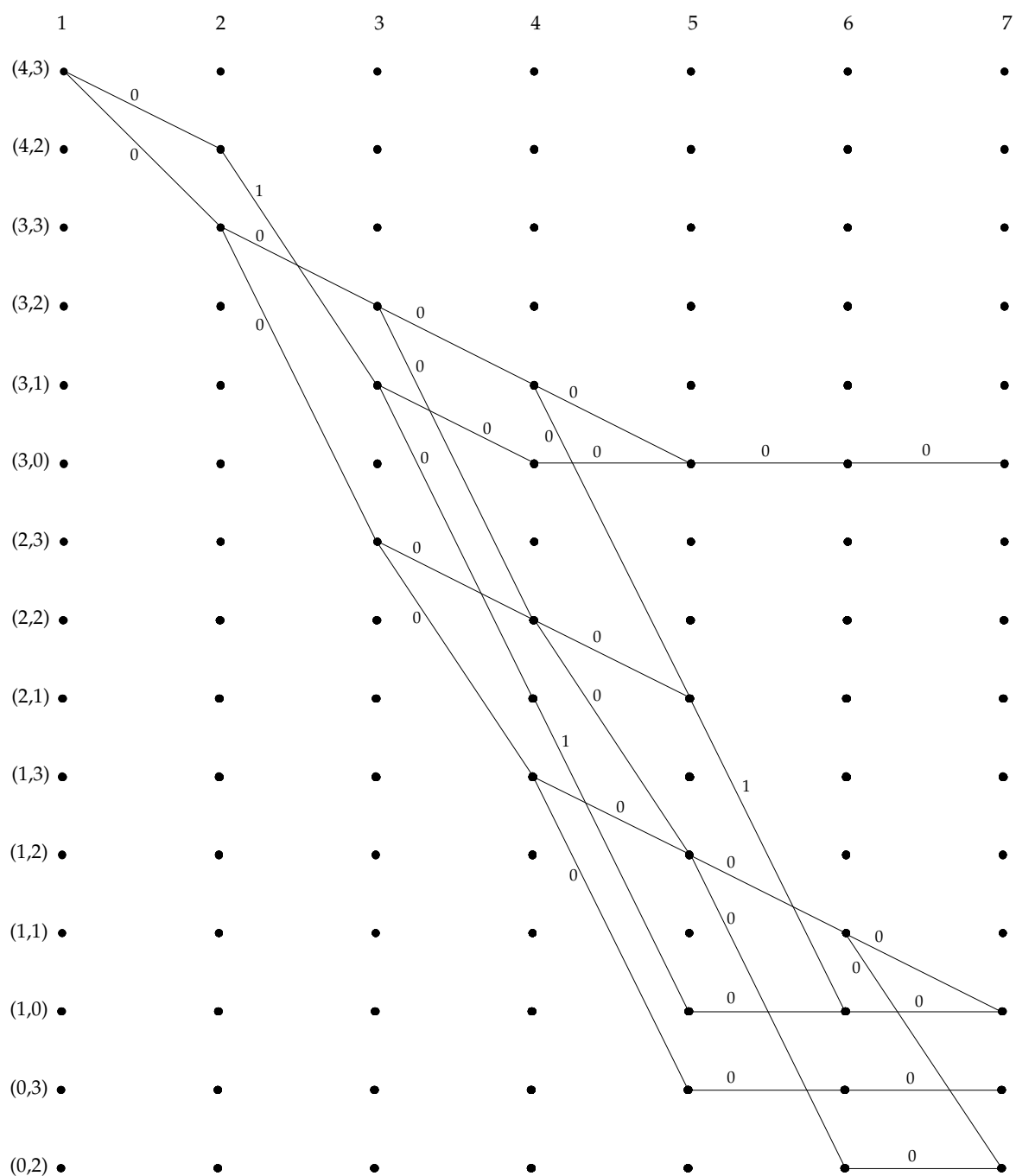


Figura 5.10: Modelo de Programação Dinâmica (Exemplo 5.9)

5.10 Conclusões

A Programação Dinâmica é usada para resolver problemas de decisão e optimização separáveis em sequências de decisões relacionadas (neste Capítulo, considerámos essencialmente problemas de optimização discreta). Aplica-se especificamente a problemas que verifiquem o princípio de optimalidade. Esse princípio determina que qualquer subsequência de decisões de uma política óptima também deve ser óptima para o respectivo subproblema. O princípio de optimalidade sugere também um método de resolução recursivo em que a solução (ou política óptima) do problema original é construída a partir de soluções dos seus subproblemas.

A Programação Dinâmica permite resolver de forma eficiente muitos problemas relevantes deste tipo. Contudo, a dimensão dos problemas pode tornar rapidamente estas abordagens muito complexas. A carga computacional associada à resolução destes problemas pode aumentar de forma significativa com o número de estados e estágios do problema: é o chamado problema da dimensionalidade. Várias abordagens têm sido propostas na literatura para contornar esse problema, algumas delas no sentido da redução do espaço de estados.

Neste Capítulo, abordámos a versão determinística da Programação Dinâmica: assumimos que a evolução do estado do sistema não era sujeita a factores aleatórios. Em qualquer estágio do processo de decisão, o estado para o qual transita o sistema depende apenas da decisão que é tomada. Esse estado é perfeitamente conhecido. Nesses casos, é possível calcular a priori uma política óptima completa para o problema. Nos problemas estocásticos, a transição entre estados não depende apenas da decisão que é tomada no estágio. Para saber qual é a decisão que deve ser tomada num estágio, é necessário primeiro observar o estado em que se encontra o sistema. As decisões da política óptima acabam assim por ser determinadas de forma sequencial à medida que é conhecido o estado corrente do sistema.

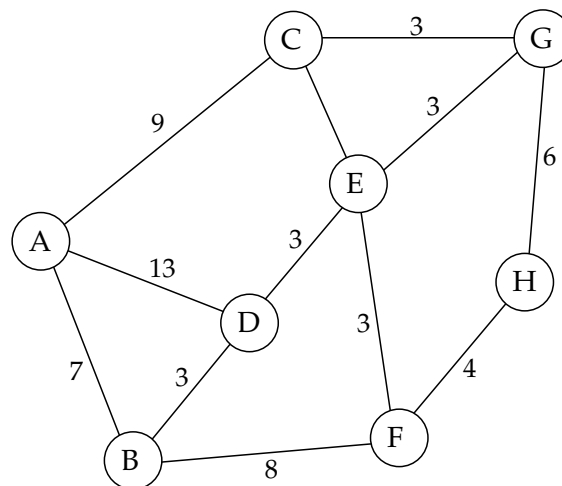
5.11 Exercícios

1. Para aumentar os seus rendimentos mensais, o Pedro decidiu procurar actividades extras em part-time. O Pedro quer dedicar uma parte do seu fim-de-semana nessas actividades, e está disposto a trabalhar no máximo 6 horas. Após ter contactado algumas agências de trabalho temporário, o Pedro identificou 4 oportunidades. Os salários auferidos em cada uma das actividades não é proporcional às horas de trabalho. A tabela seguinte indica o salário que é pago (em euros) por horas de trabalho em cada uma das actividades:

	Actividade			
	1	2	3	4
1 hora	12	13	20	25
2 horas	20	26	25	30
3 horas	30	39	30	32
4 horas	38	50	35	35

Usando Programação Dinâmica, determine o tempo que o Pedro deve dedicar a cada actividade.

2. Considere o problema de caminho mais curto entre os vértices A e H do grafo seguinte:

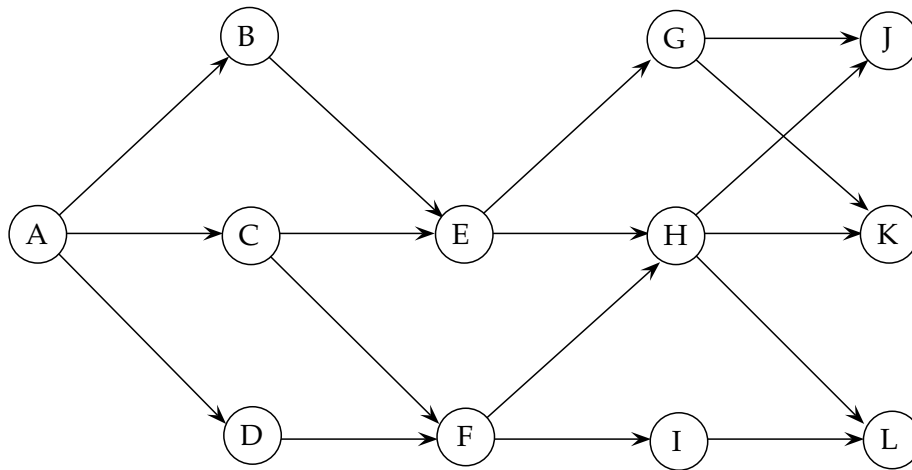


Resolva o problema usando Programação Dinâmica.

3. Existem 3 prémios localizados nos pontos J, K e L da rede representada abaixo. O valor desses prémios é de 700 euros para o ponto J, de 660 para o ponto K, e de 680 euros para o ponto L.

Suponha que a sua posição actual é o ponto A, e que o seu objectivo está em saber qual dos prémios deverá levantar sabendo que existem custos associados aos caminhos da rede: sempre que atravessa um ponto incorre num custo cujo valor (em euros) é indicado na tabela seguinte:

	B	C	D	E	F	G	H	I
Custo	10	5	15	10	5	25	30	3



Use um modelo de Programação Dinâmica para determinar o percurso que deverá seguir para maximizar o seu lucro.

4. Considere o problema de Programação Inteira seguinte:

$$\begin{aligned} \max z &= 10x_1 + 12x_2 \\ \text{sujeito a} \\ 4x_1 + 2x_2 &\leq 13 \\ 2x_1 + 3x_2 &\leq 10 \\ x_1, x_2 &\geq 0 \text{ e inteiros.} \end{aligned}$$

Resolva este problema usando Programação Dinâmica.

5. Um determinado projecto é constituído por 3 tarefas independentes (A, B, C). Existem 4 operadores aptos a realizar qualquer uma dessas tarefas. O tempo (em minutos) que os operadores levam a concluir cada uma das tarefas é indicado na tabela seguinte:

	Operador			
	1	2	3	4
Tarefa A	14	10	14	16
Tarefa B	10	7	11	8
Tarefa C	15	13	17	19

Assuma que um operador só pode realizar uma tarefa, e que as tarefas são realizadas em série.

- a) Usando Programação Dinâmica, determine que operadores deverão realizar cada uma das tarefas de modo a concluir o projecto o mais cedo possível.
- b) Suponha que as tarefas podem ser realizadas em paralelo. Determine a nova solução óptima nesse caso.
6. O Senhor Costa é gerente do maior minimercado de uma aldeia trasmontana. Na época natalícia, o Senhor Costa oferece aos seus clientes uma variedade de cabazes constituídos por garrafas de vinho do Porto, garrafas de champanhe, e bolos reis. A tabela seguinte indica a constituição exacta dos 3 cabazes comercializados pelo Senhor Costa, assim como o preço de venda (em euros) de cada cabaz:

	Cabaz 1	Cabaz 2	Cabaz 3
Vinho do Porto	3	2	2
Champanhe	2	1	2
Bolo rei	1	2	1
Preço (euros/unidade)	22	15	18

O Senhor Costa tem actualmente em stock 6 garrafas de vinho do Porto, 7 garrafas de champanhe e 5 bolos reis. Os artigos que não forem colocados num cabaz serão vendidos aos seguintes preços:

	Vinho do Porto	Champanhe	Bolo rei
Preço (euros/unidade)	2	3	1.5

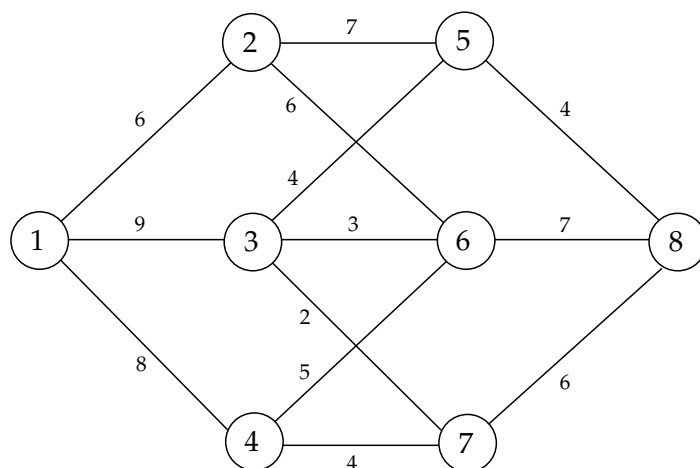
Usando Programação Dinâmica, determine quantos cabazes o Senhor Costa deverá preparar de forma a maximizar o seu lucro.

7. O Pedro está a preparar o seu plano de estudo com vista à próxima época de exames. Neste momento, falta-lhe apenas decidir o que irá fazer com 4 horas que ainda tem livres. O Pedro está inscrito em 4 provas diferentes. A probabilidade de sucesso nessas provas depende do esforço adicional que o Pedro irá fazer. Dada a natureza das provas, essas probabilidades variam de disciplina para disciplina. A tabela seguinte indica a probabilidade de sucesso numa prova em função do tempo de estudo adicional dispendido pelo Pedro:

Horas de estudo adicional	Prova 1	Prova 2	Prova 3	Prova 4
0	0.3	0.4	0.2	0.5
1	0.4	0.5	0.3	0.55
2	0.6	0.5	0.5	0.6
3	0.6	0.6	0.6	0.6
4	0.7	0.6	0.7	0.65

Determine como deverá o Pedro distribuir as suas horas de estudo de forma a maximizar a probabilidade de passar de ano sem disciplinas em atraso.

8. O presidente de uma câmara municipal está a planear a ligação de uma das suas freguesias a uma nova central eléctrica. Após terem efectuado estudos aprofundados, os técnicos da câmara determinaram os diferentes percursos onde poderiam ser instalados os cabos de transporte de energia. Atendendo à topologia variada do terreno e às características meteorológicas das diferentes zonas da freguesia, a capacidade máxima desses cabos varia ao longo dos diferentes trajectos. A rede representada abaixo ilustra os dados que foram recolhidos pelos técnicos (os valores junto aos arcos representam a capacidade máxima em megawatts dos cabos):



O presidente deve escolher o percurso pelo qual deverão ser distribuídos os cabos de modo a maximizar a quantidade de electricidade que pode ser enviada entre os pontos 1 e 8 da rede acima.

Determine a solução óptima deste problema usando Programação Dinâmica.

9. Considere um problema de gestão de inventários de um comerciante no qual se pretende determinar quando devem ser comprados determinados artigos sabendo que os preços de compra variam ao longo do tempo. Assuma que o comerciante dispõe de um depósito com capacidade para armazenar apenas 4 artigos.

Os preços de compra dos artigos dependem do número de unidades que são compradas e do mês em que é efectuada a compra. A tabela seguinte indica o valor estimado do preço de compra (em euros):

	Mês			
	1	2	3	4
1 unidade	390	430	440	470
2 unidades	720	720	750	755
3 unidades	800	805	830	875
4 unidades	850	855	875	910

Suponha que o objectivo do comerciante é revender todos artigos dentro de 5 meses. O preço pelo qual espera vender os artigos depende também do número de unidades que são vendidas. Esse preço é indicado na tabela seguinte:

	Unidades vendidas			
	1	2	3	4
Preço de venda total (em euros)	650	950	1000	1115

Sabendo que o custo de posse de stock é de 15 euros/mês/artigo, determine a política óptima de compra que o comerciante deverá adoptar usando Programação Dinâmica.

10. Uma companhia pretende planear a produção de um dos seus artigos para os próximos 5 meses. A companhia pode produzir até 8 unidades desse artigo por mês a um custo unitário de 12 euros. Associado à produção do artigo, existe um custo fixo de preparação de 7 euros (independente do número de unidades que são produzidas). Nos próximos meses, a procura do artigo será de 2, 6, 4, 7 e 4 unidades, respectivamente.

A companhia dispõe de um armazém onde pode guardar até 5 unidades do seu artigo por mês. O custo de posse de stock é de 2 euros por unidade e por mês.

Assuma que esse custo é pago no final do mês por cada unidade do artigo em stock.

Determine o plano óptimo de produção usando Programação Dinâmica.

11. Pretende-se determinar a política de aquisição e manutenção de uma única máquina num horizonte de 5 anos. O preço de compra da máquina é de 1200 euros. Esse valor aumenta de 2% todos os anos (o preço de compra é arredondado para o inteiro mais próximo). Os custos de manutenção (em euros/ano), e o valor residual da máquina em função dos seus anos de vida estão indicados na tabela seguinte:

Idade da máquina	Custo de manutenção	Valor residual
1º Ano	70	500
2º Ano	80	480
3º Ano	80	400
4º Ano	90	350
5º Ano	100	320

Determine a política óptima de aquisição e manutenção usando Programação Dinâmica.

12. Considere um problema de escalonamento de 5 tarefas num único processador. A cada tarefa está associado um prémio que é ganho apenas se a tarefa for concluída dentro de um determinado prazo. A tabela seguinte indica o valor do prémio (em unidades monetárias), o tempo de execução e o prazo de conclusão (em unidades de tempo) de cada uma das tarefas:

	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Prémio	20	30	15	10
Prazo de conclusão	1	3	3	4
Tempo de execução	1	2	1	2

Diga, justificando, se este problema pode ser formulado através de um modelo de Programação Dinâmica. Em caso afirmativo, determine a solução óptima do problema.

- 13.** Considere as duas sequências seguintes:

$$A = abcd$$

$$B = acbd$$

Usando Programação Dinâmica, determine a maior subsequência comum a A e B .

Capítulo 6

Métodos Heurísticos

6.1 Introdução

Os métodos heurísticos (ou heurísticas¹) são métodos alternativos de pesquisa de boas soluções para problemas de optimização. Esses métodos distinguem-se dos métodos exactos que temos vindo a analisar neste texto pelo facto de não garantirem que a solução óptima do problema seja encontrada no final da sua execução. Os métodos heurísticos baseiam-se em regras (muitas vezes empíricas) para a construção de soluções, e a exploração do espaço de soluções na procura de alternativas de melhor qualidade. Ao invés do que acontece com os métodos exactos, os critérios de paragem das heurísticas estão frequentemente relacionados com aspectos da sua execução, como o número de iterações, ou o tempo de total de computação já decorrido.

As heurísticas são usadas essencialmente para determinar rapidamente soluções de boa qualidade para problemas complexos. Com efeito, a complexidade inerente a muitos problemas reais faz com que os métodos exactos dificilmente consigam encontrar soluções óptimas em tempo útil. A dimensão dos problemas é outro factor que pode inviabilizar o recurso a métodos de resolução exacta. Em alguns casos, o próprio exercício que consiste em determinar uma solução válida para um problema pode ser extremamente difícil.

Existem vários tipos de métodos heurísticos. Uma classificação possível baseia-se no grau de generalidade dos métodos. Alguns métodos são definidos para resolver apenas um problema específico, enquanto outros possuem uma estrutura geral, e podem ser adaptados a cada caso particular. Os métodos deste último grupo são designados por *meta-heurísticas*. Para algumas heurísticas, existem ainda resultados de aproximação que determinam os limites para o valor das soluções geradas no pior caso (um

¹Ao longo do capítulo, usaremos indistintamente os dois termos.

resultado desse tipo poderá garantir, por exemplo, que a solução gerada pela heurística nunca é superior ao dobro do valor da solução ótima). Essas heurísticas são também designadas por algoritmos de aproximação.

Algumas heurísticas limitam-se a construir passo a passo uma solução para o problema. Outras, partindo de uma solução construída eventualmente através de um processo heurístico, procuram melhorar uma solução analisando a sua vizinhança. As primeiras são tipicamente muito eficientes. Os métodos de pesquisa na vizinhança envolvem um maior esforço computacional, mas permitem determinar geralmente soluções de melhor qualidade. O sucesso desses métodos de melhoramento depende em grande medida da forma como é definida a vizinhança de uma solução. A função de vizinhança determina as soluções que são exploradas em cada uma das iterações do método.

Um dos inconvenientes de algumas heurísticas é o facto de convergirem muitas vezes para mínimos locais, *i.e.* soluções que são melhores do que outras na vizinhança, mas que não são soluções óptimas para o problema global. As meta-heurísticas tentam escapar a esses mínimos locais permitindo que, numa dada iteração, seja escolhida uma solução de pior qualidade do que a anterior.

Neste capítulo, analisamos aspectos elementares de alguns dos métodos heurísticos mais usados na prática. Consideramos em particular a aplicação desses métodos na resolução de problemas de Programação Inteira e de Optimização Combinatória.

6.2 Avaliação de uma Heurística

O desempenho de uma heurística pode ser medido de várias formas. A análise da heurística pode conduzir, por exemplo, a resultados de aproximação no pior caso, ou a uma medida de desempenho médio. Quando esses resultados não existem, ou são difíceis de obter, recorre-se tipicamente a uma análise experimental da heurística, conduzindo testes computacionais baseados num número significativo de instâncias¹ do problema. Os dois parâmetros que se procuram avaliar são os seguintes:

- o tempo de computação necessário para atingir uma solução final;
- a qualidade da solução que é gerada pela heurística.

¹Um problema de optimização é caracterizado pelas suas instâncias, entre outros elementos. Uma instância é obtida atribuindo valores aos parâmetros do problema (num problema de caminho mais curto, por exemplo, a definição do grafo incluindo os custos dos arcos define uma instância do problema).

Idealmente, uma heurística deverá determinar uma solução de boa qualidade num espaço de tempo reduzido para um problema em que o uso de um método exacto é impraticável. As heurísticas mais simples são tipicamente as mais rápidas. As heurísticas de melhoramento, que exploram iterativamente a vizinhança de uma solução, conduzem geralmente a melhores soluções ao preço de um maior esforço computacional. Nessas heurísticas, o tempo de computação é muitas vezes usado como critério de paragem.

A qualidade das soluções geradas por uma heurística pode ser avaliada comparando o seu valor com o valor da solução óptima da instância. Essa análise é feita experimentalmente a partir de instâncias do problema para as quais é conhecida a solução óptima. Quando essa solução não é conhecida, a comparação é feita tendo por base um limite inferior ou superior para o valor do óptimo da instância.

6.3 Métodos de Construção

6.3.1 Descrição Geral

Os métodos de construção são usados para gerar de raiz uma solução para um problema. Muitos desses métodos assentam numa estratégia que é designada por estratégia miópica ou gulosa (*greedy*, na terminologia anglo-saxónica). As heurísticas gulosas determinam gradualmente uma solução válida para um problema tomando em cada passo uma decisão. Essa decisão define parcialmente a solução final, e é definitiva. O resultado da decisão não é reconsiderado em nenhuma fase posterior do processo. No final da execução, o resultado de todas as decisões que foram tomadas definem uma solução válida para o problema. As decisões são tomadas usando um critério de “satisfação imediata”. Em cada passo, é tomada a decisão mais interessante do ponto de vista do objectivo de optimização do problema. As decisões que são tomadas num dado passo dependem apenas do resultado dos passos anteriores. Esse procedimento pode eventualmente prejudicar a solução final porque não considera o impacto da decisão em fases posteriores do processo.

As heurísticas gulosas vão gradualmente reduzindo a dimensão do problema, até determinar a solução final. A título de exemplo, podemos citar a regra do custo mínimo usada para determinar uma solução válida para um problema de transportes. Nesse caso, a solução é construída escolhendo sempre a célula de menor custo, e atribuindo à respectiva variável o maior valor possível tendo em conta as restrições de oferta e procura. Ao fazer essa escolha, nada garante que não sejamos forçados a escolher uma célula de custo muito elevado numa iteração posterior.

Outra forma de construir uma solução para uma instância de um problema consiste

em reduzir essa instância a um caso especial que seja de fácil resolução. Nesse caso, a qualidade da solução estará directamente ligada à qualidade dessa aproximação.

Os métodos de construção são procedimentos tipicamente simples, e fáceis de implementar. Esses métodos são usados frequentemente para determinar soluções iniciais para outros métodos. Por definição, esses métodos não garantem que uma solução óptima seja encontrada no final da sua execução. No entanto, alguns problemas são resolvidos até à optimalidade através de procedimentos desse tipo. É o caso, por exemplo, do problema de caminho mais curto que abordámos no capítulo anterior, ou do problema da árvore de custo mínimo que apresentamos na secção seguinte. Obviamente, quando o problema é realmente complexo, os métodos de construção conduzem na maior parte dos casos a soluções aproximadas. Pelas razões que invocámos acima, essas soluções poderão estar muito afastadas do óptimo.

6.3.2 Exemplo I: o Problema da Árvore de Suporte de Custo Mínimo

Nesta secção, ilustramos o caso de um problema de optimização clássico cuja solução óptima pode ser obtida usando um método de construção baseado numa estratégia miópica.

O problema da árvore de suporte de custo mínimo pode ser definido a partir de um grafo $G = (V, A)$ não orientado e com custos associados aos arcos. O problema consiste em determinar o subgrafo de G que garante que os vértices do grafo estejam todos ligados entre eles através de arcos cujo custo total é o menor possível. Esse subgrafo constitui aquilo que é designado na terminologia dos grafos por uma árvore. Uma árvore não tem ciclos, e é constituído por exactamente $|V| - 1$ arcos.

Um método de construção simples para este problema consiste em adicionar os arcos por ordem crescente do seu custo. Quando um arco candidato cria um ciclo, esse arco é rejeitado, e prossegue-se com o arco seguinte. O método termina logo que tenham sido adicionados $|V| - 1$ arcos. O desempate entre arcos de igual custo é feito de forma arbitrária. É possível provar que uma solução construída dessa forma é sempre uma solução óptima do problema da árvore de suporte de custo mínimo. O Exemplo seguinte ilustra o funcionamento do método.

Exemplo 6.1 Considere o problema da árvore de suporte de custo mínimo definido a partir do grafo da Figura 6.1

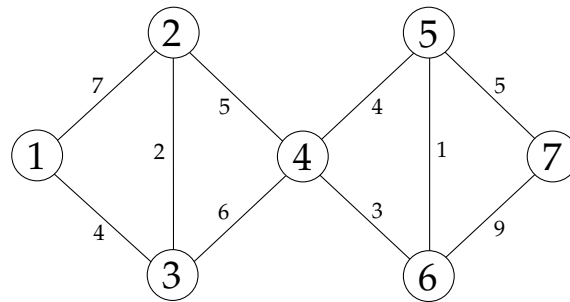


Figura 6.1: Problema da árvore de suporte de custo mínimo (Exemplo 6.1)

O primeiro arco a ser escolhido é o arco (5,6). O arco é adicionado à solução uma vez que não gera nenhum ciclo. Os arcos que são adicionados de seguida são os arcos (2,5), (4,6) e (1,3). O arco (4,5) tem um custo igual a 4, e é o próximo arco a ser considerado. No entanto, a sua inclusão na solução que vem sendo construída gera um ciclo. Por essa razão, o arco não é adicionado à árvore. A Figura 6.2 ilustra cada um dos passos do método. Os arcos a negro são os arcos que vão sendo adicionados à solução. O algoritmo termina quando foram adicionados 6 arcos. Esses arcos definem a árvore de suporte de custo mínimo do grafo.

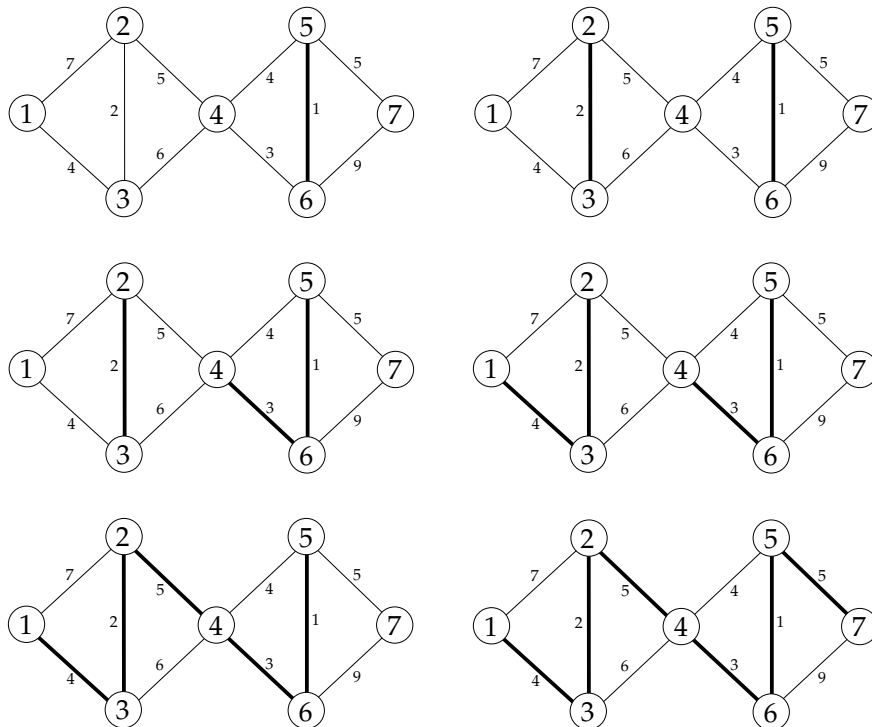


Figura 6.2: Solução do problema da árvore de suporte de custo mínimo (Exemplo 6.1)

□

Este método é conhecido na literatura por algoritmo de Kruskal.

6.3.3 Exemplo II: o Problema do Caixeiro Viajante

Na maior parte dos casos, os métodos de construção baseados em estratégias miópicas não garantem que a solução óptima de um problema seja encontrada. É o caso do problema do caixeiro viajante, outro problema clássico de optimização combinatória. Ao contrário do problema da árvore de suporte de custo mínimo, o problema do caixeiro viajante é de muito difícil resolução, mesmo se existem alguns casos especiais que podem ser resolvidos de forma eficiente. O problema do caixeiro viajante pertence a uma classe específica de complexidade. O problema é *NP-difícil*, o que, em termos práticos, significa que não é possível construir um algoritmo que resolva de forma eficiente qualquer instância do problema, atendendo ao estado actual dos conhecimentos.

O problema do caixeiro viajante consiste em determinar o circuito de menor custo que passa por todos os vértices de um grafo G uma única vez. Esse grafo poderá representar por exemplo um conjunto de cidades, e as ligações entre elas. Nesse caso, o custo dos arcos corresponderão a distâncias entre cada par de cidades. Existem algumas variantes do problema. Aqui, consideramos o caso em que o grafo $G = (V, A)$ é não orientado, e os custos associados aos seus arcos não são sujeitos a nenhuma restrição particular.

Os métodos de construção para este problema são também muito simples. Um deles é a heurística do vizinho mais próximo que consiste em adicionar arcos por ordem crescente do seu custo, partindo de um vértice arbitrário do grafo, e ligando sempre o último vértice atingido a um vértice que ainda não foi visitado. Quando todos os vértices foram visitados, o último vértice é ligado ao vértice de partida. O circuito é construído caminhando sempre no sentido do vértice adjacente (o vizinho) mais próximo.

As heurísticas de inserção são métodos alternativos que vão inserindo gradualmente vértices num circuito definido a partir de um subconjunto de vértices do grafo. Os vértices a adicionar podem ser escolhidos atendendo a diferentes critérios. Por exemplo, um vértice pode ser inserido no circuito se for o vértice que estiver mais próximo de um vértice desse circuito (o arco que o liga ao circuito tem o menor custo). O vértice escolhido é adicionado de forma a minimizar o incremento no custo total do circuito. Formalmente, seja k o vértice escolhido, e $V' \subset V$ os vértices do grafo que pertencem ao circuito. O vértice k deve ser inserido entre dois vértices $i, j \in V'$ do circuito, com $(i, j) \in A$, tal que

$$c_{ik} + c_{kj} - c_{ij}$$

seja o menor possível. Em alternativa, em vez de escolher um vértice e só depois determinar a sua posição no circuito, poder-se-ão explorar todos os vértices que não estejam no circuito assim como as diferentes posições onde eles podem ser inseridos. Entre todas as soluções possíveis (vértice e posição no circuito), escolhe-se aquela que produza o menor incremento no custo total. Essa heurística é designada por heurística de inserção de menor custo.

As duas últimas heurísticas produzem resultados com aproximação garantida para instâncias do problema que verificam a desigualdade triangular seguinte:

$$c_{ik} \leq c_{ij} + c_{jk}, \forall i, j, k \in V.$$

Para esses casos, o resultado das heurísticas nunca é pior que o dobro do valor da solução óptima.

Exemplo 6.2 Considere a instância do problema do caixeiro viajante ilustrada na Figura 6.3.

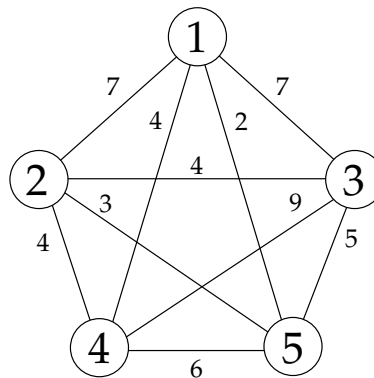


Figura 6.3: Problema do caixeiro viajante (Exemplo 6.2)

Partindo do vértice 1, a heurística do vizinho mais próximo vai adicionando o arco de menor custo que parte do último vértice que foi atingido:

- 1
- 1 – 5
- 1 – 5 – 2
- 1 – 5 – 2 – 4
- 1 – 5 – 2 – 4 – 3
- 1 – 5 – 2 – 4 – 3 – 1

O custo total dessa solução é igual a 25. O circuito final é ilustrado na Figura 6.4.

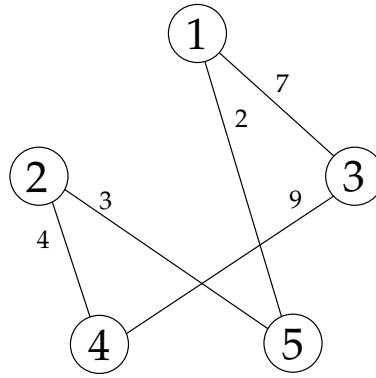


Figura 6.4: Solução do problema do caixeiro viajante (vizinho mais próximo)

A primeira heurística de inserção que descrevemos pode ser iniciada com um circuito composto por apenas um vértice. Vamos escolher o vértice 1. O primeiro vértice que é adicionado é o vértice 5 por ser o vértice mais próximo do vértice 1. O custo dessa inserção é igual 2, e o circuito que é gerado é o seguinte:

$$1 - 5 - 1.$$

O vértice mais próximo de um dos vértices do circuito é o vértice 2. Nesta fase, o vértice só pode ser inserido entre os vértices 1 e 5, dando origem ao circuito:

$$1 - 2 - 5 - 1.$$

O vértice 4 é agora o mais próximo de um dos vértices do circuito. O custo de o adicionar ao circuito depende da posição onde ele vai ser inserido. Procuramos a posição que minimiza o incremento de custo:

$$\min\{c_{14} + c_{42} - c_{12}, c_{24} + c_{45} - c_{25}, c_{54} + c_{41} - c_{51}\} = \min\{1, 7, 8\} = 1.$$

O vértice é inserido entre os vértices 1 e 2, resultando no circuito seguinte:

$$1 - 4 - 2 - 5 - 1.$$

O vértice 3 é o único vértice do grafo que não está no circuito. O vértice pode ser inserido em 4 posições diferentes. A posição é escolhida atendendo aos diferentes incrementos de custo que são gerados:

$$\min\{c_{13} + c_{34} - c_{14}, c_{43} + c_{32} - c_{42}, c_{23} + c_{35} - c_{25}, c_{53} + c_{31} - c_{51}\} = 7.$$

O circuito final tem um custo igual a 20, e corresponde ao circuito seguinte:

$$1 - 4 - 2 - 3 - 5 - 1.$$

□

6.4 Pesquisa Local

6.4.1 Descrição Geral

Ao contrário dos métodos de construção, os métodos de pesquisa local partem de uma solução inicial, e procuram iterativamente determinar soluções de melhor qualidade. Os métodos de pesquisa local baseiam-se na exploração da vizinhança da solução corrente. A função de vizinhança é determinante para o sucesso desses métodos. É ela que define o conjunto de soluções que poderão ser exploradas, e eventualmente escolhidas, numa determinada iteração do método.

No caso do problema do caixeiro viajante, a função de vizinhança poderá ser definida a partir da troca de dois vértices que sejam adjacentes num circuito. Considere, por exemplo, um circuito que consiste numa sequência de vértices $a - b - c - d - e - a$. Os circuitos seguintes pertencem à vizinhança desse circuito:

$$a - c - b - d - e - a$$

$$a - b - d - c - e - a$$

$$a - b - c - e - d - a$$

Os métodos de pesquisa local são métodos de melhoramento que em cada iteração procuram substituir a solução corrente por uma solução da sua vizinhança que seja de melhor qualidade. Esses métodos terminam quando já não é possível melhorar a solução corrente (assumindo que o tempo limite de computação não foi excedido). A solução encontrada domina as outras soluções da sua vizinhança, e nesse sentido, é considerada como um óptimo local. A optimalidade da solução do ponto de vista do problema global depende da função de vizinhança que é usada.

Uma função de vizinhança é considerada como sendo exacta, se a solução gerada pelo método de pesquisa local for a solução óptima do problema. Um exemplo célebre é o método Simplex para problemas de Programação Linear. Como vimos, em cada iteração do método Simplex, é escolhida uma solução válida de base que é adjacente à solução corrente e de melhor qualidade. A solução que é determinada no final é melhor que as soluções vizinhas, mas é também garantidamente a solução óptima do problema. Contudo, na maior parte dos casos, os métodos de pesquisa local não garantem mais do que a convergência para um óptimo local. As meta-heurísticas (que integram tipicamente métodos de pesquisa local) procuram escapar a esses mínimos locais. O princípio fundamental desses métodos consiste em permitir transições para soluções que sejam de pior qualidade.

Um método de pesquisa local é constituído pelos seguintes elementos:

- um método de geração de soluções iniciais;
- uma função de vizinhança;
- uma função de avaliação;
- uma regra de selecção da próxima solução;
- um critério de paragem.

A solução inicial pode ser obtida de diferentes formas. A abordagem mais simples consiste em gerar essa solução aleatoriamente. Em alternativa, poderá ser usado um método de construção. Note que, em alguns casos, a simples construção de uma solução válida é por si só um problema complexo.

A função de vizinhança define o espaço de soluções que pode ser explorado em cada iteração de um método de pesquisa local. Esse espaço de soluções (a vizinhança) é composto tipicamente por soluções que são próximas da solução à qual é aplicada a função de vizinhança. Na próxima secção, analisamos alguns exemplos de funções de vizinhança para alguns problemas clássicos de Optimização Combinatória.

A função de avaliação define uma ordenação entre as soluções que estão na vizinhança de uma dada solução. É com base nela que, em cada iteração, é escolhida a solução seguinte. Em problemas de optimização, a função objectivo do problema é normalmente usada com esse fim.

Em cada iteração de um método de pesquisa local, é escolhida uma nova solução na vizinhança da solução corrente cujo valor da função de avaliação é melhor do que o dessa solução corrente. Essa nova solução pode ser escolhida de formas diferentes. Ao pesquisar a vizinhança, podemos optar por escolher a primeira solução cujo valor é melhor do que o da solução corrente. Uma alternativa consiste em escolher a melhor solução da vizinhança.

Um método de pesquisa local termina naturalmente quando atinge um óptimo local, isto é, quando não consegue identificar uma solução de melhor qualidade na vizinhança da solução corrente. Para prevenir os casos em que esse óptimo local não seja identificado de forma eficiente, outros critérios poderão ser usados como o tempo de execução, ou o número de iterações efectuadas. Uma alternativa consiste ainda em terminar a pesquisa se ao longo de um dado número de iterações não tiver havido uma melhoria significativa no valor da função de avaliação.

Uma forma simples de melhorar a eficácia de uma heurística consiste em repeti-la usando parâmetros iniciais diferentes em cada execução. No contexto da pesquisa local, isso corresponde executar várias vezes o método partindo sempre de uma solução inicial que ainda não foi usada.

6.4.2 Funções de Vizinhança

Seja P um determinado problema de optimização. Uma instância I de P é completamente definida por um conjunto de soluções válidas S , e por uma função objectivo f , com a qual é possível avaliar cada uma das soluções de S . Temos assim que $I = (S, f)$. Uma função de vizinhança $V : S \rightarrow T$ determina o conjunto de soluções que irão ser consideradas como vizinhas das soluções $i \in S$. Uma solução j será vizinha de i se e só se $j \in V(i)$. Na maior parte dos casos, essas funções são simétricas: se $j \in V(i)$, então $i \in V(j)$.

As funções de vizinhança têm um impacto importante quer na eficácia, quer na eficiência dos métodos de pesquisa local. Aspectos como a dimensão da vizinhança, a sua estrutura, e a simplicidade do processo de geração de soluções vizinhas devem ser tidos em conta ao definir essas funções. Vizinhanças de grande dimensão incluem geralmente soluções de melhor qualidade, mas obrigam a um esforço computacional de pesquisa maior. As funções de vizinhança devem garantir ainda que o óptimo global do problema não deixe de poder ser atingido desde qualquer ponto do espaço de soluções.

Essas propriedades dependem invariavelmente do problema em causa, e da forma como são representadas as suas soluções. Ao longo do tempo, muitas funções têm sido exploradas. As mais simples consistem em permutar alguns elementos da solução corrente nos casos em que essas soluções podem ser representadas através de sequências de elementos. Um exemplo clássico é o caso do problema do caixeiro viajante, em que uma solução pode ser representada como uma sequência de cidades.

A título de exemplo, enumeram-se abaixo algumas funções de vizinhança simples para o problema de Programação Inteira, o problema do caixeiro viajante, e outros problemas clássicos de Optimização Combinatória.

Problema de Programação Inteira

Vamos considerar o caso em que todas as variáveis de decisão do problema são variáveis binárias. Como vimos no Capítulo 3, qualquer problema com variáveis inteiras gerais pode ser facilmente transformado num problema que inclua apenas variáveis binárias (mesmo se isso é feito ao preço de um aumento substancial do número de variáveis). As soluções de um problema de Programação Inteira com variáveis de decisão binárias $x_1, x_2, \dots, x_n$ podem ser representadas através da sequência de valores 0 e 1 atribuídos a cada uma dessas variáveis. Uma função de vizinhança que pode ser usada neste caso consiste em trocar o valor de k variáveis. Se tivermos, por exemplo, um problema com 4 variáveis (x_1, x_2, x_3, x_4) , e para $k = 2$, a solução

$$(1, 0, 0, 1),$$

terá na sua vizinhança as soluções seguintes

$$\begin{aligned} &(0, 1, 0, 1), \\ &(0, 0, 1, 1), \\ &(0, 0, 0, 0), \\ &(1, 1, 1, 1), \\ &(1, 1, 0, 0), \\ &(1, 0, 1, 0). \end{aligned}$$

assumindo que todas elas são soluções válidas do problema original.

Problema do caixeiro viajante

Considere um problema do caixeiro viajante definido a partir de um grafo $G = (V, A)$. As soluções desse problema são circuitos que podem ser representados através de sequências de $|V|$ vértices que incluam todos os vértices de V . A vizinhança de uma solução pode ser definida com base em todas as soluções que são obtidas escolhendo um vértice do circuito, e colocando-o em qualquer outra posição do circuito. Assim, se tivermos por exemplo um circuito definido pela sequência de vértices $a - b - c - d - e - a$, as soluções seguintes estarão na vizinhança dessa solução:

$$\begin{aligned} &b - a - c - d - e - b \\ &b - c - a - d - e - b \\ &b - c - d - a - e - b \end{aligned}$$

Outra função de vizinhança que pode ser usada para este problema é aquela que apresentámos no início da secção 6.3.1. Essa vizinhança corresponde às soluções que são obtidas invertendo a ordem pela qual são visitados um subconjunto de vértices adjacentes de um circuito. No exemplo que descrevemos, considerámos apenas a troca de 2 vértices. No entanto, essa troca poderá envolver mais vértices. Se considerarmos, por exemplo, a troca de 3 vértices para um circuito $a - b - c - d - e - a$, a vizinhança dessa solução incluirá as soluções seguintes:

$$\begin{aligned} &c - b - a - d - e - c \\ &a - d - c - b - e - a \\ &a - b - e - d - c - a \\ &d - b - c - a - e - d \end{aligned}$$

Uma função de vizinhança que tem dado melhores resultados do que as duas últimas que descrevemos consiste em remover k arcos do circuito (com $k \geq 2$), e voltar a ligar as

componentes com outros arcos. Partindo agora de um circuito com 7 vértices definido através da sequência $a - b - c - d - e - f - g - a$, e considerando o caso em que $k = 2$, a solução seguinte:

$$a - b - f - e - d - c - g - a$$

é vizinha do circuito inicial. É obtida retirando os arcos (b, c) e (f, g) , e ligando as componentes que são criadas através dos arcos (b, f) e (c, g) . Note que as vizinhanças geradas pela primeira função que descrevemos estão contidas nas vizinhanças desta última função com $k = 3$.

Problema de empacotamento

O problema de empacotamento consiste em determinar a forma como devem ser alocados um conjunto de itens a um conjunto de contentores de modo a minimizar o número de contentores que são usados. Uma instância do problema de empacotamento a uma dimensão é definida através de um conjunto I de itens de tamanho w_i , $i = 1, \dots, |I|$, e por um conjunto virtualmente ilimitado de contentores de largura W .

As soluções do problema de empacotamento podem ser representadas, por exemplo, através de uma sequência de itens. Essa sequência define a ordem pela qual os itens são colocados nos contentores. Sempre que a capacidade residual do último contentor preenchido não é suficiente para acomodar um item, esse item é colocado num novo contentor.

Partindo dessa representação, é possível definir várias funções de vizinhança semelhantes às que foram descritas para o problema do caixeiro viajante, uma vez que a representação das soluções é a mesma para ambos. Assim, as soluções vizinhas podem ser obtidas, por exemplo, com base na troca de dois ou mais elementos da sequência. Em alternativa, a vizinhança pode ser definida a partir das soluções que são obtidas permutando subsequências da sequência original.

Problema de coloração de grafos

O problema de coloração de grafos é definido a partir de um grafo $G = (V, A)$, e consiste em determinar o conjunto mínimo de cores com as quais é possível pintar os vértices de G de modo a que todos os vértices adjacentes tenham cores diferentes. Esse valor mínimo é também conhecido por número cromático do grafo.

Uma solução do problema de coloração de grafos corresponde a uma partição de V em subconjuntos aos quais está associada uma cor. A função de vizinhança que é usada tradicionalmente para este problema consiste em trocar a cor de apenas um dos vértices de uma solução. Essa troca corresponde a retirar um vértice do grupo onde se encontra, e adicioná-lo a outro grupo de cor diferente. Uma alternativa consiste em

definir uma vizinhança com base na troca de dois elementos que pertençam a grupos de cor distinta.

Foram propostas outras funções de vizinhança para este problema que resultam tipicamente em espaços de soluções (vizinhas) de grande dimensão. Um exemplo consiste em considerar uma sequência de vértices de cores distintas, e atribuir a cada um desses vértices a cor do vértice seguinte na sequência.

6.4.3 Exemplo: o Problema do Caixeiro Viajante

Nesta secção, ilustramos o funcionamento global de um método de pesquisa local usando como exemplo o problema do caixeiro viajante. Assumimos que a solução inicial é determinada através da heurística do vizinho mais próximo, e que a vizinhança de uma solução é definida a partir das soluções que são obtidas removendo um vértice do respectivo circuito, e colocando-o em qualquer outra posição. Em cada iteração, escolhemos a melhor solução na vizinhança da solução corrente. Vamos também assumir que a função de avaliação corresponde à função objectivo do problema.

Vamos considerar o caso da instância definida através do grafo da Figura seguinte.

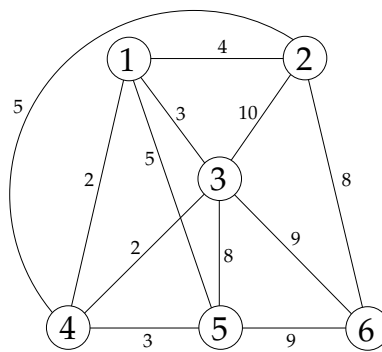


Figura 6.5: Problema do caixeiro viajante

A heurística do vizinho mais próximo gera a solução que é ilustrada na Figura 6.6, e que corresponde ao circuito:

$$1 - 4 - 3 - 5 - 6 - 2 - 1.$$

O comprimento total desse circuito é igual a 33.

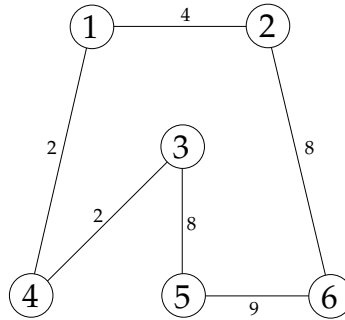


Figura 6.6: Solução inicial para o problema do caixeiro viajante

As soluções que estão na vizinhança desse circuito são obtidas trocando a posição de um vértice do circuito. O vértice escolhido só pode ser colocado em posições para as quais existem arcos correspondentes no grafo. Partindo do vértice 1, obtemos as seguintes soluções vizinhas:

$$4 - 1 - 3 - 5 - 6 - 2 - 4$$

$$4 - 3 - 1 - 5 - 6 - 2 - 4.$$

O comprimento desses circuitos é igual respectivamente a 34 e 32. Se tivéssemos optado por uma regra de selecção que consistisse em escolher a primeira solução de melhor qualidade que a solução corrente, a iteração terminaria já nesta fase, e a solução passaria a ser o circuito $4 - 3 - 1 - 5 - 6 - 2 - 4$. Neste caso, procuramos a melhor solução na vizinhança da solução corrente. Por essa razão, a pesquisa prossegue com a exploração das restantes soluções na vizinhança do circuito inicial.

A remoção do vértice 2 da sua posição actual obriga a ligar os vértices 6 e 1, qualquer que seja a posição onde o vértice 2 venha a ser inserido. Uma vez que o arco $(6, 1)$ não pertence ao conjunto de arcos do grafo, a remoção desse vértice não é considerada. O mesmo acontece no caso do vértice 6. A não existência de um arco entre dois vértices de um grafo pode ser representada através de um arco de custo muito elevado. Claramente, os circuitos que passam por arcos desse tipo serão sempre dominados por circuitos que não incluam esses arcos.

A remoção do vértice 3 conduz às seguintes soluções vizinhas:

$$1 - 4 - 5 - 3 - 6 - 2 - 1 \quad (\text{Comprimento total: } 34)$$

$$1 - 4 - 5 - 6 - 3 - 2 - 1 \quad (\text{Comprimento total: } 37)$$

$$1 - 4 - 5 - 6 - 2 - 3 - 1 \quad (\text{Comprimento total: } 35)$$

$$3 - 4 - 5 - 6 - 2 - 1 - 3 \quad (\text{Comprimento total: } 29)$$

Ao removermos o vértice 4, apenas 2 circuitos válidos são gerados:

$$1 - 3 - 4 - 5 - 6 - 2 - 1 \quad (\text{Comprimento total: } 29)$$

$$1 - 3 - 5 - 6 - 2 - 4 - 1 \quad (\text{Comprimento total: } 35)$$

Finalmente, a remoção do vértice 5 conduz às soluções vizinhas seguintes:

$$5 - 4 - 3 - 6 - 2 - 1 - 5 \quad (\text{Comprimento total: } 31)$$

$$1 - 4 - 5 - 3 - 6 - 2 - 1 \quad (\text{Comprimento total: } 35)$$

As melhores soluções na vizinhança do circuito original têm um comprimento igual a 29. O desempate é feito de forma arbitrária. No nosso caso, escolhemos o circuito $3 - 4 - 5 - 6 - 2 - 1 - 3$ que é ilustrado na Figura 6.7.

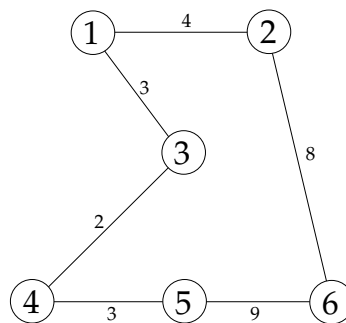


Figura 6.7: Solução válida para o problema do caixeiro viajante (ótimo local)

A exploração da vizinhança dessa solução não permite identificar nenhum circuito de comprimento inferior a 29. A solução corrente é considerada um ótimo local, e termina assim a execução do método de pesquisa local.

6.5 Pesquisa Tabu

6.5.1 Descrição Geral

Um dos grandes inconvenientes dos métodos de pesquisa local reside no facto de eles convergirem frequentemente para soluções que são apenas ótimos locais. As meta-heurísticas procuram escapar a esses ótimos locais permitindo que numa dada iteração seja escolhida uma solução cujo valor seja pior do que o da solução corrente. Os métodos baseados em pesquisa tabu enquadram-se no grupo das meta-heurísticas.

Os métodos de pesquisa tabu integram procedimentos de pesquisa local. Em cada iteração, a pesquisa de uma solução é feita na vizinhança da solução corrente. No

entanto, e ao invés do que acontece com os métodos de pesquisa local, um método de pesquisa tabu poderá prosseguir mesmo depois de ter atingido um ótimo local. Nesses casos, a nova solução é escolhida na vizinhança do ótimo local, e é, por definição, uma solução de pior qualidade. Os métodos de pesquisa tabu implementam mecanismos próprios para evitar que processo entre em ciclo. Esses mecanismos baseiam-se no princípio de memória, e são implementados através de uma ou várias *listas tabu*.

As listas tabu registam informação relativa às últimas soluções que foram escolhidas pelo método. Essa informação define aquilo que irão ser movimentos proibidos¹, ou *movimentos tabu*. Nesses casos, diz-se também que a lista regista movimentos tabu. Numa determinada iteração do método, todas as soluções associadas a movimentos tabu são sistematicamente evitadas. Na prática, a informação que é guardada nas listas tabu restringe a vizinhança que é efectivamente explorada em cada iteração. As listas tabu podem ser usadas também para guardar soluções completas. No entanto, essa alternativa é pouco usada na prática uma vez que implica um esforço computacional maior para validar os movimentos que podem ser efectuados em cada iteração.

A título de exemplo, vamos considerar o caso do problema do caixeiro viajante, e de uma função de vizinhança que consiste em eliminar dois arcos do circuito, e substituí-los por dois arcos diferentes. Uma lista tabu poderá por exemplo registar os pares de arcos que foram eliminados nas 3 últimas iterações. Nesse caso, podemos considerar um movimento como sendo tabu se corresponder a adicionar qualquer um dos arcos contidos na lista tabu.

O número de iterações ao longo das quais um movimento tabu é mantido na lista define o tamanho da lista. O tamanho das listas tabu não é necessariamente estático. Uma forma de reduzir o risco do processo entrar em ciclo consiste precisamente em variar o tamanho das listas ao longo da execução do método.

A pesquisa tabu autoriza alguma flexibilidade relativamente à rejeição dos movimentos tabu. Um movimento tabu pode ser autorizado se se verificar um determinado critério de aspiração que é frequentemente associado à qualidade da solução que resulta desse movimento. Se a solução que é obtida através de um movimento tabu for melhor que a melhor solução obtida até ao momento, o movimento tabu poderá ser autorizado, por exemplo. Outro critério de aspiração consiste em autorizar o movimento mais antigo da lista tabu, se todos os movimentos possíveis numa dada iteração forem tabu.

Os métodos de pesquisa tabu implementam mecanismos que poderão ser baseados em memória de curto prazo, ou em memória de longo prazo. Os primeiros corre-

¹Um movimento corresponde a uma transformação da solução corrente que resulta numa transição para uma solução vizinha.

spondem aos métodos de pesquisa tabu mais elementares. Nesses casos, a pesquisa é realizada num subconjunto da vizinhança da solução corrente. O objectivo consiste em evitar soluções que tenham sido geradas algumas iterações antes. Os métodos de pesquisa tabu que assentam em memória de longo prazo permitem que a pesquisa seja conduzida fora da vizinhança da solução corrente. O princípio de base dessas abordagens consiste em tentar conduzir a pesquisa para regiões do espaço de soluções que sejam potencialmente atractivas. Essas regiões podem ser identificadas, por exemplo, a partir de elementos da solução que tenham sido sistematicamente usados em várias iterações sucessivas.

O Algoritmo 6.1 sintetiza os passos gerais de um método elementar de pesquisa tabu baseado em memória de curto prazo.

Algoritmo 6.1: Pesquisa Tabu

```

1 Input: Instância  $I = (S, f)$  de um problema de optimização (minimização) ;
   /* Assumimos que a função objectivo do problema é usada para
      avaliar as soluções */
2 Seja  $t$  o tamanho da lista tabu ;
3 Determinar uma solução inicial  $x_0$  para  $I$  ;
4  $\bar{x} := x_0, \bar{f} := f(x_0), i := 1$  ;
5 Enquanto não é satisfeito o critério de paragem fazer
6   | Seja  $V'$  um subconjunto das soluções na vizinhança de  $x_{i-1}$  ;
7   |  $f' := +\infty$  ;
   | /* Determinar a melhor solução na vizinhança  $V'$  de  $x_{i-1}$ . */
8   | Para todos os  $x' \in V'$  fazer
9   |   | se (( $x'$  não está associada a um movimento tabu) ou ( $x'$  satisfaz algum
   |   | critério de aspiração)) e ( $f(x') < f'$ ) então
10  |   |   |  $x_i := x', f' := f(x')$  ;
11  |   |   | se  $f' < \bar{f}$  então
12  |   |   |   |  $\bar{x} := x', \bar{f} := f'$  ;
13  |   | Actualizar a lista tabu ;
   |   | /* Remover o movimento tabu mais antigo, e colocar o novo no
   |   | início da lista. */
14  |   |  $i := i + 1$  ;
15 Output:  $(\bar{x}, \bar{f})$  ;
16 Stop ;

```

6.5.2 Memória de Curto Prazo

A principal característica que distingue um método de pesquisa tabu de um método de pesquisa local standard é o recurso à memória. Na prática, esse conceito é implementado através das listas tabu. Os elementos de uma lista tabu dependem das soluções ou movimentos que foram realizados em iterações anteriores, e definem movimentos (tabu) que não poderão ser efectuados na próxima iteração.

Na sua versão mais simples, a pesquisa tabu implementa a chamada *memória de curto prazo*. Na maior parte dos casos, esses métodos vão guardando na lista os movimentos ou as soluções que foram gerados nas iterações mais recentes. Ao adicionar um elemento à lista, o elemento mais antigo é removido dessa lista. Esses métodos não eliminam por completo o risco do processo de pesquisa entrar em ciclo. Apenas garantem que as soluções mais recentes não voltam ser geradas nas próximas iterações.

A título de exemplo, vamos considerar o caso do Problema de Programação Inteira com variáveis binárias, e a função de vizinhança que descrevemos na Secção 6.4.2 (com $k = 2$). Vamos assumir que o problema tem 4 variáveis designadas por $x_1, x_2, \dots, x_4$, que a lista tabu L regista as últimas trocas que foram efectuadas e que não poderão ser repetidas na próxima iteração, e que L tem no máximo 2 elementos. Vamos ainda assumir que a solução corrente corresponde a

$$(1, 0, 1, 1),$$

e que a lista tabu contém os elementos seguintes

$$L = \{(x_1, x_2), (x_1, x_3)\}.$$

O elemento (x_1, x_2) dessa lista define completamente o último movimento que foi efectuado. A solução anterior à solução corrente corresponde assim à solução

$$(0, 1, 1, 1).$$

Na próxima iteração, as soluções que não serão consideradas por estarem associadas a movimentos tabu serão respectivamente:

$$(0, 1, 1, 1), \text{ e}$$

$$(0, 0, 0, 1).$$

Desta forma, evitamos que a solução anterior seja novamente gerada.

O tamanho de uma lista tabu influencia directamente a pesquisa. Quando o número de elementos que são guardados na lista é reduzido, a pesquisa tende a concentrar-se

na vizinhança do óptimo local. Listas maiores permitem escapar mais facilmente a um óptimo local.

Dependendo da função de vizinhança que é usada, o espaço de soluções a explorar em cada iteração poderá ser potencialmente muito grande. Para acelerar a exploração desse espaço, a pesquisa tabu implementa mecanismos que permitem restringir o conjunto de soluções que são efectivamente exploradas em cada iteração. Alguns desses mecanismos baseiam-se em regras a partir das quais são construídas listas de movimentos que serão escolhidos preferencialmente. Essas listas são também designadas por listas de candidatos.

Essas regras dependem na maior parte dos casos do próprio problema. Contudo, existem algumas estratégias para construir as listas de candidatos independentemente do problema em causa. Uma delas consiste em registar periodicamente um conjunto de “bons” movimentos. Em cada iteração, o próximo movimento é escolhido dentro desse conjunto, e corresponde àquele que produzir o melhor ganho. Logo que esse ganho seja inferior a um determinado limite, o conjunto é actualizado.

Uma estratégia alternativa consiste em escolher provisoriamente o primeiro movimento que conduza a uma solução cujo valor da função de avaliação seja melhor do que um determinado limite. Os métodos que usam essa estratégia efectuem geralmente mais algumas iterações (em número limitado) depois de ter encontrado essa solução na tentativa de encontrar uma solução de melhor qualidade.

6.5.3 Memória de Longo Prazo

Em muitos casos, o recurso a estratégias baseadas em memória de longo prazo permite aumentar a eficácia dos métodos de pesquisa tabu. A memória de um método de pesquisa tabu é considerada *de longo prazo* quando os elementos que são guardados dizem respeito a informação (movimentos ou qualquer outro atributo de uma solução) das iterações que foram sendo efectuadas desde o início da pesquisa. Na prática, essa informação é usada para guiar a pesquisa para regiões do espaço de soluções que não estejam na vizinhança da solução corrente.

Os métodos baseados em memória de longo prazo recorrem tipicamente a registos de frequências. Essas frequências podem ser de dois tipos:

- frequências com que determinados atributos têm permanecido na solução corrente ao longo das várias iterações: no caso do problema do caixeiro viajante, por exemplo, essa medida poderá corresponder ao número de iterações durante as quais um determinado arco tem feito parte da solução;
- frequência com que um movimento é efectuado, ou frequência com que um de-

terminado atributo é envolvido num movimento: ainda no caso do problema do caixeiro viajante, essa medida poderá estar associada ao número de vezes que um arco é adicionado ou removido do circuito.

Essas frequências são usadas para penalizar ou favorecer determinados movimentos, incluindo movimentos que conduzam a soluções fora da vizinhança da solução corrente.

Os métodos de pesquisa tabu baseados em memória de (médio e) longo prazo permitem implementar estratégias ditas de *intensificação* e de *diversificação*. A primeira consiste em concentrar a pesquisa em determinadas regiões do espaço de soluções que tenham sido identificadas como sendo promissoras. Uma forma de implementar essa estratégia consiste em forçar determinados elementos a fazer parte da solução, e explorar o espaço de soluções associado. No caso do problema do caixeiro viajante, essa estratégia pode passar por exemplo por fixar um conjunto de arcos.

As estratégias de diversificação procuram conduzir a pesquisa para regiões que não tenham sido ainda exploradas. Nesse contexto, o registo de frequências é usado por exemplo para identificar elementos da solução que tenham sido pouco usados.

Neste texto, concentramos-nos na pesquisa tabu baseada em memória de curto prazo. Os exemplos que analisaremos mais adiante referem-se a estratégias desse tipo.

6.5.4 Exemplo I: Problema de Programação Inteira

Nesta secção, ilustramos o funcionamento de um método básico de pesquisa tabu usando como exemplo um problema de Programação Inteira. Vamos considerar em particular o modelo com variáveis binárias definido da forma seguinte:

$$\begin{aligned}
 \max \quad & z = 12x_1 + 17x_2 + 7x_3 + 23x_4 + 16x_5 + 19x_6 + 13x_7 \\
 \text{s. a} \quad & \\
 & 7x_1 + 9x_2 + 3x_3 + 11x_4 + 13x_5 + 5x_6 + 7x_7 \leq 25 \\
 & 17x_1 + 23x_2 + 9x_3 + 27x_4 + 28x_5 + 14x_6 + 19x_7 \leq 60 \\
 & 8x_1 + 8x_2 + 5x_3 + 14x_4 + 11x_5 + 12x_6 + 7x_7 \leq 32 \\
 & 9x_1 + 25x_2 + 6x_3 + 22x_4 + 31x_5 + 23x_6 + 11x_7 \leq 49 \\
 & 6x_1 + 16x_2 + 7x_3 + 17x_4 + 17x_5 + 12x_6 + 8x_7 \leq 33 \\
 & x_1, x_2, x_3, x_4, x_5, x_6, x_7 \in \{0, 1\}
 \end{aligned}$$

Como vimos, as soluções desses problemas são representadas naturalmente através do valor das suas variáveis. Para avaliarmos as soluções, vamos usar simplesmente a função objectivo do modelo.

A primeira componente de um método de pesquisa tabu que é necessário definir corresponde ao procedimento que irá ser usado para determinar uma solução inicial. No nosso caso, vamos usar um método de construção simples que consiste em atribuir o valor 1 às variáveis por ordem decrescente do seu coeficiente na função objectivo. Se ao atribuírmos o valor 1 a uma variável, alguma das restrições for violada, a variável é colocada a 0, e o método prossegue com a variável seguinte. A solução que obtemos para este caso é a seguinte:

$$(x_1, x_2, x_3, x_4, x_5, x_6, x_7) = (0, 0, 0, 1, 0, 1, 0).$$

O valor associado a esta solução é igual a 42.

A vizinhança de uma solução vai ser definida conforme indicámos na Secção 6.4.2 com $k = 1$, o que corresponde a considerar como vizinhas qualquer solução que difere apenas no valor de uma das suas variáveis. Consequentemente, em cada iteração, um movimento irá consistir na troca do valor de uma variável.

Sempre que o valor de uma variável for modificado, essa variável será registada na lista tabu, impedindo assim que ela seja envolvida nos movimentos que irão ser efectuados nas próximas iterações. Neste caso, vamos assumir que a lista (L) regista no máximo três elementos.

Finalmente, um movimento tabu será efectuado apenas se o valor da respectiva solução for melhor do que todas as outras soluções que foram encontradas até ao momento.

Na 1ª iteração, a lista tabu é vazia ($L = \{\}$). Na vizinhança da solução corrente (a solução inicial), existem apenas duas soluções válidas:

$$(0, 0, 0, 0, 0, 1, 0) \quad z = 19$$

$$(0, 0, 0, 1, 0, 0, 0) \quad z = 23$$

A solução corrente domina as soluções da sua vizinhança e é portanto um óptimo local. Para escapar a esse óptimo local, o método de pesquisa tabu vai optar por escolher a segunda solução (apesar do seu valor ser inferior ao da solução corrente). Esta solução corresponde a trocar o valor da variável x_6 . A variável é inserida na lista tabu:

$$L = \{x_6\}.$$

As iterações seguintes conduzem aos resultados descritos abaixo.

Iteração 2

Solução corrente: $(0, 0, 0, 1, 0, 0, 0)$, $z = 23$

Vizinhança:

$(1, 0, 0, 1, 0, 0, 0)$	$z = 35$
$(0, 1, 0, 1, 0, 0, 0)$	$z = 40$
$(0, 0, 1, 1, 0, 0, 0)$	$z = 30$
$(0, 0, 0, 0, 0, 0, 0)$	$z = 0$
$(0, 0, 0, 1, 0, 1, 0)$	tabu
$(0, 0, 0, 1, 0, 0, 1)$	$z = 36$

A quinta solução é obtida através de um movimento tabu. Essa solução corresponde exactamente à solução da iteração anterior. Nesta iteração, escolhemos a solução com $z = 40$ uma vez que domina todas as restantes soluções da vizinhança. A variável que é trocada neste caso é a variável x_2 .

Iteração 3

Solução corrente: $(0, 1, 0, 1, 0, 0, 0)$, $z = 40$

Lista tabu: $L = \{x_2, x_6\}$

Vizinhança:

$(0, 0, 0, 1, 0, 0, 0)$	tabu
$(0, 1, 0, 0, 0, 0, 0)$	$z = 17$

Um movimento tabu pode eventualmente ser escolhido se satisfizer os critérios de aspiração que foram estipulados. No nosso caso, efectuamos um movimento tabu apenas se ele conduzir a uma solução que domine a melhor solução que foi encontrada até esta iteração. Nesta iteração, o único movimento tabu que conduz a uma solução válida é aquele que gera novamente a solução da iteração anterior. Claramente, esse movimento não satisfaz o critério de aspiração. Optamos assim por escolher a 2ª solução que corresponde novamente a escapar de um óptimo local.

Iteração 4

Solução corrente: $(0, 1, 0, 0, 0, 0, 0)$, $z = 17$

Lista tabu: $L = \{x_4, x_2, x_6\}$

Vizinhança:

$(1, 1, 0, 0, 0, 0, 0)$	$z = 29$
$(0, 0, 0, 0, 0, 0, 0)$	tabu , $z = 0$
$(0, 1, 1, 0, 0, 0, 0)$	$z = 24$
$(0, 1, 0, 1, 0, 0, 0)$	tabu , solução anterior
$(0, 1, 0, 0, 0, 1, 0)$	tabu , $z = 36$
$(0, 1, 0, 0, 0, 0, 1)$	$z = 30$

Escolhemos a última solução, adicionamos a variável x_7 à lista tabu, e removemos a variável x_6 (a mais antiga) dessa lista.

Iteração 5

Solução corrente: $(0, 1, 0, 0, 0, 0, 1)$ $z = 30$

Lista tabu: $L = \{x_7, x_4, x_2\}$

Vizinhança:

$(1, 1, 0, 0, 0, 0, 1)$	$z = 42$
$(0, 0, 0, 0, 0, 0, 1)$	tabu , $z = 13$
$(0, 1, 1, 0, 0, 0, 1)$	$z = 37$
$(0, 1, 0, 0, 0, 0, 0)$	tabu , solução anterior

A primeira solução com $z = 42$ é escolhida, e a variável x_1 é adicionada à lista tabu.

Iteração 6

Solução corrente: $(1, 1, 0, 0, 0, 0, 1)$ $z = 42$

Lista tabu: $L = \{x_1, x_7, x_4\}$

Vizinhança:

$(0, 1, 0, 0, 0, 0, 1)$	tabu , solução anterior
$(1, 0, 0, 0, 0, 0, 1)$	$z = 25$
$(1, 1, 0, 0, 0, 0, 0)$	tabu , $z = 29$

Nesta iteração, escolhemos a 2ª solução apesar do seu valor ser uma vez mais inferior ao da solução corrente. Essa solução corresponde a trocar o valor de x_2 .

Iteração 7

Solução corrente: $(1, 0, 0, 0, 0, 0, 1)$, $z = 25$

Lista tabu: $L = \{x_2, x_1, x_7\}$

Vizinhança:

$(0, 0, 0, 0, 0, 0, 1)$	tabu , $z = 13$
$(1, 1, 0, 0, 0, 0, 1)$	tabu , solução anterior
$(1, 0, 1, 0, 0, 0, 1)$	$z = 32$
$(1, 0, 0, 0, 0, 1, 1)$	$z = 44$
$(1, 0, 0, 0, 0, 0, 0)$	tabu , $z = 12$

Escolhemos a 4ª solução com $z = 44$, e adicionamos a variável x_6 à lista tabu.

Iteração 8

Solução corrente: $(1, 0, 0, 0, 0, 1, 1)$, $z = 25$

Lista tabu: $L = \{x_6, x_2, x_1\}$

Vizinhança:

$(0, 0, 0, 0, 0, 1, 1)$	tabu , $z = 32$
$(1, 0, 1, 0, 0, 1, 1)$	$z = 51$
$(1, 0, 0, 0, 0, 0, 1)$	tabu , solução anterior
$(1, 0, 0, 0, 0, 1, 0)$	$z = 31$

Nesta iteração, escolhemos a solução com $z = 51$. Esta solução já é a solução óptima do problema. No entanto, o método de pesquisa tabu prosseguiria até que o critério de paragem que tivesse sido estipulado fosse satisfeito. Isso acontece porque o método não possui nenhuma forma de detectar a optimalidade de uma solução.

Note que a enumeração completa das soluções levaria a gerar $2^7 = 128$ soluções, se considerarmos também as soluções não válidas do problema. Até à 8ª iteração, o método de pesquisa tabu analisou apenas 49 soluções. Esse valor inclui os movimentos tabu, exceptuando aqueles que conduzem à solução da iteração anterior.

6.5.5 Exemplo II: Escalonamento de Tarefas numa Máquina

Nesta secção, exploramos o caso de um problema de escalonamento de tarefas numa máquina única. O objectivo deste problema consiste em determinar a ordem pela qual as tarefas devem ser processadas na máquina. Os elementos que definem o problema particular que iremos considerar são os seguintes:

- as tarefas são independentes, podendo ser realizadas por qualquer ordem;
- cada tarefa i deve ser concluída até à data d_i ;

- a conclusão de uma tarefa i pode eventualmente atrasar-se, ao preço de um custo unitário c_i por cada unidade de tempo em atraso;
- cada tarefa i é processada em p_i unidades de tempo;
- as tarefas devem ser escalonadas de modo a minimizar o custo total associado aos atrasos.

Vamos considerar uma instância com 4 tarefas, e com os tempos de processamento seguintes:

$$(p_1, p_2, p_3, p_4) = (7, 3, 8, 2).$$

Os prazos para a conclusão das tarefas são respectivamente iguais a

$$(d_1, d_2, d_3, d_4) = (8, 5, 14, 12).$$

Vamos assumir que por cada unidade de tempo em atraso, existe um custo de duas unidades monetárias.

As soluções deste problema de escalonamento podem ser representadas através de uma sequência de tarefas que corresponde à ordem pela qual as tarefas são realizadas na máquina. Uma função de vizinhança que pode ser usada neste caso consiste em considerar todas as trocas possíveis de duas tarefas nessa sequência. A título de exemplo, a vizinhança da solução

$$(1, 2, 3, 4),$$

será composta pelas soluções seguintes:

$$(2, 1, 3, 4),$$

$$(3, 2, 1, 4),$$

$$(4, 2, 3, 1),$$

$$(1, 3, 2, 4),$$

$$(1, 4, 3, 2), \text{ e}$$

$$(1, 2, 4, 3).$$

Para avaliarmos as soluções, iremos usar a função objectivo do problema que corresponde ao total dos custos associados aos atrasos.

Uma primeira solução para este problema pode ser obtida escalonando em primeiro lugar as tarefas cujo prazo de conclusão esteja mais próximo. Neste caso, a solução que obtemos tem um custo de 28 unidades monetárias, e corresponde à sequência seguinte:

$$(2, 1, 4, 3).$$

Vamos usar a lista tabu para registar a troca que acabou de ser efectuada, e vamos assumir que essa lista guarda no máximo dois elementos. Relativamente ao critério de paragem, optamos por terminar a pesquisa se após duas iterações consecutivas a melhor solução obtida até ao momento não tiver sido alterada. O resultado de cada uma das iterações é descrito a seguir.

Iteração 1

Solução corrente: $(2, 1, 4, 3)$, custo = 28

Lista tabu: $L = \{\}$

Vizinhança:

$(1, 2, 4, 3)$	custo = 22
$(4, 1, 2, 3)$	custo = 28
$(3, 1, 4, 2)$	custo = 54
$(2, 4, 1, 3)$	custo = 20
$(2, 3, 4, 1)$	custo = 26
$(2, 1, 3, 4)$	custo = 28

Escolhemos a solução que corresponde a trocar a ordem das tarefas 1 e 4, e cujo custo é igual a 20 unidades monetárias. Esse movimento é adicionado à lista tabu. Note que esta iteração corresponde a uma iteração típica de pesquisa local, em que o movimento que é efectuado conduz a uma solução de melhor qualidade na vizinhança directa da solução corrente.

Iteração 2

Solução corrente: $(2, 4, 1, 3)$, custo = 20

Lista tabu: $L = \{(1, 4)\}$

Vizinhança:

$(4, 2, 1, 3)$	custo = 20
$(1, 4, 2, 3)$	custo = 26
$(3, 4, 1, 2)$	custo = 48
$(2, 1, 4, 3)$	tabu , solução anterior
$(2, 3, 1, 4)$	custo = 36
$(2, 4, 3, 1)$	custo = 24

A solução que corresponde ao único movimento tabu não é sequer avaliada uma vez que conduz à solução da iteração anterior. Nesta iteração, escolhemos a solução cujo

custo é igual a 20 unidades monetárias. Essa solução envolve a troca da ordem das tarefas 2 e 4. Esse movimento é adicionado à lista tabu.

Iteração 3

Solução corrente: (4, 2, 1, 3), custo = 20

Lista tabu: $L = \{(2, 4), (1, 4)\}$

Vizinhança:

(2, 4, 1, 3)	tabu , solução anterior
(1, 2, 4, 3)	tabu , custo = 22
(3, 2, 1, 4)	custo = 48
(4, 1, 2, 3)	custo = 28
(4, 3, 1, 2)	custo = 48
(4, 2, 3, 1)	custo = 24

A solução corrente é melhor que todas as suas vizinhas (ótimo local). Para escapar a essa solução, optaríamos por escolher a última solução, cujo custo é igual a 24 unidades monetárias. No entanto, uma vez que foram efectuadas duas iterações seguidas sem que tenha sido melhorada a melhor solução encontrada até ao momento, e cujo custo é igual a 20 unidades monetárias, terminamos aqui a execução do método.

Note que, para esta instância de pequenas dimensões, o método de pesquisa tabu acaba por explorar grande parte do espaço de soluções válidas. Para uma instância com n tarefas, o número de sequências que é possível definir é igual a $n!$ (número total de permutações). Com a função de vizinhança que usámos neste exemplo, cada solução tem exactamente $\frac{n \times (n-1)}{2}$ soluções vizinhas. À medida que n aumenta, a diferença entre estes dois valores aumenta significativamente.

6.6 Pesquisa por Arrefecimento Simulado

6.6.1 Descrição Geral

Outra das meta-heurísticas que é usada com frequência para resolver problemas de optimização é a chamada *pesquisa por arrefecimento simulado*. Tal como as restantes meta-heurísticas, esses métodos tentam escapar a óptimos locais escolhendo eventualmente soluções que sejam piores do que a solução corrente em determinadas iterações. Uma das particularidades dos métodos de pesquisa por arrefecimento simulado reside no facto da probabilidade de escolher soluções desse tipo decrescer à medida que vão

sendo efectuadas iterações. Inicialmente, esses métodos tendem a permitir movimentos que conduzam a soluções de má qualidade com uma elevada probabilidade, para progressivamente decrescer essa probabilidade até ao ponto em que essas soluções são escolhidas de forma muito rara.

Na literatura anglo-saxónica, a pesquisa por arrefecimento simulado é designada por *simulated annealing*. Esta abordagem tenta reproduzir processos termodinâmicos que são usados por exemplo na indústria metalúrgica e na indústria vidreira, e que consistem em aquecer um material até uma determinada temperatura, e deixar depois essa temperatura diminuir progressivamente até que o material atinja um nível mínimo de energia ao qual corresponde uma estrutura sem defeitos. Transposto para o campo da optimização, este processo consiste em permitir movimentos iniciais que piorem o valor da função de avaliação, e em aceitar cada vez menos movimentos desse tipo à medida que o processo avança.

Os métodos de pesquisa por arrefecimento simulado determinam a próxima solução a partir da vizinhança da solução corrente, e nesse sentido são uma variante dos métodos de pesquisa local. Contudo, em vez de explorar toda a vizinhança da solução corrente, os métodos de pesquisa por arrefecimento simulado vão gerar essas soluções de forma aleatória. Se a solução que tiver sido gerada for melhor do que a solução corrente, essa solução será automaticamente escolhida. Caso contrário, a solução é escolhida apenas com uma determinada probabilidade p que depende da diferença entre os valores dessa solução e da solução corrente, e de um parâmetro que é designado por *temperatura*, devido à correspondência com o processo físico. A expressão exacta dessa probabilidade é a seguinte:

$$e^{-\Delta t},$$

com

$$\Delta t = \frac{|z_{i-1} - z'_i|}{T}.$$

Na iteração i , a solução corrente é aquela que foi escolhida na iteração anterior. O valor dessa solução corrente é designado por z_{i-1} . O valor da solução que é gerada na iteração i é designado por z'_i . O valor T representa a temperatura.

A probabilidade de aceitação da solução candidata na iteração i será tanto maior quanto maior for a temperatura T , e quanto menor for a diferença entre z_{i-1} e z'_i . Inicialmente, escolhe-se um valor elevado para essa temperatura de modo a permitir que soluções piores do que a solução corrente possam ser escolhidas. Nos processos físicos, isso corresponde à fase inicial de forte aquecimento do material. Ao longo das iterações, o valor de T é diminuído gradualmente até um ponto em que o valor

da probabilidade de aceitação é muito pequeno. Essa diminuição é feita segundo um esquema de arrefecimento que é definido em função do problema. Uma das regras mais usadas consiste em actualizar o valor da temperatura segundo a expressão geométrica seguinte:

$$T_{k+1} = \alpha T_k,$$

com $0 < \alpha < 1$, sendo que os valores mais usuais estão normalmente compreendidos entre 0.8 e 0.99. A *k^{esima}* temperatura que é usada é designada por T_k . A temperatura T_k deve ser mantida durante um determinado número L_k de iterações, após as quais a probabilidade de aceitação passa a ser calculada fazendo $T = T_{k+1}$. Em muitos casos, o número de iterações que é usado para cada temperatura T_k é o mesmo qualquer que seja o valor de k ($L_k = L, \forall k$).

Não existem regras gerais que possam ser usadas para determinar o valor de L e da temperatura inicial. Esses valores dependem tipicamente do problema em causa e também da função de vizinhança que é usada, e são normalmente afinados através da realização de experiências computacionais.

O critério de paragem dos métodos de pesquisa por arrefecimento simulado pode ser fixado de várias formas. Em primeiro lugar, e no âmbito do esquema de arrefecimento, deve ser definida uma temperatura mínima que corresponderá ao último valor que será usado para realizar a pesquisa. Adicionalmente, a pesquisa pode ser terminada após um determinado número de iterações, ou se não houver nenhuma melhoria significativa do valor da solução após um dado número de iterações.

O Algoritmo 6.2 sintetiza os passos dos métodos de pesquisa por arrefecimento simulado.

6.6.2 Exemplo: o Problema de Coloração de Grafos

O problema da coloração de grafos foi introduzido na Secção 6.4.2. O problema consiste em determinar o número mínimo de cores necessárias para pintar os vértices de um grafo de modo a que dois vértices adjacentes nunca tenham cores idênticas. Nesta secção, vamos ilustrar o funcionamento da pesquisa por arrefecimento simulado usando uma instância do problema de coloração de grafos definido a partir do grafo ilustrado na Figura 6.8.

Vamos assumir que as soluções do problema são representadas através de conjuntos de vértices aos quais foi atribuída a mesma cor. Por razões de clareza, cada conjunto será designado pelo nome da cor dos seus respectivos vértices. A função de vizinhança que iremos usar consiste em trocar a cor de apenas um vértice, o que corresponde a retirar o vértice do conjunto onde se encontra, e colocá-lo noutro conjunto. Vamos

Algoritmo 6.2: Pesquisa por Arrefecimento Simulado

```

1 Input: Instância  $I = (S, f)$  de um problema de optimização ;
   /* Assumimos que a função objectivo do problema é usada para
      avaliar as soluções */
2 Input: Esquema de arrefecimento, temperatura inicial  $T_0, L$  ;
   /* Assumimos que  $L_k = L, \forall k$  */
3 Determinar uma solução inicial  $x_0$  para  $I$  ;
4  $\bar{x} := x_0, \bar{f} := f(x_0), i := 1$  ;
5  $k := 0, j := 1$  ;
6 Enquanto não é satisfeito o critério de paragem fazer
7   Gerar aleatoriamente uma solução  $x'$  na vizinhança de  $x_{i-1}$  ;
8   se  $(f(x') \leq f(x_{i-1}))$  então
9      $x_i := x'$  ;
10    se  $f(x') < \bar{f}$  então
11       $\bar{x} := x', \bar{f} := f'$  ;
12    senão
13       $p := e^{-\frac{|f(x_{i-1}) - f(x')|}{T_k}}$  ;
14      Gerar um número aleatório  $p'$  entre 0 e 1 ;
15      se  $p' \leq p$  então
16         $x_i := x'$  ;
17      senão
18         $x_i := x_{i-1}$  ;
19       $i := i + 1, j := j + 1$  ;
20      se  $j > L$  então
21         $j := 1, k := k + 1$  ;
22        Calcular  $T_k$  segundo o esquema de arrefecimento ;
23 Output:  $(\bar{x}, \bar{f})$  ;
24 Stop ;

```

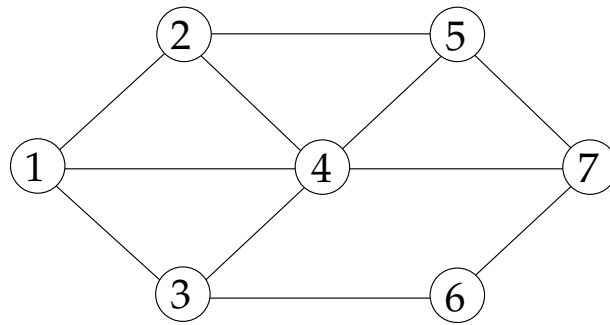


Figura 6.8: Problema de coloração de grafos

também considerar a possibilidade de atribuir a um vértice uma cor diferente daquelas que são usadas pelos outros vértices do grafo.

As soluções irão ser avaliadas com base na função objectivo do problema, *i.e.* contando o número de cores que cada uma usa.

Neste Exemplo, não iremos permitir que sejam escolhidas soluções não válidas que violem a única restrição do problema. Uma forma simples de gerar uma solução inicial que seja válida para este problema consiste em atribuir uma cor diferente a todos os vértices do grafo, o que conduz por exemplo à solução seguinte:

$$\begin{aligned} \text{Verde} &= \{1\}, \text{Azul} = \{2\}, \text{Vermelho} = \{3\}, \text{Amarelo} = \{4\}, \\ \text{Castanho} &= \{5\}, \text{Cinzento} = \{6\}, \text{Laranja} = \{7\}. \end{aligned}$$

Em cada iteração de um método de pesquisa por arrefecimento simulado, é gerada aleatoriamente uma solução candidata na vizinhança da solução corrente. No nosso caso, vamos gerar essa solução através dos passos seguintes:

- a) escolhemos um vértice assumindo que a probabilidade é a mesma qualquer que seja o vértice do grafo (no nosso Exemplo, a probabilidade de escolher qualquer um dos vértices é igual a $\frac{1}{7}$);
- b) escolhemos com igual probabilidade uma cor entre todas as cores usadas pelos outros vértices e uma “outra cor” (esta última permite que seja atribuída ao vértice uma cor diferente daquelas que foram atribuídas aos restantes vértices);
- c) atribuímos essa cor ao vértice escolhido.

Para implementarmos o primeiro desses passos, definimos intervalos de probabilidade para cada um dos vértices: para um vértice v , o intervalo corresponde a $\left[\frac{v-1}{7}, \frac{v}{7}\right]$. Para escolhermos um dos vértices, geramos um número aleatório entre 0 e 1, e escolhemos aquele cujo intervalo de probabilidades inclua esse número. A escolha da cor é

α	k	T_k	$ z_{i-1} - z'_i $	Probabilidade de aceitação
0.7	0	7.00000	1	0.86688
	1	4.90000		0.81540
	2	3.43000		0.74711
	3	2.40100		0.65936
	4	1.68070		0.55157
	5	1.17649		0.42742
	6	0.82354		0.29693
	7	0.57648		0.17646
	8	0.40354		0.08390
	9	0.28248		0.02901
	10	0.19773		0.00636

Tabela 6.1: Temperaturas e probabilidades de aceitação (esquema de arrefecimento)

feita de forma semelhante: se z_{i-1} for o valor da solução corrente, consideramos que existem z_{i-1} cores candidatas (incluindo a “outra cor”).

O esquema de arrefecimento que iremos usar baseia-se na expressão geométrica de actualização da temperatura que introduzimos na secção anterior. Vamos usar um valor de α igual a 0.7, e uma temperatura inicial T_0 igual ao número de vértices do grafo. A Tabela 6.1 ilustra a evolução da temperatura, e da respectiva probabilidade de aceitar uma solução pior do que a solução corrente no caso em que a diferença entre os valores dessas soluções é igual a uma unidade. Note que, com a função de vizinhança que definimos, a diferença entre a solução corrente e qualquer uma das suas vizinhas nunca é superior a 1.

Vamos ainda assumir que uma temperatura se mantém ao longo de 3 iterações consecutivas ($L = 3$). Um possível critério de paragem consiste em limitar o número máximo de iterações que irão ser realizadas. Ao fixarmos esse valor, fixamos automaticamente o valor da temperatura mínima que irá ser usada no método.

Descrevemos abaixo as iterações iniciais do método de pesquisa por arrefecimento simulado. Os valores aleatórios são dados a título ilustrativo. Se repetirmos a execução do método, esses valores serão inevitavelmente diferentes.

Iteração 1

Solução corrente:

$$\text{Verde} = \{1\}$$

Azul	=	{2}
Vermelho	=	{3}
Amarelo	=	{4}
Castanho	=	{5}
Cinzentos	=	{6}
Laranja	=	{7}

Custo da solução corrente: 7

Solução candidata:

a) Escolher vértice:

- Número aleatório: 0.17724
- Vértice escolhido: 2¹

b) Escolher cor²:

- Número aleatório: 0.20193
- Cor escolhida: Vermelho³

Solução candidata é válida? *sim*⁴

Custo da solução candidata: 6. A solução é automaticamente escolhida.

Número de iterações com T_0 : 1

Iteração 2

Solução corrente:

Verde	=	{1}
Vermelho	=	{2, 3}
Amarelo	=	{4}
Castanho	=	{5}
Cinzentos	=	{6}

¹Intervalo de probabilidades: [0.14286, 0.28571[.

²Assumimos que as cores estão ordenadas conforme indicado na solução corrente. A “outra cor” é adicionada no fim da lista.

³Intervalo de probabilidades: [0.14286, 0.28571[.

⁴Se a solução candidata não fosse válida, gerar-se-iam novos números aleatórios até encontrar uma solução válida ou atingir um critério de paragem.

$$\text{Laranja} = \{7\}$$

Custo da solução corrente: 6

Solução candidata:

a) Escolher vértice:

- Número aleatório: 0.59710
- Vértice escolhido: 5¹

b) Escolher cor:

- Número aleatório: 0.51524
- Cor escolhida: Cinzento²

Solução candidata é válida? *sim*

Custo da solução candidata: 5. A solução é automaticamente escolhida.

Número de iterações com T_0 : 2

Iteração 3

Solução corrente:

$$\begin{aligned}\text{Verde} &= \{1\} \\ \text{Vermelho} &= \{2, 3\} \\ \text{Amarelo} &= \{4\} \\ \text{Cinzento} &= \{5, 6\} \\ \text{Laranja} &= \{7\}\end{aligned}$$

Custo da solução corrente: 5

Solução candidata:

a) Escolher vértice:

- Número aleatório: 0.45562
- Vértice escolhido: 4³

¹Intervalo de probabilidades: [0.57143, 0.71429[

²Intervalo de probabilidades: [0.50000, 0.66667[

³Intervalo de probabilidades: [0.28571, 0.42857[

b) Escolher cor:

- Número aleatório: 0.87103
- Cor escolhida: “Outra cor”⁴

Solução candidata é válida? *sim*

Custo da solução candidata: 6. Decidir se deve ser aceite a solução candidata:

- Probabilidade de aceitação da solução candidata: $e^{-\frac{1}{7}} = 0.86688$
- Número aleatório: 0.23017; como $0.23017 \leq 0.86688$, a solução é aceite.

Número de iterações com T_0 : 3

A temperatura é actualizada. Passamos a usar a temperatura T_1 (4.9)

O método prosseguiria desta forma até que fosse satisfeito um dos critérios de paragem. Tal como acontece com as outras heurísticas, o método não tem forma de saber se a solução que obteve é de facto a solução óptima global.

6.7 Conclusões

Os métodos heurísticos são usados com muita frequência para resolver problemas de optimização. Esses métodos são usados quando o problema é demasiado complexo para ser resolvido através de um método exacto, ou quando a obtenção de uma solução óptima não é um requisito forte. Neste capítulo, analisámos alguns desses métodos começando pelos mais simples que consistem em construir de raiz uma solução válida para um problema. Descrevemos os métodos de pesquisa local, cujo princípio é usado de forma integrada em muitos outros métodos. Um dos principais inconvenientes desses métodos é o facto de obterem muitas vezes soluções que são apenas óptimos locais, *i.e.* que dominam as soluções na sua vizinhança directa, mas que não são necessariamente soluções óptimas para o problema global. As meta-heurísticas são métodos heurísticos genéricos que procuram escapar a esses óptimos locais autorizando transições para soluções que sejam piores do que a solução corrente. A pesquisa tabu e a pesquisa por arrefecimento simulado são dois exemplos de meta-heurísticas que foram aqui abordados.

O desenvolvimento de uma meta-heurística envolve normalmente um elevado grau de parametrização. A afinação dos parâmetros é realizada através de experiências computacionais, em que um conjunto (idealmente grande) de instâncias do problema em causa são resolvidas usando diferentes valores para os parâmetros.

⁴Intervalo de probabilidades: $[0.80000, 1.00000[$

Uma das formas de melhorar o desempenho de um método heurístico consiste em combiná-lo com outras abordagens heurísticas, dando assim origem a métodos híbridos. As possibilidades são inúmeras. Neste capítulo, descrevemos a estrutura básica de alguns dos métodos heurísticos mais usados na prática. Deixamos de lado muitos outros métodos, variantes de métodos, e estratégias de hibridização cuja análise sai claramente fora do âmbito deste texto.

6.8 Exercícios

1. Uma universidade quer ligar as suas escolas através de uma rede de comunicação de dados e voz. A configuração da rede é do tipo *peer-to-peer*: para garantir a ligação entre dois pontos da rede, é necessário que exista apenas um caminho entre eles. As várias possibilidades de interligação das escolas são ilustrada no grafo da Figura 6.9.

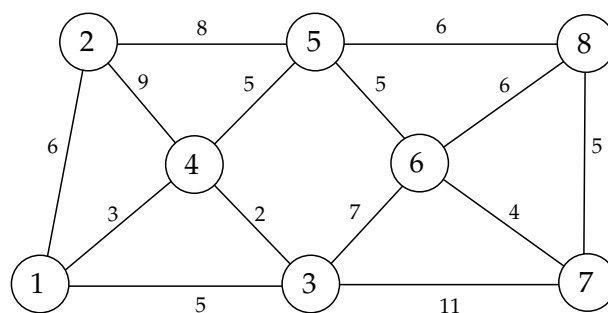


Figura 6.9: Estrutura da rede

Os valores junto aos arcos indicam o custo de instalação da respectiva ligação. A universidade pretende escolher as ligações que deverão ser instaladas de modo a minimizar o custo total de instalação e a garantir a conectividade da rede.

- a) Identifique o problema de optimização a que corresponde este problema.
 - b) Determine que ligações deverão ser instaladas para minimizar o custo total de instalação.
2. Considere uma instância do problema do caixeiro viajante definido a partir de um grafo completo com 6 vértices cuja matriz de custos é representada a seguir:

	2	3	4	5	6
1	3	6	5	1	6
2		5	10	7	4
3			7	5	8
4				6	9
5					3

Assuma que a matriz de custos é simétrica.

- a) Determine uma solução válida para esta instância usando a heurística do vizinho mais próximo.
 - b) Repita a heurística do vizinho mais próximo escolhendo como vértice inicial cada um dos vértices do grafo.
 - c) Determine uma solução usando a heurística de inserção de menor custo.
3. Considere uma instância do problema de empacotamento com contentores de largura $W = 100$, e 10 itens diferentes, cada um com procura unitária e tamanhos iguais respectivamente a 53, 50, 46, 37, 32, 24, 22, 15, 13 e 5.
- a) Obtenha uma solução válida para o problema usando uma heurística de construção que consiste em colocar os itens um a um nos contentores por ordem decrescente do seu tamanho (heurística *first fit decreasing*). Quantos contentores são usados?
 - b) Assuma que as soluções do problema são representadas através de sequências de itens que representam a ordem pela qual os itens são colocados nos contentores, e considere uma função de vizinhança que consiste em trocar a posição de dois itens nessa sequência.

Usando essa função de vizinhança, aplique o método de pesquisa local partindo da sequência que usou em a). Faça no máximo 3 iterações.

4. Considere o problema de caixeiro viajante definido num grafo completo com 5 vértices e com os custos representados na tabela seguinte:

	2	3	4	5
1	2	1	7	4
2		3	3	5
3			2	1
4				6

- a) Determine uma solução válida para esta instância usando a heurística do vizinho mais próximo e partindo do vértice 1.
- b) Assuma que as soluções do problema são representadas pela sequência de vértices que são atravessados ao percorrer o circuito. Considere uma função de vizinhança que consiste em trocar a ordem de dois vértices no circuito. Use o método de pesquisa local para melhorar a solução que obteve em a). Faça no máximo 3 iterações.
5. Considere a instância do problema de coloração de grafos definido a partir do grafo ilustrado na Figura 6.10.

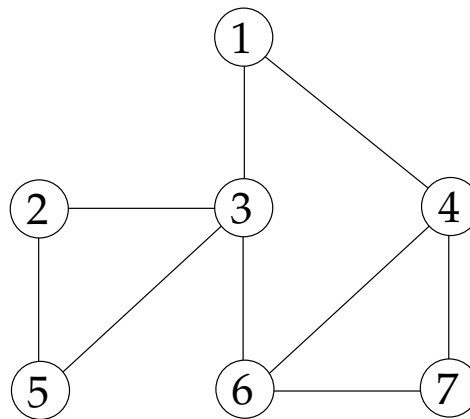


Figura 6.10: Problema de coloração de grafos

- a) Sugira uma forma de representar as soluções diferente daquela que foi usada no exemplo da Secção 6.6.2.
- b) Partindo da representação que escolheu em a), defina uma função de vizinhança para este problema. Enumere todas as soluções da instância representada acima que são vizinhas da solução que corresponde a pintar os vértices da forma seguinte:

$1, 2 \rightarrow \textit{verde}$
 $3 \rightarrow \textit{azul}$
 $4, 5 \rightarrow \textit{vermelho}$
 $6 \rightarrow \textit{amarelo}$
 $7 \rightarrow \textit{branco}$

- c) Defina os elementos de um método de pesquisa tabu para este problema.

- d) Aplique o método que definiu em c) partindo da solução descrita na alínea b). Faça no máximo 4 iterações.

6. Considere o problema de Programação Inteira seguinte:

$$\max z = 15x_1 + 7x_2 + 19x_3 + 14x_4 + 11x_5$$

s. a

$$4x_1 + 2x_2 + 5x_3 + 4x_4 + 3x_5 \leq 10$$

$$5x_1 + 2x_2 + 6x_3 + 5x_4 + 4x_5 \leq 13$$

$$x_1 + x_3 \leq 1$$

$$x_2 + 2x_4 + 5x_5 \leq 11$$

$$x_1, x_2, x_3, x_4, x_5 \in \{0, 1\}$$

Resolva este problema usando um método de pesquisa por arrefecimento simulado, e partindo da solução inicial seguinte:

$$(x_1, x_2, x_3, x_4, x_5) = (1, 0, 1, 0, 0).$$

Descreva todos os elementos do método que usar, e faça no máximo 3 iterações.

Bibliografia

- [1] E. Aarts e J. Lenstra. *Local Search in Combinatorial Optimization*. Princeton University Press, 2003.
- [2] R. Ahuja, T. Magnanti, e J. Orlin. *Network Flows: Theory, Algorithms and Applications*. Prentice Hall, 1993.
- [3] M. Bazaraa, J. Jarvis, H. Sherali. *Linear Programming and Network Flows*. John Wiley & Sons, 3rd Edition, 2005.
- [4] C. Blum e A. Roli. Metaheuristics in Combinatorial Optimization: Overview and Conceptual Comparison. *ACM Computing Surveys*, 35(3):268–308, 2003.
- [5] G. Dantzig, M. Thapa. *Linear Programming: Introduction*. Springer, 1997.
- [6] E. Denardo. *Dynamic Programming: Models and Applications*. Prentice Hall, 1982.
- [7] H. A. Eiselt, C. -L. Sandblom. *Integer Programming and Network Models*. Springer, 2000.
- [8] M. Garey e D. S. Johnson. *Computers and Intractability, A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979.
- [9] F. Glover e M. Laguna. *Tabu Search*. Kluwer Academic Publishers, 1997.
- [10] F. Hillier e G. Lieberman. *Introduction to Operations Research*. McGraw-Hill, 8th Edition, 2005.
- [11] B. Korte e J. Vygen. *Combinatorial Optimization: Theory and Algorithms*. Springer, 2nd Edition, 2002.
- [12] Z. Michalewicz, D. B. Fogel. *How to Solve it: Modern Heuristics*. Springer, 2nd Edition, 2004.
- [13] K. Murty. *Linear Programming*. John Wiley & Sons, 1983.

- [14] G. Nemhauser, A. Rinnooy Kan, e M. Todd. *Handbooks in Operations Research and Management Science: Volume 1: Optimization*. Elsevier, 1989.
- [15] G. Nemhauser, L. Wolsey. *Integer and Combinatorial Optimization*. John Wiley & Sons, 1999.
- [16] P. M. Pardalos e M. G. C. Resende. *Handbook of Applied Optimization*. Oxford University Press, 2002.
- [17] Y. Pochet e L. Wolsey. *Production Planning by Mixed Integer Programming*. Springer, 2006.
- [18] A. Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1998.
- [19] A. Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*. Springer, 2003.
- [20] G. Sierksma. *Linear and Integer Programming: Theory and Practice*. CRC Press, 2nd edition, 2001.
- [21] H. A. Taha. *Operations Research: An Introduction*. Prentice Hall, 8th Edition, 2006.
- [22] H. Williams. *Model Building in Mathematical Programming*. John Wiley & Sons, 1999.
- [23] L. Wolsey. *Integer Programming*. John Wiley & Sons, 1998.